

Full Runnable Gut Microbiome Project

This notebook combines the selected notebooks and Python files into one runnable notebook.

Run this notebook from top to bottom.

1. Create helper Python files and imports

In [121...

```
# Imports

from pathlib import Path
import sys
from config import DATA_DIR
import glob
import warnings
import missingno as msno
import os

# Data manipulation
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Statistics
from scipy.stats import pearsonr, spearmanr
from scipy.stats import ttest_ind
from scipy.stats import mannwhitneyu

# Model imports

# Regression
import statsmodels.api as sm
import statsmodels.formula.api as smf

from sklearn.model_selection import GroupKFold
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report, roc_auc_score
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.linear_model import LinearRegression
from xgboost import XGBRegressor
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
from sklearn.base import clone
from sklearn.model_selection import train_test_split
from scipy import stats
from sklearn.model_selection import cross_val_score, GridSearchCV
from sklearn.linear_model import Ridge
from sklearn.ensemble import RandomForestRegressor
```

```

from sklearn.svm import SVC
from sklearn.calibration import CalibratedClassifierCV
from lightgbm import LGBMClassifier
from sklearn.model_selection import GroupKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from scipy import stats
import seaborn as sns
print("Libraries loaded.")

```

Libraries loaded.

```

In [122... # Display settings
pd.set_option("display.max_columns", None)
pd.set_option("display.max_columns", 25)
pd.set_option("display.float_format", "{:.4f}".format)
sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams["figure.dpi"] = 100
print("Ready.")

```

Ready.

```

In [123... # Locate repo root (directory containing config.py) regardless of launch directo
_root = next(p for p in [Path.cwd(), *Path.cwd().parents] if (p / "config.py").e
DATA_DIR = _root / "data" # robust: does not depend on __file__ inside Jupyter
sys.path.insert(0, str(_root))

```

```

In [124... #Debugging code to check paths
print("cwd:", Path.cwd())
root = next(p for p in [Path.cwd(), *Path.cwd().parents] if (p / "config.py").ex
print("config.py found at:", root)
print("DATA_DIR would be:", root / "data")
print("data/ exists:", (root / "data").exists())
print("microbiome_filtered.csv exists:", (root / "data" / "microbiome_filtered.c

```

```

cwd: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3
config.py found at: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3
DATA_DIR would be: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data
data/ exists: True
microbiome_filtered.csv exists: True

```

Python helper file: config.py

```

In [125... Path('config.py').write_text('from pathlib import Path\n\nPROJECT_ROOT = Path(__
print("Created helper file:", 'config.py')

```

Created helper file: config.py

model.py

```

In [126... #Load data
df_fme = pd.read_csv(DATA_DIR / "fme_statistical_dataset_extended.csv", index_co
print(f"Loaded dataset: {df_fme.shape[0]} samples, {df_fme.shape[1]} columns")
print(f"Participants: {df_fme['participant_id'].nunique()}")
print("Sample of Data:")
print(df_fme.head())

#Turn shannon diversity from contiues to binary

```

```

median_shannon = df_fme["shannon_diversity"].median()
df_fme["High_Diversity_Class"] = (df_fme["shannon_diversity"] > median_shannon).
print(f"\nTarget distribution (High=1 / Low=0):")
print(df_fme["High_Diversity_Class"].value_counts())

#Features
feature_cols = [
    "fme_score_daily",
    "fme_participant_mean",
    "fme_within_person",
    "fme_weighted_3day",
    "KCAL",
    "FIBE",
    "CARB",
    "TFAT",
    "PROT",
    "Age",
    "BMI"
]

feature_cols_lagged = [
    "fme_score_daily",
    "fme_participant_mean",
    "fme_within_person",
    "fme_lag1",
    "fme_lag2",
    "KCAL",
    "FIBE",
    "CARB",
    "TFAT",
    "PROT",
    "Age",
    "BMI"
]

feature_cols_full_fme = [
    "fme_score_daily",
    "fme_participant_mean",
    "fme_within_person",
    "fme_weighted_3day",
    "fme_lag1",
    "fme_lag2",
    "KCAL",
    "FIBE",
    "CARB",
    "TFAT",
    "PROT",
    "Age",
    "BMI"
]

feature_cols_full_fme = [f for f in feature_cols_full_fme if f in df_fme.columns]

feature_cols_lagged = [f for f in feature_cols_lagged if f in df_fme.columns]

feature_cols = [f for f in feature_cols if f in df_fme.columns]

#Drop rows with missing values in features or target
df_fme_clean = df_fme[feature_cols + ["High_Diversity_Class", "participant_id"]]

```

```

#Prepare data for modeling
X = df_fme_clean[feature_cols]
print(X.columns.tolist())
y = df_fme_clean['High_Diversity_Class']
groups = df_fme_clean['participant_id']

#Define models
models = {
    "Logistic Regression": LogisticRegression(),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "XGBoost": XGBClassifier(eval_metric='logloss', random_state=42),
    "SVM": CalibratedClassifierCV(SVC(kernel='rbf'), ensemble=False),
    "LightGBM": LGBMClassifier(n_estimators=100, random_state=42, verbose=-1)
}

#Cross validation
gkf = GroupKFold(n_splits=5)

#Train and evaluate models
print("\nModel Comparison Results:")
best_model_name = ""
best_auc = 0
best_model_obj = None
classification_results = []

for name, model in models.items():
    acc_scores = []
    auc_scores = []

    #Make sure subjects are not split between train and test sets
    for train_idx, test_idx in gkf.split(X, y, groups=groups):
        X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
        y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

        #Scale features
        scaler = StandardScaler()
        # X_train_scaled = scaler.fit_transform(X_train)
        # X_test_scaled = scaler.transform(X_test)
        X_train_scaled = pd.DataFrame(scaler.fit_transform(X_train), columns=feature_cols)
        X_test_scaled = pd.DataFrame(scaler.transform(X_test), columns=feature_cols)

        #Train
        #Clone model to avoid reusing fitted state between folds
        fold_model = clone(model)
        fold_model.fit(X_train_scaled, y_train)

        #Predictions
        y_pred = fold_model.predict(X_test_scaled)
        y_proba = fold_model.predict_proba(X_test_scaled)[: , 1]

        #Calculate metrics
        acc_scores.append(accuracy_score(y_test, y_pred))
        auc_scores.append(roc_auc_score(y_test, y_proba))

    #Compute mean metrics across folds
    mean_acc = np.mean(acc_scores)
    std_acc = np.std(acc_scores)
    mean_auc = np.mean(auc_scores)
    std_auc = np.std(auc_scores)

```



```

print(f"\n[{name}]\n")
print(f"Mean Accuracy: {mean_acc:.3f} ± {std_acc:.3f}")
print(f"Mean ROC-AUC: {mean_auc:.3f} ± {std_auc:.3f}")

classification_results.append({
    "Model": name,
    "Mean Accuracy": mean_acc,
    "Std Accuracy": std_acc,
    "Mean ROC-AUC": mean_auc,
    "Std ROC-AUC": std_auc
})

#Save best model based on AUC
if mean_auc > best_auc:
    best_auc = mean_auc
    best_model_name = name
    best_model_obj = clone(model)

print(f"\n Best Model: {best_model_name} (Mean AUC = {best_auc:.3f})")

# Save and plot classification model comparison
classification_results_df = pd.DataFrame(classification_results)

classification_results_path = DATA_DIR / "classification_model_comparison.csv"
classification_results_df.to_csv(classification_results_path, index=False)

print(f"Classification results saved: {classification_results_path}")
print(classification_results_df)

# Plot Accuracy and ROC-AUC comparison
plot_df = classification_results_df.copy()

x = np.arange(len(plot_df["Model"]))
width = 0.35

plt.figure(figsize=(9, 5))

plt.bar(
    x - width / 2,
    plot_df["Mean Accuracy"],
    width,
    label="Accuracy"
)

plt.bar(
    x + width / 2,
    plot_df["Mean ROC-AUC"],
    width,
    label="ROC-AUC"
)

plt.axhline(
    y=0.5,
    linestyle="--",
    linewidth=1.5,
    label="Random baseline"
)

plt.xticks(x, plot_df["Model"], rotation=20, ha="right")
plt.ylabel("Score")

```

```

min_score = plot_df[["Mean Accuracy", "Mean ROC-AUC"]].min().min()
max_score = plot_df[["Mean Accuracy", "Mean ROC-AUC"]].max().max()

plt.ylim(
    max(0, min_score - 0.05),
    min(1, max_score + 0.08)
)
plt.title("Classification Model Performance")
plt.legend()

for i, row in plot_df.iterrows():
    plt.text(
        i - width / 2,
        row["Mean Accuracy"] + 0.005,
        f"{row['Mean Accuracy']:.3f}",
        ha="center",
        va="bottom",
        fontsize=9
    )

    plt.text(
        i + width / 2,
        row["Mean ROC-AUC"] + 0.005,
        f"{row['Mean ROC-AUC']:.3f}",
        ha="center",
        va="bottom",
        fontsize=9
    )

plt.tight_layout()
plt.savefig(DATA_DIR / "classification_model_performance.png", dpi=300, bbox_inches="tight")
plt.show()

print(f"Classification performance plot saved: {DATA_DIR / 'classification_model_performance.png'}")

#Refit best model on the full clean regression dataset
final_reg_scaler = StandardScaler()
X_scaled = final_reg_scaler.fit_transform(X)
best_model_obj.fit(X_scaled, y)

#Feature importance for best model
if best_model_name == "Logistic Regression":
    importances = np.abs(best_model_obj.coef_[0])
    title = "Feature Importance (Logistic Regression - Absolute Coefficients)"
    xlabel = "Absolute Coefficient Value"
elif best_model_name == "SVM":
    print("SVM does not support feature importance with rbf kernel - skipping plot")
    importances = None
else:
    importances = best_model_obj.feature_importances_
    title = f"Feature Importance ({best_model_name})"
    xlabel = "Importance Score"

if importances is not None:
    indices = np.argsort(importances)[::-1]
    feature_arr = np.array(feature_cols)

    plt.figure(figsize=(10, 6))
    sns.barplot(x=importances[indices], y=feature_arr[indices], hue=feature_arr[indices])

```

```

plt.title(title, fontsize=14)
plt.xlabel(xlabel, fontsize=12)
plt.ylabel("Feature", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "feature_importance.png", dpi=150)      #Save before
plt.show()
print(f"Plot saved: {DATA_DIR / 'feature_importance.png'}")

#Regression: predict continuous Shannon diversity
print("\n Regression: Predict Shannon Diversity")

#Use the same features, but keep Shannon diversity as a continuous target
df_reg_clean = df_fme[feature_cols + ["shannon_diversity", "participant_id"]].dr

#Prepare regression data
X_reg = df_reg_clean[feature_cols]
y_reg = df_reg_clean["shannon_diversity"]
groups_reg = df_reg_clean["participant_id"]

print(f"Regression dataset: {df_reg_clean.shape[0]} samples")
print(f"Features used: {feature_cols}")
print(f"Target mean Shannon diversity: {y_reg.mean():.3f}")
print(f"Target std Shannon diversity: {y_reg.std():.3f}")

# Define regression models
regression_models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0),
    "Random Forest Regressor": RandomForestRegressor(
        n_estimators=100,
        random_state=42
    ),
    "XGBoost Regressor": XGBRegressor(
        objective="reg:squarederror",
        n_estimators=100,
        random_state=42
    )
}

# Use grouped cross validation to avoid participant leakage
gkf_reg = GroupKFold(n_splits=5)

#Train and evaluate regression models
print("\nRegression Model Comparison Results")

best_reg_model_name = ""
best_reg_r2 = -np.inf
best_reg_model_template = None

regression_results = []
all_y_true_best = []
all_y_pred_best = []

for name, model in regression_models.items():
    r2_scores = []
    rmse_scores = []
    mae_scores = []
    all_y_true = []

```

```

all_y_pred = []

#Make sure subjects are not split between train and test sets
for train_idx, test_idx in gkf_reg.split(X_reg, y_reg, groups=groups_reg):
    X_train, X_test = X_reg.iloc[train_idx], X_reg.iloc[test_idx]
    y_train, y_test = y_reg.iloc[train_idx], y_reg.iloc[test_idx]

    #Scale features using train data only
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    #Clone model to avoid reusing fitted state between folds
    fold_model = clone(model)

    #Train
    fold_model.fit(X_train_scaled, y_train)

    #Predict continuous Shannon diversity
    y_pred = fold_model.predict(X_test_scaled)

    #Calculate regression metrics
    r2_scores.append(r2_score(y_test, y_pred))
    rmse_scores.append(np.sqrt(mean_squared_error(y_test, y_pred)))
    mae_scores.append(mean_absolute_error(y_test, y_pred))
    all_y_true.extend(y_test)
    all_y_pred.extend(y_pred)

#Compute mean metrics across folds
mean_r2 = np.mean(r2_scores)
std_r2 = np.std(r2_scores)
mean_rmse = np.mean(rmse_scores)
mean_mae = np.mean(mae_scores)

regression_results.append({
    "Model": name,
    "Mean R2": mean_r2,
    "Std R2": std_r2,
    "Mean RMSE": mean_rmse,
    "Mean MAE": mean_mae
})

print(f"\n[{name}]")
print(f"Mean R^2:    {mean_r2:.3f} ± {std_r2:.3f}")
print(f"Mean RMSE:  {mean_rmse:.3f}")
print(f"Mean MAE:    {mean_mae:.3f}")

#Save best regression model based on R2
if mean_r2 > best_reg_r2:
    best_reg_r2 = mean_r2
    best_reg_model_name = name
    best_reg_model_template = model
    all_y_true_best = all_y_true
    all_y_pred_best = all_y_pred

print(f"\nBest Regression Model: {best_reg_model_name} (Mean R^2 = {best_reg_r2:.3f})")

# Save regression results table
regression_results_df = pd.DataFrame(regression_results)
regression_results_path = DATA_DIR / "regression_model_comparison.csv"

```

```

regression_results_df.to_csv(regression_results_path, index=False)
print(f"Regression results saved: {regression_results_path}")

#Refit best regression model on the full clean regression dataset
final_reg_scaler = StandardScaler()
X_reg_scaled = final_reg_scaler.fit_transform(X_reg)

best_reg_model_obj = clone(best_reg_model_template)
best_reg_model_obj.fit(X_reg_scaled, y_reg)

#Feature importance for best regression model
if best_reg_model_name == "Linear Regression" or best_reg_model_name == "Ridge R
    reg_importances = np.abs(best_reg_model_obj.coef_)
    reg_title = "Feature Importance (" + best_reg_model_name + " - Absolute Coeffici
    reg_xlabel = "Absolute Coefficient Value"
else:
    reg_importances = best_reg_model_obj.feature_importances_
    reg_title = f"Feature Importance ({best_reg_model_name})"
    reg_xlabel = "Importance Score"

reg_indices = np.argsort(reg_importances)[::-1]
feature_arr = np.array(feature_cols)

plt.figure(figsize=(10, 6))
sns.barplot(
    x=reg_importances[reg_indices],
    y=feature_arr[reg_indices],
    hue=feature_arr[reg_indices],
    palette="viridis",
    legend=False
)
plt.title(reg_title, fontsize=14)
plt.xlabel(reg_xlabel, fontsize=12)
plt.ylabel("Feature", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "regression_feature_importance.png", dpi=150)
plt.show()
print(f"Regression feature importance plot saved: {DATA_DIR / 'regression_featur

all_y_true = np.array(all_y_true_best)
all_y_pred = np.array(all_y_pred_best)

#Plot predicted vs actual Shannon diversity
plt.figure(figsize=(7, 7))
sns.scatterplot(x=all_y_true, y=all_y_pred)
plt.plot(
    [all_y_true.min(), all_y_true.max()],
    [all_y_true.min(), all_y_true.max()],
    linestyle="--"
)
plt.title(f"Predicted vs Actual Shannon Diversity ({best_reg_model_name})", font
plt.xlabel("Actual Shannon Diversity", fontsize=12)
plt.ylabel("Predicted Shannon Diversity", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "regression_predicted_vs_actual.png", dpi=150)
plt.show()
print(f"Prediction plot saved: {DATA_DIR / 'regression_predicted_vs_actual.png'})

#Plot residuals

```

```

residuals = all_y_true - all_y_pred

plt.figure(figsize=(8, 6))
sns.scatterplot(x=all_y_pred, y=residuals)
plt.axhline(0, linestyle="--")
plt.title(f"Residual Plot ({best_reg_model_name})", fontsize=14)
plt.xlabel("Predicted Shannon Diversity", fontsize=12)
plt.ylabel("Residuals", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "regression_residuals.png", dpi=150)
plt.show()
print(f"Residual plot saved: {DATA_DIR / 'regression_residuals.png'}")

# Try model distance from average shannon instead of predict shannon
# Calculate avg shannon score
df_fme['shannon_avg'] = df_fme.groupby('participant_id')['shannon_diversity'].transform('mean')
# Calculate sample shannon deviation
df_fme['shannon_deviation'] = df_fme['shannon_diversity'] - df_fme['shannon_avg']

#Regression: predict continuous distance from avg Shannon diversity
print("\n Regression: predict continuous distance from avg Shannon diversity")

#Use the same features, but keep Shannon diversity as a continuous target
df_reg_dev_clean = df_fme[feature_cols + ["shannon_deviation", "participant_id"]]

#Prepare regression data
X_dev_reg = df_reg_dev_clean[feature_cols]
y_dev_reg = df_reg_dev_clean["shannon_deviation"]
groups_dev_reg = df_reg_dev_clean["participant_id"]

print(f"Regression dataset: {df_reg_dev_clean.shape[0]} samples")
print(f"Features used: {feature_cols}")
print(f"Target mean Shannon deviation diversity: {y_dev_reg.mean():.3f}")
print(f"Target std Shannon deviation diversity: {y_dev_reg.std():.3f}")

#Define regression models
regression_dev_models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0),
    "Random Forest Regressor": RandomForestRegressor(
        n_estimators=100,
        random_state=42
    ),
    "XGBoost Regressor": XGBRegressor(
        objective="reg:squarederror",
        n_estimators=100,
        random_state=42
    )
}

#Use grouped cross validation to avoid participant leakage
gkf_reg = GroupKFold(n_splits=5)

#Train and evaluate regression models
print("\nRegression Model for deviation Comparison Results")

best_dev_reg_model_name = ""
best_dev_reg_r2 = -np.inf
best_dev_reg_model_template = None

```

```

regression_dev_results = []
all_dev_y_true_best = []
all_dev_y_pred_best = []

for name, model in regression_dev_models.items():
    r2_scores = []
    rmse_scores = []
    mae_scores = []
    all_y_true = []
    all_y_pred = []

    #Make sure subjects are not split between train and test sets
    for train_idx, test_idx in gkf_reg.split(X_dev_reg, y_dev_reg, groups=groups):
        X_train, X_test = X_dev_reg.iloc[train_idx], X_dev_reg.iloc[test_idx]
        y_train, y_test = y_dev_reg.iloc[train_idx], y_dev_reg.iloc[test_idx]

        #Scale features using train data only
        scaler = StandardScaler()
        X_train_scaled = scaler.fit_transform(X_train)
        X_test_scaled = scaler.transform(X_test)

        #Clone model to avoid reusing fitted state between folds
        fold_model = clone(model)

        #Train
        fold_model.fit(X_train_scaled, y_train)

        #Predict continuous Shannon diversity
        y_pred = fold_model.predict(X_test_scaled)

        #Calculate regression metrics
        r2_scores.append(r2_score(y_test, y_pred))
        rmse_scores.append(np.sqrt(mean_squared_error(y_test, y_pred)))
        mae_scores.append(mean_absolute_error(y_test, y_pred))
        all_y_true.extend(y_test)
        all_y_pred.extend(y_pred)

    #Compute mean metrics across folds
    mean_r2 = np.mean(r2_scores)
    std_r2 = np.std(r2_scores)
    mean_rmse = np.mean(rmse_scores)
    mean_mae = np.mean(mae_scores)

    regression_dev_results.append({
        "Model": name,
        "Mean R2": mean_r2,
        "Std R2": std_r2,
        "Mean RMSE": mean_rmse,
        "Mean MAE": mean_mae
    })

    print(f"\n[{name}]\n")
    print(f"Mean R^2:    {mean_r2:.3f} ± {std_r2:.3f}")
    print(f"Mean RMSE:  {mean_rmse:.3f}")
    print(f"Mean MAE:   {mean_mae:.3f}")

    #Save best regression model based on R2
    if mean_r2 > best_dev_reg_r2:
        best_dev_reg_r2 = mean_r2
        best_dev_reg_model_name = name

```

```

        best_dev_reg_model_template = model
        all_dev_y_true_best = all_y_true
        all_dev_y_pred_best = all_y_pred

print(f"\nBest Regression Model: {best_dev_reg_model_name} (Mean R2 = {best_dev_

#Save regression deviation from avg results table
regression_dev_results_df = pd.DataFrame(regression_dev_results)
regression_dev_results_path = DATA_DIR / "regression_dev_model_comparison.csv"
regression_dev_results_df.to_csv(regression_dev_results_path, index=False)
print(f"Regression results saved: {regression_dev_results_path}")

#Refit best regression model on the full clean regression dataset
final_dev_reg_scaler = StandardScaler()
X_dev_reg_scaled = final_dev_reg_scaler.fit_transform(X_dev_reg)

best_dev_reg_model_obj = clone(best_dev_reg_model_template)
best_dev_reg_model_obj.fit(X_dev_reg_scaled, y_dev_reg)

#Feature importance for best regression model
if best_dev_reg_model_name == "Linear Regression" or best_dev_reg_model_name ==
    reg_dev_importances = np.abs(best_dev_reg_model_obj.coef_)
    reg_dev_title = "Feature Importance (" + best_dev_reg_model_name + " - Absolute
    reg_dev_xlabel = "Absolute Coefficient Value"
else:
    reg_dev_importances = best_dev_reg_model_obj.feature_importances_
    reg_dev_title = f"Feature Importance ({best_dev_reg_model_name})"
    reg_dev_xlabel = "Importance Score"

reg_dev_indices = np.argsort(reg_dev_importances)[::-1]
feature_dev_arr = np.array(feature_cols)

# Bar plot of distribution of Shannon Deviation from Personal Average
plt.figure(figsize=(8, 5))
sns.histplot(df_reg_dev_clean["shannon_deviation"], bins=30, kde=True)
plt.axvline(0, linestyle="--", color="red")
plt.title("Distribution of Shannon Deviation from Personal Average", fontsize=14)
plt.xlabel("Shannon Deviation", fontsize=12)
plt.ylabel("Count", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "dev_shannon_deviation_distribution.png", dpi=150)
plt.show()

# Bar plot of feature importance
plt.figure(figsize=(10, 6))
sns.barplot(
    x=reg_dev_importances[reg_dev_indices],
    y=feature_dev_arr[reg_dev_indices],
    hue=feature_dev_arr[reg_dev_indices],
    palette="viridis",
    legend=False
)
plt.title(reg_dev_title, fontsize=14)
plt.xlabel(reg_dev_xlabel, fontsize=12)
plt.ylabel("Feature", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "dev_regression_feature_importance.png", dpi=150)
plt.show()
print(f"Regression feature importance plot saved: {DATA_DIR / 'dev_regression_fe

```



```
all_dev_y_true = np.array(all_dev_y_true_best)
all_dev_y_pred = np.array(all_dev_y_pred_best)

#Plot predicted vs actual Shannon diversity
plt.figure(figsize=(7, 7))
sns.scatterplot(x=all_dev_y_true, y=all_dev_y_pred)
plt.plot(
    [all_dev_y_true.min(), all_dev_y_true.max()],
    [all_dev_y_true.min(), all_dev_y_true.max()],
    linestyle="--"
)
plt.title(f"Predicted vs Actual Shannon Deviation ({best_dev_reg_model_name})",
plt.xlabel("Actual Shannon Deviation", fontsize=12)
plt.ylabel("Predicted Shannon Deviation", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "dev_regression_predicted_vs_actual.png", dpi=150)
plt.show()
print(f"Prediction plot saved: {DATA_DIR / 'dev_regression_predicted_vs_actual.p

#Plot residuals
dev_residuals = all_dev_y_true - all_dev_y_pred

plt.figure(figsize=(8, 6))
sns.scatterplot(x=all_dev_y_pred, y=dev_residuals)
plt.axhline(0, linestyle="--")
plt.title(f"Residual Plot ({best_dev_reg_model_name})", fontsize=14)
plt.xlabel("Predicted Shannon Deviation", fontsize=12)
plt.ylabel("Residuals", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "dev_regression_residuals.png", dpi=150)
plt.show()
print(f"Residual plot saved: {DATA_DIR / 'dev_regression_residuals.png'})")
```

Loaded dataset: 475 samples, 35 columns

Participants: 34

Sample of Data:

fecal_sample_id	participant_id	study_day	Gender	Age	BMI	Weight	\
MCT.f.0002	MCTs01	2	F	25.9000	25.0000	74.6000	
MCT.f.0003	MCTs01	3	F	25.9000	25.0000	74.6000	
MCT.f.0004	MCTs01	4	F	25.9000	25.0000	74.6000	
MCT.f.0005	MCTs01	5	F	25.9000	25.0000	74.6000	
MCT.f.0006	MCTs01	6	F	25.9000	25.0000	74.6000	

fecal_sample_id	Supplement	Medications	fme_score_daily	\
MCT.f.0002	EVOO	Nexplanon (bc implant)	67.1961	
MCT.f.0003	EVOO	Nexplanon (bc implant)	81.2412	
MCT.f.0004	EVOO	Nexplanon (bc implant)	126.7711	
MCT.f.0005	EVOO	Nexplanon (bc implant)	31.4846	
MCT.f.0006	EVOO	Nexplanon (bc implant)	61.2266	

fecal_sample_id	fme_participant_mean	fme_within_person	fme_lag1	...	\
MCT.f.0002	71.2710	-4.0749	NaN	...	
MCT.f.0003	71.2710	9.9703	67.1961	...	
MCT.f.0004	71.2710	55.5001	81.2412	...	
MCT.f.0005	71.2710	-39.7864	126.7711	...	
MCT.f.0006	71.2710	-10.0444	31.4846	...	

fecal_sample_id	PROT	TFAT	CARB	FIBE	SUGR	SODI	\
MCT.f.0002	109.6198	53.6153	242.9190	14.4194	74.2081	4300.6165	
MCT.f.0003	89.9930	50.5614	234.2251	25.9961	67.4747	4225.8133	
MCT.f.0004	94.6120	92.0216	257.5390	26.1784	83.9603	5329.8821	
MCT.f.0005	110.2684	95.5703	165.6081	13.0114	64.9220	3527.5014	
MCT.f.0006	161.8178	108.3985	332.6372	21.5291	119.8657	5428.5730	

fecal_sample_id	D_TOTAL	D_YOGURT	D_CHEESE	PF_MEAT	PF_SEAFD_HI	G_WHOLE
MCT.f.0002	4.1560	0.6460	1.1535	0.0000	3.9690	0.9718
MCT.f.0003	2.9797	0.6460	1.2076	0.0000	0.0000	2.9318
MCT.f.0004	3.6345	0.0000	1.4826	0.0000	0.0000	0.9718
MCT.f.0005	3.3797	0.0000	1.0590	1.8144	0.0000	0.9718
MCT.f.0006	1.5022	0.0000	0.0000	6.2960	0.0000	0.4698

[5 rows x 35 columns]

Target distribution (High=1 / Low=0):

High_Diversity_Class

0 238

1 237

Name: count, dtype: int64

['fme_score_daily', 'fme_participant_mean', 'fme_within_person', 'fme_weighted_3day', 'KCAL', 'FIBE', 'CARB', 'TFAT', 'PROT', 'Age', 'BMI']

Model Comparison Results:

[Logistic Regression]

Mean Accuracy: 0.417 ± 0.052

Mean ROC-AUC: 0.366 ± 0.051

[Random Forest]

Mean Accuracy: 0.549 ± 0.035

Mean ROC-AUC: 0.528 ± 0.071

[XGBoost]

Mean Accuracy: 0.547 ± 0.017

Mean ROC-AUC: 0.552 ± 0.127

[SVM]

Mean Accuracy: 0.406 ± 0.044

Mean ROC-AUC: 0.429 ± 0.029

[LightGBM]

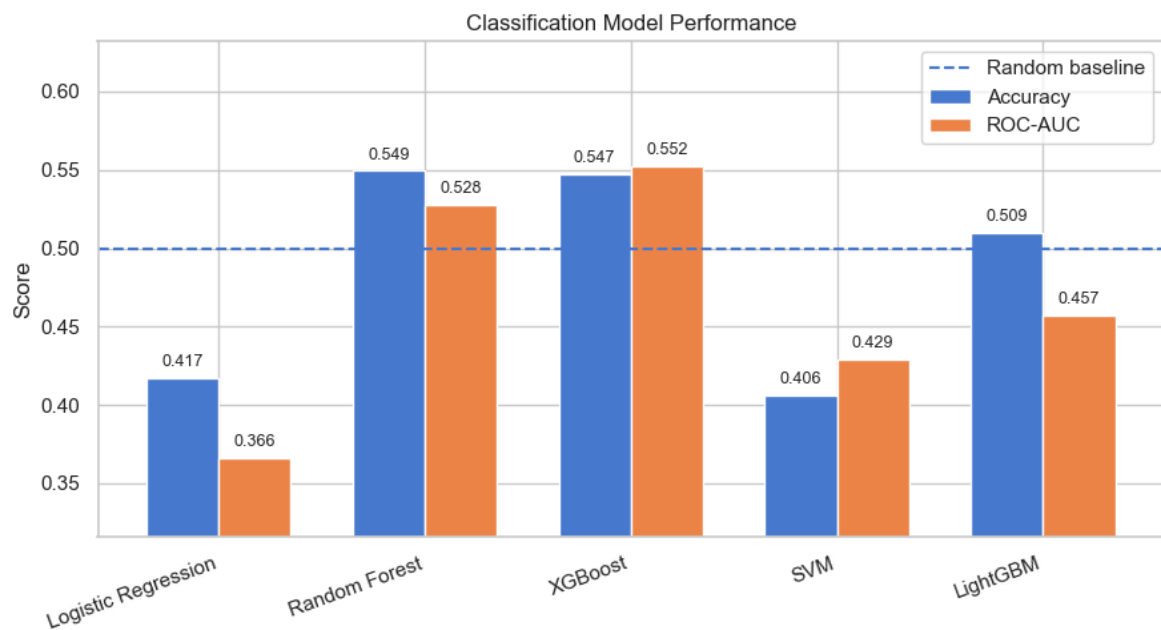
Mean Accuracy: 0.509 ± 0.112

Mean ROC-AUC: 0.457 ± 0.164

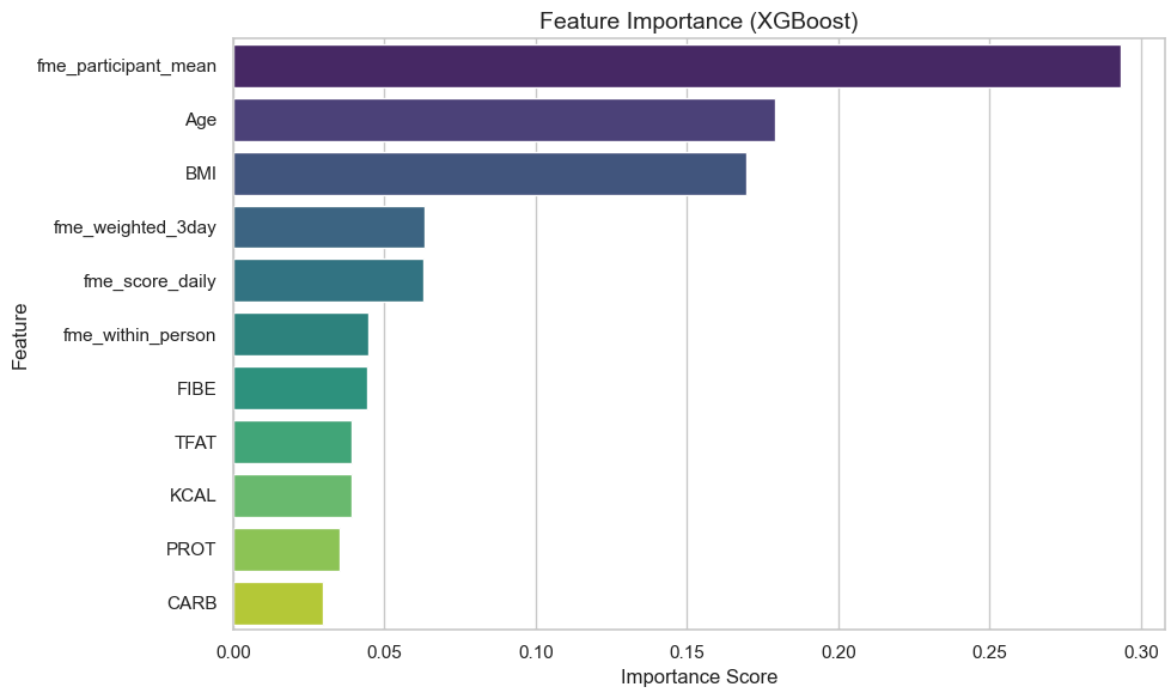
Best Model: XGBoost (Mean AUC = 0.552)

Classification results saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\classification_model_comparison.csv

	Model	Mean Accuracy	Std Accuracy	Mean ROC-AUC	Std ROC-AUC
0	Logistic Regression	0.4173	0.0521	0.3659	0.0506
1	Random Forest	0.5494	0.0348	0.5278	0.0710
2	XGBoost	0.5475	0.0171	0.5520	0.1268
3	SVM	0.4061	0.0437	0.4292	0.0291
4	LightGBM	0.5095	0.1117	0.4574	0.1643



Classification performance plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\classification_model_performance.png



Plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\feature_importance.png

Regression: Predict Shannon Diversity
 Regression dataset: 475 samples
 Features used: ['fme_score_daily', 'fme_participant_mean', 'fme_within_person', 'fme_weighted_3day', 'KCAL', 'FIBE', 'CARB', 'TFAT', 'PROT', 'Age', 'BMI']
 Target mean Shannon diversity: 2.865
 Target std Shannon diversity: 0.413

Regression Model Comparison Results

[Linear Regression]

Mean R^2 : -0.320 ± 0.201
 Mean RMSE: 0.453
 Mean MAE: 0.372

[Ridge Regression]

Mean R^2 : -0.316 ± 0.200
 Mean RMSE: 0.453
 Mean MAE: 0.371

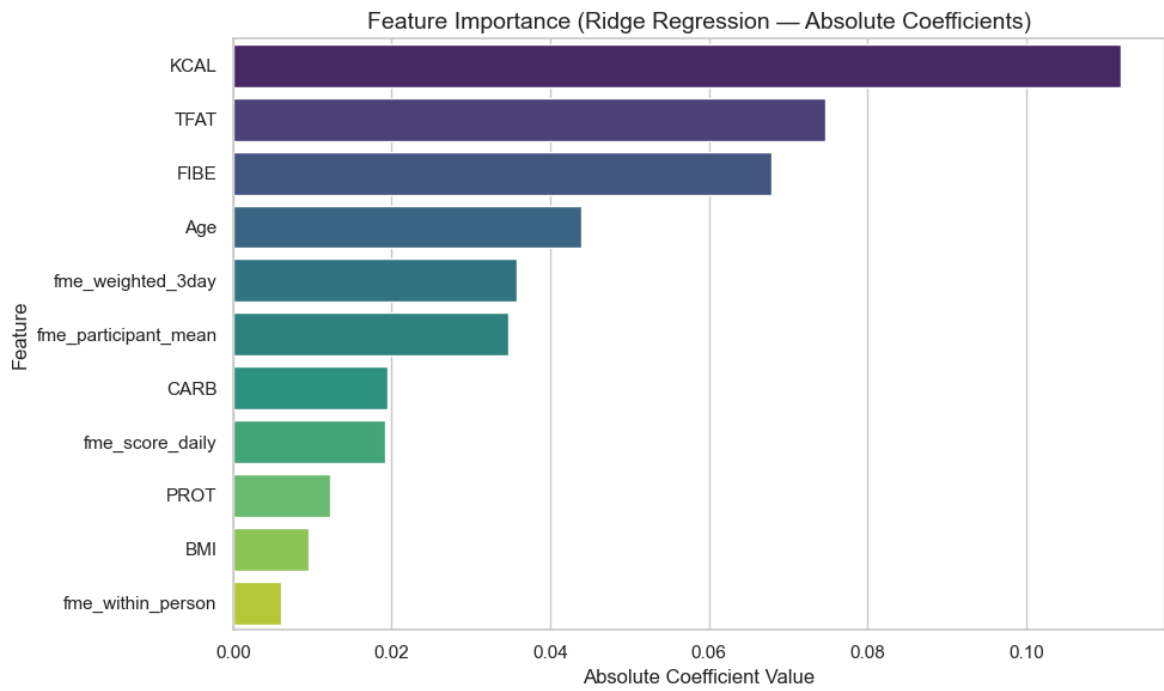
[Random Forest Regressor]

Mean R^2 : -0.619 ± 0.870
 Mean RMSE: 0.476
 Mean MAE: 0.390

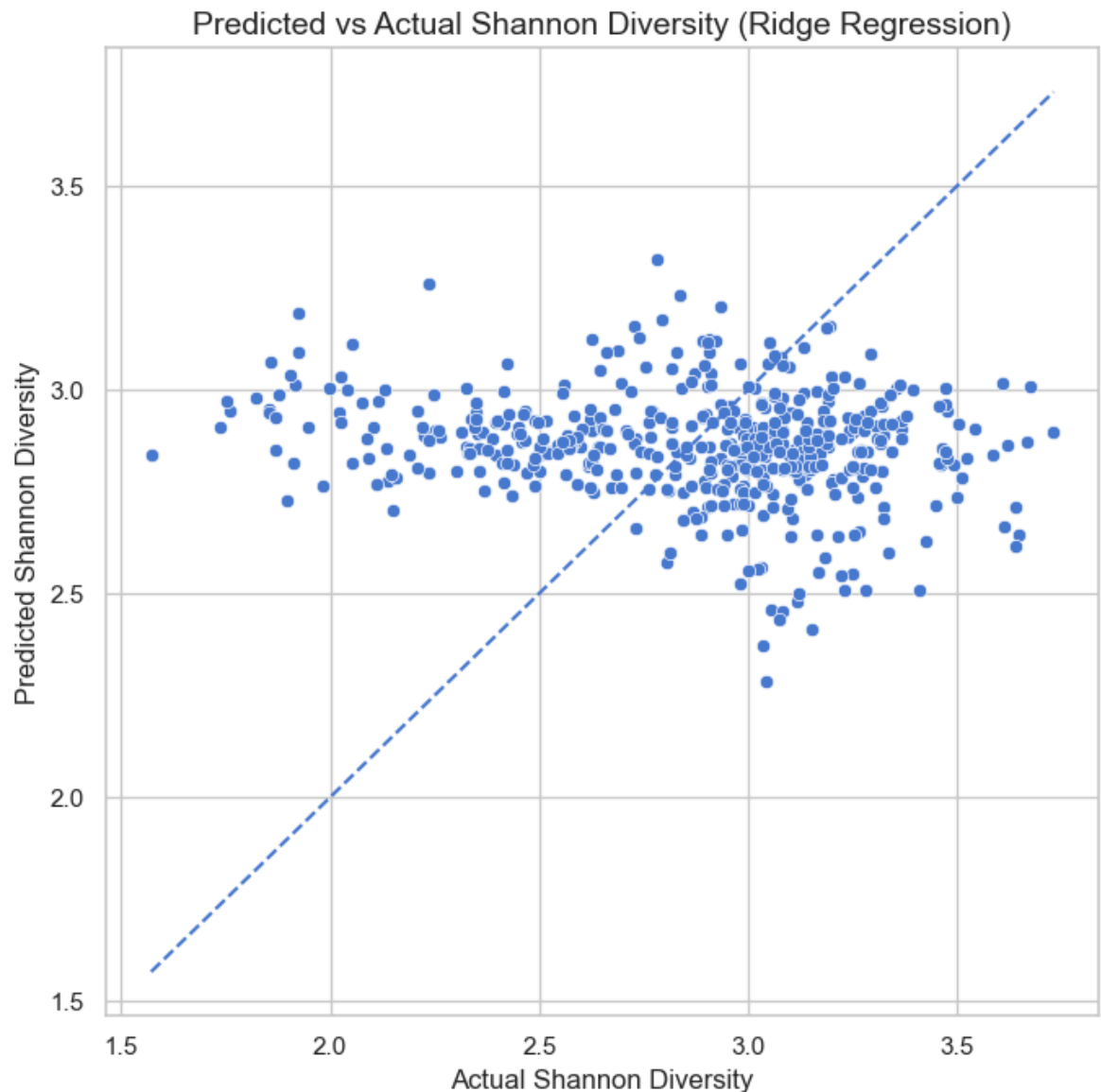
[XGBoost Regressor]

Mean R^2 : -0.367 ± 0.826
 Mean RMSE: 0.436
 Mean MAE: 0.331

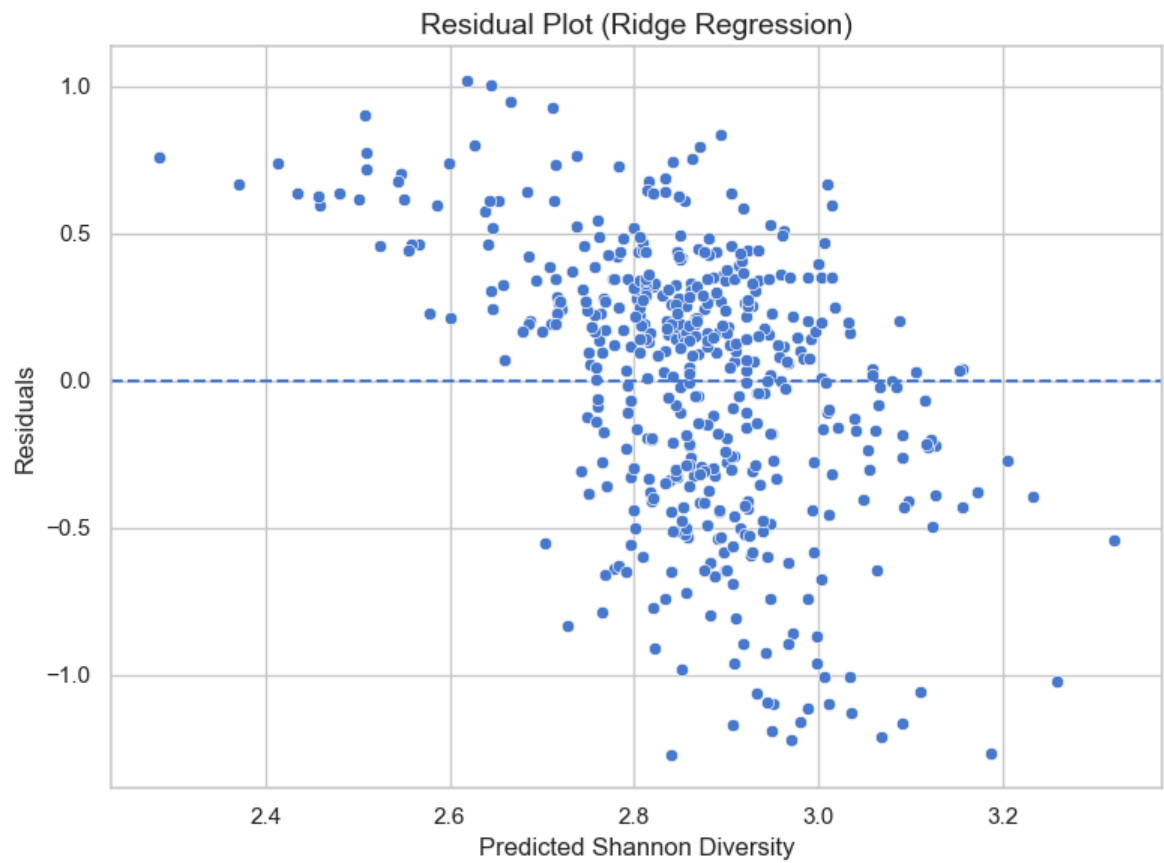
Best Regression Model: Ridge Regression (Mean $R^2 = -0.316$)
 Regression results saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\regression_model_comparison.csv



Regression feature importance plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\regression_feature_importance.png



Prediction plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\regression_predicted_vs_actual.png



Residual plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\regression_residuals.png

Regression: predict continuous distance from avg Shannon diversity
Regression dataset: 475 samples
Features used: ['fme_score_daily', 'fme_participant_mean', 'fme_within_person', 'fme_weighted_3day', 'KCAL', 'FIBE', 'CARB', 'TFAT', 'PROT', 'Age', 'BMI']
Target mean Shannon deviation diversity: 0.000
Target std Shannon deviation diversity: 0.187

Regression Model for deviation Comparison Results

[Linear Regression]

Mean R^2 : -0.045 ± 0.025
Mean RMSE: 0.189
Mean MAE: 0.137

[Ridge Regression]

Mean R^2 : -0.044 ± 0.024
Mean RMSE: 0.189
Mean MAE: 0.137

[Random Forest Regressor]

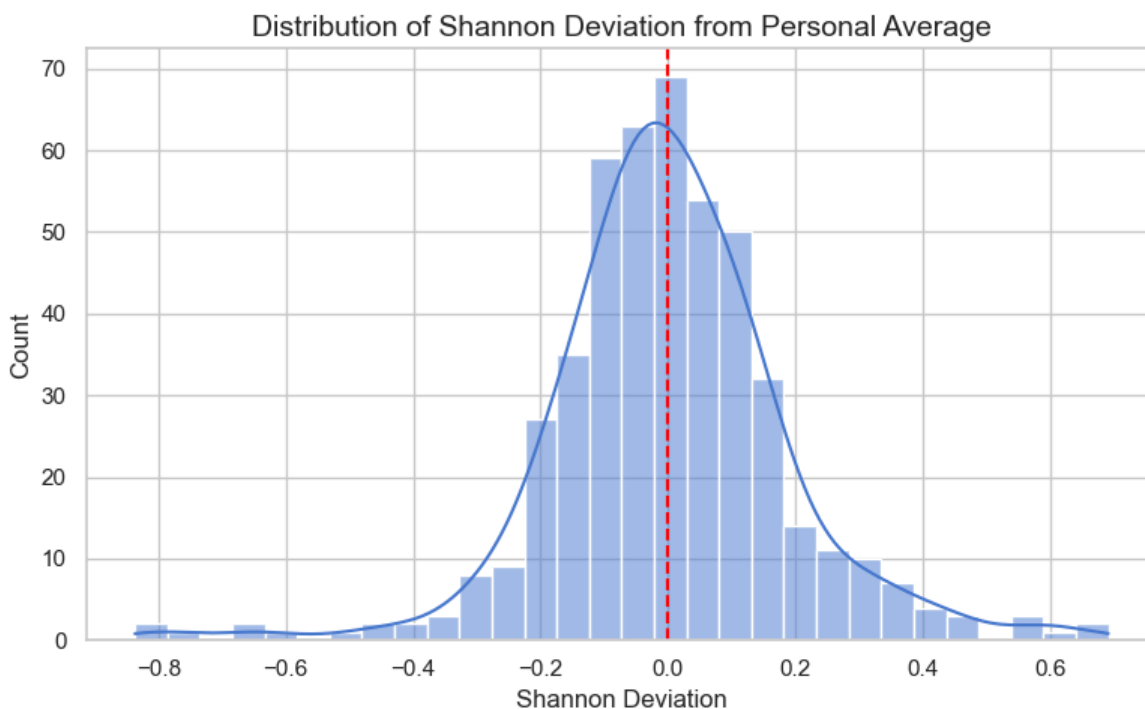
Mean R^2 : -0.125 ± 0.045
Mean RMSE: 0.196
Mean MAE: 0.145

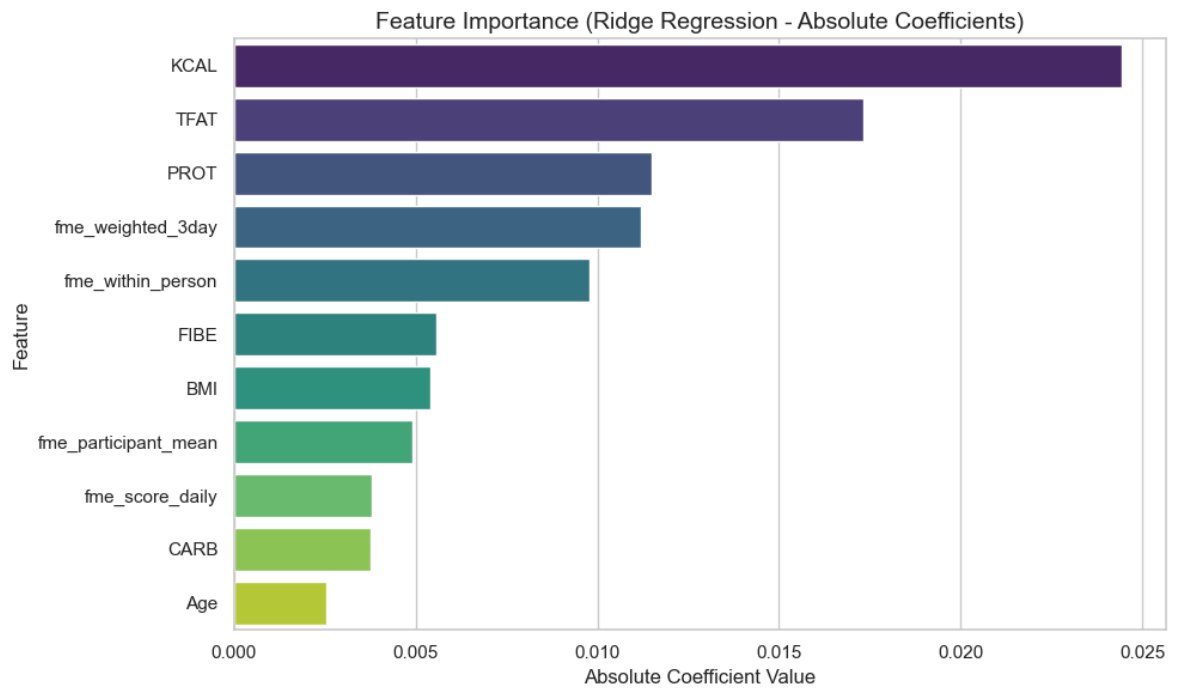
[XGBoost Regressor]

Mean R^2 : -0.435 ± 0.231
Mean RMSE: 0.220
Mean MAE: 0.164

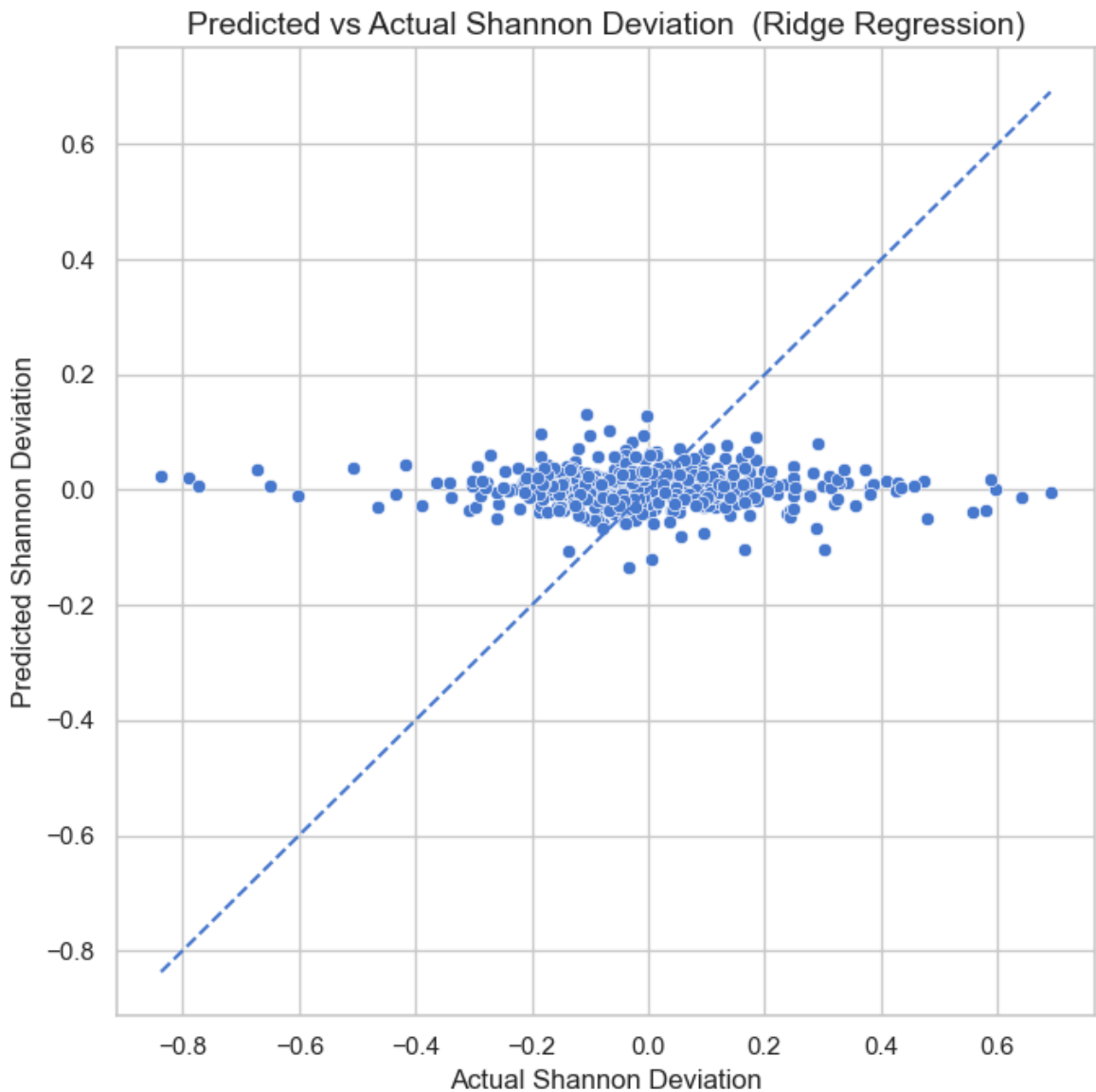
Best Regression Model: Ridge Regression (Mean $R^2 = -0.044$)

Regression results saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\regression_dev_model_comparison.csv

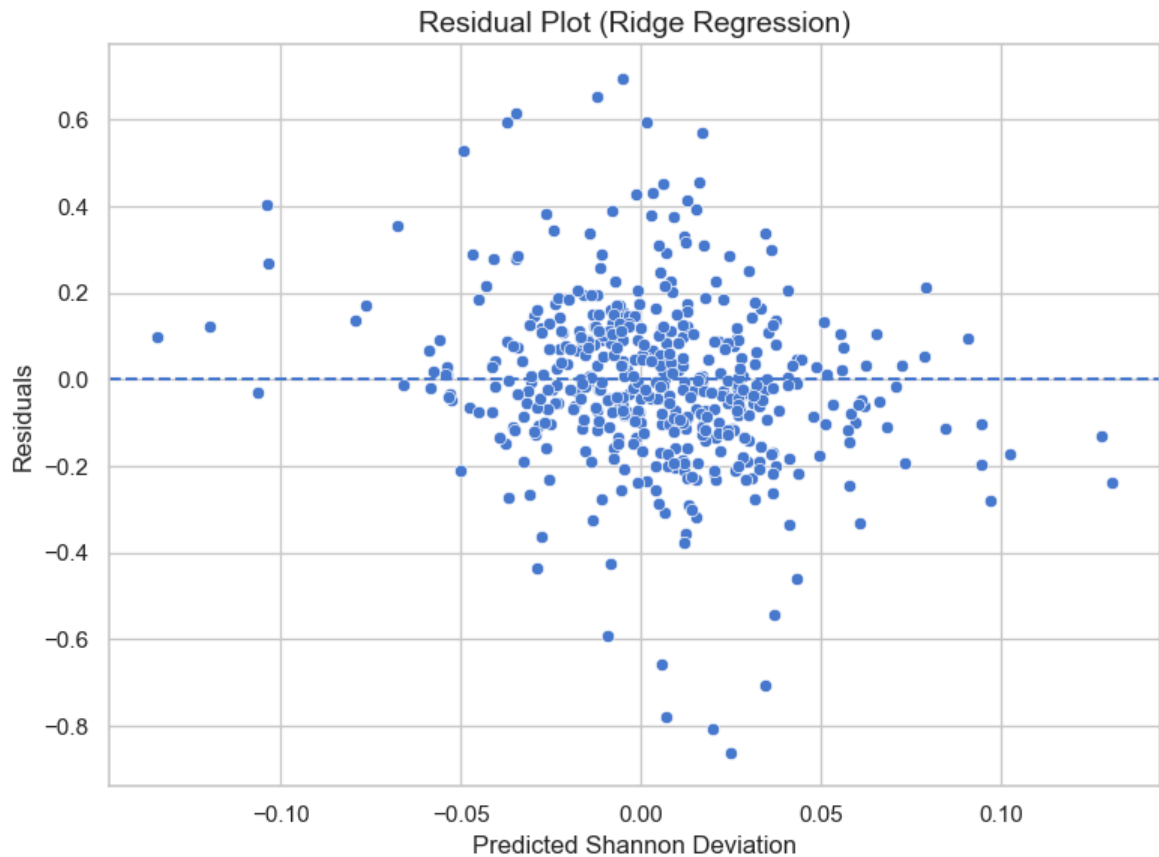




Regression feature importance plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\dev_regression_feature_importance.png



Prediction plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\dev_regression_predicted_vs_actual.png



Residual plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\dev_regression_residuals.png

model_lagged.py

In [127...

```
#Load data
df_fme = pd.read_csv(DATA_DIR / "fme_statistical_dataset_extended.csv", index_co
print(f"Loaded dataset: {df_fme.shape[0]} samples, {df_fme.shape[1]} columns")
print(f"Participants: {df_fme['participant_id'].nunique()}")
print("Sample of Data:")
print(df_fme.head())

#Turn shannon diversity from contiues to binary
median_shannon = df_fme["shannon_diversity"].median()
df_fme["High_Diversity_Class"] = (df_fme["shannon_diversity"] > median_shannon).
print(f"\nTarget distribution (High=1 / Low=0):")
print(df_fme["High_Diversity_Class"].value_counts())

#Features
feature_cols_lagged = [
    "fme_score_daily",
    "fme_participant_mean",
    "fme_within_person",
    "fme_lag1",
    "fme_lag2",
    "KCAL",
    "FIBE",
    "CARB",
    "TFAT",
    "PROT",
    "Age",
    "BMI"
]
```

```

feature_cols_lagged = [f for f in feature_cols_lagged if f in df_fme.columns]

#Drop rows with missing values in features or target
df_fme_clean = df_fme[feature_cols_lagged + ["High_Diversity_Class", "participant_id"]]

#Check how many samples are left
print(f"Lagged classification clean dataset: {df_fme_clean.shape[0]} samples")
print(f"Participants after cleaning: {df_fme_clean['participant_id'].nunique()}")

#Prepare data for modeling
X = df_fme_clean[feature_cols_lagged]
print(X.columns.tolist())
y = df_fme_clean['High_Diversity_Class']
groups = df_fme_clean['participant_id']

#Define models
models = {
    "Logistic Regression": LogisticRegression(),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "XGBoost": XGBClassifier(eval_metric='logloss', random_state=42),
    "SVM": CalibratedClassifierCV(SVC(kernel='rbf'), ensemble=False),
    "LightGBM": LGBMClassifier(n_estimators=100, random_state=42, verbose=-1)
}

#Cross validation
gkf = GroupKFold(n_splits=5)

#Train and evaluate models
print("\nModel Comparison Results:")
best_model_name = ""
best_auc = 0
best_model_obj = None
classification_results = []

for name, model in models.items():
    acc_scores = []
    auc_scores = []

    #Make sure subjects are not split between train and test sets
    for train_idx, test_idx in gkf.split(X, y, groups=groups):
        X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
        y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

        #Scale features
        scaler = StandardScaler()
        # X_train_scaled = scaler.fit_transform(X_train)
        # X_test_scaled = scaler.transform(X_test)
        X_train_scaled = pd.DataFrame(scaler.fit_transform(X_train), columns=feature_cols_lagged)
        X_test_scaled = pd.DataFrame(scaler.transform(X_test), columns=feature_cols_lagged)

        #Train
        #Clone model to avoid reusing fitted state between folds
        fold_model = clone(model)
        fold_model.fit(X_train_scaled, y_train)

        #Predictions
        y_pred = fold_model.predict(X_test_scaled)
        y_proba = fold_model.predict_proba(X_test_scaled)[:, 1]

```

```

        #Calculate metrics
        acc_scores.append(accuracy_score(y_test, y_pred))
        auc_scores.append(roc_auc_score(y_test, y_proba))

    #Compute mean metrics across folds
    mean_acc = np.mean(acc_scores)
    std_acc = np.std(acc_scores)
    mean_auc = np.mean(auc_scores)
    std_auc = np.std(auc_scores)

    print(f"\n[{name}]]")
    print(f"Mean Accuracy: {mean_acc:.3f} ± {std_acc:.3f}")
    print(f"Mean ROC-AUC: {mean_auc:.3f} ± {std_auc:.3f}")

    classification_results.append({
        "Model": name,
        "Mean Accuracy": mean_acc,
        "Std Accuracy": std_acc,
        "Mean ROC-AUC": mean_auc,
        "Std ROC-AUC": std_auc
    })

    #Save best model based on AUC
    if mean_auc > best_auc:
        best_auc = mean_auc
        best_model_name = name
        best_model_obj = clone(model)

print(f"\n Best Model: {best_model_name} (Mean AUC = {best_auc:.3f})")

# Save and plot classification model comparison
classification_results_df = pd.DataFrame(classification_results)

classification_results_path = DATA_DIR / "classification_model_comparison.csv"
classification_results_df.to_csv(classification_results_path, index=False)

print(f"Classification results saved: {classification_results_path}")
print(classification_results_df)

# Plot Accuracy and ROC-AUC comparison
plot_df = classification_results_df.copy()

x = np.arange(len(plot_df["Model"]))
width = 0.35

plt.figure(figsize=(9, 5))

plt.bar(
    x - width / 2,
    plot_df["Mean Accuracy"],
    width,
    label="Accuracy"
)

plt.bar(
    x + width / 2,
    plot_df["Mean ROC-AUC"],
    width,
    label="ROC-AUC"
)

```

```

plt.axhline(
    y=0.5,
    linestyle="--",
    linewidth=1.5,
    label="Random baseline"
)

plt.xticks(x, plot_df["Model"], rotation=20, ha="right")
plt.ylabel("Score")
min_score = plot_df[["Mean Accuracy", "Mean ROC-AUC"]].min().min()
max_score = plot_df[["Mean Accuracy", "Mean ROC-AUC"]].max().max()

plt.ylim(
    max(0, min_score - 0.05),
    min(1, max_score + 0.08)
)
plt.title("Classification Model Performance")
plt.legend()

for i, row in plot_df.iterrows():
    plt.text(
        i - width / 2,
        row["Mean Accuracy"] + 0.005,
        f"{row['Mean Accuracy']:.3f}",
        ha="center",
        va="bottom",
        fontsize=9
    )

    plt.text(
        i + width / 2,
        row["Mean ROC-AUC"] + 0.005,
        f"{row['Mean ROC-AUC']:.3f}",
        ha="center",
        va="bottom",
        fontsize=9
    )

plt.tight_layout()
plt.savefig(DATA_DIR / "classification_model_performance.png", dpi=300, bbox_inches="tight")
plt.show()

print(f"Classification performance plot saved: {DATA_DIR / 'classification_model_performance.png'}")

#Refit best model on the full clean regression dataset
final_reg_scaler = StandardScaler()
X_scaled = final_reg_scaler.fit_transform(X)
best_model_obj.fit(X_scaled, y)

#Feature importance for best model
if best_model_name == "Logistic Regression":
    importances = np.abs(best_model_obj.coef_[0])
    title = "Feature Importance (Logistic Regression - Absolute Coefficients)"
    xlabel = "Absolute Coefficient Value"
elif best_model_name == "SVM":
    print("SVM does not support feature importance with rbf kernel - skipping plot")
    importances = None
else:

```

```

importances = best_model_obj.feature_importances_
title       = f"Feature Importance ({best_model_name})"
xlabel      = "Importance Score"

if importances is not None:
    indices    = np.argsort(importances)[::-1]
    feature_arr = np.array(feature_cols_lagged)

    plt.figure(figsize=(10, 6))
    sns.barplot(x=importances[indices], y=feature_arr[indices], hue=feature_arr[indices])
    plt.title(title, fontsize=14)
    plt.xlabel(xlabel, fontsize=12)
    plt.ylabel("Feature", fontsize=12)
    plt.tight_layout()
    plt.savefig(DATA_DIR / "lagged_feature_importance.png", dpi=150)      #Save
    plt.show()
    print(f"Plot saved: {DATA_DIR / 'lagged_feature_importance.png'}")

#Regression: predict continuous Shannon diversity
print("\n Regression: Predict Shannon Diversity")

#Use the same features, but keep Shannon diversity as a continuous target
df_reg_clean = df_fme[feature_cols_lagged + ["shannon_diversity", "participant_id"]]

#Prepare regression data
X_reg = df_reg_clean[feature_cols_lagged]
y_reg = df_reg_clean["shannon_diversity"]
groups_reg = df_reg_clean["participant_id"]

print(f"Regression dataset: {df_reg_clean.shape[0]} samples")
print(f"Features used: {feature_cols_lagged}")
print(f"Target mean Shannon diversity: {y_reg.mean():.3f}")
print(f"Target std Shannon diversity: {y_reg.std():.3f}")

# Define regression models
regression_models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0),
    "Random Forest Regressor": RandomForestRegressor(
        n_estimators=100,
        random_state=42
    ),
    "XGBoost Regressor": XGBRegressor(
        objective="reg:squarederror",
        n_estimators=100,
        random_state=42
    )
}

# Use grouped cross validation to avoid participant leakage
gkf_reg = GroupKFold(n_splits=5)

#Train and evaluate regression models
print("\nRegression Model Comparison Results")

best_reg_model_name = ""
best_reg_r2 = -np.inf
best_reg_model_template = None

```

```

regression_results = []
all_y_true_best = []
all_y_pred_best = []

for name, model in regression_models.items():
    r2_scores = []
    rmse_scores = []
    mae_scores = []
    all_y_true = []
    all_y_pred = []

    #Make sure subjects are not split between train and test sets
    for train_idx, test_idx in gkf_reg.split(X_reg, y_reg, groups=groups_reg):
        X_train, X_test = X_reg.iloc[train_idx], X_reg.iloc[test_idx]
        y_train, y_test = y_reg.iloc[train_idx], y_reg.iloc[test_idx]

        #Scale features using train data only
        scaler = StandardScaler()
        X_train_scaled = scaler.fit_transform(X_train)
        X_test_scaled = scaler.transform(X_test)

        #Clone model to avoid reusing fitted state between folds
        fold_model = clone(model)

        #Train
        fold_model.fit(X_train_scaled, y_train)

        #Predict continuous Shannon diversity
        y_pred = fold_model.predict(X_test_scaled)

        #Calculate regression metrics
        r2_scores.append(r2_score(y_test, y_pred))
        rmse_scores.append(np.sqrt(mean_squared_error(y_test, y_pred)))
        mae_scores.append(mean_absolute_error(y_test, y_pred))
        all_y_true.extend(y_test)
        all_y_pred.extend(y_pred)

    #Compute mean metrics across folds
    mean_r2 = np.mean(r2_scores)
    std_r2 = np.std(r2_scores)
    mean_rmse = np.mean(rmse_scores)
    mean_mae = np.mean(mae_scores)

    regression_results.append({
        "Model": name,
        "Mean R2": mean_r2,
        "Std R2": std_r2,
        "Mean RMSE": mean_rmse,
        "Mean MAE": mean_mae
    })

    print(f"\n[{name}]\n")
    print(f"Mean R^2:    {mean_r2:.3f} ± {std_r2:.3f}")
    print(f"Mean RMSE:   {mean_rmse:.3f}")
    print(f"Mean MAE:    {mean_mae:.3f}")

    #Save best regression model based on R2
    if mean_r2 > best_reg_r2:
        best_reg_r2 = mean_r2

```

```

        best_reg_model_name = name
        best_reg_model_template = model
        all_y_true_best = all_y_true
        all_y_pred_best = all_y_pred

print(f"\nBest Regression Model: {best_reg_model_name} (Mean R2 = {best_reg_r2:.

# Save regression results table
regression_results_df = pd.DataFrame(regression_results)
regression_results_path = DATA_DIR / "lagged_regression_model_comparison.csv"
regression_results_df.to_csv(regression_results_path, index=False)
print(f"lagged_regression_model_comparison: {regression_results_path}")

#Refit best regression model on the full clean regression dataset
final_reg_scaler = StandardScaler()
X_reg_scaled = final_reg_scaler.fit_transform(X_reg)

best_reg_model_obj = clone(best_reg_model_template)
best_reg_model_obj.fit(X_reg_scaled, y_reg)

#Feature importance for best regression model
if best_reg_model_name == "Linear Regression" or best_reg_model_name == "Ridge R
    reg_importances = np.abs(best_reg_model_obj.coef_)
    reg_title = "Feature Importance (" + best_reg_model_name + " - Absolute Coeffici
    reg_xlabel = "Absolute Coefficient Value"
else:
    reg_importances = best_reg_model_obj.feature_importances_
    reg_title = f"Feature Importance ({best_reg_model_name})"
    reg_xlabel = "Importance Score"

reg_indices = np.argsort(reg_importances)[::-1]
feature_arr = np.array(feature_cols_lagged)

plt.figure(figsize=(10, 6))
sns.barplot(
    x=reg_importances[reg_indices],
    y=feature_arr[reg_indices],
    hue=feature_arr[reg_indices],
    palette="viridis",
    legend=False
)
plt.title(reg_title, fontsize=14)
plt.xlabel(reg_xlabel, fontsize=12)
plt.ylabel("Feature", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "lagged_regression_feature_importance.png", dpi=150)
plt.show()
print(f"Regression feature importance plot saved: {DATA_DIR / 'lagged_regression

all_y_true = np.array(all_y_true_best)
all_y_pred = np.array(all_y_pred_best)

#Plot predicted vs actual Shannon diversity
plt.figure(figsize=(7, 7))
sns.scatterplot(x=all_y_true, y=all_y_pred)
plt.plot(
    [all_y_true.min(), all_y_true.max()],
    [all_y_true.min(), all_y_true.max()],
    linestyle="--"

```

```

)
plt.title(f"Predicted vs Actual Shannon Diversity ({best_reg_model_name})", font
plt.xlabel("Actual Shannon Diversity", fontsize=12)
plt.ylabel("Predicted Shannon Diversity", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "lagged_regression_predicted_vs_actual.png", dpi=150)
plt.show()
print(f"Prediction plot saved: {DATA_DIR / 'lagged_regression_predicted_vs_actua

#Plot residuals
residuals = all_y_true - all_y_pred

plt.figure(figsize=(8, 6))
sns.scatterplot(x=all_y_pred, y=residuals)
plt.axhline(0, linestyle="--")
plt.title(f"Residual Plot ({best_reg_model_name})", fontsize=14)
plt.xlabel("Predicted Shannon Diversity", fontsize=12)
plt.ylabel("Residuals", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "lagged_regression_residuals.png", dpi=150)
plt.show()
print(f"Residual plot saved: {DATA_DIR / 'lagged_regression_residuals.png'}")

# Try model distance from avarge shanon instead of predict shanon
# Calculate avg shanon score
df_fme['shannon_avg'] = df_fme.groupby('participant_id')['shannon_diversity'].tr
# Calculate sample shanon deviation
df_fme['shannon_deviation'] = df_fme['shannon_diversity'] - df_fme['shannon_avg']

#Regression: predict continuous distance from avg Shannon diversity
print("\n Regression: predict continuous distance from avg Shannon diversity")

#Use the same features, but keep Shannon diversity as a continuous target
df_reg_dev_clean = df_fme[feature_cols_lagged + ["shannon_deviation", "participa

#Prepare regression data
X_dev_reg = df_reg_dev_clean[feature_cols_lagged]
y_dev_reg = df_reg_dev_clean["shannon_deviation"]
groups_dev_reg = df_reg_dev_clean["participant_id"]

print(f"Regression dataset: {df_reg_dev_clean.shape[0]} samples")
print(f"Features used: {feature_cols_lagged}")
print(f"Target mean Shannon deviation diversity: {y_dev_reg.mean():.3f}")
print(f"Target std Shannon deviation diversity: {y_dev_reg.std():.3f}")

#Define regression models
regression_dev_models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0),
    "Random Forest Regressor": RandomForestRegressor(
        n_estimators=100,
        random_state=42
    ),
    "XGBoost Regressor": XGBRegressor(
        objective="reg:squarederror",
        n_estimators=100,
        random_state=42
    )
}

```



```

#Use grouped cross validation to avoid participant Leakage
gkf_reg = GroupKFold(n_splits=5)

#Train and evaluate regression models
print("\nRegression Model for deviation- comparison Results")

best_dev_reg_model_name = ""
best_dev_reg_r2 = -np.inf
best_dev_reg_model_template = None

regression_dev_results = []
all_dev_y_true_best = []
all_dev_y_pred_best = []

for name, model in regression_dev_models.items():
    r2_scores = []
    rmse_scores = []
    mae_scores = []
    all_y_true = []
    all_y_pred = []

    #Make sure subjects are not split between train and test sets
    for train_idx, test_idx in gkf_reg.split(X_dev_reg, y_dev_reg, groups=groups):
        X_train, X_test = X_dev_reg.iloc[train_idx], X_dev_reg.iloc[test_idx]
        y_train, y_test = y_dev_reg.iloc[train_idx], y_dev_reg.iloc[test_idx]

        #Scale features using train data only
        scaler = StandardScaler()
        X_train_scaled = scaler.fit_transform(X_train)
        X_test_scaled = scaler.transform(X_test)

        #Clone model to avoid reusing fitted state between folds
        fold_model = clone(model)

        #Train
        fold_model.fit(X_train_scaled, y_train)

        #Predict continuous Shannon diversity
        y_pred = fold_model.predict(X_test_scaled)

        #Calculate regression metrics
        r2_scores.append(r2_score(y_test, y_pred))
        rmse_scores.append(np.sqrt(mean_squared_error(y_test, y_pred)))
        mae_scores.append(mean_absolute_error(y_test, y_pred))
        all_y_true.extend(y_test)
        all_y_pred.extend(y_pred)

    #Compute mean metrics across folds
    mean_r2 = np.mean(r2_scores)
    std_r2 = np.std(r2_scores)
    mean_rmse = np.mean(rmse_scores)
    mean_mae = np.mean(mae_scores)

    regression_dev_results.append({
        "Model": name,
        "Mean R2": mean_r2,
        "Std R2": std_r2,
        "Mean RMSE": mean_rmse,
        "Mean MAE": mean_mae
    })

```

```

print(f"\n[{name}]]")
print(f"Mean R^2:    {mean_r2:.3f} ± {std_r2:.3f}")
print(f"Mean RMSE:  {mean_rmse:.3f}")
print(f"Mean MAE:    {mean_mae:.3f}")

#Save best regression model based on R2
if mean_r2 > best_dev_reg_r2:
    best_dev_reg_r2 = mean_r2
    best_dev_reg_model_name = name
    best_dev_reg_model_template = model
    all_dev_y_true_best = all_y_true
    all_dev_y_pred_best = all_y_pred

print(f"\nBest Regression Model: {best_dev_reg_model_name} (Mean R2 = {best_dev_

#Save regression deviation from avg results table
regression_dev_results_df = pd.DataFrame(regression_dev_results)
regression_dev_results_path = DATA_DIR / "lagged_regression_dev_model_comparison
regression_dev_results_df.to_csv(regression_dev_results_path, index=False)
print(f"Regression results saved: {regression_dev_results_path}")

#Refit best regression model on the full clean regression dataset
final_dev_reg_scaler = StandardScaler()
X_dev_reg_scaled = final_dev_reg_scaler.fit_transform(X_dev_reg)

best_dev_reg_model_obj = clone(best_dev_reg_model_template)
best_dev_reg_model_obj.fit(X_dev_reg_scaled, y_dev_reg)

#Feature importance for best regression model
if best_dev_reg_model_name == "Linear Regression" or best_dev_reg_model_name ==
    reg_dev_importances = np.abs(best_dev_reg_model_obj.coef_)
    reg_dev_title = "Feature Importance (" + best_dev_reg_model_name + " - Absolute
    reg_dev_xlabel = "Absolute Coefficient Value"
else:
    reg_dev_importances = best_dev_reg_model_obj.feature_importances_
    reg_dev_title = f"Feature Importance ({best_dev_reg_model_name})"
    reg_dev_xlabel = "Importance Score"

reg_dev_indices = np.argsort(reg_dev_importances)[::-1]
feature_dev_arr = np.array(feature_cols_lagged)

# Bar plot of distribution of Shannon Deviation from Personal Average
plt.figure(figsize=(8, 5))
sns.histplot(df_reg_dev_clean["shannon_deviation"], bins=30, kde=True)
plt.axvline(0, linestyle="--", color="red")
plt.title("Distribution of Shannon Deviation from Personal Average", fontsize=14)
plt.xlabel("Shannon Deviation", fontsize=12)
plt.ylabel("Count", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "lagged_dev_shannon_deviation_distribution.png", dpi=150)
plt.show()

# Bar plot of feature importance
plt.figure(figsize=(10, 6))
sns.barplot(
    x=reg_dev_importances[reg_dev_indices],
    y=feature_dev_arr[reg_dev_indices],
    hue=feature_dev_arr[reg_dev_indices],
    palette="viridis",

```

```

        legend=False
    )
    plt.title(reg_dev_title, fontsize=14)
    plt.xlabel(reg_dev_xlabel, fontsize=12)
    plt.ylabel("Feature", fontsize=12)
    plt.tight_layout()
    plt.savefig(DATA_DIR / "lagged_dev_regression_feature_importance.png", dpi=150)
    plt.show()
    print(f"Regression feature importance plot saved: {DATA_DIR / 'lagged_dev_regres

all_dev_y_true = np.array(all_dev_y_true_best)
all_dev_y_pred = np.array(all_dev_y_pred_best)

#Plot predicted vs actual Shannon diversity
plt.figure(figsize=(7, 7))
sns.scatterplot(x=all_dev_y_true, y=all_dev_y_pred)
plt.plot(
    [all_dev_y_true.min(), all_dev_y_true.max()],
    [all_dev_y_true.min(), all_dev_y_true.max()],
    linestyle="--"
)
plt.title(f"Predicted vs Actual Shannon Deviation ({best_dev_reg_model_name})",
plt.xlabel("Actual Shannon Deviation", fontsize=12)
plt.ylabel("Predicted Shannon Deviation", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "lagged_dev_regression_predicted_vs_actual.png", dpi=150)
plt.show()
print(f"Prediction plot saved: {DATA_DIR / 'lagged_dev_regression_predicted_vs_a

#Plot residuals
dev_residuals = all_dev_y_true - all_dev_y_pred

plt.figure(figsize=(8, 6))
sns.scatterplot(x=all_dev_y_pred, y=dev_residuals)
plt.axhline(0, linestyle="--")
plt.title(f"Residual Plot ({best_dev_reg_model_name})", fontsize=14)
plt.xlabel("Predicted Shannon Deviation", fontsize=12)
plt.ylabel("Residuals", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "lagged_dev_regression_residuals.png", dpi=150)
plt.show()
print(f"Residual plot saved: {DATA_DIR / 'lagged_dev_regression_residuals.png'}")

```

Loaded dataset: 475 samples, 35 columns

Participants: 34

Sample of Data:

fecal_sample_id	participant_id	study_day	Gender	Age	BMI	Weight	\
MCT.f.0002	MCTs01	2	F	25.9000	25.0000	74.6000	
MCT.f.0003	MCTs01	3	F	25.9000	25.0000	74.6000	
MCT.f.0004	MCTs01	4	F	25.9000	25.0000	74.6000	
MCT.f.0005	MCTs01	5	F	25.9000	25.0000	74.6000	
MCT.f.0006	MCTs01	6	F	25.9000	25.0000	74.6000	

fecal_sample_id	Supplement	Medications	fme_score_daily	\
MCT.f.0002	EVOO	Nexplanon (bc implant)	67.1961	
MCT.f.0003	EVOO	Nexplanon (bc implant)	81.2412	
MCT.f.0004	EVOO	Nexplanon (bc implant)	126.7711	
MCT.f.0005	EVOO	Nexplanon (bc implant)	31.4846	
MCT.f.0006	EVOO	Nexplanon (bc implant)	61.2266	

fecal_sample_id	fme_participant_mean	fme_within_person	fme_lag1	...	\
MCT.f.0002	71.2710	-4.0749	NaN	...	
MCT.f.0003	71.2710	9.9703	67.1961	...	
MCT.f.0004	71.2710	55.5001	81.2412	...	
MCT.f.0005	71.2710	-39.7864	126.7711	...	
MCT.f.0006	71.2710	-10.0444	31.4846	...	

fecal_sample_id	PROT	TFAT	CARB	FIBE	SUGR	SODI	\
MCT.f.0002	109.6198	53.6153	242.9190	14.4194	74.2081	4300.6165	
MCT.f.0003	89.9930	50.5614	234.2251	25.9961	67.4747	4225.8133	
MCT.f.0004	94.6120	92.0216	257.5390	26.1784	83.9603	5329.8821	
MCT.f.0005	110.2684	95.5703	165.6081	13.0114	64.9220	3527.5014	
MCT.f.0006	161.8178	108.3985	332.6372	21.5291	119.8657	5428.5730	

fecal_sample_id	D_TOTAL	D_YOGURT	D_CHEESE	PF_MEAT	PF_SEAFD_HI	G_WHOLE
MCT.f.0002	4.1560	0.6460	1.1535	0.0000	3.9690	0.9718
MCT.f.0003	2.9797	0.6460	1.2076	0.0000	0.0000	2.9318
MCT.f.0004	3.6345	0.0000	1.4826	0.0000	0.0000	0.9718
MCT.f.0005	3.3797	0.0000	1.0590	1.8144	0.0000	0.9718
MCT.f.0006	1.5022	0.0000	0.0000	6.2960	0.0000	0.4698

[5 rows x 35 columns]

Target distribution (High=1 / Low=0):

High_Diversity_Class

0 238

1 237

Name: count, dtype: int64

Lagged classification clean dataset: 407 samples

Participants after cleaning: 34

['fme_score_daily', 'fme_participant_mean', 'fme_within_person', 'fme_lag1', 'fme_lag2', 'KCAL', 'FIBE', 'CARB', 'TFAT', 'PROT', 'Age', 'BMI']

Model Comparison Results:

[Logistic Regression]

Mean Accuracy: 0.425 ± 0.073

Mean ROC-AUC: 0.339 ± 0.057

[Random Forest]

Mean Accuracy: 0.546 ± 0.042

Mean ROC-AUC: 0.487 ± 0.079

[XGBoost]

Mean Accuracy: 0.529 ± 0.065

Mean ROC-AUC: 0.473 ± 0.077

[SVM]

Mean Accuracy: 0.373 ± 0.059

Mean ROC-AUC: 0.412 ± 0.048

[LightGBM]

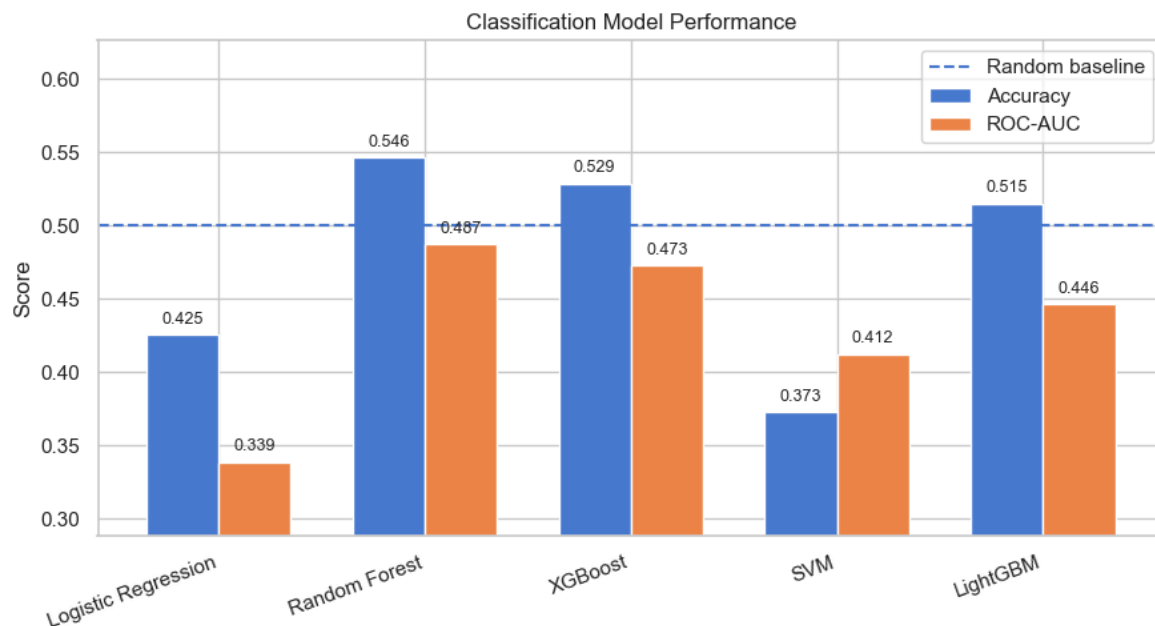
Mean Accuracy: 0.515 ± 0.128

Mean ROC-AUC: 0.446 ± 0.192

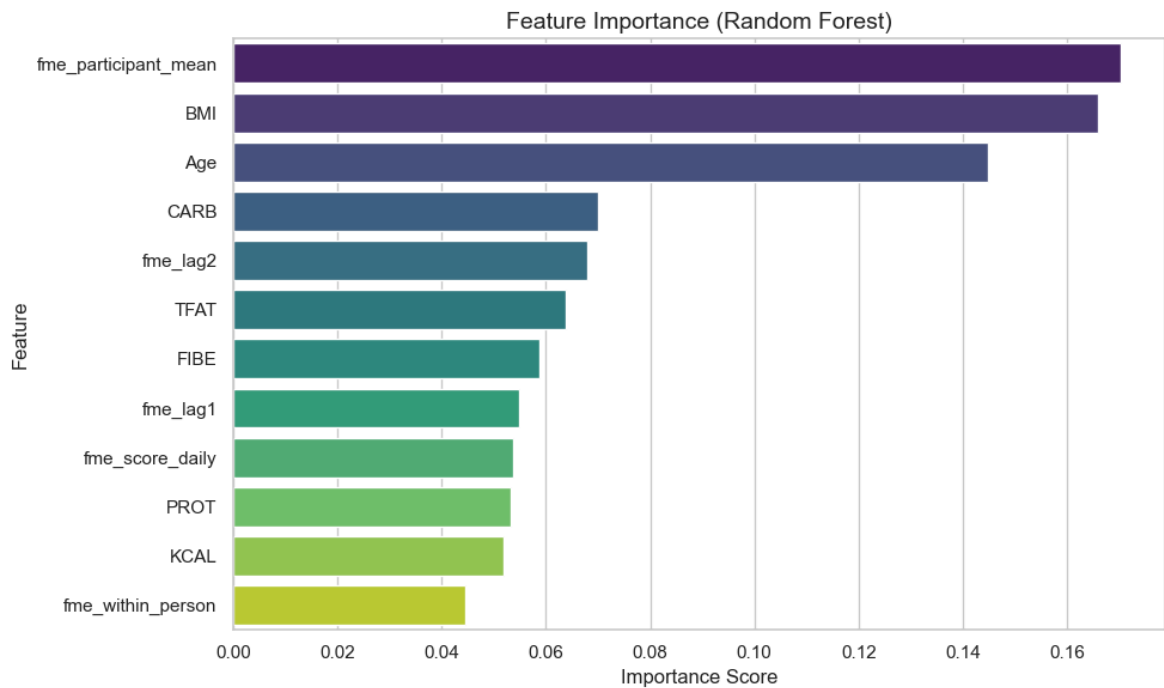
Best Model: Random Forest (Mean AUC = 0.487)

Classification results saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\classification_model_comparison.csv

	Model	Mean Accuracy	Std Accuracy	Mean ROC-AUC	Std ROC-AUC
0	Logistic Regression	0.4254	0.0729	0.3385	0.0568
1	Random Forest	0.5462	0.0423	0.4873	0.0791
2	XGBoost	0.5288	0.0651	0.4727	0.0772
3	SVM	0.3728	0.0594	0.4122	0.0478
4	LightGBM	0.5152	0.1281	0.4465	0.1923



Classification performance plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\classification_model_performance.png



Plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\lagged_feature_importance.png

Regression: Predict Shannon Diversity
Regression dataset: 407 samples
Features used: ['fme_score_daily', 'fme_participant_mean', 'fme_within_person', 'fme_lag1', 'fme_lag2', 'KCAL', 'FIBE', 'CARB', 'TFAT', 'PROT', 'Age', 'BMI']
Target mean Shannon diversity: 2.868
Target std Shannon diversity: 0.411

Regression Model Comparison Results

[Linear Regression]

Mean R^2 : -0.337 ± 0.256
Mean RMSE: 0.455
Mean MAE: 0.374

[Ridge Regression]

Mean R^2 : -0.332 ± 0.254
Mean RMSE: 0.454
Mean MAE: 0.373

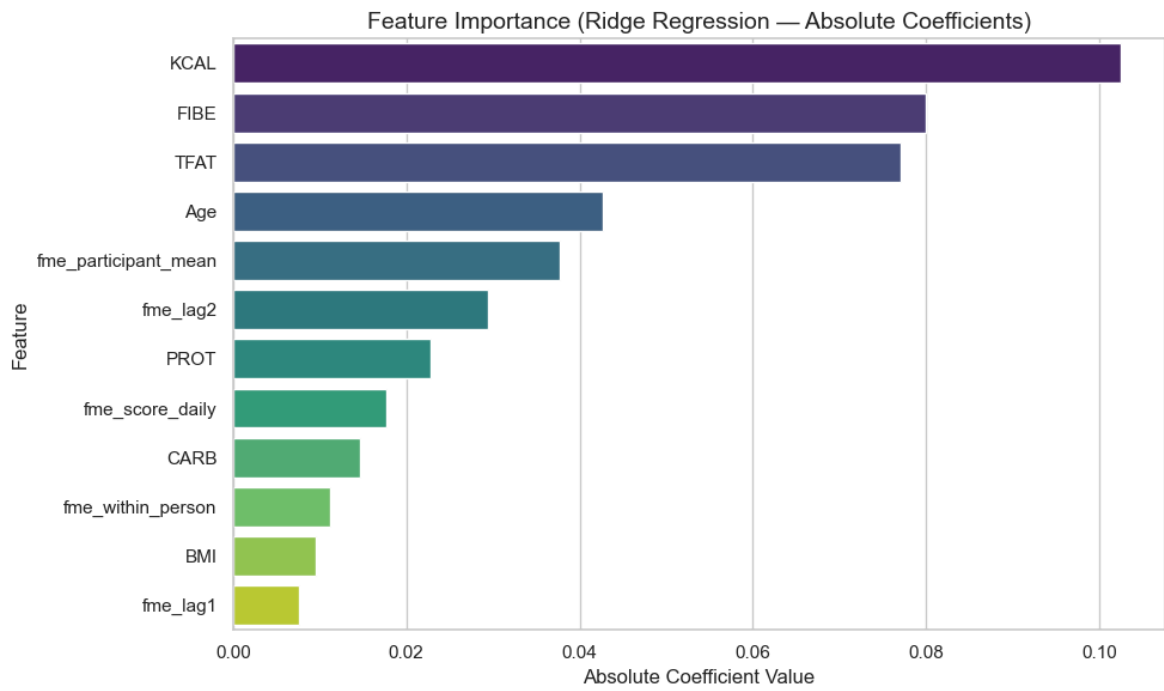
[Random Forest Regressor]

Mean R^2 : -0.536 ± 0.729
Mean RMSE: 0.468
Mean MAE: 0.388

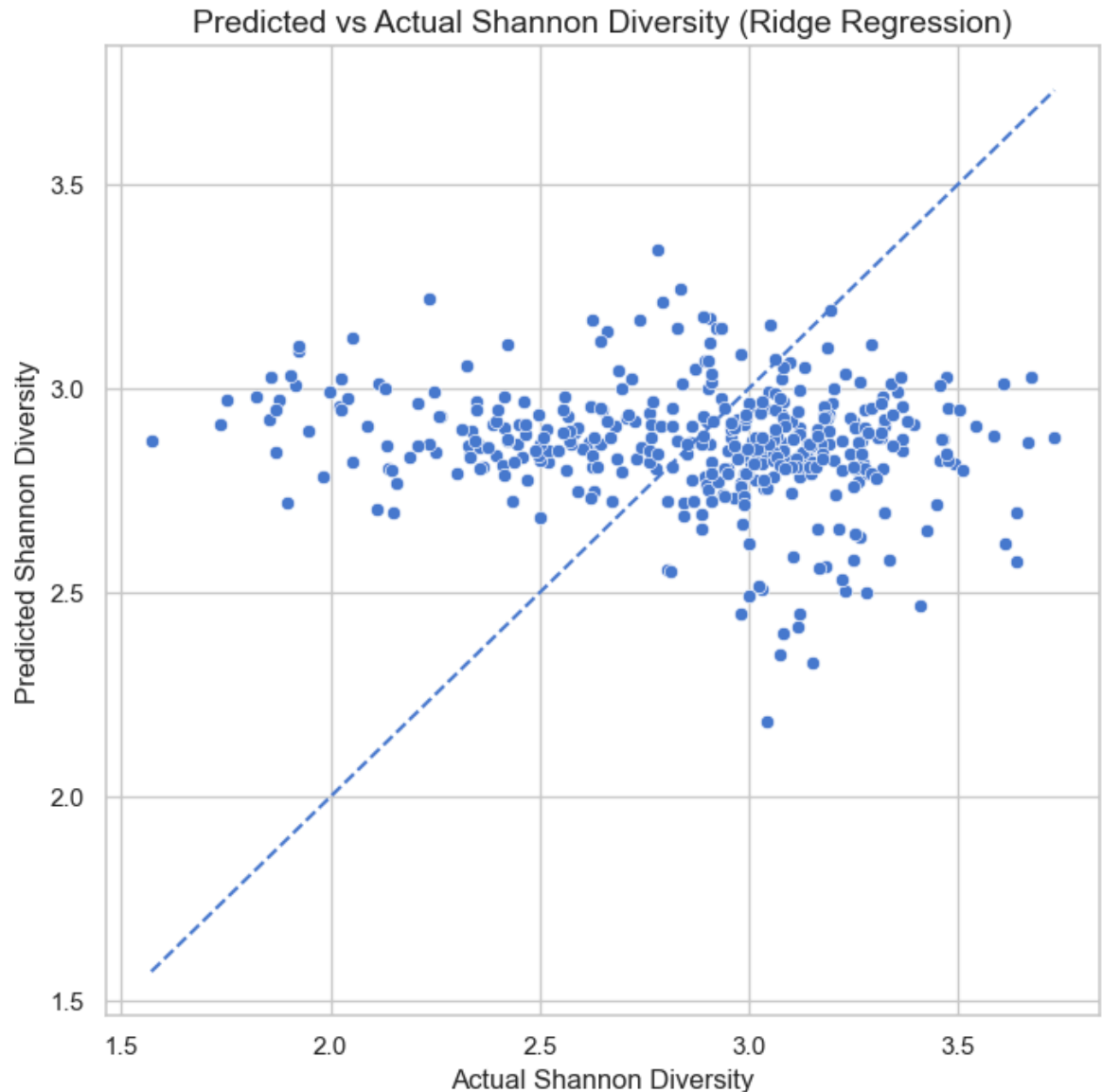
[XGBoost Regressor]

Mean R^2 : -0.384 ± 1.005
Mean RMSE: 0.433
Mean MAE: 0.329

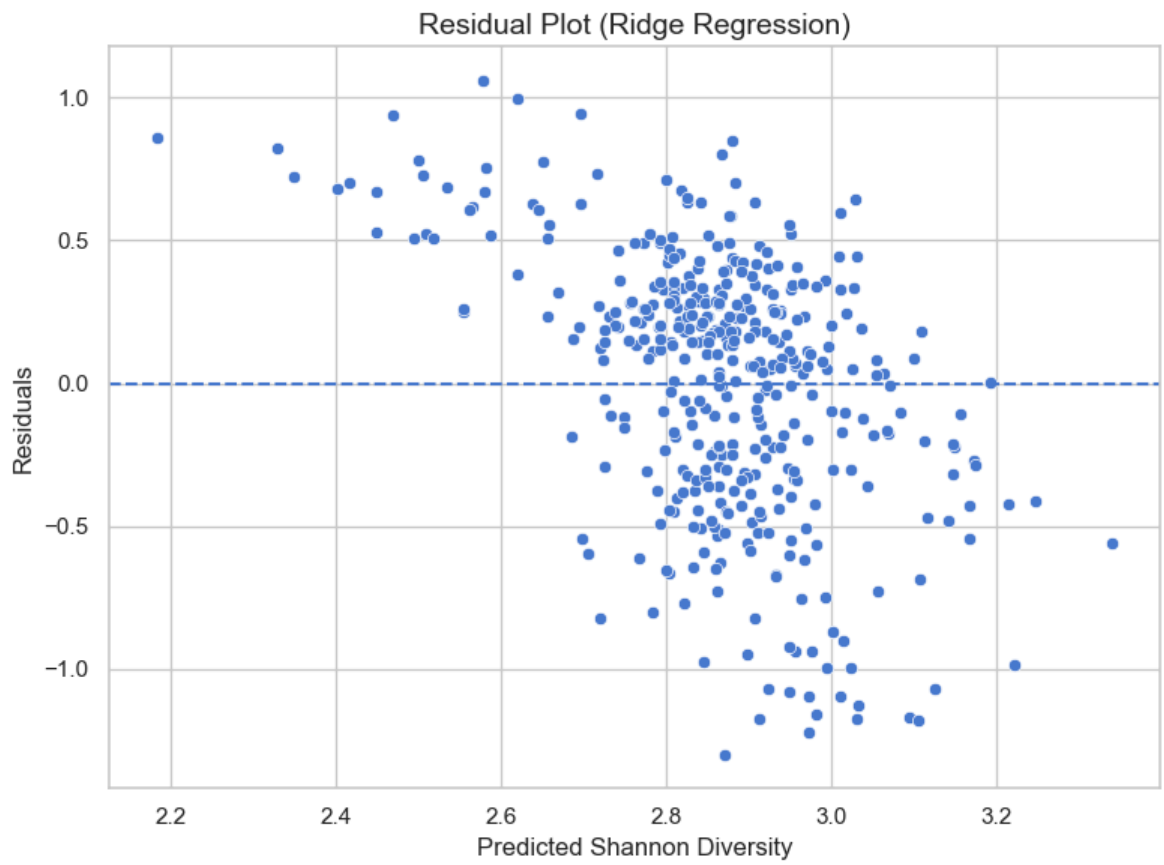
Best Regression Model: Ridge Regression (Mean $R^2 = -0.332$)
lagged_regression_model_comparison: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\lagged_regression_model_comparison.csv



Regression feature importance plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\lagged_regression_feature_importance.png



Prediction plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\lagged_regression_predicted_vs_actual.png



Residual plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\lagged_regression_residuals.png

Regression: predict continuous distance from avg Shannon diversity
Regression dataset: 407 samples
Features used: ['fme_score_daily', 'fme_participant_mean', 'fme_within_person', 'fme_lag1', 'fme_lag2', 'KCAL', 'FIBE', 'CARB', 'TFAT', 'PROT', 'Age', 'BMI']
Target mean Shannon deviation diversity: 0.007
Target std Shannon deviation diversity: 0.189

Regression Model for deviation- comparison Results

[Linear Regression]

Mean R^2 : -0.043 ± 0.036
Mean RMSE: 0.191
Mean MAE: 0.136

[Ridge Regression]

Mean R^2 : -0.042 ± 0.036
Mean RMSE: 0.191
Mean MAE: 0.136

[Random Forest Regressor]

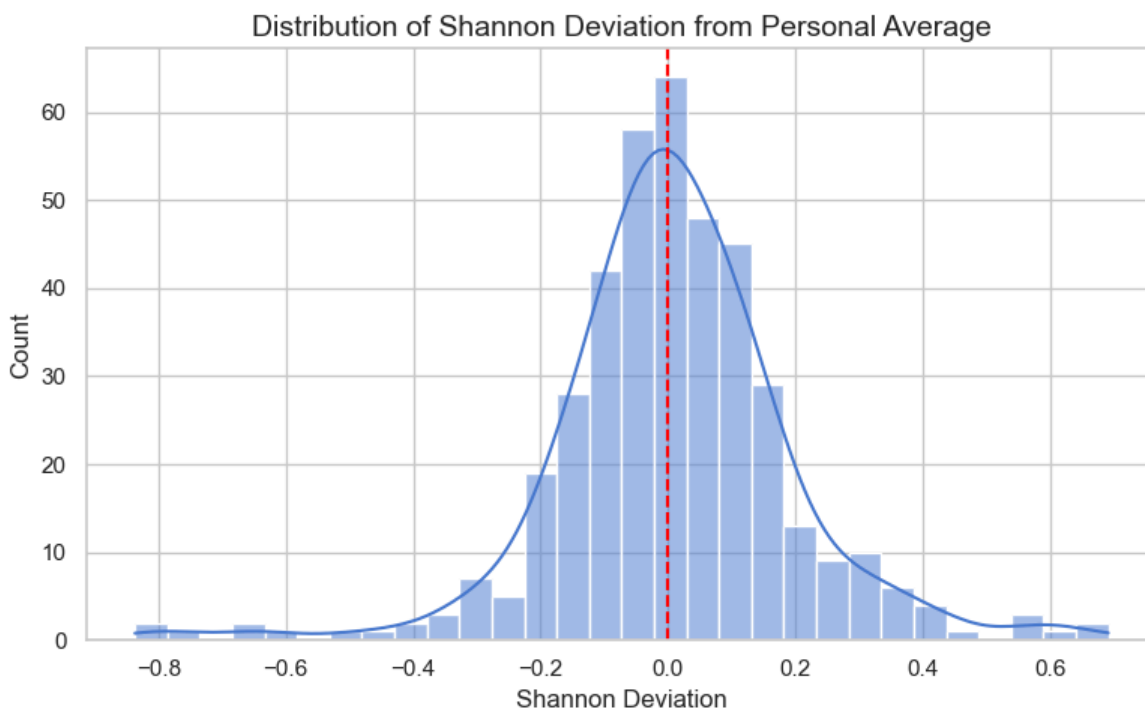
Mean R^2 : -0.133 ± 0.080
Mean RMSE: 0.199
Mean MAE: 0.145

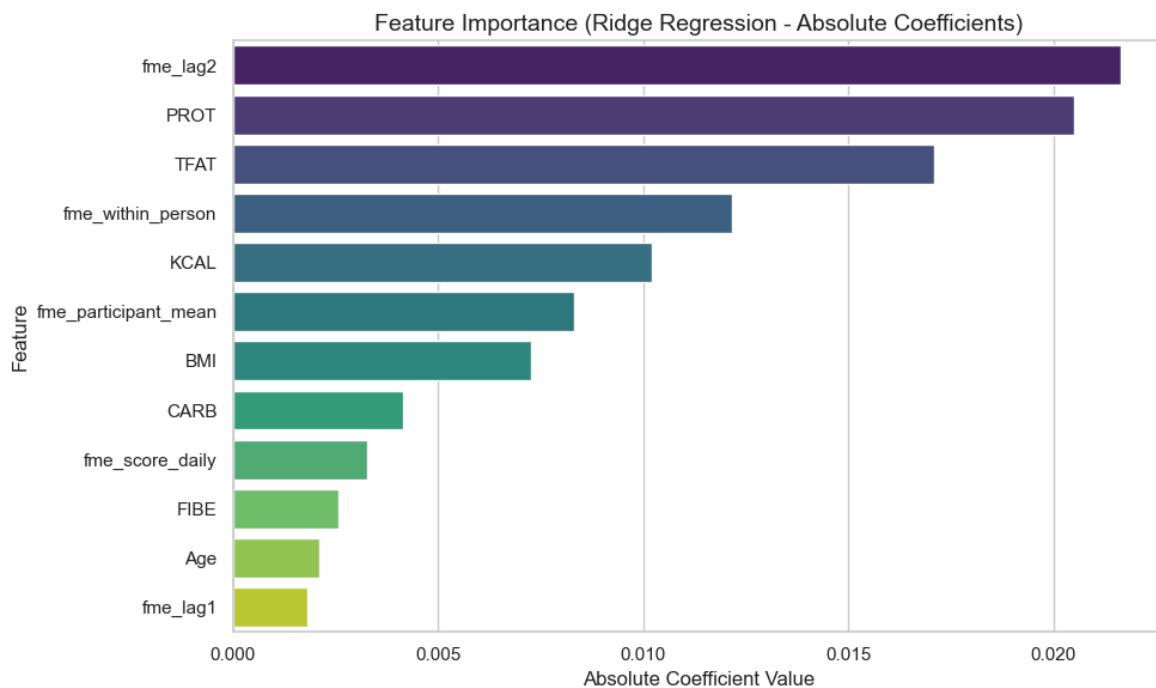
[XGBoost Regressor]

Mean R^2 : -0.385 ± 0.216
Mean RMSE: 0.218
Mean MAE: 0.162

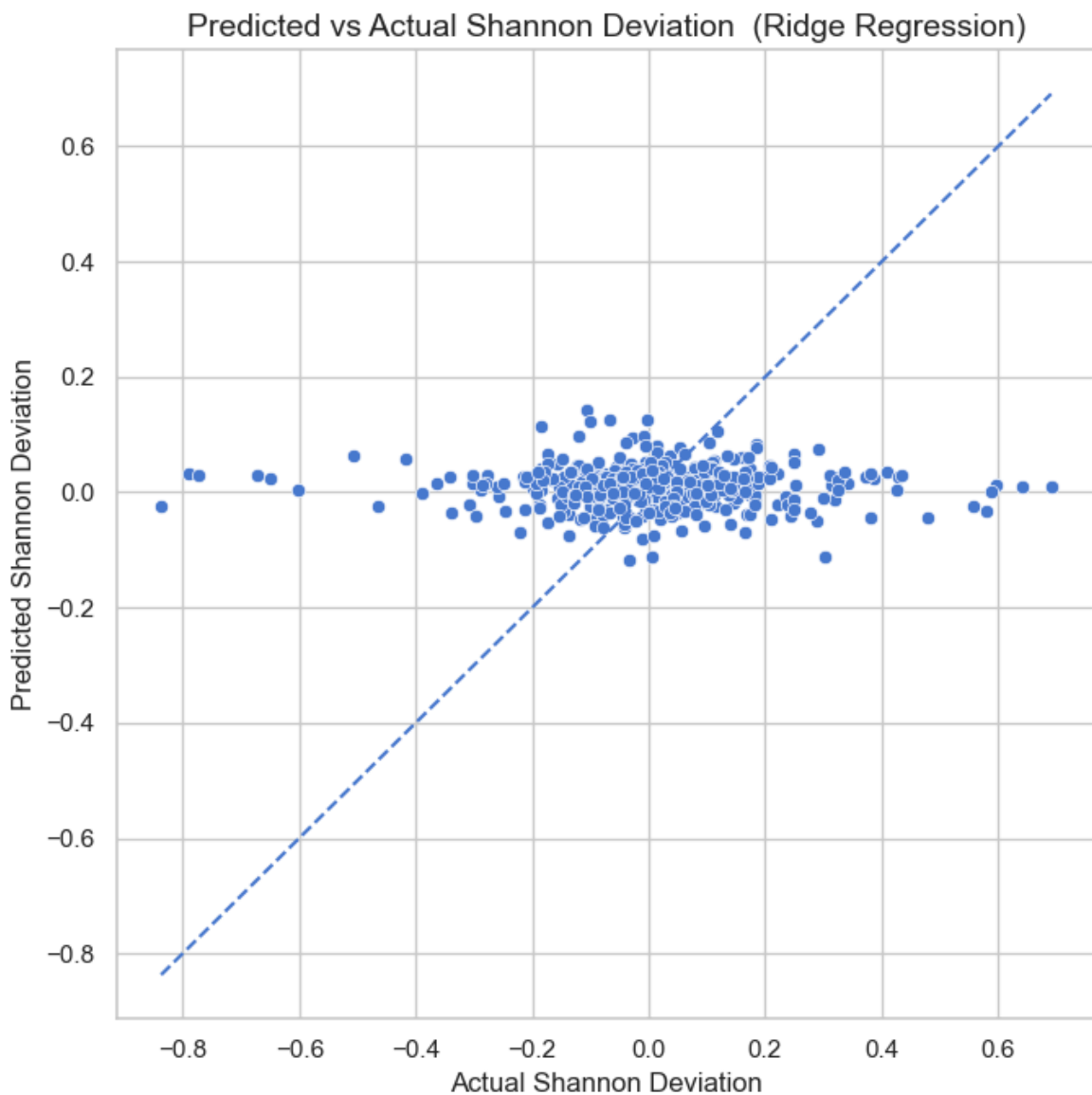
Best Regression Model: Ridge Regression (Mean $R^2 = -0.042$)

Regression results saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\lagged_regression_dev_model_comparison.csv

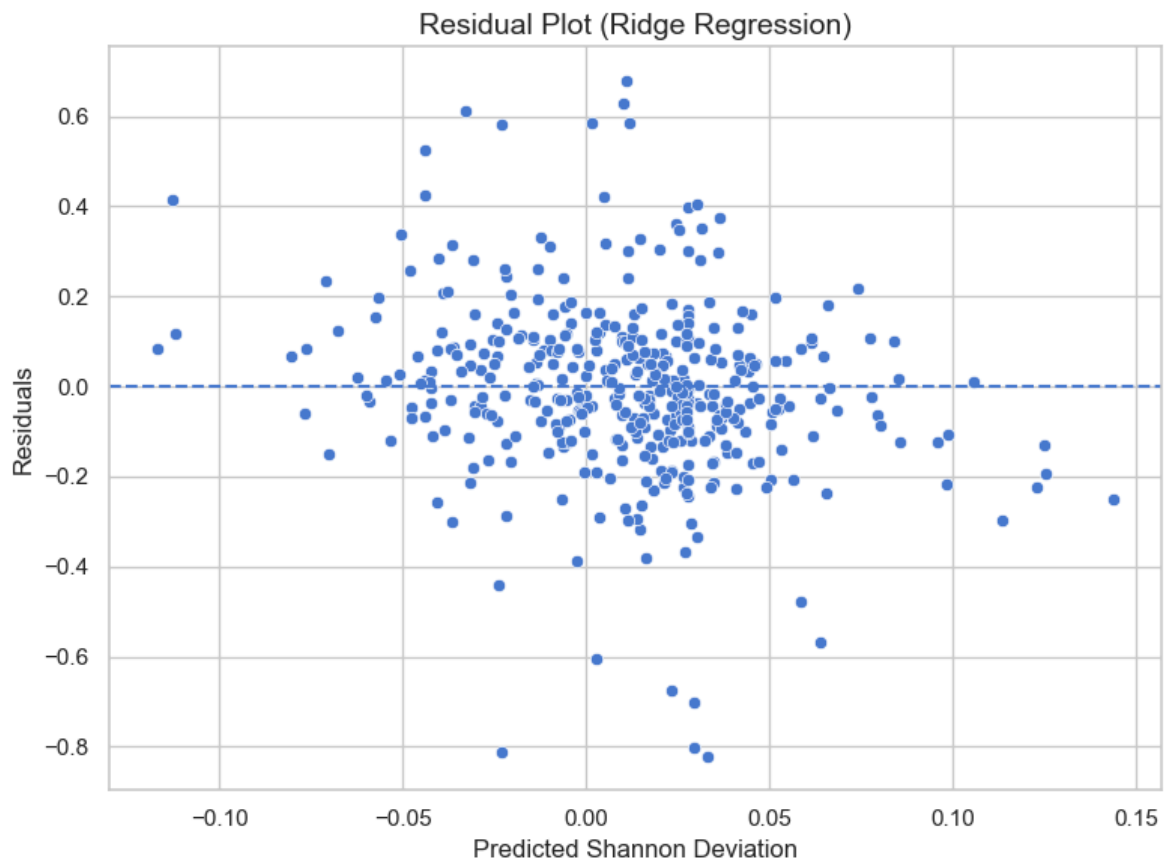




Regression feature importance plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\lagged_dev_regression_feature_importance.png



Prediction plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\lagged_dev_regression_predicted_vs_actual.png



Residual plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\lagged_dev_regression_residuals.png

model_lagged_full.py

In [128...

```
#Load data
df_fme = pd.read_csv(DATA_DIR / "fme_statistical_dataset_extended.csv", index_co
print(f"Loaded dataset: {df_fme.shape[0]} samples, {df_fme.shape[1]} columns")
print(f"Participants: {df_fme['participant_id'].nunique()}")
print("Sample of Data:")
print(df_fme.head())

#Turn shannon diversity from contiues to binary
median_shannon = df_fme["shannon_diversity"].median()
df_fme["High_Diversity_Class"] = (df_fme["shannon_diversity"] > median_shannon).
print(f"\nTarget distribution (High=1 / Low=0):")
print(df_fme["High_Diversity_Class"].value_counts())

#Features
feature_cols_full_fme = [
    "fme_score_daily",
    "fme_participant_mean",
    "fme_within_person",
    "fme_weighted_3day",
    "fme_lag1",
    "fme_lag2",
    "KCAL",
    "FIBE",
    "CARB",
    "TFAT",
    "PROT",
    "Age",
    "BMI"
]
```

```

feature_cols_full_fme = [f for f in feature_cols_full_fme if f in df_fme.columns]

#Drop rows with missing values in features or target
df_fme_clean = df_fme[feature_cols_full_fme + ["High_Diversity_Class", "particip

print(f"Full FME classification clean dataset: {df_fme_clean.shape[0]} samples")
print(f"Participants after cleaning: {df_fme_clean['participant_id'].nunique()}")

#Prepare data for modeling
X = df_fme_clean[feature_cols_full_fme]
print(X.columns.tolist())
y = df_fme_clean['High_Diversity_Class']
groups = df_fme_clean['participant_id']

#Define models
models = {
    "Logistic Regression": LogisticRegression(),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "XGBoost": XGBClassifier(eval_metric='logloss', random_state=42),
    "SVM": CalibratedClassifierCV(SVC(kernel='rbf'), ensemble=False),
    "LightGBM": LGBMClassifier(n_estimators=100, random_state=42, verbose=-1)
}

#Cross validation
gkf = GroupKFold(n_splits=5)

#Train and evaluate models
print("\nModel Comparison Results:")
best_model_name = ""
best_auc = 0
best_model_obj = None
classification_results = []

for name, model in models.items():
    acc_scores = []
    auc_scores = []

    #Make sure subjects are not split between train and test sets
    for train_idx, test_idx in gkf.split(X, y, groups=groups):
        X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
        y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

        #Scale features
        scaler = StandardScaler()
        # X_train_scaled = scaler.fit_transform(X_train)
        # X_test_scaled = scaler.transform(X_test)
        X_train_scaled = pd.DataFrame(scaler.fit_transform(X_train), columns=fea
        X_test_scaled = pd.DataFrame(scaler.transform(X_test), columns=feature_c

        #Train
        #Clone model to avoid reusing fitted state between folds
        fold_model = clone(model)
        fold_model.fit(X_train_scaled, y_train)

        #Predictions
        y_pred = fold_model.predict(X_test_scaled)
        y_proba = fold_model.predict_proba(X_test_scaled)[: , 1]

        #Calculate metrics

```

```

        acc_scores.append(accuracy_score(y_test, y_pred))
        auc_scores.append(roc_auc_score(y_test, y_proba))

#Compute mean metrics across folds
mean_acc = np.mean(acc_scores)
std_acc = np.std(acc_scores)
mean_auc = np.mean(auc_scores)
std_auc = np.std(auc_scores)

print(f"\n[{name}]")
print(f"Mean Accuracy: {mean_acc:.3f} ± {std_acc:.3f}")
print(f"Mean ROC-AUC: {mean_auc:.3f} ± {std_auc:.3f}")

classification_results.append({
    "Model": name,
    "Mean Accuracy": mean_acc,
    "Std Accuracy": std_acc,
    "Mean ROC-AUC": mean_auc,
    "Std ROC-AUC": std_auc
})

#Save best model based on AUC
if mean_auc > best_auc:
    best_auc = mean_auc
    best_model_name = name
    best_model_obj = clone(model)

print(f"\n Best Model: {best_model_name} (Mean AUC = {best_auc:.3f})")

# Save and plot classification model comparison
classification_results_df = pd.DataFrame(classification_results)

classification_results_path = DATA_DIR / "classification_model_comparison.csv"
classification_results_df.to_csv(classification_results_path, index=False)

print(f"Classification results saved: {classification_results_path}")
print(classification_results_df)

# Plot Accuracy and ROC-AUC comparison
plot_df = classification_results_df.copy()

x = np.arange(len(plot_df["Model"]))
width = 0.35

plt.figure(figsize=(9, 5))

plt.bar(
    x - width / 2,
    plot_df["Mean Accuracy"],
    width,
    label="Accuracy"
)

plt.bar(
    x + width / 2,
    plot_df["Mean ROC-AUC"],
    width,
    label="ROC-AUC"
)

```

```

plt.axhline(
    y=0.5,
    linestyle="--",
    linewidth=1.5,
    label="Random baseline"
)

plt.xticks(x, plot_df["Model"], rotation=20, ha="right")
plt.ylabel("Score")
min_score = plot_df[["Mean Accuracy", "Mean ROC-AUC"]].min().min()
max_score = plot_df[["Mean Accuracy", "Mean ROC-AUC"]].max().max()

plt.ylim(
    max(0, min_score - 0.05),
    min(1, max_score + 0.08)
)
plt.title("Classification Model Performance")
plt.legend()

for i, row in plot_df.iterrows():
    plt.text(
        i - width / 2,
        row["Mean Accuracy"] + 0.005,
        f"{row['Mean Accuracy']:.3f}",
        ha="center",
        va="bottom",
        fontsize=9
    )

    plt.text(
        i + width / 2,
        row["Mean ROC-AUC"] + 0.005,
        f"{row['Mean ROC-AUC']:.3f}",
        ha="center",
        va="bottom",
        fontsize=9
    )

plt.tight_layout()
plt.savefig(DATA_DIR / "classification_model_performance.png", dpi=300, bbox_inches="tight")
plt.show()

print(f"Classification performance plot saved: {DATA_DIR / 'classification_model_performance.png'}")

#Refit best model on the full clean regression dataset
final_reg_scaler = StandardScaler()
X_scaled = final_reg_scaler.fit_transform(X)
best_model_obj.fit(X_scaled, y)

#Feature importance for best model
if best_model_name == "Logistic Regression":
    importances = np.abs(best_model_obj.coef_[0])
    title = "Feature Importance (Logistic Regression - Absolute Coefficient)"
    xlabel = "Absolute Coefficient Value"
elif best_model_name == "SVM":
    print("SVM does not support feature importance with rbf kernel - skipping plot")
    importances = None
else:
    importances = best_model_obj.feature_importances_

```

```

title      = f"Feature Importance ({best_model_name})"
xlabel     = "Importance Score"

if importances is not None:
    indices      = np.argsort(importances)[::-1]
    feature_arr  = np.array(feature_cols_full_fme)

    plt.figure(figsize=(10, 6))
    sns.barplot(x=importances[indices], y=feature_arr[indices], hue=feature_arr[
    plt.title(title, fontsize=14)
    plt.xlabel(xlabel, fontsize=12)
    plt.ylabel("Feature", fontsize=12)
    plt.tight_layout()
    plt.savefig(DATA_DIR / "full_fme_feature_importance.png", dpi=150)      #Sav
    plt.show()
    print(f"Plot saved: {DATA_DIR / 'full_fme_feature_importance.png'}")

#Regression: predict continuous Shannon diversity
print("\n Regression: Predict Shannon Diversity")

#Use the same features, but keep Shannon diversity as a continuous target
df_reg_clean = df_fme[feature_cols_full_fme + ["shannon_diversity", "participant

#Prepare regression data
X_reg = df_reg_clean[feature_cols_full_fme]
y_reg = df_reg_clean["shannon_diversity"]
groups_reg = df_reg_clean["participant_id"]

print(f"Regression dataset: {df_reg_clean.shape[0]} samples")
print(f"Features used: {feature_cols_full_fme}")
print(f"Target mean Shannon diversity: {y_reg.mean():.3f}")
print(f"Target std Shannon diversity: {y_reg.std():.3f}")

# Define regression models
regression_models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0),
    "Random Forest Regressor": RandomForestRegressor(
        n_estimators=100,
        random_state=42
    ),
    "XGBoost Regressor": XGBRegressor(
        objective="reg:squarederror",
        n_estimators=100,
        random_state=42
    )
}

# Use grouped cross validation to avoid participant leakage
gkf_reg = GroupKFold(n_splits=5)

#Train and evaluate regression models
print("\nRegression Model Comparison Results")

best_reg_model_name = ""
best_reg_r2 = -np.inf
best_reg_model_template = None

```

```

regression_results = []
all_y_true_best = []
all_y_pred_best = []

for name, model in regression_models.items():
    r2_scores = []
    rmse_scores = []
    mae_scores = []
    all_y_true = []
    all_y_pred = []

    #Make sure subjects are not split between train and test sets
    for train_idx, test_idx in gkf_reg.split(X_reg, y_reg, groups=groups_reg):
        X_train, X_test = X_reg.iloc[train_idx], X_reg.iloc[test_idx]
        y_train, y_test = y_reg.iloc[train_idx], y_reg.iloc[test_idx]

        #Scale features using train data only
        scaler = StandardScaler()
        X_train_scaled = scaler.fit_transform(X_train)
        X_test_scaled = scaler.transform(X_test)

        #Clone model to avoid reusing fitted state between folds
        fold_model = clone(model)

        #Train
        fold_model.fit(X_train_scaled, y_train)

        #Predict continuous Shannon diversity
        y_pred = fold_model.predict(X_test_scaled)

        #Calculate regression metrics
        r2_scores.append(r2_score(y_test, y_pred))
        rmse_scores.append(np.sqrt(mean_squared_error(y_test, y_pred)))
        mae_scores.append(mean_absolute_error(y_test, y_pred))
        all_y_true.extend(y_test)
        all_y_pred.extend(y_pred)

    #Compute mean metrics across folds
    mean_r2 = np.mean(r2_scores)
    std_r2 = np.std(r2_scores)
    mean_rmse = np.mean(rmse_scores)
    mean_mae = np.mean(mae_scores)

    regression_results.append({
        "Model": name,
        "Mean R2": mean_r2,
        "Std R2": std_r2,
        "Mean RMSE": mean_rmse,
        "Mean MAE": mean_mae
    })

    print(f"\n[{name}]\n")
    print(f"Mean R^2:    {mean_r2:.3f} ± {std_r2:.3f}")
    print(f"Mean RMSE:  {mean_rmse:.3f}")
    print(f"Mean MAE:   {mean_mae:.3f}")

    #Save best regression model based on R2
    if mean_r2 > best_reg_r2:
        best_reg_r2 = mean_r2
        best_reg_model_name = name

```



```

        best_reg_model_template = model
        all_y_true_best = all_y_true
        all_y_pred_best = all_y_pred

print(f"\nBest Regression Model: {best_reg_model_name} (Mean R2 = {best_reg_r2:.

# Save regression results table
regression_results_df = pd.DataFrame(regression_results)
regression_results_path = DATA_DIR / "full_fme_regression_model_comparison.csv"
regression_results_df.to_csv(regression_results_path, index=False)
print(f"Regression results saved: {regression_results_path}")

#Refit best regression model on the full clean regression dataset
final_reg_scaler = StandardScaler()
X_reg_scaled = final_reg_scaler.fit_transform(X_reg)

best_reg_model_obj = clone(best_reg_model_template)
best_reg_model_obj.fit(X_reg_scaled, y_reg)

#Feature importance for best regression model
if best_reg_model_name == "Linear Regression" or best_reg_model_name == "Ridge R
    reg_importances = np.abs(best_reg_model_obj.coef_)
    reg_title = "Feature Importance (" + best_reg_model_name + " - Absolute Coeffici
    reg_xlabel = "Absolute Coefficient Value"
else:
    reg_importances = best_reg_model_obj.feature_importances_
    reg_title = f"Feature Importance ({best_reg_model_name})"
    reg_xlabel = "Importance Score"

reg_indices = np.argsort(reg_importances)[::-1]
feature_arr = np.array(feature_cols_full_fme)

plt.figure(figsize=(10, 6))
sns.barplot(
    x=reg_importances[reg_indices],
    y=feature_arr[reg_indices],
    hue=feature_arr[reg_indices],
    palette="viridis",
    legend=False
)
plt.title(reg_title, fontsize=14)
plt.xlabel(reg_xlabel, fontsize=12)
plt.ylabel("Feature", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "full_fme_regression_feature_importance.png", dpi=150)
plt.show()
print(f"Regression feature importance plot saved: {DATA_DIR / 'full_fme_regessi

all_y_true = np.array(all_y_true_best)
all_y_pred = np.array(all_y_pred_best)

#Plot predicted vs actual Shannon diversity
plt.figure(figsize=(7, 7))
sns.scatterplot(x=all_y_true, y=all_y_pred)
plt.plot(
    [all_y_true.min(), all_y_true.max()],
    [all_y_true.min(), all_y_true.max()],
    linestyle="--"
)

```

```

plt.title(f"Predicted vs Actual Shannon Diversity ({best_reg_model_name})", font
plt.xlabel("Actual Shannon Diversity", fontsize=12)
plt.ylabel("Predicted Shannon Diversity", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "full_fme_regression_predicted_vs_actual.png", dpi=150)
plt.show()
print(f"Prediction plot saved: {DATA_DIR / 'full_fme_regression_predicted_vs_act

#Plot residuals
residuals = all_y_true - all_y_pred

plt.figure(figsize=(8, 6))
sns.scatterplot(x=all_y_pred, y=residuals)
plt.axhline(0, linestyle="--")
plt.title(f"Residual Plot ({best_reg_model_name})", fontsize=14)
plt.xlabel("Predicted Shannon Diversity", fontsize=12)
plt.ylabel("Residuals", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "full_fme_regression_residuals.png", dpi=150)
plt.show()
print(f"Residual plot saved: {DATA_DIR / 'full_fme_regression_residuals.png'}")

# Try model distance from avarge shanon instead of predict shanon
# Calculate avg shanon score
df_fme['shannon_avg'] = df_fme.groupby('participant_id')['shannon_diversity'].tr
# Calculate sample shanon deviation
df_fme['shannon_deviation'] = df_fme['shannon_diversity'] - df_fme['shannon_avg']

#Regression: predict continuous distance from avg Shannon diversity
print("\n Regression: predict continuous distance from avg Shannon diversity")

#Use the same features, but keep Shannon diversity as a continuous target
df_reg_dev_clean = df_fme[feature_cols_full_fme + ["shannon_deviation", "partici

#Prepare regression data
X_dev_reg = df_reg_dev_clean[feature_cols_full_fme]
y_dev_reg = df_reg_dev_clean["shannon_deviation"]
groups_dev_reg = df_reg_dev_clean["participant_id"]

print(f"Regression dataset: {df_reg_dev_clean.shape[0]} samples")
print(f"Features used: {feature_cols_full_fme}")
print(f"Target mean Shannon deviation diversity: {y_dev_reg.mean():.3f}")
print(f"Target std Shannon deviation diversity: {y_dev_reg.std():.3f}")

#Define regression models
regression_dev_models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0),
    "Random Forest Regressor": RandomForestRegressor(
        n_estimators=100,
        random_state=42
    ),
    "XGBoost Regressor": XGBRegressor(
        objective="reg:squarederror",
        n_estimators=100,
        random_state=42
    )
}

#Use grouped cross validation to avoid participant leakage

```

```

gkf_reg = GroupKFold(n_splits=5)

#Train and evaluate regression models
print("\nRegression Model for deviation Comparison Results")

best_dev_reg_model_name = ""
best_dev_reg_r2 = -np.inf
best_dev_reg_model_template = None

regression_dev_results = []
all_dev_y_true_best = []
all_dev_y_pred_best = []

for name, model in regression_dev_models.items():
    r2_scores = []
    rmse_scores = []
    mae_scores = []
    all_y_true = []
    all_y_pred = []

    #Make sure subjects are not split between train and test sets
    for train_idx, test_idx in gkf_reg.split(X_dev_reg, y_dev_reg, groups=groups):
        X_train, X_test = X_dev_reg.iloc[train_idx], X_dev_reg.iloc[test_idx]
        y_train, y_test = y_dev_reg.iloc[train_idx], y_dev_reg.iloc[test_idx]

        #Scale features using train data only
        scaler = StandardScaler()
        X_train_scaled = scaler.fit_transform(X_train)
        X_test_scaled = scaler.transform(X_test)

        #Clone model to avoid reusing fitted state between folds
        fold_model = clone(model)

        #Train
        fold_model.fit(X_train_scaled, y_train)

        #Predict continuous Shannon diversity
        y_pred = fold_model.predict(X_test_scaled)

        #Calculate regression metrics
        r2_scores.append(r2_score(y_test, y_pred))
        rmse_scores.append(np.sqrt(mean_squared_error(y_test, y_pred)))
        mae_scores.append(mean_absolute_error(y_test, y_pred))
        all_y_true.extend(y_test)
        all_y_pred.extend(y_pred)

    #Compute mean metrics across folds
    mean_r2 = np.mean(r2_scores)
    std_r2 = np.std(r2_scores)
    mean_rmse = np.mean(rmse_scores)
    mean_mae = np.mean(mae_scores)

    regression_dev_results.append({
        "Model": name,
        "Mean R2": mean_r2,
        "Std R2": std_r2,
        "Mean RMSE": mean_rmse,
        "Mean MAE": mean_mae
    })

```

```

print(f"\n[{name}]")
print(f"Mean R^2:    {mean_r2:.3f} ± {std_r2:.3f}")
print(f"Mean RMSE:  {mean_rmse:.3f}")
print(f"Mean MAE:    {mean_mae:.3f}")

#Save best regression model based on R2
if mean_r2 > best_dev_reg_r2:
    best_dev_reg_r2 = mean_r2
    best_dev_reg_model_name = name
    best_dev_reg_model_template = model
    all_dev_y_true_best = all_y_true
    all_dev_y_pred_best = all_y_pred

print(f"\nBest Regression Model: {best_dev_reg_model_name} (Mean R² = {best_dev_

#Save regression deviation from avg results table
regression_dev_results_df = pd.DataFrame(regression_dev_results)
regression_dev_results_path = DATA_DIR / "full_fme_regression_dev_model_comparis
regression_dev_results_df.to_csv(regression_dev_results_path, index=False)
print(f"Regression results saved: {regression_dev_results_path}")

#Refit best regression model on the full clean regression dataset
final_dev_reg_scaler = StandardScaler()
X_dev_reg_scaled = final_dev_reg_scaler.fit_transform(X_dev_reg)

best_dev_reg_model_obj = clone(best_dev_reg_model_template)
best_dev_reg_model_obj.fit(X_dev_reg_scaled, y_dev_reg)

#Feature importance for best regression model
if best_dev_reg_model_name == "Linear Regression" or best_dev_reg_model_name ==
    reg_dev_importances = np.abs(best_dev_reg_model_obj.coef_)
    reg_dev_title = "Feature Importance (" + best_dev_reg_model_name + " - Absolute
    reg_dev_xlabel = "Absolute Coefficient Value"
else:
    reg_dev_importances = best_dev_reg_model_obj.feature_importances_
    reg_dev_title = f"Feature Importance ({best_dev_reg_model_name})"
    reg_dev_xlabel = "Importance Score"

reg_dev_indices = np.argsort(reg_dev_importances)[::-1]
feature_dev_arr = np.array(feature_cols_full_fme)

# Bar plot of distribution of Shannon Deviation from Personal Average
plt.figure(figsize=(8, 5))
sns.histplot(df_reg_dev_clean["shannon_deviation"], bins=30, kde=True)
plt.axvline(0, linestyle="--", color="red")
plt.title("Distribution of Shannon Deviation from Personal Average", fontsize=14)
plt.xlabel("Shannon Deviation", fontsize=12)
plt.ylabel("Count", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "full_fme_dev_shannon_deviation_distribution.png", dpi=15)
plt.show()

# Bar plot of feature importance
plt.figure(figsize=(10, 6))
sns.barplot(
    x=reg_dev_importances[reg_dev_indices],
    y=feature_dev_arr[reg_dev_indices],
    hue=feature_dev_arr[reg_dev_indices],
    palette="viridis",

```

```

        legend=False
    )
    plt.title(reg_dev_title, fontsize=14)
    plt.xlabel(reg_dev_xlabel, fontsize=12)
    plt.ylabel("Feature", fontsize=12)
    plt.tight_layout()
    plt.savefig(DATA_DIR / "full_fme_dev_regression_feature_importance.png", dpi=150)
    plt.show()
    print(f"Regression feature importance plot saved: {DATA_DIR / 'dev_regression_fe

all_dev_y_true = np.array(all_dev_y_true_best)
all_dev_y_pred = np.array(all_dev_y_pred_best)

#Plot predicted vs actual Shannon diversity
plt.figure(figsize=(7, 7))
sns.scatterplot(x=all_dev_y_true, y=all_dev_y_pred)
plt.plot(
    [all_dev_y_true.min(), all_dev_y_true.max()],
    [all_dev_y_true.min(), all_dev_y_true.max()],
    linestyle="--"
)
plt.title(f"Predicted vs Actual Shannon Deviation ({best_dev_reg_model_name})",
plt.xlabel("Actual Shannon Deviation", fontsize=12)
plt.ylabel("Predicted Shannon Deviation", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "full_fme_dev_regression_predicted_vs_actual.png", dpi=150)
plt.show()
print(f"Prediction plot saved: {DATA_DIR / 'full_fme_dev_regression_predicted_vs

#Plot residuals
dev_residuals = all_dev_y_true - all_dev_y_pred

plt.figure(figsize=(8, 6))
sns.scatterplot(x=all_dev_y_pred, y=dev_residuals)
plt.axhline(0, linestyle="--")
plt.title(f"Residual Plot ({best_dev_reg_model_name})", fontsize=14)
plt.xlabel("Predicted Shannon Deviation", fontsize=12)
plt.ylabel("Residuals", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "full_fme_dev_regression_residuals.png", dpi=150)
plt.show()
print(f"Residual plot saved: {DATA_DIR / 'full_fme_dev_regression_residuals.png'

```

Loaded dataset: 475 samples, 35 columns

Participants: 34

Sample of Data:

fecal_sample_id	participant_id	study_day	Gender	Age	BMI	Weight	\
MCT.f.0002	MCTs01	2	F	25.9000	25.0000	74.6000	
MCT.f.0003	MCTs01	3	F	25.9000	25.0000	74.6000	
MCT.f.0004	MCTs01	4	F	25.9000	25.0000	74.6000	
MCT.f.0005	MCTs01	5	F	25.9000	25.0000	74.6000	
MCT.f.0006	MCTs01	6	F	25.9000	25.0000	74.6000	

fecal_sample_id	Supplement	Medications	fme_score_daily	\
MCT.f.0002	EVOO	Nexplanon (bc implant)	67.1961	
MCT.f.0003	EVOO	Nexplanon (bc implant)	81.2412	
MCT.f.0004	EVOO	Nexplanon (bc implant)	126.7711	
MCT.f.0005	EVOO	Nexplanon (bc implant)	31.4846	
MCT.f.0006	EVOO	Nexplanon (bc implant)	61.2266	

fecal_sample_id	fme_participant_mean	fme_within_person	fme_lag1	...	\
MCT.f.0002	71.2710	-4.0749	NaN	...	
MCT.f.0003	71.2710	9.9703	67.1961	...	
MCT.f.0004	71.2710	55.5001	81.2412	...	
MCT.f.0005	71.2710	-39.7864	126.7711	...	
MCT.f.0006	71.2710	-10.0444	31.4846	...	

fecal_sample_id	PROT	TFAT	CARB	FIBE	SUGR	SODI	\
MCT.f.0002	109.6198	53.6153	242.9190	14.4194	74.2081	4300.6165	
MCT.f.0003	89.9930	50.5614	234.2251	25.9961	67.4747	4225.8133	
MCT.f.0004	94.6120	92.0216	257.5390	26.1784	83.9603	5329.8821	
MCT.f.0005	110.2684	95.5703	165.6081	13.0114	64.9220	3527.5014	
MCT.f.0006	161.8178	108.3985	332.6372	21.5291	119.8657	5428.5730	

fecal_sample_id	D_TOTAL	D_YOGURT	D_CHEESE	PF_MEAT	PF_SEAFD_HI	G_WHOLE
MCT.f.0002	4.1560	0.6460	1.1535	0.0000	3.9690	0.9718
MCT.f.0003	2.9797	0.6460	1.2076	0.0000	0.0000	2.9318
MCT.f.0004	3.6345	0.0000	1.4826	0.0000	0.0000	0.9718
MCT.f.0005	3.3797	0.0000	1.0590	1.8144	0.0000	0.9718
MCT.f.0006	1.5022	0.0000	0.0000	6.2960	0.0000	0.4698

[5 rows x 35 columns]

Target distribution (High=1 / Low=0):

High_Diversity_Class

0 238

1 237

Name: count, dtype: int64

Full FME classification clean dataset: 407 samples

Participants after cleaning: 34

['fme_score_daily', 'fme_participant_mean', 'fme_within_person', 'fme_weighted_3d
ay', 'fme_lag1', 'fme_lag2', 'KCAL', 'FIBE', 'CARB', 'TFAT', 'PROT', 'Age', 'BM
I']

Model Comparison Results:

[Logistic Regression]

Mean Accuracy: 0.425 ± 0.073

Mean ROC-AUC: 0.339 ± 0.057

[Random Forest]

Mean Accuracy: 0.558 ± 0.038

Mean ROC-AUC: 0.499 ± 0.102

[XGBoost]

Mean Accuracy: 0.519 ± 0.071

Mean ROC-AUC: 0.479 ± 0.094

[SVM]

Mean Accuracy: 0.368 ± 0.059

Mean ROC-AUC: 0.430 ± 0.071

[LightGBM]

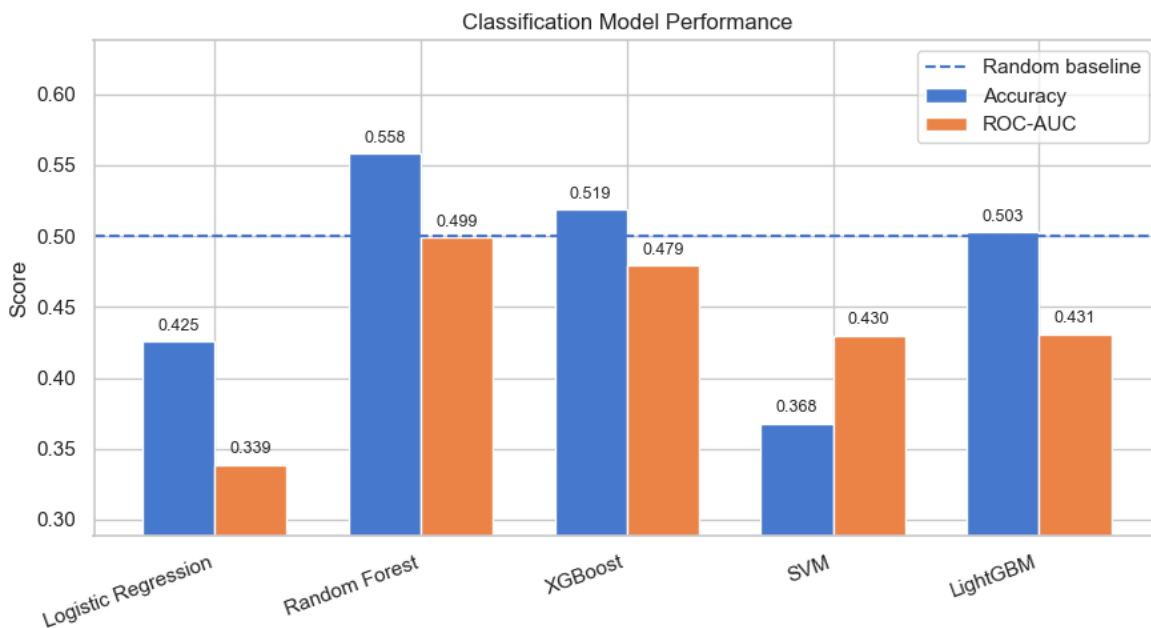
Mean Accuracy: 0.503 ± 0.090

Mean ROC-AUC: 0.431 ± 0.181

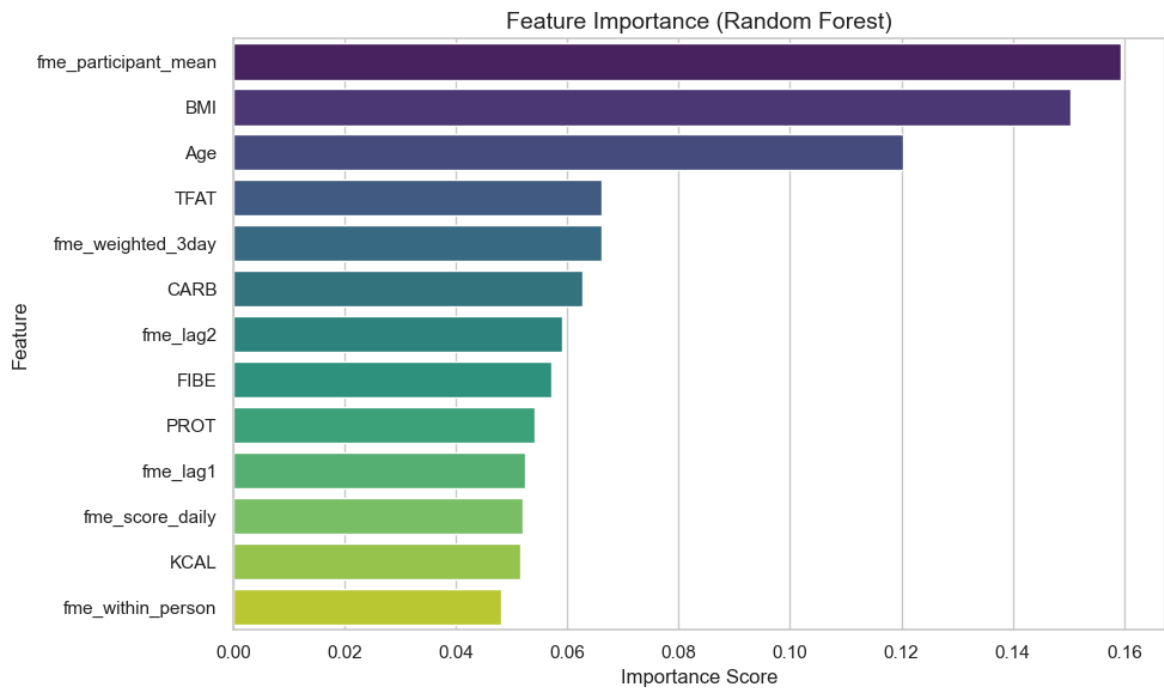
Best Model: Random Forest (Mean AUC = 0.499)

Classification results saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\classification_model_comparison.csv

	Model	Mean Accuracy	Std Accuracy	Mean ROC-AUC	Std ROC-AUC
0	Logistic Regression	0.4254	0.0729	0.3387	0.0568
1	Random Forest	0.5579	0.0379	0.4987	0.1019
2	XGBoost	0.5187	0.0714	0.4794	0.0944
3	SVM	0.3678	0.0594	0.4298	0.0707
4	LightGBM	0.5030	0.0899	0.4308	0.1813



Classification performance plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\classification_model_performance.png



Plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\full_fme_feature_importance.png

Regression: Predict Shannon Diversity
Regression dataset: 407 samples
Features used: ['fme_score_daily', 'fme_participant_mean', 'fme_within_person', 'fme_weighted_3day', 'fme_lag1', 'fme_lag2', 'KCAL', 'FIBE', 'CARB', 'TFAT', 'PROT', 'Age', 'BMI']
Target mean Shannon diversity: 2.868
Target std Shannon diversity: 0.411

Regression Model Comparison Results

[Linear Regression]

Mean R^2 : -0.337 ± 0.256
Mean RMSE: 0.455
Mean MAE: 0.374

[Ridge Regression]

Mean R^2 : -0.332 ± 0.254
Mean RMSE: 0.454
Mean MAE: 0.374

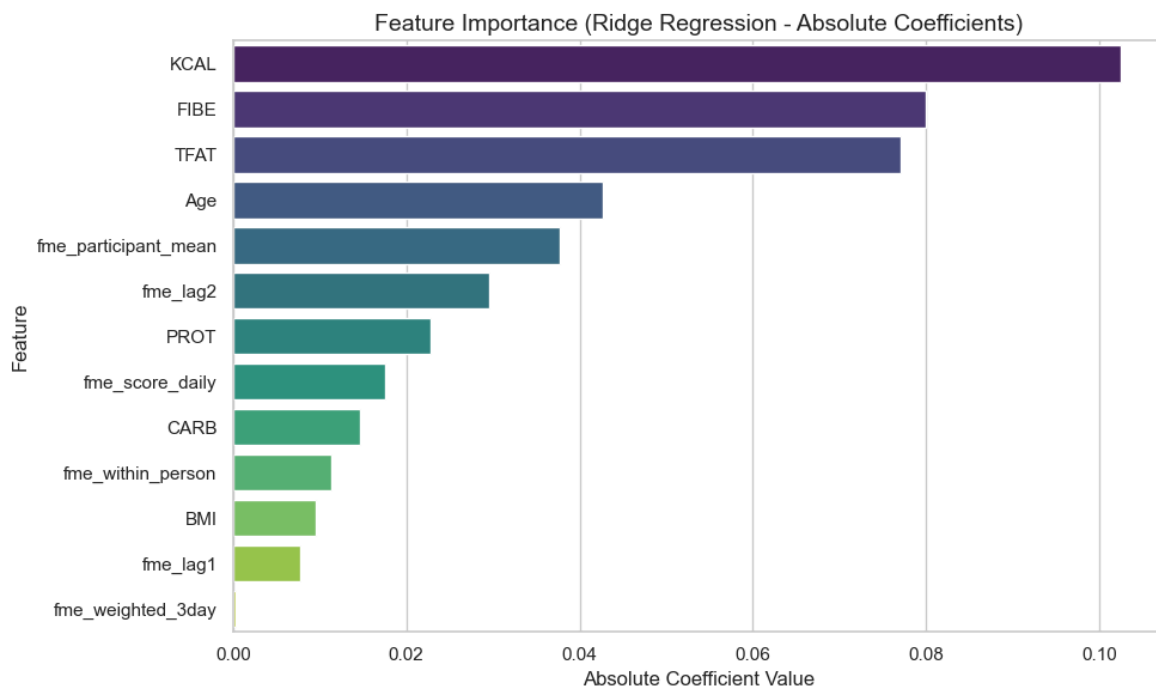
[Random Forest Regressor]

Mean R^2 : -0.534 ± 0.724
Mean RMSE: 0.469
Mean MAE: 0.389

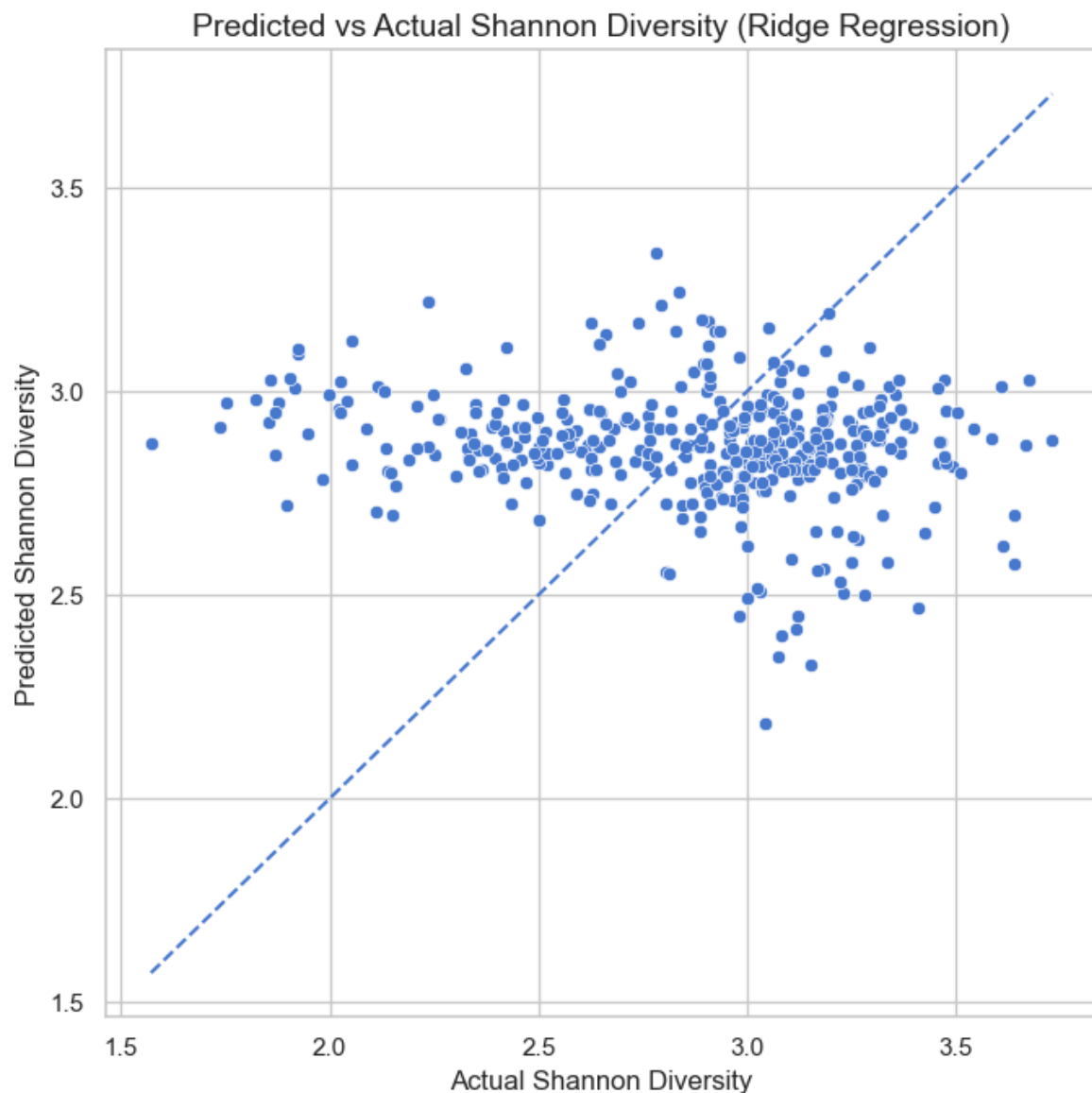
[XGBoost Regressor]

Mean R^2 : -0.389 ± 1.021
Mean RMSE: 0.432
Mean MAE: 0.327

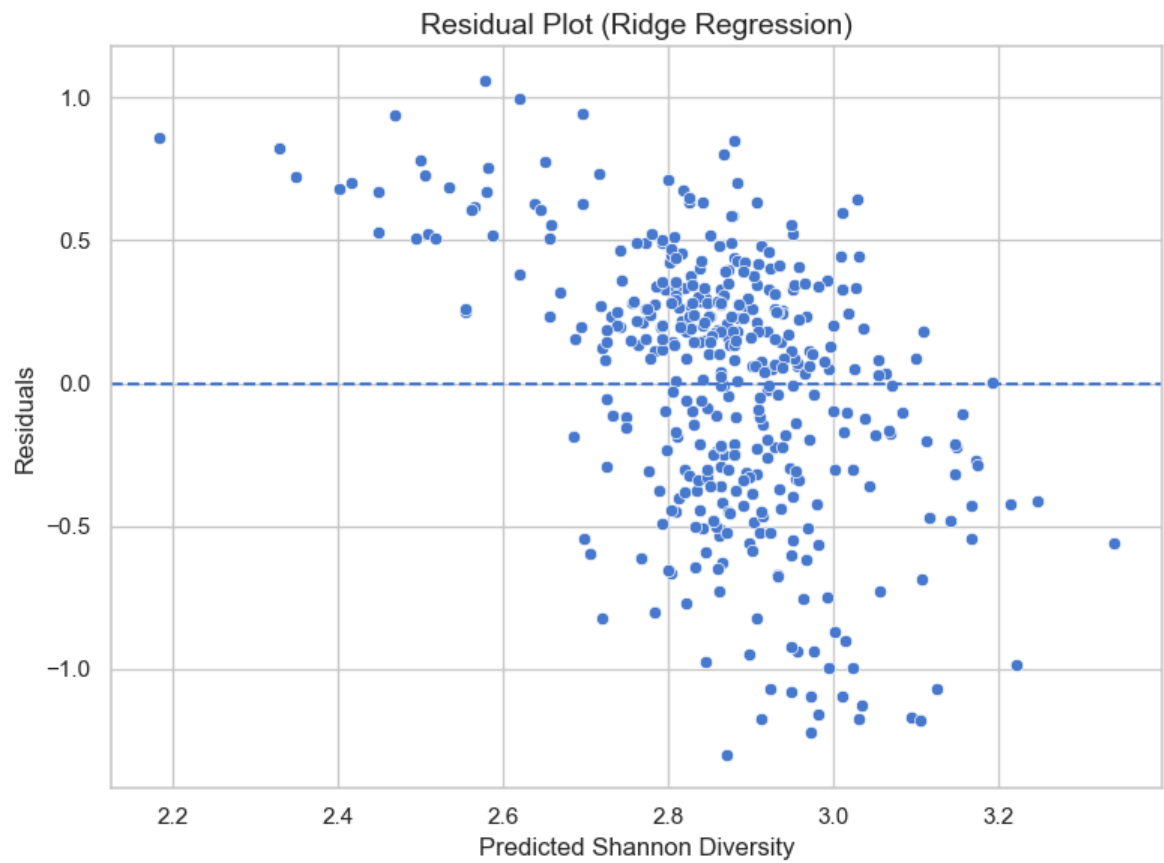
Best Regression Model: Ridge Regression (Mean $R^2 = -0.332$)
Regression results saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\full_fme_regression_model_comparison.csv



Regression feature importance plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\full_fme_regression_feature_importance.png



Prediction plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\full_fme_regression_predicted_vs_actual.png



Residual plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\full_fme_regression_residuals.png

Regression: predict continuous distance from avg Shannon diversity
Regression dataset: 407 samples
Features used: ['fme_score_daily', 'fme_participant_mean', 'fme_within_person', 'fme_weighted_3day', 'fme_lag1', 'fme_lag2', 'KCAL', 'FIBE', 'CARB', 'TFAT', 'PRO T', 'Age', 'BMI']
Target mean Shannon deviation diversity: 0.007
Target std Shannon deviation diversity: 0.189

Regression Model for deviation Comparison Results

[Linear Regression]

Mean R^2 : -0.043 ± 0.036
Mean RMSE: 0.191
Mean MAE: 0.136

[Ridge Regression]

Mean R^2 : -0.042 ± 0.036
Mean RMSE: 0.191
Mean MAE: 0.136

[Random Forest Regressor]

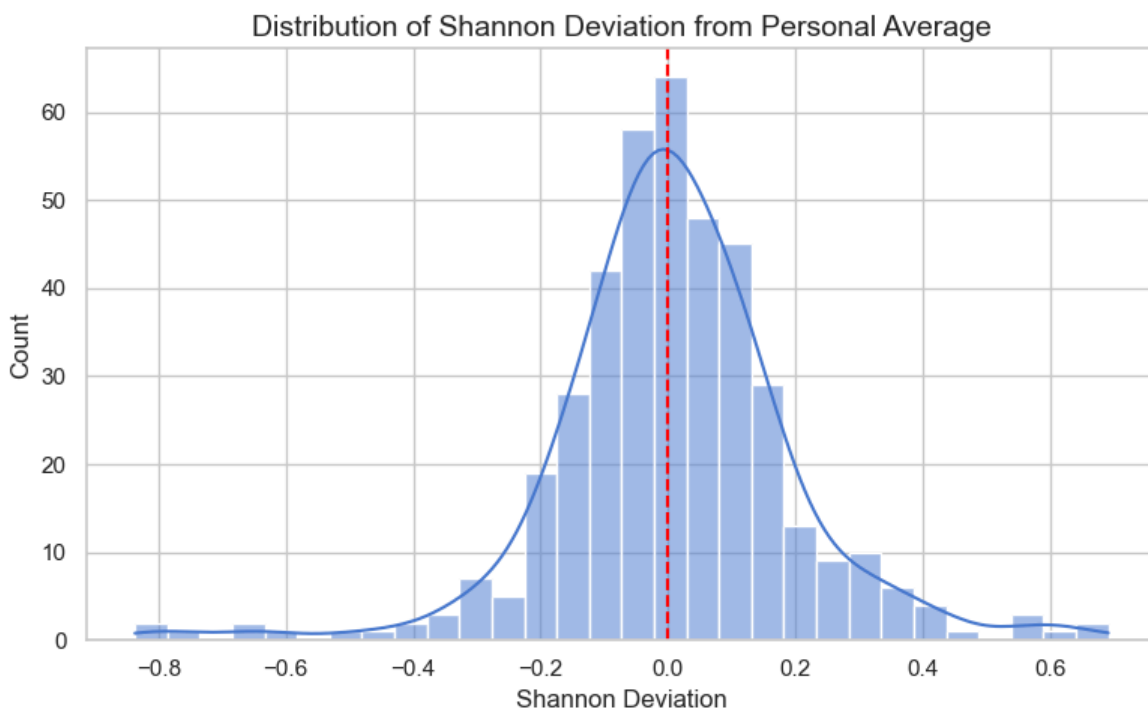
Mean R^2 : -0.132 ± 0.055
Mean RMSE: 0.199
Mean MAE: 0.145

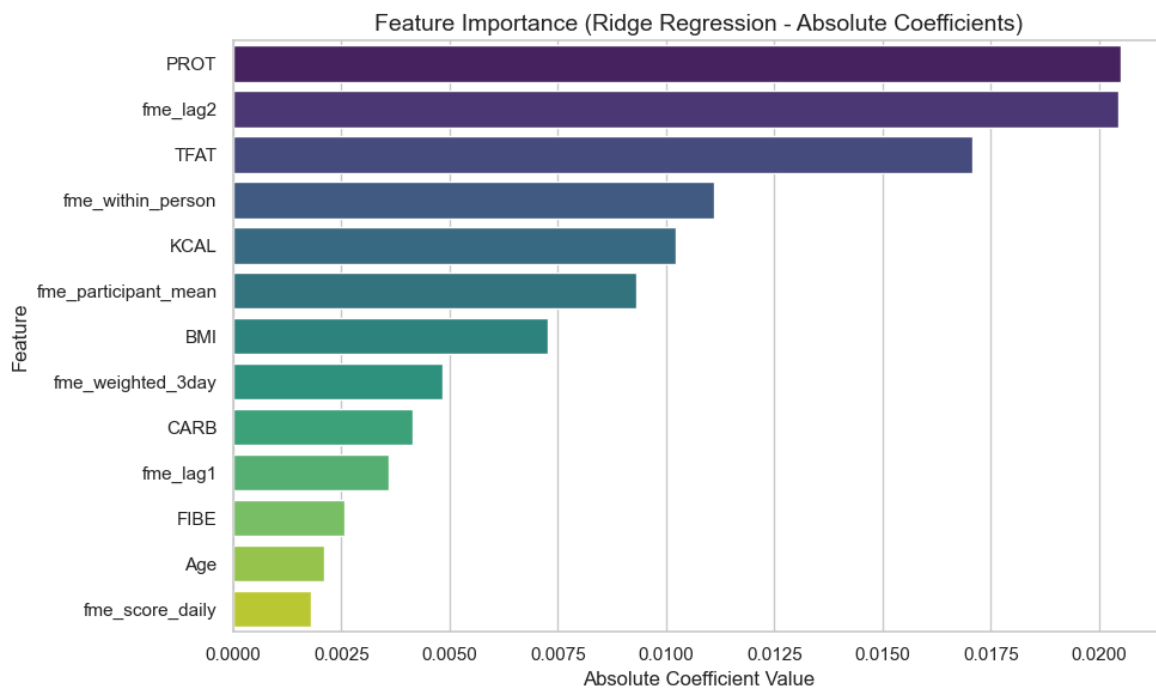
[XGBoost Regressor]

Mean R^2 : -0.375 ± 0.234
Mean RMSE: 0.218
Mean MAE: 0.161

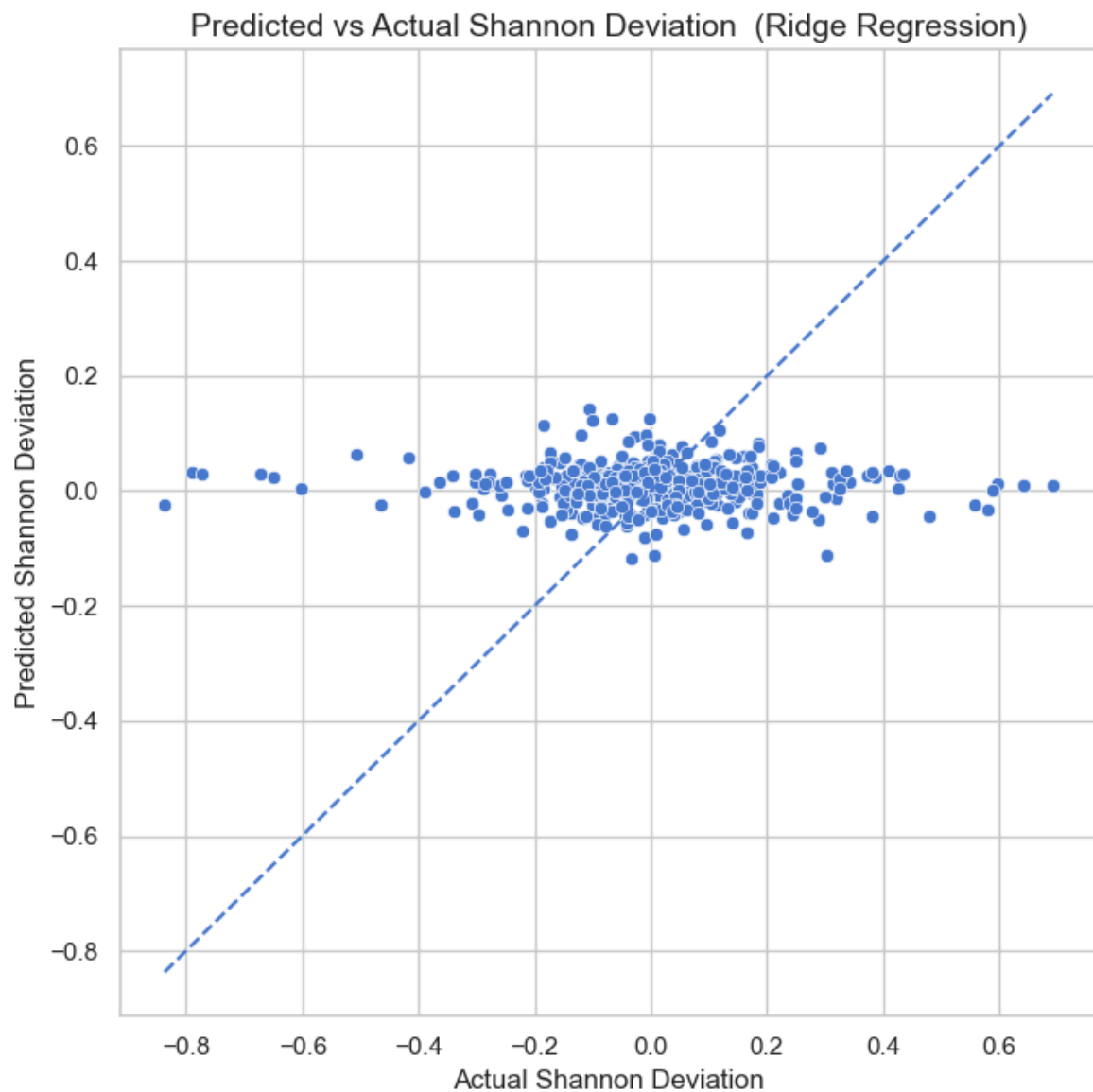
Best Regression Model: Ridge Regression (Mean $R^2 = -0.042$)

Regression results saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\full_fme_regression_dev_model_comparison.csv

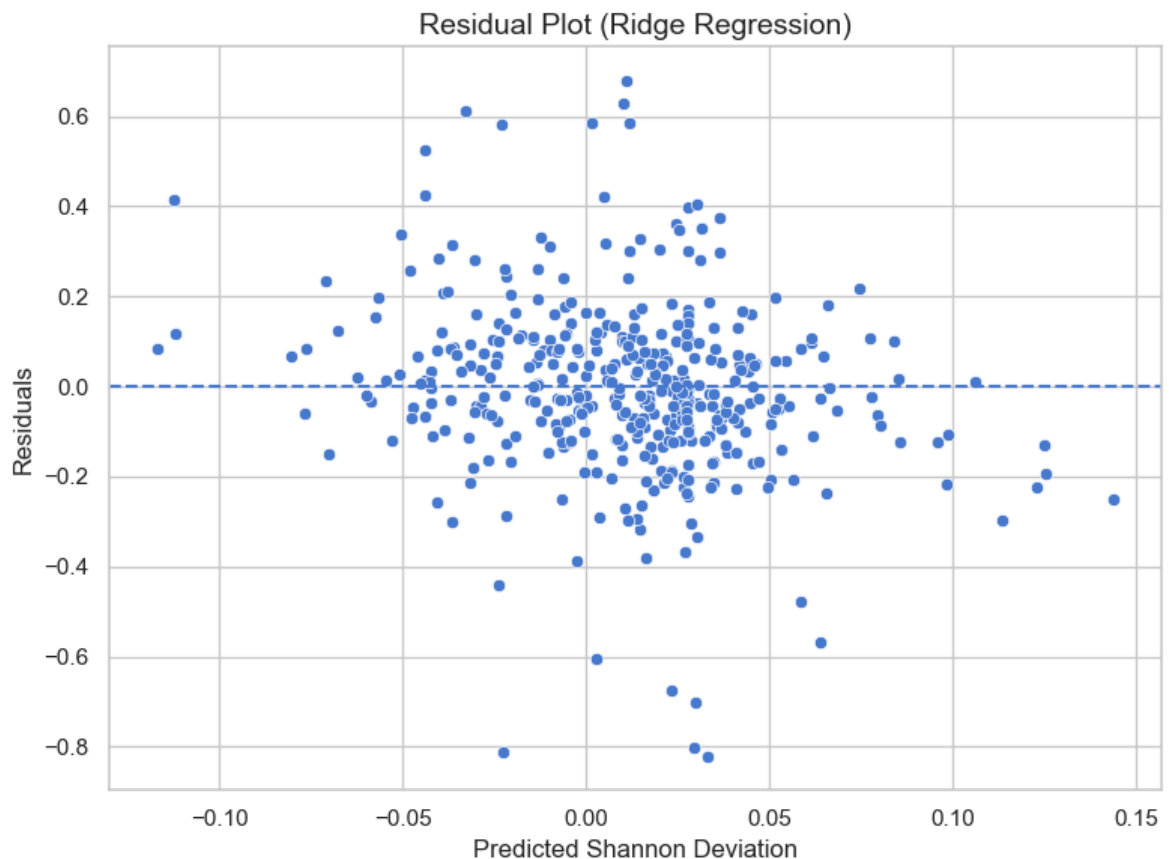




Regression feature importance plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\dev_regression_feature_importance.png



Prediction plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\full_fme_dev_regression_predicted_vs_actual.png



Residual plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\full_fme_dev_regression_residuals.png

Python helper file:

`stability_features_stat_models.py`

In [129... Path('stability_features_stat_models.py').write_text('import pandas as pd\nimport print("Created helper file:", 'stability_features_stat_models.py')

Created helper file: stability_features_stat_models.py

2. Combined notebooks

Start sprint 1 notebook

Notebook:

`Merge_data_Analysis.ipynb`

EDA on people data

Summary

This notebook profiles the dietary records and participant metadata.

The analysis includes:

- Loading daily dietary records from `nutrition_totals.txt`.
- Selecting key dietary variables: calories (`KCAL`), fiber (`FIBE`), carbohydrates (`CARB`), and total fat (`TFAT`).
- Calculating missing-value percentages for dietary variables.
- Visualizing missing values using `missingno`.
- Aggregating dietary records into average intake per participant.
- Loading participant metadata from `food_map.txt`.
- Adding participant-level characteristics: `Age` and `BMI`.
- Plotting histograms and boxplots for nutrient intake and participant characteristics.

These variables can later be used as control variables / confounders in downstream microbiome analysis.

In [130]...

```
# Define file paths

nutrition_path = DATA_DIR / "dietstudy_analyses-master" / "data" / "diet" / "nut
metadata_path = DATA_DIR / "dietstudy_analyses-master" / "data" / "maps" / "fo

print("Nutrition path exists:", nutrition_path.exists())
print("Nutrition path:", nutrition_path)

print("Metadata path exists:", metadata_path.exists())
print("Metadata path:", metadata_path)
```

Nutrition path exists: True

Nutrition path: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\dietstudy_analyses-master\data\diet\nutrition_totals.txt

Metadata path exists: True

Metadata path: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\dietstudy_analyses-master\data\maps\food_map.txt

In [131]...

```
# Load the nutrition totals data

df_nutrients = pd.read_csv(nutrition_path, sep="\t")

df_nutrients.columns = df_nutrients.columns.str.strip()

print("Nutrients shape:", df_nutrients.shape)

display(df_nutrients.head())

print("Nutrient columns:")
print(df_nutrients.columns.tolist())
```

Nutrients shape: (580, 107)

	X.SampleID	UserName	StudyDayNo	RecordDayNo	KCAL	PROT	TFAT	
0	MCT.f.0001	MCTs01	Day.01	1	1707.0402	95.6687	55.1853	187
1	MCT.f.0002	MCTs01	Day.02	2	1970.0436	109.6198	53.6153	242
2	MCT.f.0003	MCTs01	Day.03	3	1714.8953	89.9930	50.5614	234
3	MCT.f.0004	MCTs01	Day.04	4	2487.2326	94.6120	92.0216	257
4	MCT.f.0005	MCTs01	Day.05	5	2260.2112	110.2684	95.5703	165

5 rows × 107 columns

Nutrient columns:

```
[ 'X.SampleID', 'UserName', 'StudyDayNo', 'RecordDayNo', 'KCAL', 'PROT', 'TFAT',
  'CARB', 'MOIS', 'ALC', 'CAFF', 'THEO', 'SUGR', 'FIBE', 'CALC', 'IRON', 'MAGN', 'P
HOS', 'POTA', 'SODI', 'ZINC', 'COPP', 'SELE', 'VC', 'VB1', 'VB2', 'NIAC', 'VB6',
  'FOLA', 'FA', 'FF', 'FDFE', 'VB12', 'VARA', 'RET', 'BCAR', 'ACAR', 'CRYP', 'LYC
O', 'LZ', 'ATOC', 'VK', 'CHOLE', 'SFAT', 'S040', 'S060', 'S080', 'S100', 'S120',
  'S140', 'S160', 'S180', 'MFAT', 'M161', 'M181', 'M201', 'M221', 'PFAT', 'P182',
  'P183', 'P184', 'P204', 'P205', 'P225', 'P226', 'VITD', 'CHOLN', 'VITE_ADD', 'B12
_ADD', 'F_TOTAL', 'F_CITMLB', 'F_OTHER', 'F_JUICE', 'V_TOTAL', 'V_DRKGR', 'V_REDO
R_TOTAL', 'V_REDOR_TOMATO', 'V_REDOR_OTHER', 'V_STARCHY_TOTAL', 'V_STARCHY_POTAT
O', 'V_STARCHY_OTHER', 'V_OTHER', 'V_LEGUMES', 'G_TOTAL', 'G_WHOLE', 'G_REFINED',
  'PF_TOTAL', 'PF_MPS_TOTAL', 'PF_MEAT', 'PF_CUREDMEAT', 'PF_ORGAN', 'PF_POULT', 'P
F_SEAFD_HI', 'PF_SEAFD_LOW', 'PF_EGGS', 'PF_SOY', 'PF_NUTSDS', 'PF_LEGUMES', 'D_T
OTAL', 'D_MILK', 'D_YOGURT', 'D_CHEESE', 'OILS', 'SOLID_FATS', 'ADD_SUGARS', 'A_D
RINKS', 'Replicate']
```

```
In [132... # Define expected nutrient columns and check which ones are present

nutrient_columns = ["KCAL", "FIBE", "CARB"]

# Try to add fat column automatically if it exists under a common name
fat_candidates = ["FAT", "TFAT", "TotalFat", "Fat", "Total_Fat"]

for col in fat_candidates:
    if col in df_nutrients.columns:
        nutrient_columns.append(col)
        break

existing_nutrient_columns = [col for col in nutrient_columns if col in df_nutrie

print("Selected nutrient columns:")
print(existing_nutrient_columns)

print("\nMissing expected columns:")
print(set(nutrient_columns) - set(existing_nutrient_columns))
```

Selected nutrient columns:
['KCAL', 'FIBE', 'CARB', 'TFAT']

Missing expected columns:
set()

```
In [133... # Analyze missing data in the selected nutrient columns

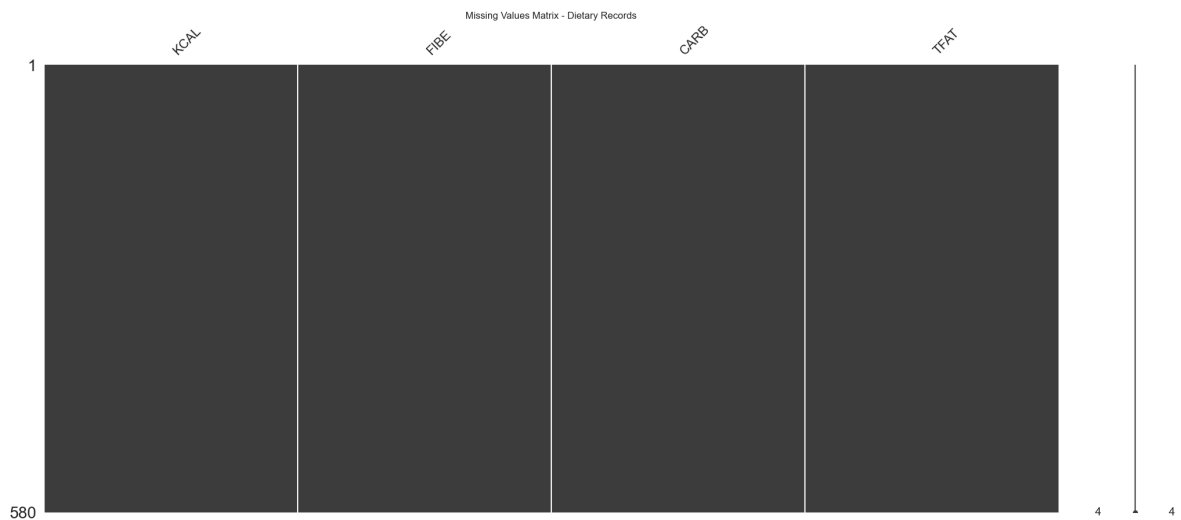
missing_percentages = (
    df_nutrients[existing_nutrient_columns]
```

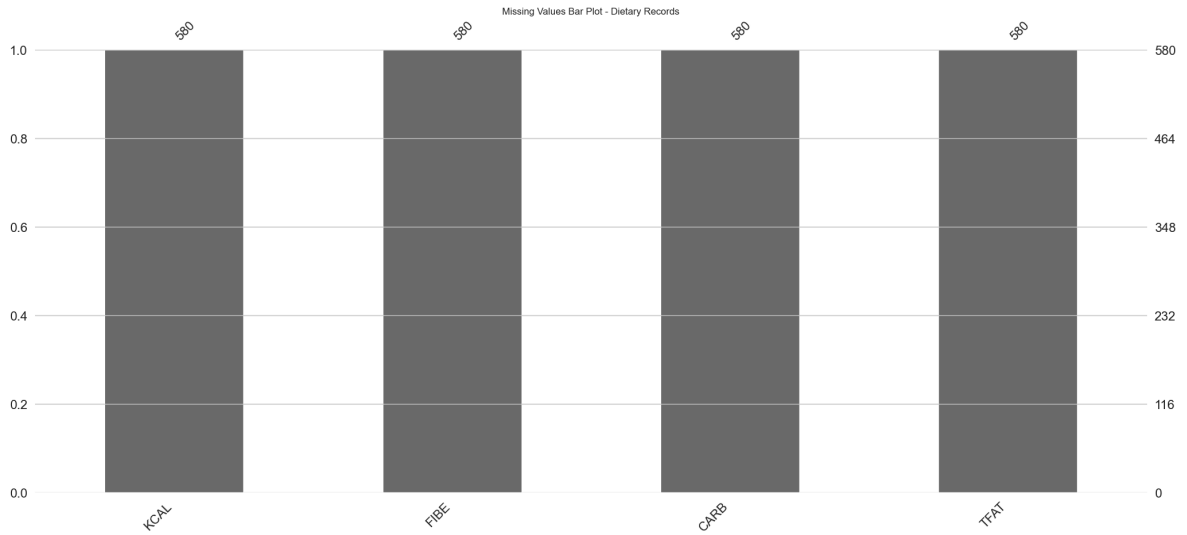
```
.isnull()  
.mean()  
.mul(100)  
.sort_values(ascending=False)  
)  
  
missing_df = missing_percentages.reset_index()  
missing_df.columns = ["Feature", "Missing Percentage"]  
  
display(missing_df)
```

	Feature	Missing Percentage
0	KCAL	0.0000
1	FIBE	0.0000
2	CARB	0.0000
3	TFAT	0.0000

In [134...

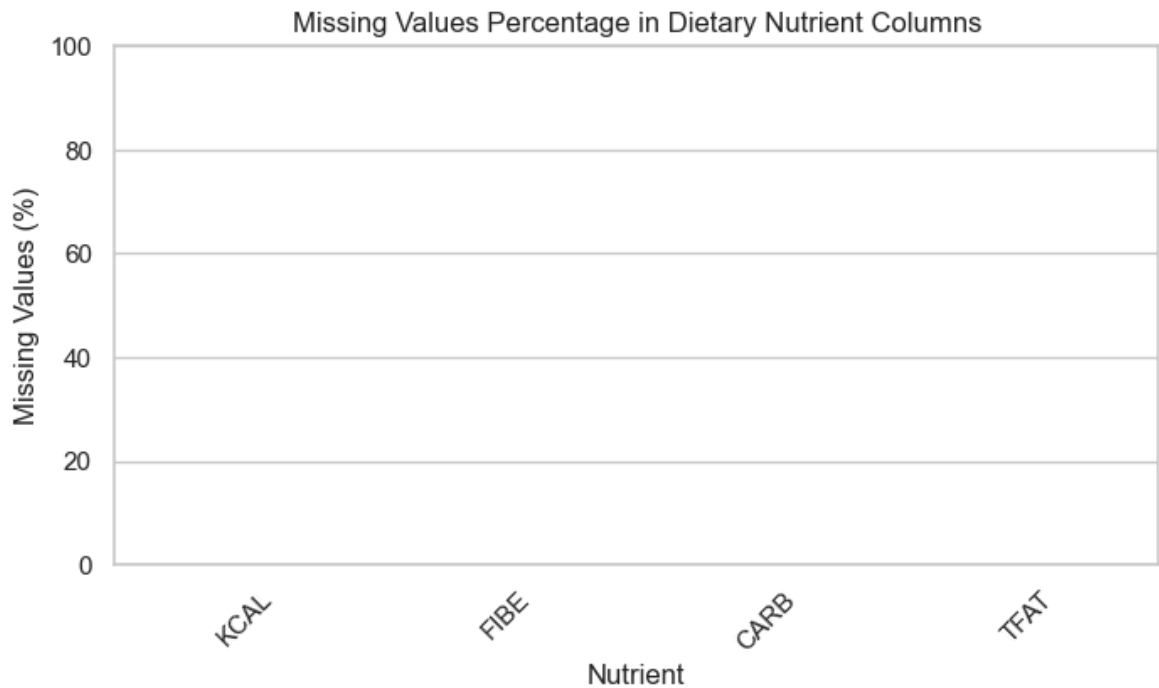
```
# Visualize missing data patterns  
  
msno.matrix(df_nutrients[existing_nutrient_columns])  
plt.title("Missing Values Matrix - Dietary Records")  
plt.show()  
  
msno.bar(df_nutrients[existing_nutrient_columns])  
plt.title("Missing Values Bar Plot - Dietary Records")  
plt.show()
```





In [135... *# Create a bar plot for missing percentages*

```
plt.figure(figsize=(8, 4))
sns.barplot(data=missing_df, x="Feature", y="Missing Percentage")
plt.title("Missing Values Percentage in Dietary Nutrient Columns")
plt.xlabel("Nutrient")
plt.ylabel("Missing Values (%)")
plt.ylim(0, 100)
plt.xticks(rotation=45)
plt.show()
```



In [136... *# Load participant metadata containing Age and BMI*

```
df_metadata = pd.read_csv(metadata_path, sep="\t")
df_metadata.columns = df_metadata.columns.str.strip()

print("Metadata shape:", df_metadata.shape)
display(df_metadata.head())

print("Metadata columns:")
print(df_metadata.columns.tolist())
```

Metadata shape: (566, 29)

	#SampleID	UserName	StudyDayNo	StudyDate	Sample.Week.Day	DietDayNo	Diet.R
0	MCT.f.0001	MCTs01	1	1/31/17	Tuesday	0	
1	MCT.f.0002	MCTs01	2	2/1/17	Wednesday	1	
2	MCT.f.0003	MCTs01	3	2/2/17	Thursday	2	
3	MCT.f.0004	MCTs01	4	2/3/17	Friday	3	
4	MCT.f.0005	MCTs01	5	2/4/17	Saturday	4	

5 rows × 29 columns

Metadata columns:
 ['#SampleID', 'UserName', 'StudyDayNo', 'StudyDate', 'Sample.Week.Day', 'DietDayNo', 'Diet.Record.Date', 'Diet.Week.Day', 'Gender', 'Age', 'Weight', 'Height', 'BMI', 'Supplement', 'Waist.Circumference', 'Study.Status', 'oilGrams.assigned', 'fecal.status', 'fecal.time', 'BMI.1', 'Weight.Change', 'Run1.Plate', 'Run2.Rerun', 'SampleOrder', 'SampleType', 'Timing', 'Activity.Factor', 'Medications', 'Dietary.Supp']

In [137...

```
# Select participant-level metadata columns

profile_columns = ["Age", "BMI"]

df_profile = (
    df_metadata[["UserName"] + profile_columns]
    .drop_duplicates(subset="UserName")
    .copy()
)

print("Profile dataframe shape:", df_profile.shape)
display(df_profile.head())

print("Missing values in profile columns:")
display(df_profile[profile_columns].isnull().mean().mul(100))
```

Profile dataframe shape: (34, 3)

	UserName	Age	BMI
0	MCTs01	25.9000	25.0000
17	MCTs03	25.4000	26.6000
33	MCTs04	24.0000	18.9000
50	MCTs05	23.1000	19.9000
57	MCTs06	23.7000	22.7000

Missing values in profile columns:

Age 0.0000
 BMI 0.0000
 dtype: float64

```
In [138... # Aggregate daily dietary records into average nutrient intake per participant

df_dietary_analysis = (
    df_nutrients
    .groupby("UserName")[existing_nutrient_columns]
    .mean()
    .reset_index()
)

print("Dietary analysis shape:", df_dietary_analysis.shape)
display(df_dietary_analysis.head())
```

Dietary analysis shape: (37, 5)

	UserName	KCAL	FIBE	CARB	TFAT
0	MCTs01	2233.9717	23.5308	246.3153	72.7932
1	MCTs02	1406.0945	12.2058	102.5730	88.8896
2	MCTs03	3797.8396	32.5283	379.4849	179.3785
3	MCTs04	2554.8960	33.8229	259.6359	123.5212
4	MCTs05	1459.1681	13.0463	193.9457	51.8696

```
In [139... # Merge dietary averages with participant metadata

df_analysis_full = df_dietary_analysis.merge(
    df_profile,
    on="UserName",
    how="left"
)

print("Full analysis dataframe shape:", df_analysis_full.shape)
display(df_analysis_full.head())

print("Missing values after merge:")
display(df_analysis_full[existing_nutrient_columns + profile_columns].isnull().m
```

Full analysis dataframe shape: (37, 7)

	UserName	KCAL	FIBE	CARB	TFAT	Age	BMI
0	MCTs01	2233.9717	23.5308	246.3153	72.7932	25.9000	25.0000
1	MCTs02	1406.0945	12.2058	102.5730	88.8896	NaN	NaN
2	MCTs03	3797.8396	32.5283	379.4849	179.3785	25.4000	26.6000
3	MCTs04	2554.8960	33.8229	259.6359	123.5212	24.0000	18.9000
4	MCTs05	1459.1681	13.0463	193.9457	51.8696	23.1000	19.9000

Missing values after merge:

```
KCAL    0.0000
FIBE    0.0000
CARB    0.0000
TFAT    0.0000
Age      8.1081
BMI      8.1081
dtype: float64
```

```
In [140... # Missing values summary after merging dietary records with participant metadata

missing_after_merge = (
    df_analysis_full[existing_nutrient_columns + profile_columns]
    .isnull()
    .sum()
    .reset_index()
)

missing_after_merge.columns = ["Feature", "Missing Count"]
missing_after_merge["Missing Percentage"] = (
    missing_after_merge["Missing Count"] / len(df_analysis_full) * 100
)

display(missing_after_merge)
```

	Feature	Missing Count	Missing Percentage
0	KCAL	0	0.0000
1	FIBE	0	0.0000
2	CARB	0	0.0000
3	TFAT	0	0.0000
4	Age	3	8.1081
5	BMI	3	8.1081

```
In [141... # Identify participants with dietary records but missing participant metadata

missing_profile_users = df_analysis_full[
    df_analysis_full["Age"].isna() | df_analysis_full["BMI"].isna()
]["UserName"].tolist()

print("Number of participants with missing Age/BMI:", len(missing_profile_users))
print("Participants with missing Age/BMI:")
print(missing_profile_users)
```

```
Number of participants with missing Age/BMI: 3
Participants with missing Age/BMI:
['MCTs02', 'MCTs17', 'MCTs30']
```

```
In [142... # Compare participants available in dietary records vs. participant metadata

diet_users = set(df_dietary_analysis["UserName"])
profile_users = set(df_profile["UserName"])

users_only_in_diet = sorted(diet_users - profile_users)
users_only_in_profile = sorted(profile_users - diet_users)

print("Number of participants in dietary records:", len(diet_users))
```

```
print("Number of participants in metadata:", len(profile_users))

print("\nParticipants found in dietary records but missing from metadata:")
print(users_only_in_diet)

print("\nParticipants found in metadata but missing from dietary records:")
print(users_only_in_profile)
```

Number of participants in dietary records: 37

Number of participants in metadata: 34

Participants found in dietary records but missing from metadata:
['MCTs02', 'MCTs17', 'MCTs30']

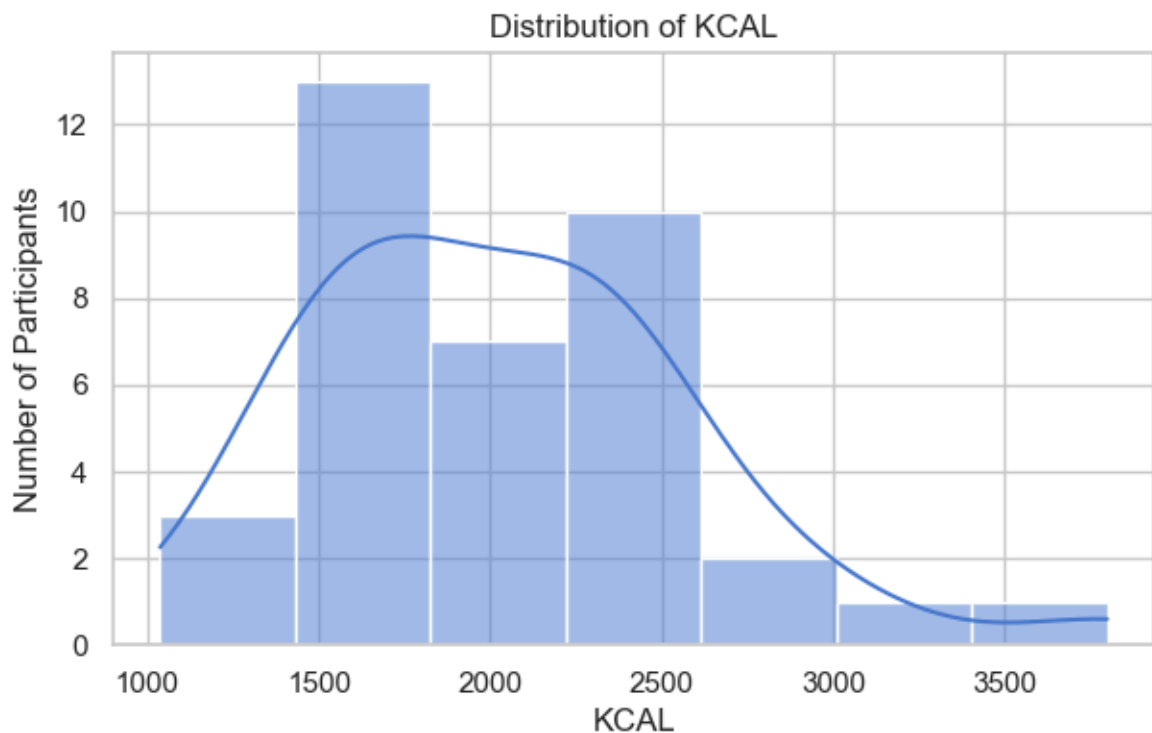
Participants found in metadata but missing from dietary records:
[]

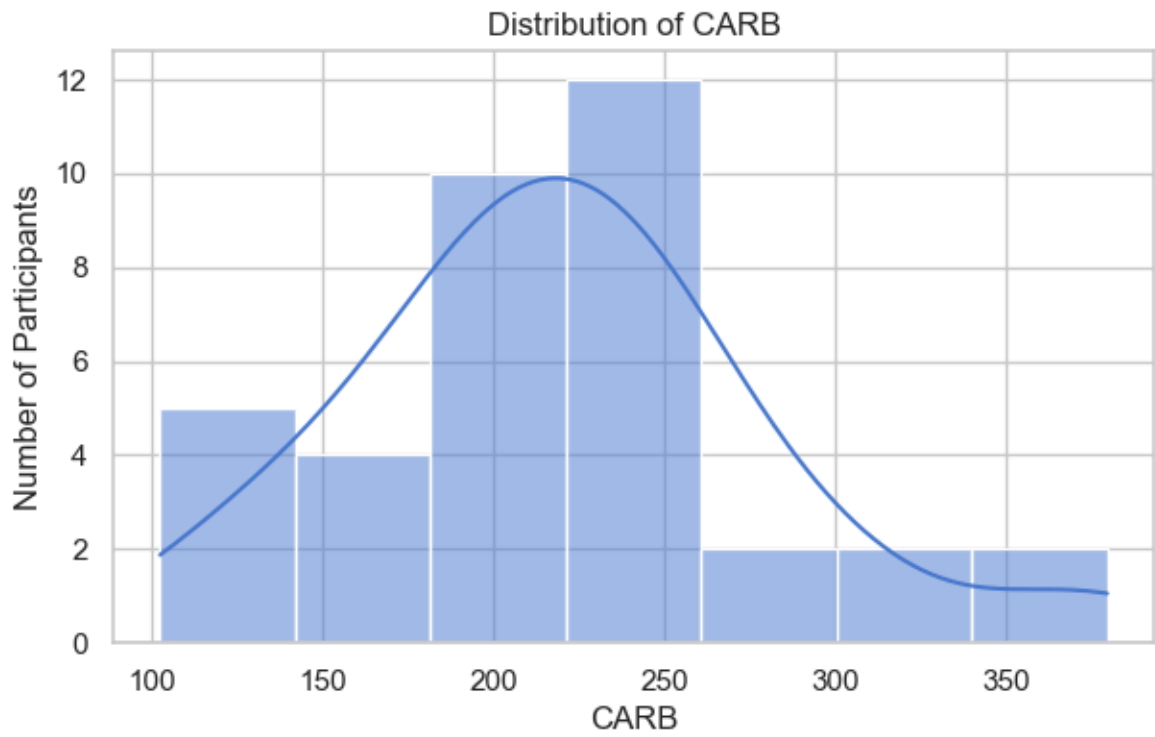
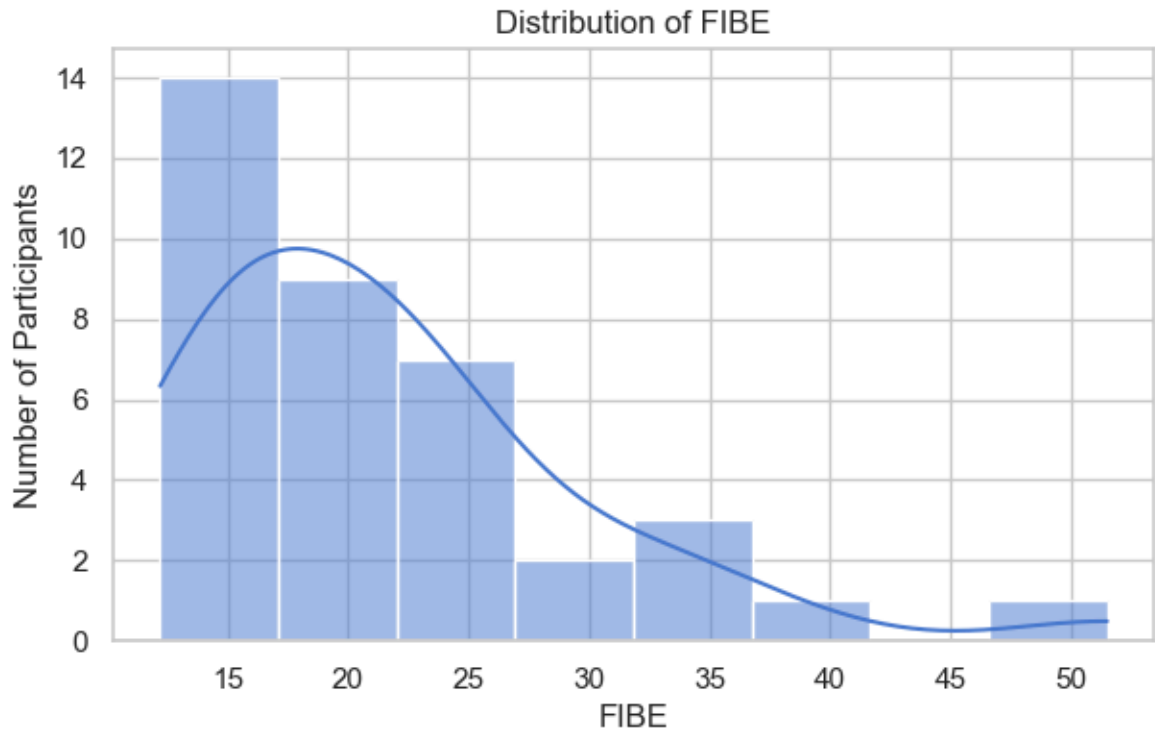
In [143...

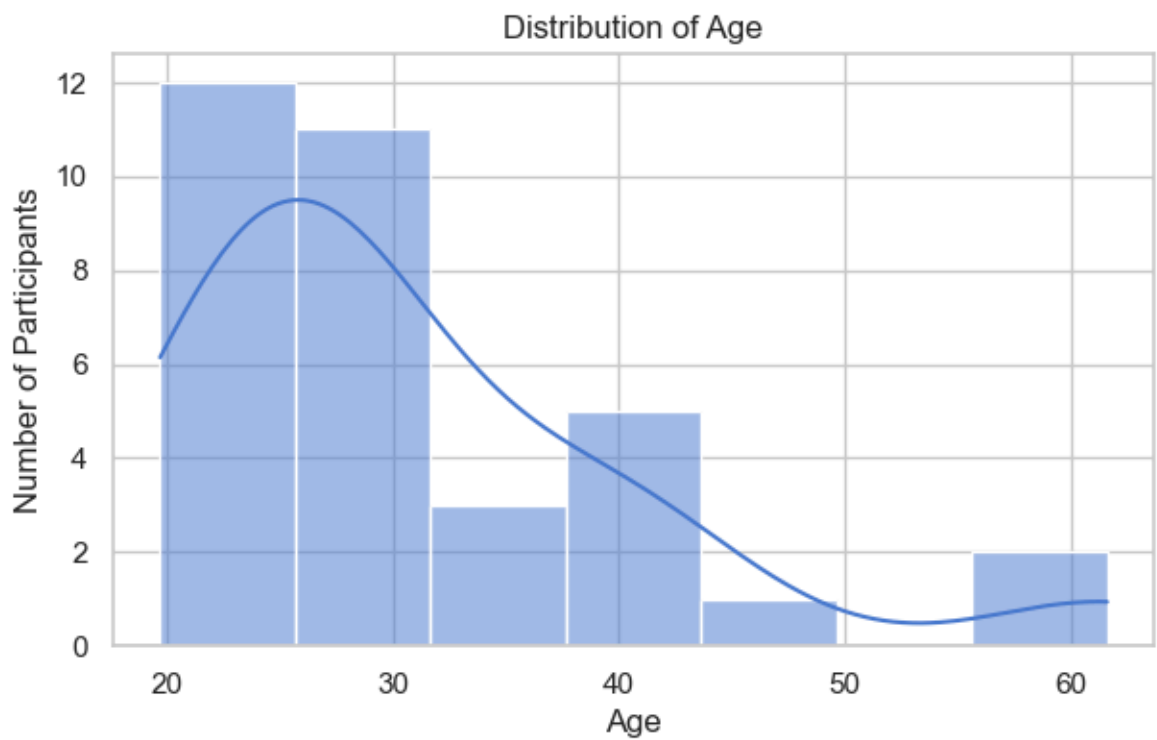
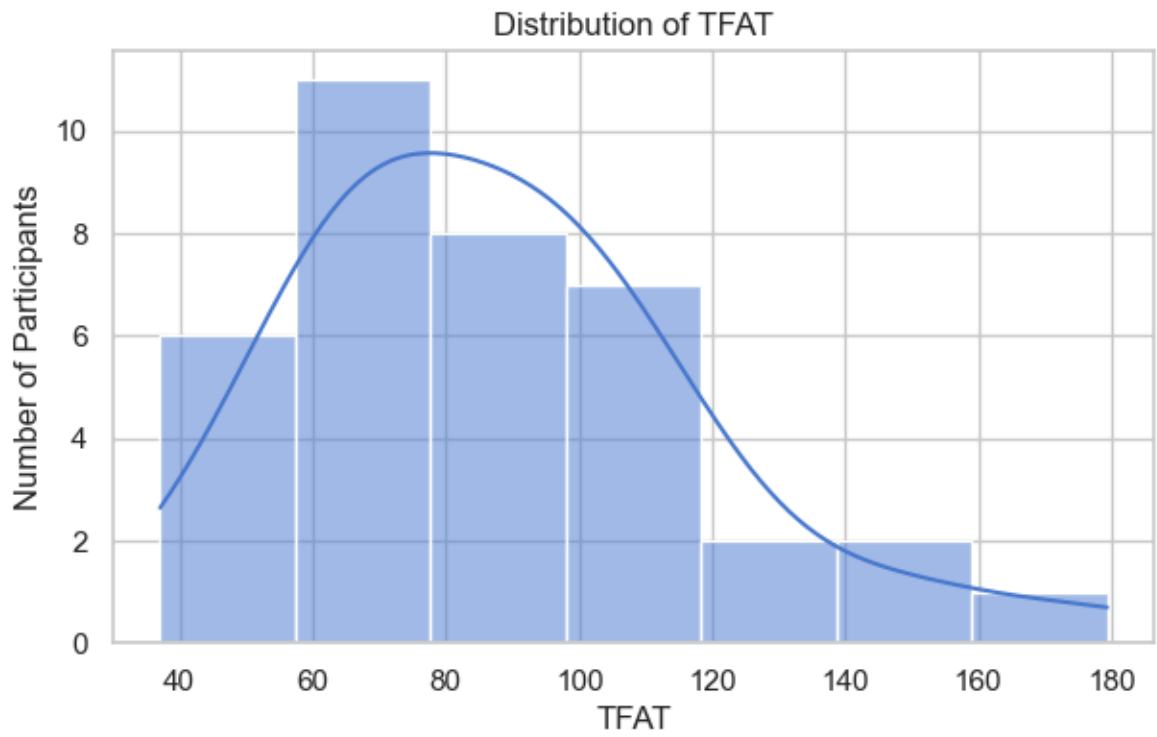
```
# Histograms for nutrient intake and participant characteristics

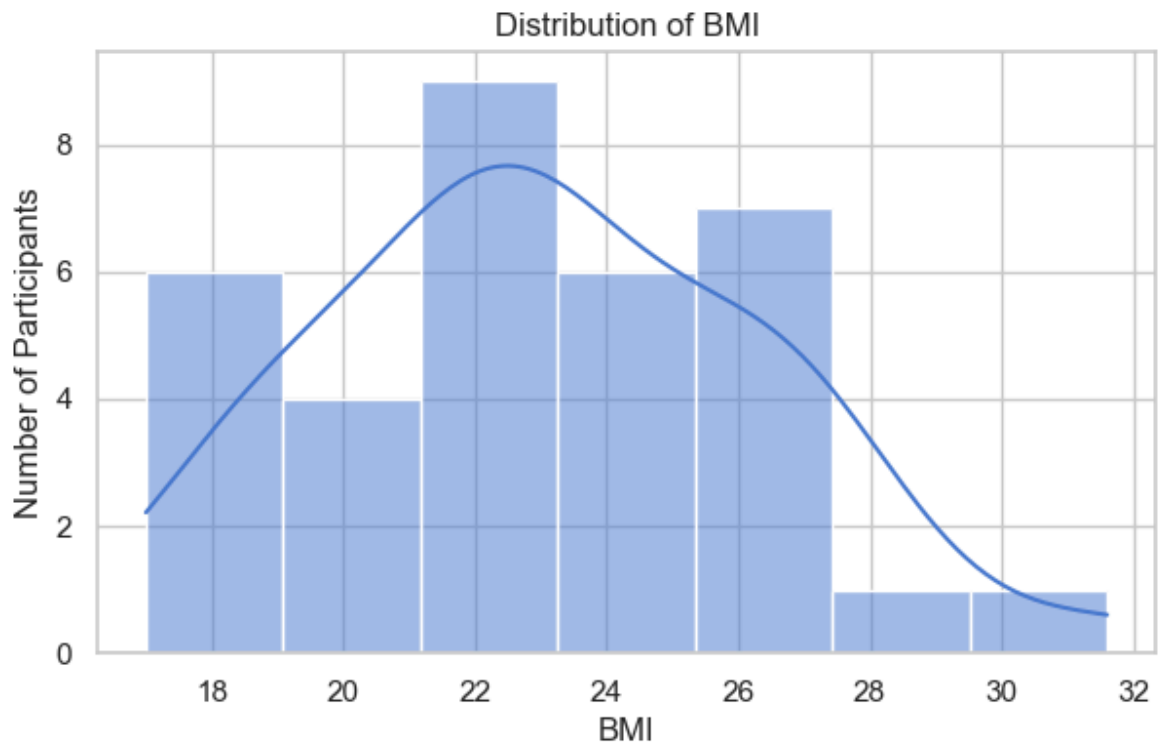
analysis_columns = existing_nutrient_columns + profile_columns

for col in analysis_columns:
    plt.figure(figsize=(7, 4))
    sns.histplot(df_analysis_full[col].dropna(), kde=True)
    plt.title(f"Distribution of {col}")
    plt.xlabel(col)
    plt.ylabel("Number of Participants")
    plt.show()
```



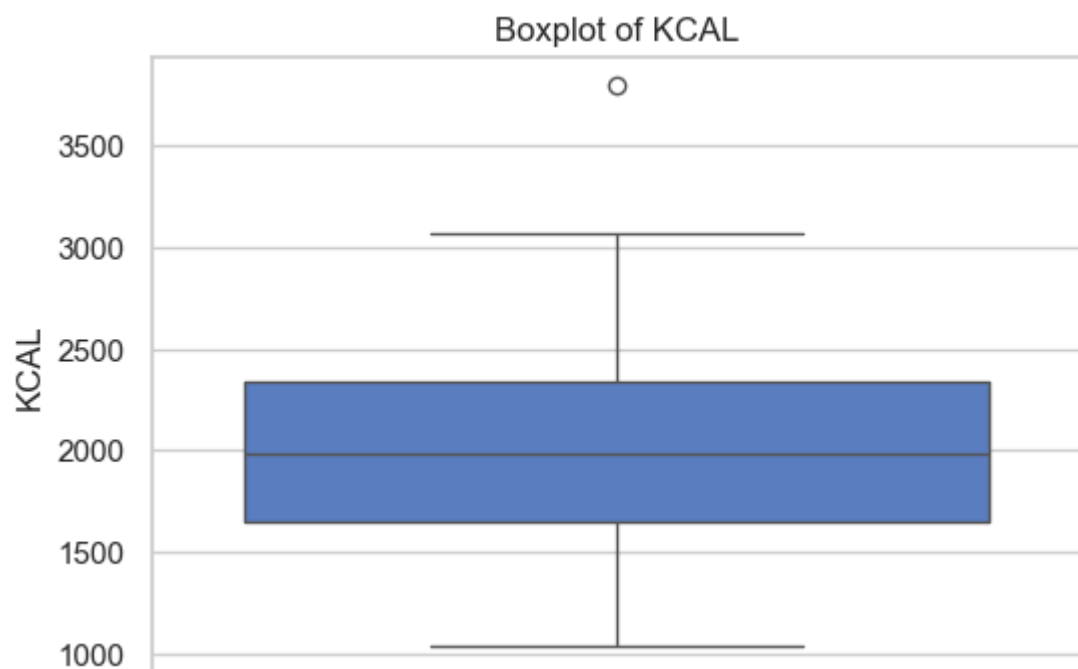


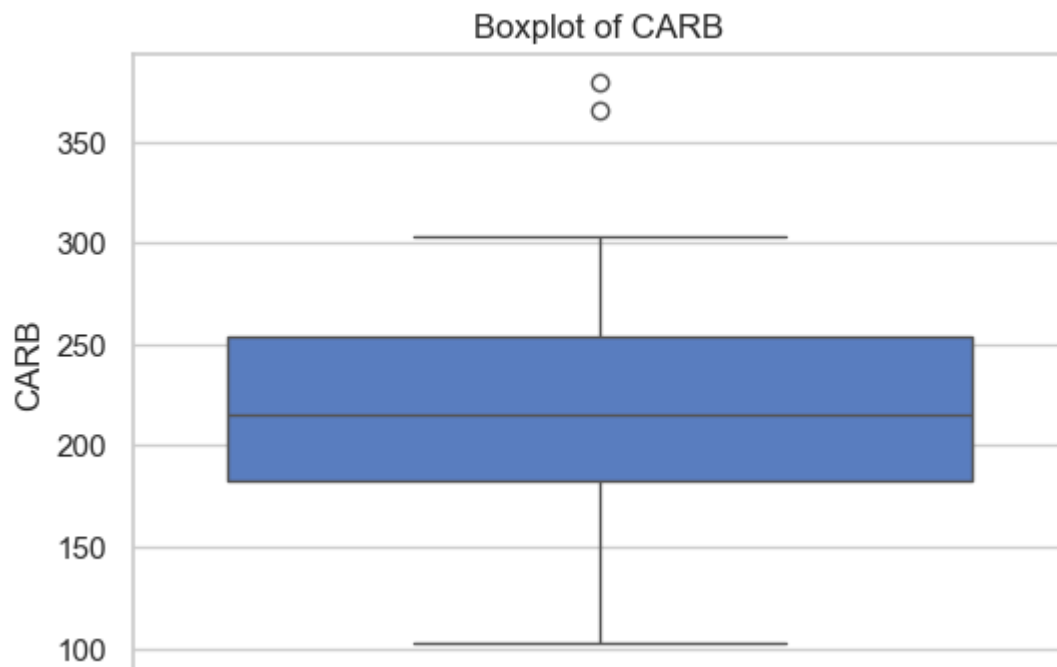
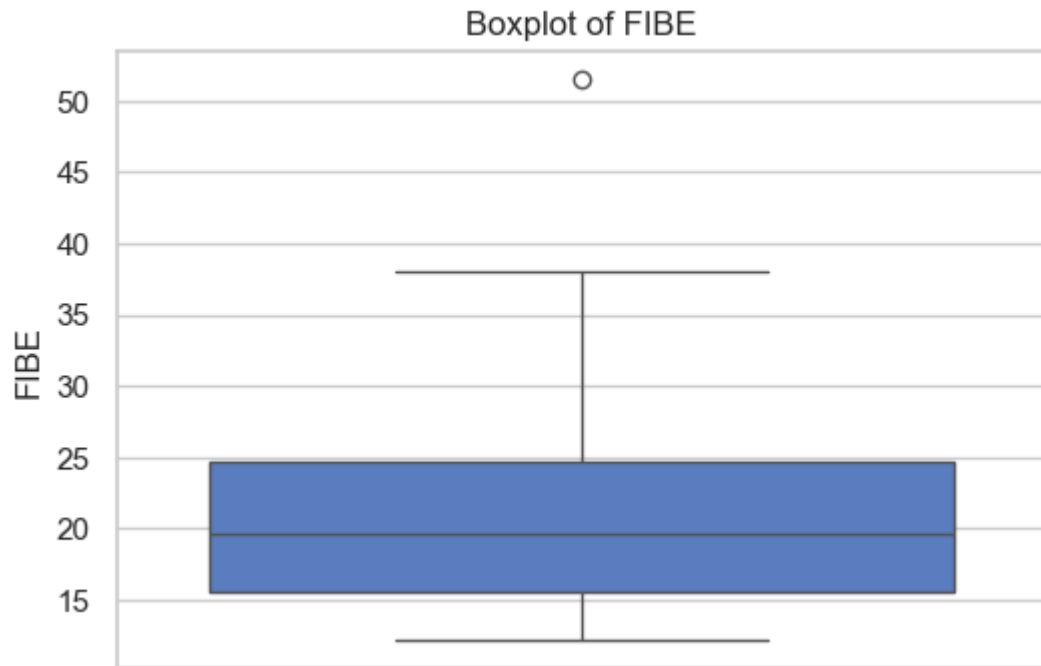


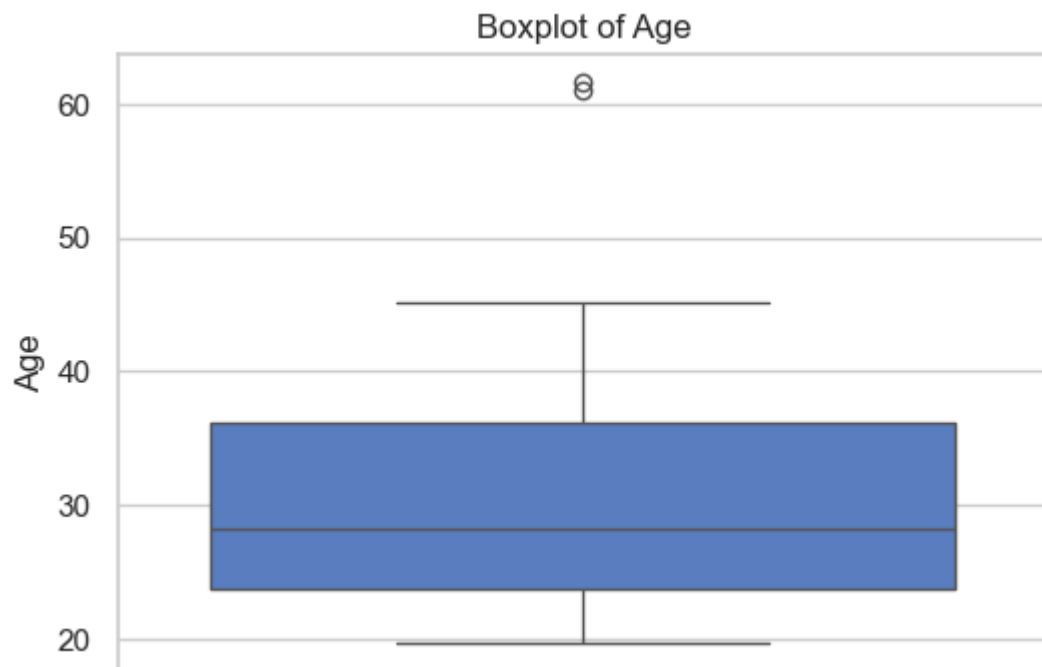
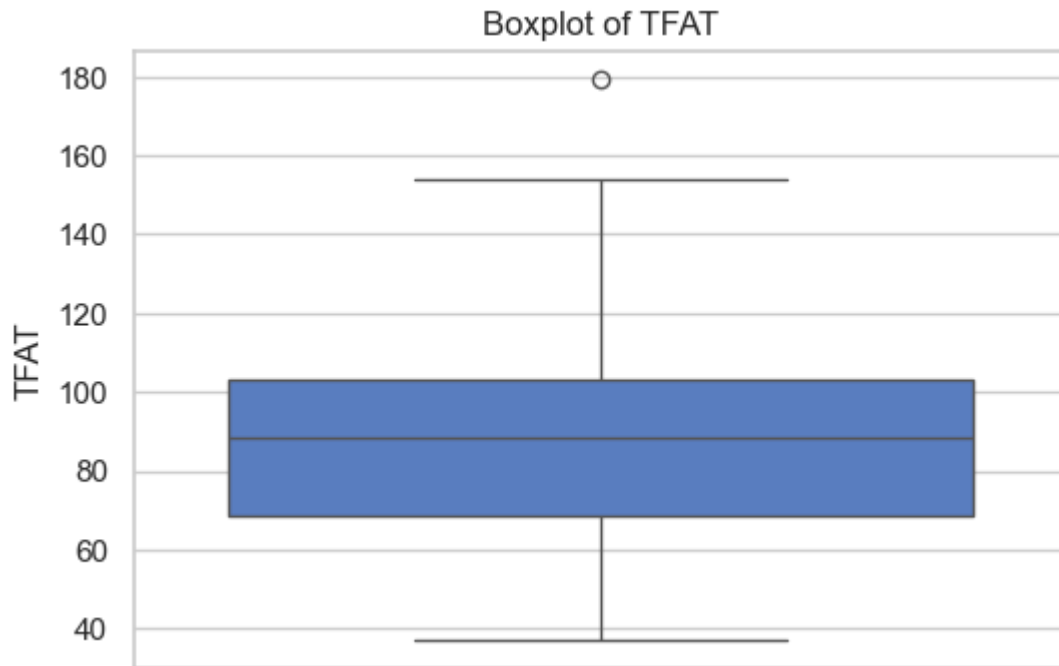


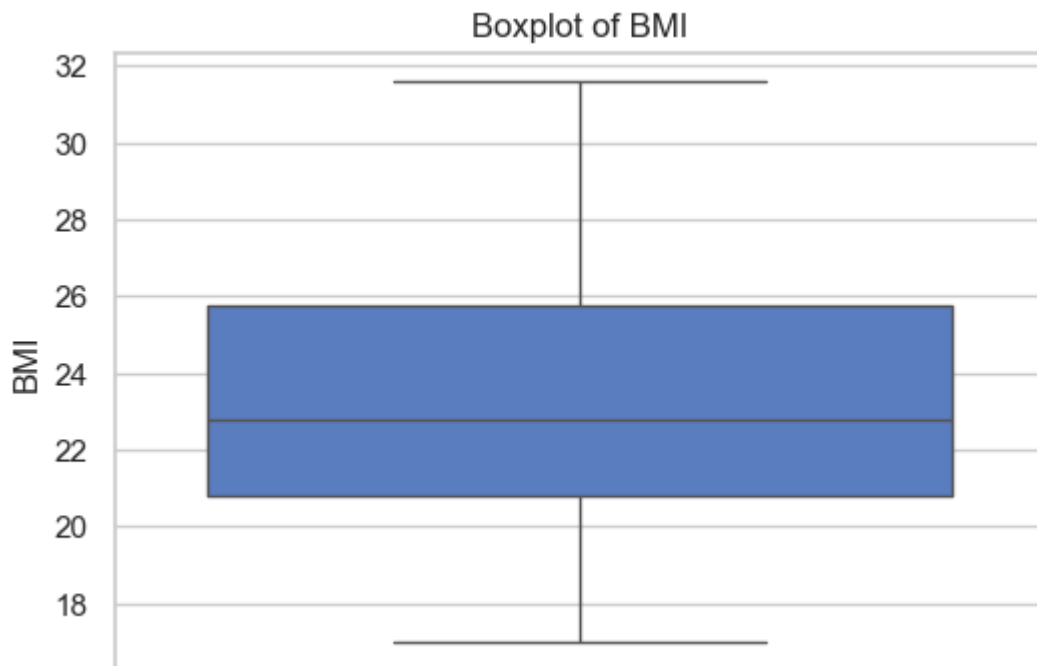
In [144... *# Boxplots for nutrient intake and participant characteristics*

```
for col in analysis_columns:
    plt.figure(figsize=(6, 4))
    sns.boxplot(y=df_analysis_full[col].dropna())
    plt.title(f"Boxplot of {col}")
    plt.ylabel(col)
    plt.show()
```









In [145... *# Create a complete-case dataset for analyses requiring participant metadata*

```
df_analysis_complete = df_analysis_full.dropna(subset=["Age", "BMI"]).copy()

print("Original merged dataset shape:", df_analysis_full.shape)
print("Complete-case dataset shape:", df_analysis_complete.shape)

display(df_analysis_complete.head())
```

Original merged dataset shape: (37, 7)

Complete-case dataset shape: (34, 7)

	UserName	KCAL	FIBE	CARB	TFAT	Age	BMI
0	MCTs01	2233.9717	23.5308	246.3153	72.7932	25.9000	25.0000
2	MCTs03	3797.8396	32.5283	379.4849	179.3785	25.4000	26.6000
3	MCTs04	2554.8960	33.8229	259.6359	123.5212	24.0000	18.9000
4	MCTs05	1459.1681	13.0463	193.9457	51.8696	23.1000	19.9000
5	MCTs06	2821.1574	25.0103	300.5661	119.2294	23.7000	22.7000

In [146... *# Descriptive statistics for dietary and participant-level variables*

```
summary_stats = df_analysis_full[analysis_columns].describe().T
display(summary_stats)
```

	count	mean	std	min	25%	50%	75%	max
KCAL	37.0000	2033.2975	555.3414	1038.4115	1647.9699	1986.3144	2338.8370	3797.839
FIBE	37.0000	21.5020	8.2279	12.2058	15.5557	19.6145	24.7420	51.493
CARB	37.0000	219.8896	60.2076	102.5730	183.1356	215.3049	253.8767	379.484
TFAT	37.0000	88.1145	30.3073	37.0642	68.7704	88.3925	103.0461	179.378
Age	34.0000	30.8235	10.2633	19.7000	23.7500	28.2000	36.1250	61.600
BMI	34.0000	23.0912	3.3155	17.0000	20.8000	22.8000	25.7500	31.600

In [147...

```
# Descriptive statistics for dietary and participant-level variables without mis

summary_stats = df_analysis_complete[analysis_columns].describe().T
display(summary_stats)
```

	count	mean	std	min	25%	50%	75%	max
KCAL	34.0000	2080.2830	554.3712	1038.4115	1720.4759	2053.1710	2381.9370	3797.839
FIBE	34.0000	21.9145	8.3631	12.5786	16.0169	20.1444	24.9432	51.493
CARB	34.0000	226.3086	57.3760	118.5393	194.1939	224.1961	255.3407	379.484
TFAT	34.0000	89.6828	30.8844	37.0642	69.3915	89.0266	105.2184	179.378
Age	34.0000	30.8235	10.2633	19.7000	23.7500	28.2000	36.1250	61.600
BMI	34.0000	23.0912	3.3155	17.0000	20.8000	22.8000	25.7500	31.600

EDA on CFMD

CFMD Food Microbiome Analysis

Data source: CuratedFoodMetagenomicData (CFMD)

- cFMD-main/cFMD-main/cFMD_metadata.tsv — sample metadata with category , sample_id
- cFMD-main/cFMD-main/cFMD_data/**/*_taxonomic_profiles.tsv — per-dataset abundance matrices

Tasks:

1. Explore CFMD and analyze food microbial richness per category
2. Check species overlap between the top-2 richest categories

Cell 1 — Setup and Metadata Loading

In [148...

```
warnings.filterwarnings('ignore')
sns.set_theme(style='whitegrid')
plt.rcParams['figure.dpi'] = 120

# Load metadata
#metadata = pd.read_csv(DATA_DIR / 'cFMD-main' / 'cFMD-main' / 'cFMD_metadata.ts
metadata = pd.read_csv(DATA_DIR / 'cFMD-main' / 'cFMD-main' / 'cFMD_metadata.tsv
metadata.columns = metadata.columns.astype(str).str.strip()
metadata['sample_id'] = metadata['sample_id'].astype(str).str.strip()

print('=== CFMD Metadata ===')
print(f'Shape : {metadata.shape}')
print(f'Columns : {metadata.columns.tolist()}')
print()

# EDA: category distribution -
cat_counts = metadata['category'].value_counts()
print(f"'category' - {cat_counts.shape[0]} unique values:")
print(cat_counts.to_string())

fig, ax = plt.subplots(figsize=(13, 4))
cat_counts.head(15).plot(kind='bar', ax=ax, color='steelblue', edgecolor='white')
ax.set_title('CFMD: Sample Count per Food Category (top 15)', fontsize=13)
ax.set_xlabel('Category')
ax.set_ylabel('Sample Count')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.savefig(DATA_DIR / 'category_distribution.png', dpi=150, bbox_inches='tight')
plt.show()
print('Plot saved → category_distribution.png')

# EDA: macrocategory distribution -
print()
print('Macrocategory counts:')
print(metadata['macrocategory'].value_counts().to_string())

# EDA: sequencing platform
print()
print('Sequencing platform counts:')
print(metadata['sequencing_platform'].value_counts().to_string())
```

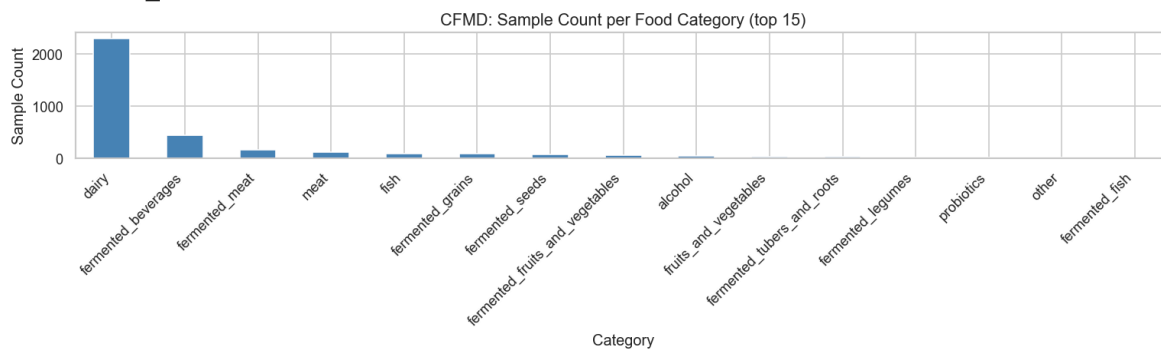
=== CFMD Metadata ===

Shape : (3444, 32)

Columns : ['dataset_name', 'sample_id', 'macrocategory', 'category', 'type', 'sub
type', 'commercial_name', 'fermented/non-fermented', 'country', 'sample_accessio
n', 'run_accession', 'experiment_accession', 'study_accession', 'project_accessio
n', 'database_origin', 'library_layout', 'sequencing_platform', 'DNA_extraction_k
it', 'collection_date', 'n_of_bases', 'n_of_reads', 'min_read_len', 'median_read_
len', 'mean_read_len', 'max_read_len', 'n_contigs', 'n_MAGs_MQ_prok', 'n_MAGs_HQ_
prok', 'n_MAGs_MQ_euk', 'n_MAGs_HQ_euk', 'filtered', 'curator']

'category' – 15 unique values:

category	
dairy	2296
fermented_beverages	435
fermented_meat	157
meat	119
fish	93
fermented_grains	83
fermented_seeds	65
fermented_fruits_and_vegetables	52
alcohol	48
fruits_and_vegetables	25
fermented_tubers_and_roots	24
fermented_legumes	20
probiotics	14
other	8
fermented_fish	5



Plot saved → category_distribution.png

Macrocategory counts:

```
macrocategory
food      3444
```

Sequencing platform counts:

```
sequencing_platform
Illumina_NovaSeq_6000      1616
NextSeq_500                1053
Illumina_MiSeq             283
HiSeq_X_Ten                119
Illumina_HiSeq_2500        102
Illumina_HiSeq_4000         75
NextSeq_550                71
Illumina_HiSeq_2000         41
HiSeq_X_Ten                20
Illumina_HiSeq_1500         15
Illumina_HiSeq_XTen         14
Illumina_HiSeq_4000         13
Ion_Torrent_PGM            10
Illumina_NovaSeq_6000         4
Illumina_HiSeq_1000         2
Illumina_Genome_Analyzer_II  2
Illumina_MiSeq              2
Illumina_HiSeq_X            1
Sequel;Illumina_MiSeq        1
```

Cell 2 — Load Taxonomic Profiles and Merge with Metadata

Each `*_taxonomic_profiles.tsv` file is a wide matrix: rows = taxa, columns = samples. Column names are formatted as `DatasetName__SampleAccession`. We extract the accession part (after `__`) to match `sample_id` in the metadata.

```
In [149... # profile_files = glob.glob(
#     './cFMD-main/cFMD-main/cFMD_data/**/*.taxonomic_profiles.tsv',
#     recursive=True
# )
profile_files = glob.glob(
    str(DATA_DIR / 'cFMD-main' / 'cFMD-main' / 'cFMD_data' / '**' / '*_taxonomic_profiles.tsv'),
    recursive=True
)
print(f'Found {len(profile_files)} taxonomic profile files')

all_profiles = []
skipped = []

for path in profile_files:
    try:
        df = pd.read_csv(path, sep='\t')
        # First column is always the taxonomy column
        tax_col = df.columns[0]
        df = df.rename(columns={tax_col: '#taxonomy'})

        # Melt wide → long
        long = df.melt(
```

```

        id_vars=['#taxonomy'],
        var_name='sample_col',
        value_name='abundance'
    )

    # Column names: 'DatasetName__SampleAccession' → extract accession after
    long['sample_id'] = long['sample_col'].str.split('__').str[-1]
    long = long.drop(columns=['sample_col'])
    all_profiles.append(long)

    except Exception as e:
        skipped.append((path, str(e)))

if skipped:
    print(f'\nSkipped {len(skipped)} files:')
    for p, e in skipped:
        print(f'  {p}: {e}')

# Combine all datasets
long_df = pd.concat(all_profiles, ignore_index=True)
long_df['sample_id'] = long_df['sample_id'].astype(str).str.strip()
long_df['abundance'] = pd.to_numeric(long_df['abundance'], errors='coerce').fillna(0)

print(f'\nCombined long-format shape : {long_df.shape}')
print(f'Unique sample IDs in profiles : {long_df["sample_id"].nunique()}')
print(f'Unique taxa in profiles      : {long_df["#taxonomy"].nunique()}')

# Merge with metadata
merged = long_df.merge(
    metadata[['sample_id', 'category', 'macrocategory', 'country', 'fermented/no
on='sample_id',
how='left'
)

# Filter: keep only rows with positive abundance
merged_filtered = merged[merged['abundance'] > 0].copy()

print(f'\nRows after filtering (abundance > 0) : {len(merged_filtered):,}')
print(f'Unique taxa retained                      : {merged_filtered["#taxonomy"].nunique()}')
print(f'Missing category (unmatched samples) : {merged_filtered["category"].isna().sum()}')
print(f'Categories present                        : {merged_filtered["category"].nunique()}')
print()
print('Sample category distribution:')
print(merged_filtered.dropna(subset=['category'])["category"].value_counts().to_

display(merged_filtered.head())

```


Found 85 taxonomic profile files

Combined long-format shape : (1614609, 3)

Unique sample IDs in profiles : 3392

Unique taxa in profiles : 5057

Rows after filtering (abundance > 0) : 94,491

Unique taxa retained : 5051

Missing category (unmatched samples) : 0

Categories present : 15

Sample category distribution:

category	
dairy	57653
fermented_beverages	7663
fermented_meat	5649
meat	4976
fish	4558
fermented_seeds	4537
fruits_and_vegetables	2883
fermented_fruits_and_vegetables	1637
fermented_tubers_and_roots	1465
fermented_grains	1161
alcohol	944
fermented_legumes	936
other	223
probiotics	114
fermented_fish	92

	#taxonomy	abundance	sample_id	category
0	k_Bacteria p_Firmicutes c_Bacilli o_Lactob...	0.1411	SRR8742576	fermented_
3	k_Bacteria p_Pseudomonadota c_Gammaproteoba...	0.1039	SRR8742576	fermented_
7	k_Bacteria p_Firmicutes c_Bacilli o_Lactob...	14.7489	SRR8742576	fermented_
12	k_Bacteria p_Firmicutes c_Bacilli o_Lactob...	0.3360	SRR8742576	fermented_
20	k_Eukaryota p_Ascomycota c_Saccharomycetes ...	0.8975	SRR8742576	fermented_

Cell 3 — Task 1: Microbial Richness per Food Category

Richness = number of unique taxa with abundance > 0 per food category.

In [150...]

```
# Guard: verify merge succeeded
if 'category' not in merged_filtered.columns or merged_filtered['category'].isna().any():
    raise RuntimeError('category data is missing - merge did not succeed. Fix Cell 2')

richness = (
    merged_filtered
    .dropna(subset=['category'])
    .groupby('category')['#taxonomy']
    .nunique()
    .reset_index(name='microbial_richness')
```

```

        .sort_values('microbial_richness', ascending=False)
        .reset_index(drop=True)
    )

    print('=== Microbial Richness per Food Category ===')
    print(richness.to_string(index=False))

    # Bar chart: top 10 —
    top10 = richness.head(10)
    palette = sns.color_palette('viridis', len(top10))

    fig, ax = plt.subplots(figsize=(13, 5))
    bars = ax.bar(
        top10['category'],
        top10['microbial_richness'],
        color=palette,
        edgecolor='white'
    )
    for bar, val in zip(bars, top10['microbial_richness']):
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 0.3,
            str(int(val)),
            ha='center', va='bottom', fontsize=10, fontweight='bold'
        )

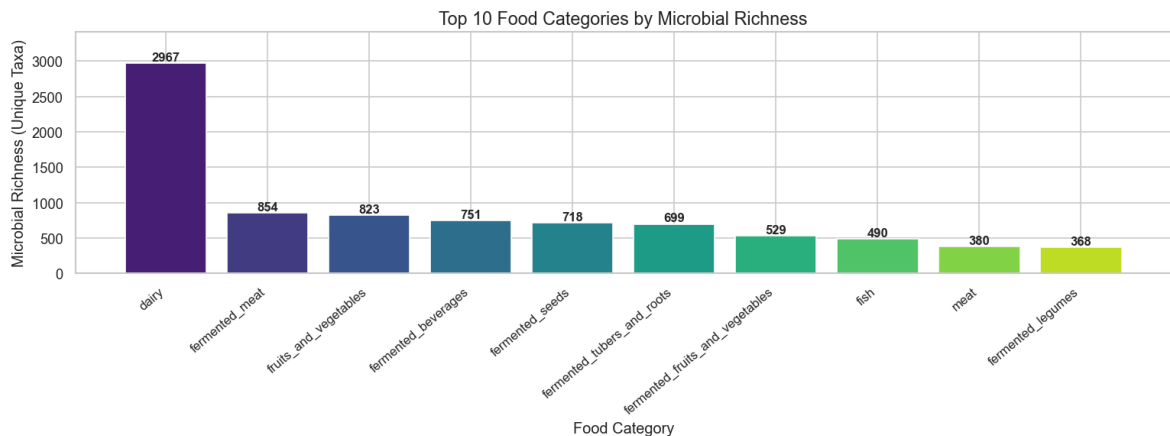
    ax.set_title('Top 10 Food Categories by Microbial Richness', fontsize=14)
    ax.set_xlabel('Food Category', fontsize=12)
    ax.set_ylabel('Microbial Richness (Unique Taxa)', fontsize=12)
    ax.set_ylim(0, top10['microbial_richness'].max() * 1.15)
    plt.xticks(rotation=40, ha='right', fontsize=10)
    plt.tight_layout()
    plt.savefig(DATA_DIR / 'richness_by_category.png', dpi=150, bbox_inches='tight')
    plt.show()
    print('Plot saved → richness_by_category.png')

```

```

=== Microbial Richness per Food Category ===
              category  microbial_richness
              dairy          2967
        fermented_meat          854
    fruits_and_vegetables          823
        fermented_beverages          751
        fermented_seeds          718
    fermented_tubers_and_roots          699
    fermented_fruits_and_vegetables          529
              fish          490
              meat          380
        fermented_legumes          368
        fermented_grains          361
              alcohol          330
              other          158
        fermented_fish          44
        probiotics          36

```



Plot saved → richness_by_category.png

Cell 4 — Task 2: Species Overlap between Top-2 Categories (Venn Diagram)

We compare the two richest food categories using a Venn diagram and the Jaccard similarity index.

In [151...

```
try:
    from matplotlib_venn import venn2
except ImportError:
    import subprocess, sys
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', 'matplotlib-venn'])
    from matplotlib_venn import venn2

cat1, cat2 = richness.head(2)['category'].tolist()
print(f"Comparing: '{cat1}' vs '{cat2}'")

sp1 = set(merged_filtered.loc[merged_filtered['category'] == cat1, '#taxonomy'])
sp2 = set(merged_filtered.loc[merged_filtered['category'] == cat2, '#taxonomy'])
shared = sp1 & sp2
union = sp1 | sp2

print(f"\nSpecies unique to '{cat1}' : {len(sp1 - sp2):,}")
print(f"Species unique to '{cat2}' : {len(sp2 - sp1):,}")
print(f"Shared species : {len(shared):,}")
print(f"Jaccard similarity index : {len(shared) / len(union):.4f}")

if shared:
    print(f'\nTop shared species (last taxonomic level):')
    for s in sorted(shared)[:20]: # show first 20
        sep = '|' if '|' in s else ';'
        label = s.split(sep)[-1].strip()
        print(f' {label}')
    if len(shared) > 20:
        print(f' ... and {len(shared) - 20} more')

# Venn diagram -
fig, ax = plt.subplots(figsize=(8, 6))
v = venn2(
    subsets=[sp1, sp2],
    set_labels=(cat1, cat2),
    ax=ax,
    set_colors=('4C72B0', 'DD8452'),
```

```

        alpha=0.65
    )
    for region_id in ('10', '01', '11'):
        lbl = v.get_label_by_id(region_id)
        if lbl:
            lbl.set_fontsize(14)
    for set_lbl in v.set_labels:
        if set_lbl:
            set_lbl.set_fontsize(11)
            set_lbl.set_fontweight('bold')

    ax.set_title(
        f'Microbial Species Overlap\n"{cat1}" vs "{cat2}"\nJaccard = {len(shared)/
        fontsize=12
    )
    plt.tight_layout()
    plt.savefig(DATA_DIR / 'venn_species_overlap.png', dpi=150, bbox_inches='tight')
    plt.show()
    print('Plot saved → venn_species_overlap.png')

```

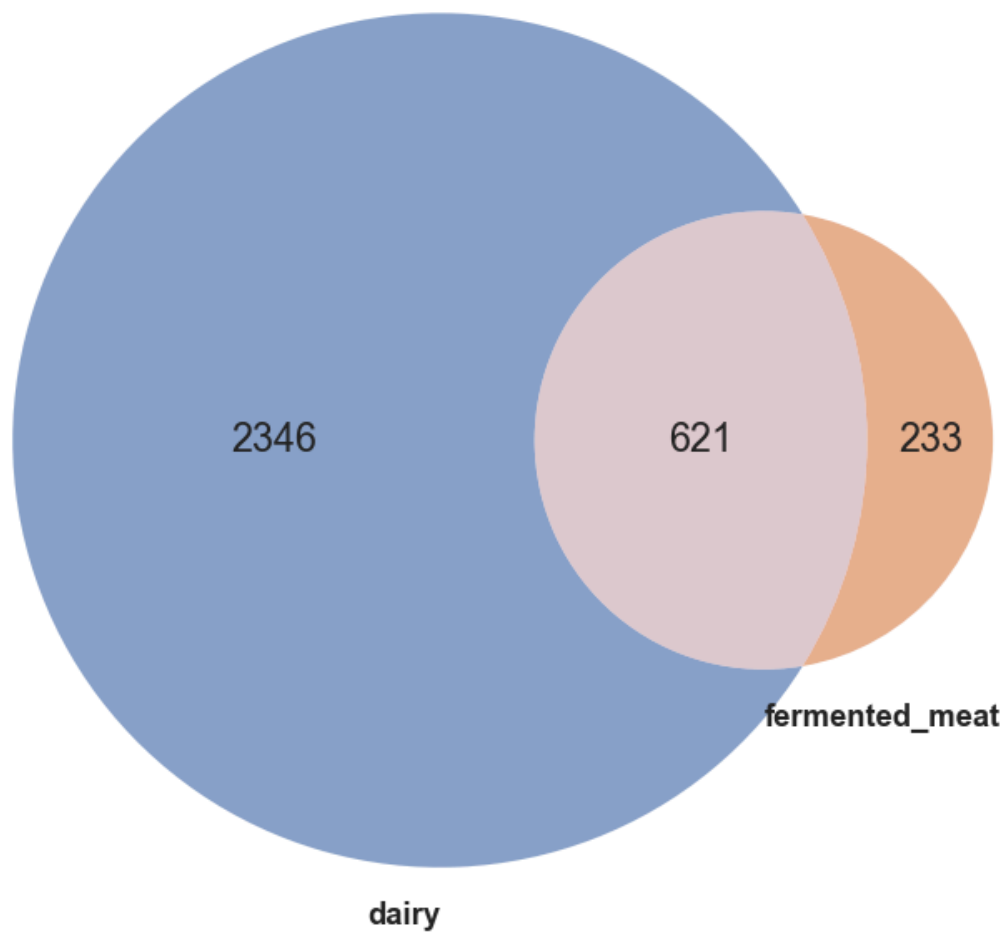
Comparing: 'dairy' vs 'fermented_meat'

Species unique to 'dairy' : 2,346
 Species unique to 'fermented_meat' : 233
 Shared species : 621
 Jaccard similarity index : 0.1941

Top shared species (last taxonomic level):

t__SGB257
 t__SGB203
 t__SGB282
 t__SGB39400
 t__SGB33761
 t__SGB23979
 t__SGB98684
 t__SGB86828
 t__SGB17183
 t__SGB17232
 t__SGB93302
 t__SGB98927
 t__SGB99001
 t__SGB99002
 t__SGB16645
 t__SGB98807
 t__SGB16638
 t__SGB98794
 t__SGB16639
 t__SGB88049
 ... and 601 more

Microbial Species Overlap
"dairy" vs "fermented_meat"
Jaccard = 0.1941



Plot saved → venn_species_overlap.png

Loading and Exploring the Paper 1 Microbiome Dataset

In [152...

```
base = DATA_DIR / "dietstudy_analyses-master" / "data"

print(base)
print("Base exists:", os.path.exists(base))

microbiome = pd.read_csv(
    base / "microbiome/processed_sample/taxonomy_counts_s.txt",
    sep="\t",
    index_col=0
)

sample_meta = pd.read_csv(
    base / "maps/food_map.txt",
    sep="\t"
).rename(columns={"#SampleID": "SampleID"})

diet_raw = pd.read_csv(
    base / "diet/processed_food/dhydrt.txt",
```

```
        sep="\t",
        index_col=0
    )

    diet_decay = pd.read_csv(
        base / "diet/processed_food/masterdecaydiet.txt",
        sep="\t",
        index_col=0
    )

    nutrition_totals = pd.read_csv(
        base / "diet/nutrition_totals.txt",
        sep="\t",
        index_col=0
    )

    microbiome_clr = pd.read_csv(
        base / "microbiome/processed_sample/taxonomy_clr_s.txt",
        sep="\t",
        index_col=0
    )

    diet_fiber = pd.read_csv(
        base / "diet/diet.fiber.txt",
        sep="\t",
        index_col=0
    )

    fiber_avg = pd.read_csv(
        base / "diet/fiber/fiber.smry.otu.txt",
        sep="\t",
        index_col=0
    )

    micro_avg_counts = pd.read_csv(
        base / "microbiome/processed_average/UN_taxonomy_counts_s.txt",
        sep="\t",
        index_col=0
    )

    micro_avg_clr = pd.read_csv(
        base / "microbiome/processed_average/UN_taxonomy_clr_s.txt",
        sep="\t",
        index_col=0
    )

    micro_avg_norm = pd.read_csv(
        base / "microbiome/processed_average/UN_taxonomy_norm_s.txt",
        sep="\t",
        index_col=0
    )

    print("=== microbiome ===")
    print(microbiome.shape)
    print(microbiome.head())

    print("=== sample_meta ===")
    print(sample_meta.shape)
    print(sample_meta.head())
```

```
print("=== diet_raw ===")
print(diet_raw.shape)
print(diet_raw.head())

print("=== diet_decay ===")
print(diet_decay.shape)
print(diet_decay.head())

print("=== nutrition_totals ===")
print(nutrition_totals.shape)
print(nutrition_totals.head())

print("=== microbiome_clr ===")
print(microbiome_clr.shape)
print(microbiome_clr.head())

print("=== diet_fiber ===")
print(diet_fiber.shape)
print(diet_fiber.head())

print("=== fiber_avg ===")
print(fiber_avg.shape)
print(fiber_avg.head())

print("=== micro_avg_counts ===")
print(micro_avg_counts.shape)
print(micro_avg_counts.head())

print("=== micro_avg_clr ===")
print(micro_avg_clr.shape)
print(micro_avg_clr.head())

print("=== micro_avg_norm ===")
print(micro_avg_norm.shape)
print(micro_avg_norm.head())

microbiome.to_csv(DATA_DIR / "microbiome.csv")
sample_meta.to_csv(DATA_DIR / "sample_meta.csv", index=False)
diet_raw.to_csv(DATA_DIR / "diet_raw.csv")
diet_decay.to_csv(DATA_DIR / "diet_decay.csv")
nutrition_totals.to_csv(DATA_DIR / "nutrition_totals.csv")
microbiome_clr.to_csv(DATA_DIR / "microbiome_clr.csv")
diet_fiber.to_csv(DATA_DIR / "diet_fiber.csv")
fiber_avg.to_csv(DATA_DIR / "fiber_avg.csv")
micro_avg_counts.to_csv(DATA_DIR / "micro_avg_counts.csv")
micro_avg_clr.to_csv(DATA_DIR / "micro_avg_clr.csv")
micro_avg_norm.to_csv(DATA_DIR / "micro_avg_norm.csv")

print("Done! CSV files were saved.")
```

c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\dietstudy_analyses-master\data
 Base exists: True
 === microbiome ===
 (223, 483)

	MCT.f.0002	MCT.f.0003	\
#taxonomy			
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	79055	33364	
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	2421	16944	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	39779	9983	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	1	5	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	18393	561	

	MCT.f.0004	MCT.f.0005	\
#taxonomy			
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	64027	70720	
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	43511	18335	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	31539	44880	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	4	4	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	11742	15831	

	MCT.f.0006	MCT.f.0007	\
#taxonomy			
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	63361	58636	
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	36493	43987	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	29901	31975	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	7	2	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	14319	16541	

	MCT.f.0008	MCT.f.0009	\
#taxonomy			
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	73520	69225	
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	15944	20583	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	41607	39981	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	13	3	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	8328	10082	

	MCT.f.0010	MCT.f.0011	\
#taxonomy			
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	58080	75592	
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	5493	23064	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	32774	41183	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	9	4	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	40723	16670	

	MCT.f.0012	MCT.f.0013	\
#taxonomy			
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	66472	59605	
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	30620	38315	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	35785	31152	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	62	2	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	18614	17576	

	...	MCT.f.0616	\
#taxonomy	...		
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	...	106169	
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	...	8	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	...	34073	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	...	0	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	...	0	

	MCT.f.0617	MCT.f.0618	\
#taxonomy			
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	80434	116037	
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	47	11	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	20982	25278	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0	7	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	31	11	

	MCT.f.0619	MCT.f.0620	\
#taxonomy			
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	93723	105413	
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	62	3	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	26608	24590	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	10	1	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	52	6	

	MCT.f.0621	MCT.f.0622	\
#taxonomy			
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	100387	97278	
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	5	23	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	26421	24128	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0	184	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	9	13	

	MCT.f.0623	MCT.f.0624	\
#taxonomy			
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	117400	123705	
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	2	7	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	23532	20010	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	2	2	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	5	10	

	MCT.f.0625	MCT.f.0627	\
#taxonomy			
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	96470	117867	
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	18	2206	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	22195	9340	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	6	142	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	9	1246	

	MCT.f.0628
#taxonomy	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	93290
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	5
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	21397
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	1
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	4

[5 rows x 483 columns]
 === sample_meta ===
 (566, 29)

	SampleID	UserName	StudyDayNo	StudyDate	Sample.Week.Day	DietDayNo	\
0	MCT.f.0001	MCTs01	1	1/31/17	Tuesday	0	
1	MCT.f.0002	MCTs01	2	2/1/17	Wednesday	1	
2	MCT.f.0003	MCTs01	3	2/2/17	Thursday	2	
3	MCT.f.0004	MCTs01	4	2/3/17	Friday	3	
4	MCT.f.0005	MCTs01	5	2/4/17	Saturday	4	

Diet.Record.Date	Diet.Week.Day	Gender	Age	Weight	Height	...	\
------------------	---------------	--------	-----	--------	--------	-----	---

0	1/30/17	Monday	F	25.9000	74.6000	172.7000	...
1	1/31/17	Tuesday	F	25.9000	74.6000	172.7000	...
2	2/1/17	Wednesday	F	25.9000	74.6000	172.7000	...
3	2/2/17	Thursday	F	25.9000	74.6000	172.7000	...
4	2/3/17	Friday	F	25.9000	74.6000	172.7000	...

	fecal.status	fecal.time	BMI.1	Weight.Change	Run1.Plate	Run2.Rerun	\
0	1.0000	12:00 PM	25.0000	-0.4000	5.0000	NaN	
1	1.0000	9:00 AM	25.0000	-0.4000	1.0000	NaN	
2	1.0000	10:30 AM	25.0000	-0.4000	1.0000	NaN	
3	1.0000	8:55 AM	25.0000	-0.4000	5.0000	NaN	
4	1.0000	1:40 PM	25.0000	-0.4000	2.0000	Yes	

	SampleOrder	SampleType	Timing	Activity.Factor	Medications	\
0	54.0000	Sample	Pre	1.3750	Nexplanon (bc implant)	
1	39.0000	Sample	Pre	1.3750	Nexplanon (bc implant)	
2	59.0000	Sample	Pre	1.3750	Nexplanon (bc implant)	
3	32.0000	Sample	Pre	1.3750	Nexplanon (bc implant)	
4	89.0000	Sample	Pre	1.3750	Nexplanon (bc implant)	

	Dietary.Supp
0	Multivitamin;VitD;VitC;Zinc
1	Multivitamin;VitD;VitC;Zinc
2	Multivitamin;VitD;VitC;Zinc
3	Multivitamin;VitD;VitC;Zinc
4	Multivitamin;VitD;VitC;Zinc

[5 rows x 29 columns]

=== diet_raw ===

(1452, 567)

	MCT.f.0065	MCT.f.0070	\
#FoodID			
Milk NFS	6.5575	104.9200	
Milk cows fluid whole	0.0000	0.0000	
Milk cows fluid 2 fat	0.0000	0.0000	
Milk cows fluid 1 fat	0.0000	0.0000	
Milk cows fluid skim or nonfat 0 5 or less butt...	0.0000	0.0000	

	MCT.f.0071	MCT.f.0072	\
#FoodID			
Milk NFS	11.8035	6.5575	
Milk cows fluid whole	0.0000	0.0000	
Milk cows fluid 2 fat	0.0000	0.0000	
Milk cows fluid 1 fat	0.0000	0.0000	
Milk cows fluid skim or nonfat 0 5 or less butt...	0.0000	0.0000	

	MCT.f.0073	MCT.f.0074	\
#FoodID			
Milk NFS	6.5575	26.2300	
Milk cows fluid whole	0.0000	0.0000	
Milk cows fluid 2 fat	0.0000	0.0000	
Milk cows fluid 1 fat	0.0000	0.0000	
Milk cows fluid skim or nonfat 0 5 or less butt...	0.0000	0.0000	

	MCT.f.0095	MCT.f.0099	\
#FoodID			
Milk NFS	41.9680	52.4600	
Milk cows fluid whole	59.7358	0.0000	
Milk cows fluid 2 fat	0.0000	0.0000	
Milk cows fluid 1 fat	0.0000	0.0000	

Milk cows fluid skim or nonfat 0 5 or less butt...	0.0000	0.0000	
	MCT.f.0100	MCT.f.0260	\
#FoodID			
Milk NFS	39.3450	19.6725	
Milk cows fluid whole	0.0000	0.0000	
Milk cows fluid 2 fat	0.0000	0.0000	
Milk cows fluid 1 fat	0.0000	0.0000	
Milk cows fluid skim or nonfat 0 5 or less butt...	0.0000	0.0000	
	MCT.f.0269	MCT.f.0449	\
#FoodID			
Milk NFS	26.2300	6.5575	
Milk cows fluid whole	0.0000	0.0000	
Milk cows fluid 2 fat	3.2910	0.0000	
Milk cows fluid 1 fat	0.0000	0.0000	
Milk cows fluid skim or nonfat 0 5 or less butt...	22.9866	0.0000	
	...	MCT.f.0198	\
#FoodID	...		
Milk NFS	...	0.0000	
Milk cows fluid whole	...	0.0000	
Milk cows fluid 2 fat	...	0.0000	
Milk cows fluid 1 fat	...	0.0000	
Milk cows fluid skim or nonfat 0 5 or less butt...	...	0.0000	
	MCT.f.0203	MCT.f.0204	\
#FoodID			
Milk NFS	0.0000	0.0000	
Milk cows fluid whole	0.0000	0.0000	
Milk cows fluid 2 fat	0.0000	0.0000	
Milk cows fluid 1 fat	0.0000	0.0000	
Milk cows fluid skim or nonfat 0 5 or less butt...	0.0000	0.0000	
	MCT.f.0175	MCT.f.0176	\
#FoodID			
Milk NFS	0.0000	0.0000	
Milk cows fluid whole	0.0000	0.0000	
Milk cows fluid 2 fat	0.0000	0.0000	
Milk cows fluid 1 fat	0.0000	0.0000	
Milk cows fluid skim or nonfat 0 5 or less butt...	0.0000	0.0000	
	MCT.f.0183	MCT.f.0184	\
#FoodID			
Milk NFS	0.0000	0.0000	
Milk cows fluid whole	0.0000	0.0000	
Milk cows fluid 2 fat	0.0000	0.0000	
Milk cows fluid 1 fat	0.0000	0.0000	
Milk cows fluid skim or nonfat 0 5 or less butt...	0.0000	0.0000	
	MCT.f.0193	MCT.f.0194	\
#FoodID			
Milk NFS	0.0000	0.0000	
Milk cows fluid whole	0.0000	0.0000	
Milk cows fluid 2 fat	0.0000	0.0000	
Milk cows fluid 1 fat	0.0000	0.0000	
Milk cows fluid skim or nonfat 0 5 or less butt...	0.0000	0.0000	
	MCT.f.0199	MCT.f.0200	\
#FoodID			

Milk NFS	0.0000	0.0000
Milk cows fluid whole	0.0000	0.0000
Milk cows fluid 2 fat	0.0000	0.0000
Milk cows fluid 1 fat	0.0000	0.0000
Milk cows fluid skim or nonfat 0 5 or less butt...	0.0000	0.0000

taxonomy

#FoodID

Milk NFS	L1_Milk_and_Milk_Products;L2_
Milks_and_milk_dr...	
Milk cows fluid whole	L1_Milk_and_Milk_Products;L2_
Milks_and_milk_dr...	
Milk cows fluid 2 fat	L1_Milk_and_Milk_Products;L2_
Milks_and_milk_dr...	
Milk cows fluid 1 fat	L1_Milk_and_Milk_Products;L2_
Milks_and_milk_dr...	
Milk cows fluid skim or nonfat 0 5 or less butt...	L1_Milk_and_Milk_Products;L2_
Milks_and_milk_dr...	

[5 rows x 567 columns]

=== diet_decay ===

(1452, 564)

MCT.f.0001 MCT.f.0002 \

#SampleID

Milk NFS	0.0000	0.0000
Milk cows fluid whole	0.0000	0.0000
Milk cows fluid 2 fat	0.0000	0.0000
Milk cows fluid 1 fat	0.0000	0.0000
Milk cows fluid skim or nonfat 0 5 or less butt...	44.8474	67.2710

MCT.f.0003 MCT.f.0004 \

#SampleID

Milk NFS	0.0000	0.0000
Milk cows fluid whole	0.0000	0.0000
Milk cows fluid 2 fat	0.0000	0.0000
Milk cows fluid 1 fat	0.0000	0.0000
Milk cows fluid skim or nonfat 0 5 or less butt...	56.0592	72.8770

MCT.f.0005 MCT.f.0006 \

#SampleID

Milk NFS	0.0000	0.0000
Milk cows fluid whole	0.0000	0.0000
Milk cows fluid 2 fat	0.0000	0.0000
Milk cows fluid 1 fat	0.0000	0.0000
Milk cows fluid skim or nonfat 0 5 or less butt...	81.2858	40.6429

MCT.f.0007 MCT.f.0008 \

#SampleID

Milk NFS	0.0000	0.0000
Milk cows fluid whole	0.0000	0.0000
Milk cows fluid 2 fat	0.0000	0.0000
Milk cows fluid 1 fat	0.0000	0.0000
Milk cows fluid skim or nonfat 0 5 or less butt...	65.1688	77.4318

MCT.f.0009 MCT.f.0010 \

#SampleID

Milk NFS	0.0000	0.0000
Milk cows fluid whole	0.0000	0.0000
Milk cows fluid 2 fat	0.0000	0.0000

Milk cows fluid 1 fat	0.0000	0.0000
Milk cows fluid skim or nonfat 0 5 or less butt...	83.5632	86.6290

	MCT.f.0011	MCT.f.0012	\
--	------------	------------	---

#SampleID		
Milk NFS	0.0000	0.0000
Milk cows fluid whole	0.0000	0.0000
Milk cows fluid 2 fat	0.0000	0.0000
Milk cows fluid 1 fat	0.0000	0.0000
Milk cows fluid skim or nonfat 0 5 or less butt...	88.1619	88.9283

	...	MCT.f.0619	\
--	-----	------------	---

#SampleID	...	
Milk NFS	...	0.0000
Milk cows fluid whole	...	0.0000
Milk cows fluid 2 fat	...	2.4682
Milk cows fluid 1 fat	...	0.0000
Milk cows fluid skim or nonfat 0 5 or less butt...	...	0.1753

	MCT.f.0620	MCT.f.0621	\
--	------------	------------	---

#SampleID		
Milk NFS	0.0000	6.5575
Milk cows fluid whole	0.0000	0.0000
Milk cows fluid 2 fat	1.2341	0.6171
Milk cows fluid 1 fat	0.0000	0.0000
Milk cows fluid skim or nonfat 0 5 or less butt...	0.0877	0.0438

	MCT.f.0622	MCT.f.0623	\
--	------------	------------	---

#SampleID		
Milk NFS	4.5903	2.2951
Milk cows fluid whole	0.0000	0.0000
Milk cows fluid 2 fat	0.3085	0.1543
Milk cows fluid 1 fat	0.0000	0.0000
Milk cows fluid skim or nonfat 0 5 or less butt...	0.0219	0.0110

	MCT.f.0624	MCT.f.0625	\
--	------------	------------	---

#SampleID		
Milk NFS	7.7051	7.1313
Milk cows fluid whole	0.0000	0.0000
Milk cows fluid 2 fat	0.0771	0.0386
Milk cows fluid 1 fat	0.0000	0.0000
Milk cows fluid skim or nonfat 0 5 or less butt...	0.0055	0.0027

	MCT.f.0626	MCT.f.0627	\
--	------------	------------	---

#SampleID		
Milk NFS	3.5656	1.7828
Milk cows fluid whole	0.0000	0.0000
Milk cows fluid 2 fat	0.0193	0.0096
Milk cows fluid 1 fat	0.0000	0.0000
Milk cows fluid skim or nonfat 0 5 or less butt...	0.0014	0.0007

	MCT.f.0628	MCT.f.0629	\
--	------------	------------	---

#SampleID		
Milk NFS	0.8914	0.4457
Milk cows fluid whole	0.0000	0.0000
Milk cows fluid 2 fat	0.0048	0.0024
Milk cows fluid 1 fat	0.0000	0.0000
Milk cows fluid skim or nonfat 0 5 or less butt...	0.0003	0.0002

```

taxonomy
#SampleID
Milk NFS L1_Milk_and_Milk_Products;L2_
Milks_and_milk_dr...
Milk cows fluid whole L1_Milk_and_Milk_Products;L2_
Milks_and_milk_dr...
Milk cows fluid 2 fat L1_Milk_and_Milk_Products;L2_
Milks_and_milk_dr...
Milk cows fluid 1 fat L1_Milk_and_Milk_Products;L2_
Milks_and_milk_dr...
Milk cows fluid skim or nonfat 0 5 or less butt... L1_Milk_and_Milk_Products;L2_
Milks_and_milk_dr...

```

[5 rows x 564 columns]

=== nutrition_totals ===

(580, 106)

	UserName	StudyDayNo	RecordDayNo	KCAL	PROT	TFAT	\
X.SampleID							
MCT.f.0001	MCTs01	Day.01	1	1707.0402	95.6687	55.1853	
MCT.f.0002	MCTs01	Day.02	2	1970.0436	109.6198	53.6153	
MCT.f.0003	MCTs01	Day.03	3	1714.8953	89.9930	50.5614	
MCT.f.0004	MCTs01	Day.04	4	2487.2326	94.6120	92.0216	
MCT.f.0005	MCTs01	Day.05	5	2260.2112	110.2684	95.5703	

	CARB	MOIS	ALC	CAFF	THEO	SUGR	...	PF_SOY	\
X.SampleID							...		
MCT.f.0001	187.9654	3848.6055	14.9587	331.5200	0.0000	84.4685	...	0.0000	
MCT.f.0002	242.9190	3808.0057	14.0400	378.8800	0.0000	74.2081	...	0.0000	
MCT.f.0003	234.2251	4183.6224	0.0000	378.8800	9.4720	67.4747	...	0.0000	
MCT.f.0004	257.5390	4816.2115	42.4480	379.4300	20.2400	83.9603	...	0.0000	
MCT.f.0005	165.6081	4404.2439	43.9306	534.5450	20.7350	64.9220	...	0.0000	

	PF_NUTSDS	PF_LEGUMES	D_TOTAL	D_MILK	D_YOGURT	D_CHEESE	\
X.SampleID							
MCT.f.0001	1.0000	0.0000	3.2656	2.0578	0.6460	0.5618	
MCT.f.0002	1.0000	0.0000	4.1560	2.3566	0.6460	1.1535	
MCT.f.0003	1.0000	0.0000	2.9797	1.1261	0.6460	1.2076	
MCT.f.0004	1.0000	0.0000	3.6345	2.1519	0.0000	1.4826	
MCT.f.0005	1.0000	0.0000	3.3797	2.3207	0.0000	1.0590	

	OILS	SOLID_FATS	ADD_SUGARS	A_DRINKS	Replicate
X.SampleID					
MCT.f.0001	14.6689	21.7959	4.8224	1.0725	1
MCT.f.0002	15.8862	19.1398	4.3260	1.0080	1
MCT.f.0003	11.8369	17.7116	4.7583	0.0000	1
MCT.f.0004	55.3081	19.9378	5.9171	3.0352	1
MCT.f.0005	24.2312	54.6136	5.7939	3.1164	1

[5 rows x 106 columns]

=== microbiome_clr ===

(223, 483)

	MCT.f.0002	MCT.f.0003	\
#taxonomy			
k__Bacteria;NA;NA;NA;NA;NA;NA	1.1863	1.6253	
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	1.0804	3.0227	
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	3.4697	5.5913	
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	-3.7742	-1.7939	
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	-0.8124	1.0605	

	MCT.f.0004	MCT.f.0005	\
--	------------	------------	---

#taxonomy		
k__Bacteria;NA;NA;NA;NA;NA;NA	0.7413	0.3923
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	1.2554	0.3471
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	3.9533	2.9413
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	-3.6572	-3.3715
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	-0.9946	-1.1192
MCT.f.0006 MCT.f.0007 \		
#taxonomy		
k__Bacteria;NA;NA;NA;NA;NA;NA	0.9768	1.0627
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	0.5617	1.2157
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	2.9753	3.7625
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	-3.7619	-3.0399
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	-1.1286	-0.9922
MCT.f.0008 MCT.f.0009 \		
#taxonomy		
k__Bacteria;NA;NA;NA;NA;NA;NA	1.0831	0.9981
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	0.8941	1.1893
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	3.3268	3.7631
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	-3.1401	-2.9106
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	-1.4353	-0.8737
MCT.f.0010 MCT.f.0011 \		
#taxonomy		
k__Bacteria;NA;NA;NA;NA;NA;NA	0.7037	0.9198
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	2.3239	1.1150
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	5.0367	3.6882
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	-2.2141	-3.6753
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	-0.1346	-0.8226
MCT.f.0012 MCT.f.0013 \		
#taxonomy		
k__Bacteria;NA;NA;NA;NA;NA;NA	0.7139	1.0205
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	0.8600	1.2033
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	3.3247	3.7826
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	-3.2751	-3.1384
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	-1.2602	-0.9228
... MCT.f.0616 \		
#taxonomy		
k__Bacteria;NA;NA;NA;NA;NA;NA	...	2.9694
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	...	1.3148
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	...	-1.6653
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	...	-1.9481
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	...	-0.8446
MCT.f.0617 MCT.f.0618 \		
#taxonomy		
k__Bacteria;NA;NA;NA;NA;NA;NA	1.2707	1.3025
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	3.2961	2.6635
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	1.3220	0.3862
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	-1.3514	-2.6368
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	2.1549	1.2881
MCT.f.0619 MCT.f.0620 \		
#taxonomy		
k__Bacteria;NA;NA;NA;NA;NA;NA	0.8011	2.2349
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	1.8997	2.8623
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	-1.0842	0.6192

k__Bacteria;p__Actinobacteria;c__Actinobacteria...	-1.1111	-3.3143	
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	1.0888	1.7333	
	MCT.f.0621	MCT.f.0622	\
#taxonomy			
k__Bacteria;NA;NA;NA;NA;NA;NA	1.1115	1.2338	
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	3.4539	3.3590	
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	1.0470	1.5391	
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	-1.9927	-1.4053	
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	2.2998	2.3264	
	MCT.f.0623	MCT.f.0624	\
#taxonomy			
k__Bacteria;NA;NA;NA;NA;NA;NA	2.4192	1.1160	
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	3.3256	2.7127	
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	1.2938	0.4602	
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	-2.0619	-2.9262	
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	2.0970	1.4471	
	MCT.f.0625	MCT.f.0627	\
#taxonomy			
k__Bacteria;NA;NA;NA;NA;NA;NA	0.7038	1.4949	
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	2.5561	3.8529	
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	0.8215	1.6607	
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	-2.2767	-1.7218	
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	1.7452	2.6685	
	MCT.f.0628		
#taxonomy			
k__Bacteria;NA;NA;NA;NA;NA;NA	0.9911		
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	2.7642		
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	0.5431		
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	-2.1450		
k__Bacteria;p__Actinobacteria;c__Actinobacteria...	1.6703		
[5 rows x 483 columns]			
=== diet_fiber ===			
(1452, 581)			
	MCT.f.0065	MCT.f.0070	\
#FOODID			
Milk NFS	0.0000	0.0000	
Milk cows fluid whole	0.0000	0.0000	
Milk cows fluid 2 fat	0.0000	0.0000	
Milk cows fluid 1 fat	0.0000	0.0000	
Milk cows fluid skim or nonfat 0 5 or less butt...	0.0000	0.0000	
	MCT.f.0071	MCT.f.0072	\
#FOODID			
Milk NFS	0.0000	0.0000	
Milk cows fluid whole	0.0000	0.0000	
Milk cows fluid 2 fat	0.0000	0.0000	
Milk cows fluid 1 fat	0.0000	0.0000	
Milk cows fluid skim or nonfat 0 5 or less butt...	0.0000	0.0000	
	MCT.f.0073	MCT.f.0074	\
#FOODID			
Milk NFS	0.0000	0.0000	
Milk cows fluid whole	0.0000	0.0000	
Milk cows fluid 2 fat	0.0000	0.0000	
Milk cows fluid 1 fat	0.0000	0.0000	

Milk cows fluid skim or nonfat 0 5 or less butt...	0.0000	0.0000	
	MCT.f.0076	MCT.f.0085	\
#FOODID			
Milk NFS	0.0000	0.0000	
Milk cows fluid whole	0.0000	0.0000	
Milk cows fluid 2 fat	0.0000	0.0000	
Milk cows fluid 1 fat	0.0000	0.0000	
Milk cows fluid skim or nonfat 0 5 or less butt...	0.0000	0.0000	
	MCT.f.0095	MCT.f.0099	\
#FOODID			
Milk NFS	0.0000	0.0000	
Milk cows fluid whole	0.0000	0.0000	
Milk cows fluid 2 fat	0.0000	0.0000	
Milk cows fluid 1 fat	0.0000	0.0000	
Milk cows fluid skim or nonfat 0 5 or less butt...	0.0000	0.0000	
	MCT.f.0100	MCT.f.0260	\
#FOODID			
Milk NFS	0.0000	0.0000	
Milk cows fluid whole	0.0000	0.0000	
Milk cows fluid 2 fat	0.0000	0.0000	
Milk cows fluid 1 fat	0.0000	0.0000	
Milk cows fluid skim or nonfat 0 5 or less butt...	0.0000	0.0000	
	...	MCT.f.0198	\
#FOODID	...		
Milk NFS	...	0	
Milk cows fluid whole	...	0	
Milk cows fluid 2 fat	...	0	
Milk cows fluid 1 fat	...	0	
Milk cows fluid skim or nonfat 0 5 or less butt...	...	0	
	MCT.f.0203	MCT.f.0204	\
#FOODID			
Milk NFS	0	0	
Milk cows fluid whole	0	0	
Milk cows fluid 2 fat	0	0	
Milk cows fluid 1 fat	0	0	
Milk cows fluid skim or nonfat 0 5 or less butt...	0	0	
	MCT.f.0175	MCT.f.0176	\
#FOODID			
Milk NFS	0.0000	0.0000	
Milk cows fluid whole	0.0000	0.0000	
Milk cows fluid 2 fat	0.0000	0.0000	
Milk cows fluid 1 fat	0.0000	0.0000	
Milk cows fluid skim or nonfat 0 5 or less butt...	0.0000	0.0000	
	MCT.f.0183	MCT.f.0184	\
#FOODID			
Milk NFS	0.0000	0.0000	
Milk cows fluid whole	0.0000	0.0000	
Milk cows fluid 2 fat	0.0000	0.0000	
Milk cows fluid 1 fat	0.0000	0.0000	
Milk cows fluid skim or nonfat 0 5 or less butt...	0.0000	0.0000	
	MCT.f.0193	MCT.f.0194	\
#FOODID			

Milk NFS	0	0
Milk cows fluid whole	0	0
Milk cows fluid 2 fat	0	0
Milk cows fluid 1 fat	0	0
Milk cows fluid skim or nonfat 0 5 or less butt...	0	0

MCT.f.0199 MCT.f.0200 \

#FOODID

Milk NFS	0	0
Milk cows fluid whole	0	0
Milk cows fluid 2 fat	0	0
Milk cows fluid 1 fat	0	0
Milk cows fluid skim or nonfat 0 5 or less butt...	0	0

taxonomy

#FOODID

Milk NFS	L1_Milk_and_Milk_Products;L2_
Milks_and_milk_dr...	
Milk cows fluid whole	L1_Milk_and_Milk_Products;L2_
Milks_and_milk_dr...	
Milk cows fluid 2 fat	L1_Milk_and_Milk_Products;L2_
Milks_and_milk_dr...	
Milk cows fluid 1 fat	L1_Milk_and_Milk_Products;L2_
Milks_and_milk_dr...	
Milk cows fluid skim or nonfat 0 5 or less butt...	L1_Milk_and_Milk_Products;L2_
Milks_and_milk_dr...	

[5 rows x 581 columns]

=== fiber_avg ===

(1452, 35)

MCTs01 MCTs03 MCTs04 MCTs05 MCTs06 MCTs07 \

#FoodID

Chili with beans without meat	0.0000	0.0000	0.0000	0.0000	0.0000	2.2414
Falafel	0.1300	0.0000	0.0000	0.0000	0.0000	0.0000
Hummus	0.3618	0.0000	0.6150	0.0000	0.0000	0.0000
Pork and beans	0.0000	0.0000	0.0000	0.0000	0.4800	0.0000
Refried beans	0.3404	0.0000	0.3101	0.0000	0.0000	0.0000

MCTs08 MCTs09 MCTs10 MCTs11 MCTs12 MCTs13 \

#FoodID

Chili with beans without meat	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Falafel	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Hummus	0.1447	0.0000	0.0000	0.0000	0.0000	0.0000
Pork and beans	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Refried beans	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

... MCTs26 MCTs27 MCTs28 MCTs29 MCTs31 \

#FoodID

Chili with beans without meat	...	0.0000	0.0000	0.0000	0.0000	0.8036
Falafel	...	0.0000	0.0000	0.0000	0.0000	0.0000
Hummus	...	0.0000	0.0000	0.0000	0.0000	0.1085
Pork and beans	...	0.0000	0.0000	0.0000	0.0000	0.0000
Refried beans	...	0.0000	0.2270	0.0000	0.0000	0.0000

MCTs32 MCTs33 MCTs34 MCTs35 MCTs36 MCTs37 \

#FoodID

Chili with beans without meat	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Falafel	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Hummus	0.0000	0.4341	0.0000	0.0000	0.1537	0.0000

Pork and beans	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Refried beans	0.9078	0.0000	0.0000	0.0000	0.0000	0.0000

taxonomy

#FoodID

Chili with beans without meat	L1_Dry_Beans_Peas_Other_Legumes_Nuts_and_Seeds...
Falafel	L1_Dry_Beans_Peas_Other_Legumes_Nuts_and_Seeds...
Hummus	L1_Dry_Beans_Peas_Other_Legumes_Nuts_and_Seeds...
Pork and beans	L1_Dry_Beans_Peas_Other_Legumes_Nuts_and_Seeds...
Refried beans	L1_Dry_Beans_Peas_Other_Legumes_Nuts_and_Seeds...

[5 rows x 35 columns]

=== micro_avg_counts ===

(226, 34)

MCTs01 MCTs03 MCTs04 \

#taxonomy

k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	2455	2977	2796
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	984	704	312
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	1272	841	91
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0	1	1
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	577	1	13

MCTs05 MCTs06 MCTs07 \

#taxonomy

k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	4394	1701	2682
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	918	1328	895
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	901	374	373
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0	133	0
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0	22	423

MCTs08 MCTs09 MCTs10 \

#taxonomy

k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	3861	2507	1604
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	361	738	632
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	984	990	317
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	1	0	0
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	5	289	864

MCTs11 MCTs12 MCTs13 \

#taxonomy

k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	2562	2417	2239
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	385	451	362
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	324	133	317
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	2	2	1
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	515	553	739

... MCTs25 MCTs26 \

#taxonomy

k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	...	2623	2977
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	...	634	739
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	...	621	800
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	...	0	0
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	...	372	391

MCTs27 MCTs28 MCTs29 \

#taxonomy

k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	2135	1927	1243
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	1019	723	1151
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	834	214	320
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0	7	1

k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	494	764	212	
	MCTs31	MCTs32	MCTs33	\
#taxonomy				
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	1305	1907	2463	
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	285	1183	263	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	12	150	274	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	4475	643	0	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	3	26	1277	
	MCTs34	MCTs35	MCTs36	\
#taxonomy				
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	914	228	2890	
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	308	862	1281	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	15	25	596	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0	5383	2	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	1592	3	494	
	MCTs37			
#taxonomy				
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	4014			
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	11			
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	760			
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	1			
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	6			
[5 rows x 34 columns]				
=== micro_avg_clr ===				
(226, 34)				
	MCTs01	MCTs03	MCTs04	\
#taxonomy				
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	6.8593	7.2016	7.2870	
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	5.9450	5.7598	5.0923	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	6.2019	5.9377	3.8656	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	-2.1007	-1.0382	-0.8328	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	5.4116	-0.7221	1.8991	
	MCTs05	MCTs06	MCTs07	\
#taxonomy				
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	8.5081	5.8034	6.9401	
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	6.9429	5.5561	5.8422	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	6.9238	4.2893	4.9681	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	-1.8219	3.2555	-1.8906	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	-1.2719	1.4668	5.0936	
	MCTs08	MCTs09	MCTs10	\
#taxonomy				
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	8.1252	7.1033	6.3748	
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	5.7555	5.8800	5.4438	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	6.7577	6.1744	4.7529	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	-0.2152	-1.7464	-1.7834	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	1.4089	4.9445	5.7562	
	MCTs11	MCTs12	MCTs13	\
#taxonomy				
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	6.9635	7.3358	6.6972	
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	5.0688	5.6573	4.8751	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	4.8972	4.4333	4.7420	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	-0.1598	0.1498	-0.9028	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	5.3600	5.8613	5.5894	

```

... MCTs25 MCTs26 \
#taxonomy
...
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... ... 7.4198 7.1740
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos... ... 5.9995 5.7804
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... ... 5.9786 5.8605
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... ... -1.9312 -1.8573
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... ... 5.4672 5.1451

MCTs27 MCTs28 MCTs29 \
#taxonomy
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... 6.8185 6.0500 5.6104
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos... 6.0793 5.0692 5.5333
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... 5.8785 3.8532 4.2535
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... -2.5647 0.4385 -1.3881
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... 5.3549 5.1253 3.8421

MCTs31 MCTs32 MCTs33 \
#taxonomy
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... 6.7642 6.0633 6.9630
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos... 5.2422 5.5864 4.7253
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... 2.1118 3.5185 4.7676
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... 7.9963 4.9764 -2.1752
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... 0.6535 1.7690 6.3057

MCTs34 MCTs35 MCTs36 \
#taxonomy
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... 6.2754 4.8376 7.2627
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos... 5.1863 6.1684 6.4489
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... 2.1652 2.6122 5.6842
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... -1.6961 7.9997 0.1431
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... 6.8308 0.5151 5.4954

MCTs37
#taxonomy
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... 8.6497
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos... 2.7224
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... 6.9857
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... 0.4323
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... 2.1736

[5 rows x 34 columns]
=== micro_avg_norm ===
(226, 34)

MCTs01 MCTs03 MCTs04 \
#taxonomy
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... 0.2455 0.2977 0.2796
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos... 0.0984 0.0704 0.0312
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... 0.1272 0.0841 0.0091
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... 0.0000 0.0001 0.0001
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... 0.0577 0.0001 0.0013

MCTs05 MCTs06 MCTs07 \
#taxonomy
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... 0.4394 0.1701 0.2682
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos... 0.0918 0.1328 0.0895
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... 0.0901 0.0374 0.0373
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... 0.0000 0.0133 0.0000
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__... 0.0000 0.0022 0.0423

```

	MCTs08	MCTs09	MCTs10	\
#taxonomy				
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0.3861	0.2507	0.1604	
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	0.0361	0.0738	0.0632	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0.0984	0.0990	0.0317	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0.0001	0.0000	0.0000	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0.0005	0.0289	0.0864	
	MCTs11	MCTs12	MCTs13	\
#taxonomy				
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0.2562	0.2417	0.2239	
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	0.0385	0.0451	0.0362	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0.0324	0.0133	0.0317	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0.0002	0.0002	0.0001	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0.0515	0.0553	0.0739	
	...	MCTs25	MCTs26	\
#taxonomy	...			
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	...	0.2623	0.2977	
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	...	0.0634	0.0739	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	...	0.0621	0.0800	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	...	0.0000	0.0000	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	...	0.0372	0.0391	
	MCTs27	MCTs28	MCTs29	\
#taxonomy				
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0.2135	0.1927	0.1243	
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	0.1019	0.0723	0.1151	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0.0834	0.0214	0.0320	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0.0000	0.0007	0.0001	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0.0494	0.0764	0.0212	
	MCTs31	MCTs32	MCTs33	\
#taxonomy				
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0.1305	0.1907	0.2463	
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	0.0285	0.1183	0.0263	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0.0012	0.0150	0.0274	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0.4475	0.0643	0.0000	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0.0003	0.0026	0.1277	
	MCTs34	MCTs35	MCTs36	\
#taxonomy				
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0.0914	0.0228	0.2890	
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	0.0308	0.0862	0.1281	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0.0015	0.0025	0.0596	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0.0000	0.5383	0.0002	
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0.1592	0.0003	0.0494	
	MCTs37			
#taxonomy				
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0.4014			
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clos...	0.0011			
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0.0760			
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0.0001			
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__...	0.0006			

[5 rows x 34 columns]

Done! CSV files were saved.

Profiling the Paper 1 Gut Microbiome Abundance Dataset

In [153...

```
microbiome = pd.read_csv(DATA_DIR / "microbiome.csv", index_col=0)

print("Shape:", microbiome.shape)
print("Number of microbial features:", microbiome.shape[0])
print("Number of samples:", microbiome.shape[1])

print("\nFirst rows:")
display(microbiome.head())

print("\nBasic statistics:")
display(microbiome.describe())
```

Shape: (223, 483)
 Number of microbial features: 223
 Number of samples: 483

First rows:

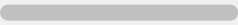
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__Bacteroidales;f__Bacteroidacea
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clostridiales;f__Ruminococcaceae;g__Faecalibacterium

k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__Bacteroidales;f__Bacteroidaceae;g__Bacter

k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__Bacteroidales;f__Prevotellaceae;g__Prevotell

k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__Bacteroidales;f__Rikenellaceae;g__Alistipes;s__

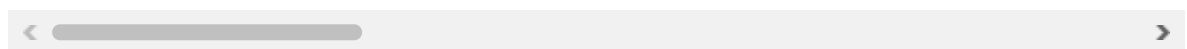
5 rows × 483 columns

<  >

Basic statistics:

	MCT.f.0002	MCT.f.0003	MCT.f.0004	MCT.f.0005	MCT.f.0006	MCT.f.0007	MCT.f.0
count	223.0000	223.0000	223.0000	223.0000	223.0000	223.0000	223.0
mean	1196.1166	1196.1300	1196.1435	1196.0852	1196.1076	1196.0987	1196.0
std	6292.6284	3929.3673	5764.6143	6018.0095	5532.6329	5599.8566	5909.8
min	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0
25%	11.0000	13.0000	18.0000	15.0000	18.0000	16.0000	18.0
50%	40.0000	91.0000	53.0000	57.0000	62.0000	53.0000	56.0
75%	145.5000	429.0000	231.5000	267.0000	224.5000	221.0000	285.0
max	79055.0000	33364.0000	64027.0000	70720.0000	63361.0000	58636.0000	73520.0

8 rows × 483 columns



EDA on microbiome data from the Johnson et al. 2019 research

Gut Microbiome Data Profiling

In this step, we loaded the gut microbiome abundance table (`microbiome.csv`) generated from the Johnson et al. 2019 dataset.

We performed an initial profiling of the dataset to better understand its structure and characteristics before preprocessing and filtering.

What we examined

- Dataset dimensions
- Number of microbial features (taxa/species)
- Number of samples
- First rows of the abundance table
- Descriptive statistics for abundance values

Results

- The dataset contains:
 - **223 microbial features**
 - **483 fecal samples**
- Rows represent microbial taxa/species.
- Columns represent individual microbiome samples.
- The abundance values are highly variable across samples, with many low values and zeros, which is common in microbiome datasets.

Key observations

- Some bacterial species are highly abundant in specific samples.

- Many taxa appear at very low abundance levels.
- The dataset is expected to be sparse, meaning many entries contain zeros due to absent or rare microbes.

This profiling step provides the foundation for the next stages:

1. Low-prevalence feature filtering
2. Sparsity analysis
3. Data preprocessing for downstream microbiome analysis

Analyzing Sparsity and Feature Prevalence

```
In [154... # Calculate sparsity (percentage of zeros)

total_values = microbiome.size
zero_values = (microbiome == 0).sum().sum()

sparsity = (zero_values / total_values) * 100

print("Total values in dataset:", total_values)
print("Number of zero values:", zero_values)
print(f"Sparsity: {sparsity:.2f}%")
```

```
Total values in dataset: 107709
Number of zero values: 9467
Sparsity: 8.79%
```

```
In [155... # Prevalence of each microbial feature
# (fraction of samples where feature appears)

prevalence = (microbiome > 0).sum(axis=1) / microbiome.shape[1]

print(prevalence.describe())

# Show first prevalence values
display(prevalence.head())
```

```
count    223.0000
mean      0.9121
std       0.1456
min       0.2712
25%      0.9110
50%      0.9731
75%      0.9948
max       1.0000
dtype: float64
```

```
#taxonomy
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__Bacteroidales;f__Bacteroidaceae;g__
_Bacteroides;NA 1.0000
k__Bacteria;p__Firmicutes;c__Clostridia;o__Clostridiales;f__Ruminococcaceae;g__Fa
ecalibacterium;s__Faecalibacterium prausnitzii 1.0000
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__Bacteroidales;f__Bacteroidaceae;g__
_Bacteroides;s__Bacteroides uniformis 1.0000
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__Bacteroidales;f__Prevotellaceae;g__
_Prevotella;s__Prevotella copri 0.9110
k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o__Bacteroidales;f__Rikenellaceae;g__
Alistipes;s__Alistipes putredinis 0.9938
dtype: float64
```

Sparsity and Prevalence Analysis

In this step, we analyzed the sparsity and prevalence of the gut microbiome abundance table.

Sparsity Analysis

We calculated the percentage of zero values in the dataset to measure sparsity.

Results

- Total values in the dataset: **107,709**
- Zero values: **9,467**
- Dataset sparsity: **8.79%**

This indicates that most microbial features are present in many samples, while a smaller portion of the matrix contains absent microbes (zeros).

Prevalence Analysis

We then calculated the prevalence of each microbial feature, defined as the fraction of samples in which a feature appears.

Key observations

- The average prevalence across taxa is very high (~0.91).
- Many microbial species appear in nearly all samples.
- Some taxa are less common and appear only in a subset of participants.
- The minimum prevalence observed was approximately **27%** of samples.

These results suggest that the dataset is relatively dense compared to many microbiome datasets, but still contains lower-prevalence taxa that may be filtered in the next preprocessing stage.

Filtering Low-Prevalence Microbial Features

```
In [156... # Filter low-prevalence microbial features

threshold = 0.10 # keep taxa appearing in at least 10% of samples

filtered_microbiome = microbiome.loc[prevalence >= threshold]

print("Original shape:", microbiome.shape)
print("Filtered shape:", filtered_microbiome.shape)

print("Removed features:", microbiome.shape[0] - filtered_microbiome.shape[0])
```

Original shape: (223, 483)
 Filtered shape: (223, 483)
 Removed features: 0

```
In [157... filtered_microbiome.to_csv(DATA_DIR / "microbiome_filtered.csv")

print("Filtered microbiome dataset saved.")
```

Filtered microbiome dataset saved.

```
In [158... """
paper2_data.py
Loads and exports datasets from Carlino et al. 2024 (Cell)
Source: https://github.com/SegataLab/cFMD
"""

# Base link to database
#base_cfmd = DATA_DIR / "cFMD-main" / "cFMD-main" / "cFMD_data"
base_cfmd = DATA_DIR / "cFMD-main" / "cFMD-main" / "cFMD_data"

# Dataset of microbial species vs fecal samples
data_base = []
# Iterate on each dataset in the database
for dataset in os.listdir(base_cfmd):
    # Create path to dataset
    path = base_cfmd / dataset / f"{dataset}_taxonomic_profiles.tsv"
    # If the path is correct add the dataset to the data base
    if path.exists():
        df = pd.read_csv(path, sep="\t", index_col=0)
        data_base.append(df)

# Join the data sets to one single data set
combined = pd.concat(data_base, axis=1)
# Fill null value with zero
combined = combined.fillna(0)
print("Shape:", combined.shape)

# save the combined datasets to csv
combined.to_csv(DATA_DIR / "combined_datasets.csv")

# Metadata on the microbiome
#metadata = pd.read_csv(DATA_DIR / "cFMD-main" / "cFMD-main" / "cFMD_metadata.ts
metadata = pd.read_csv(DATA_DIR / "cFMD-main" / "cFMD-main" / "cFMD_metadata.tsv
```

```
# Export the metadata to csv
metadata.to_csv(DATA_DIR / "cfmd_metadata.csv", index=False)
```

Shape: (5057, 3393)

```
In [159... #for item in os.listdir(r"C:\Users\rtajz\Downloads"):
for item in os.listdir(DATA_DIR):
    if "cFMD" in item or "cfmd" in item:
        print(item)
```

cfMD-main

cfmd_metadata.csv

Loading and Combining the Paper 2 (cFMD) Microbiome Dataset

```
In [ ]: # Paper 2: Carlino et al. 2024 / cFMD dataset
#base_cfmd = DATA_DIR / "cFMD-main" / "cFMD-main" / "cFMD_data"
#metadata_path = DATA_DIR / "cFMD-main" / "cFMD-main" / "cFMD_metadata.tsv"
base_cfmd = DATA_DIR / "cFMD-main" / "cFMD-main" / "cFMD_data"
metadata_path = DATA_DIR / "cFMD-main" / "cFMD-main" / "cFMD_metadata.tsv"

print("Base exists:", os.path.exists(base_cfmd))
print("Metadata exists:", os.path.exists(metadata_path))

data_base = []

for dataset in os.listdir(base_cfmd):
    path = base_cfmd / dataset / f"{dataset}_taxonomic_profiles.tsv"

    if path.exists():
        df = pd.read_csv(path, sep="\t", index_col=0)
        data_base.append(df)

combined = pd.concat(data_base, axis=1)
combined = combined.fillna(0)

print("=== Paper 2 combined dataset ===")
print("Shape:", combined.shape)
display(combined.head())

combined.to_csv(str(DATA_DIR / "combined_datasets.csv"))

metadata = pd.read_csv(metadata_path, sep="\t")

print("=== Paper 2 metadata ===")
print("Shape:", metadata.shape)
display(metadata.head())

metadata.to_csv(DATA_DIR / "cfmd_metadata.csv", index=False)

print("Done! Paper 2 CSV files were saved.")
```

Base exists: True

Metadata exists: True

=== Paper 2 combined dataset ===

Shape: (5057, 3393)

k_Bacteria p_Firmicutes c_Bacilli o_Lactobacillales f_Streptococ
k_Bacteria p_Actinomycetota c_Actinomycetes o_Propionibacteriales f_Propionibacteriaceae
k_Bacteria p_Firmicutes c_Bacilli o_Lactobacillales f_Lactobacillaceae g_Lactipl
k_Bacteria p_Pseudomonadota c_Gammaproteobacteria o_Enterobacterales f_Erw
k_Bacteria p_Firmicutes c_Bacilli o_Bacillales f_Paenibacilla

5 rows × 3393 columns

Paper 2 Dataset Integration and Exploration

In this step, we loaded and combined multiple microbiome datasets from the cFMD database (Carlino et al. 2024).

What was done

- Loaded all taxonomic profile tables from the cFMD repository.
- Merged all datasets into one combined abundance matrix.
- Filled missing values with zeros.
- Loaded and explored the accompanying metadata table.
- Exported both the combined microbiome dataset and metadata to CSV files.

Results

- Combined microbiome dataset shape:
 - **5057 microbial features**
 - **3393 samples**
- Metadata table shape:
 - **3444 samples**
 - **32 metadata variables**

Key observations

- The combined dataset contains thousands of microbial taxa collected from many food-related microbiome samples.
- Missing values were replaced with zeros to create a complete abundance matrix for downstream analysis.
- The metadata includes detailed information such as:
 - dataset source
 - food category
 - fermentation status

- country
- sequencing information

This integrated dataset will be used for profiling, sparsity analysis, prevalence filtering, and food microbiome exposure analysis in later stages of the project.

Profiling the Paper 2 Food Microbiome Dataset

```
In [ ]: combined = pd.read_csv(DATA_DIR / "combined_datasets.csv", index_col=0)

print("Shape:", combined.shape)
print("Number of microbial features:", combined.shape[0])
print("Number of samples:", combined.shape[1])

print("\nFirst rows:")
display(combined.head())

print("\nBasic statistics:")
display(combined.describe())
```

Shape: (5057, 3393)
 Number of microbial features: 5057
 Number of samples: 3393

First rows:

```
k_Bacteria|p_Firmicutes|c_Bacilli|o_Lactobacillales|f_Streptococcus|g_Lactobacillus|s_1
k_Bacteria|p_Actinomycetota|c_Actinomycetes|o_Propionibacteriales|f_Propionibacteriaceae|g_Propionibacterium|s_2
k_Bacteria|p_Firmicutes|c_Bacilli|o_Lactobacillales|f_Lactobacillaceae|g_Lactiplanibaculum|s_3
k_Bacteria|p_Pseudomonadota|c_Gammaproteobacteria|o_Enterobacterales|f_Enterobacteriaceae|g_Enterobacter|s_4
k_Bacteria|p_Firmicutes|c_Bacilli|o_Bacillales|f_Paenibacillales|g_Paenibacillus|s_5
```

5 rows × 3393 columns

```
< Basic statistics:
>
```

	AlmeidaO_2020__SRR8742576	AlmeidaO_2020__SRR8742577	AlmeidaO_2020__SRR8742578
count	5057.0000	5057.0000	5057.0000
mean	0.0198	0.0198	0.0198
std	0.4641	0.8094	0.8094
min	0.0000	0.0000	0.0000
25%	0.0000	0.0000	0.0000
50%	0.0000	0.0000	0.0000
75%	0.0000	0.0000	0.0000
max	14.8993	50.9473	50.9473

8 rows × 3392 columns

Profiling the Paper 2 Food Microbiome Dataset

In this step, we performed an initial exploration and profiling of the combined cFMD microbiome abundance dataset.

What was examined

- Dataset dimensions
- Number of microbial features
- Number of samples
- First rows of the abundance matrix
- Descriptive statistics of microbial abundance values

Results

- The dataset contains:
 - **5057 microbial features**
 - **3393 microbiome samples**
- Rows represent microbial taxa/species.
- Columns represent individual food microbiome samples collected from multiple datasets.

Key observations

- The dataset is substantially larger than the Paper 1 microbiome dataset.
- Many abundance values are very small or zero, indicating the presence of rare microbial taxa.
- The abundance distributions are highly skewed, which is common in microbiome sequencing data.

- Some microbial taxa appear frequently across samples, while others are highly sparse.

This profiling step provides the basis for further preprocessing, including sparsity analysis, prevalence filtering, and downstream microbiome exposure analysis.

Sparsity and Prevalence Analysis of the Paper 2 Dataset

```
In [ ]: # Calculate sparsity

total_values = combined.size
zero_values = (combined == 0).sum().sum()

sparsity = (zero_values / total_values) * 100

print("Total values in dataset:", total_values)
print("Number of zero values:", zero_values)
print(f"Sparsity: {sparsity:.2f}%")
```

```
Total values in dataset: 17158401
Number of zero values: 17063753
Sparsity: 99.45%
```

```
In [ ]: # Convert all values to numeric
combined_numeric = combined.apply(pd.to_numeric, errors='coerce')

# Replace NaN with zero
combined_numeric = combined_numeric.fillna(0)

# Calculate prevalence
prevalence = (combined_numeric > 0).sum(axis=1) / combined_numeric.shape[1]

print(prevalence.describe())

display(prevalence.head())
```

```
count    5057.0000
mean      0.0055
std       0.0211
min       0.0000
25%       0.0003
50%       0.0009
75%       0.0027
max       0.5617
dtype: float64
```



```

sample
k__Bacteria|p__Firmicutes|c__Bacilli|o__Lactobacillales|f__Streptococcaceae|g__Lactococcus|s__Lactococcus_lactis|t__SGB7985      0.5617
k__Bacteria|p__Actinomycetota|c__Actinomycetes|o__Propionibacteriales|f__Propionibacteriaceae|g__Cutibacterium|s__Cutibacterium_acnes|t__SGB16955      0.0719
k__Bacteria|p__Firmicutes|c__Bacilli|o__Lactobacillales|f__Lactobacillaceae|g__Lactiplantibacillus|s__Lactiplantibacillus_plantarum|t__SGB7202      0.1813
k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria|o__Enterobacterales|f__Erwinia|g__Pantoea|s__Pantoea_coffeiphila|t__SGB23192      0.0071
k__Bacteria|p__Firmicutes|c__Bacilli|o__Bacillales|f__Paenibacillaceae|g__Paenibacillus|s__Paenibacillus_cookii|t__SGB7404      0.0038
dtype: float64

```

Prevalence Analysis of the Paper 2 Dataset

We analyzed the prevalence of microbial features in the Paper 2 microbiome dataset after converting all values to numeric format. Since the dataset originally contained mixed data types, non-numeric values were converted to `NaN` and then replaced with zeros to enable numerical analysis.

The prevalence calculation measured the fraction of samples in which each microbial feature appeared (abundance > 0).

The results show that the dataset is highly sparse:

- The average prevalence is very low (~0.0055).
- Many microbial taxa appear in only a very small number of samples.
- Some taxa are present in more samples, with the maximum prevalence reaching ~56%.

These findings are consistent with large-scale microbiome datasets, where most microbial species are rare and only a subset are commonly observed across samples.

End sprint 1 notebook

Start sprint 2 notebook

Notebook:

Merge_python_notebook_sprint2.ipynb

Association Testing

Objective

The goal of this analysis is to investigate the relationship between Food Microbiome Exposure (FME) and gut microbiome characteristics.

Specifically, we examine whether FME is associated with:

- Alpha diversity (Shannon diversity)
- Microbiome stability
- Community composition metrics

This analysis is based on the dataset generated in the FME analysis stage.

```
In [ ]: # Locate repo root (directory containing config.py) regardless of launch directo
_root = next(p for p in [Path.cwd(), *Path.cwd().parents] if (p / "config.py").e
DATA_DIR = _root / "data" # robust: does not depend on __file__ inside Jupyter
sys.path.insert(0, str(_root))
```

Dataset Overview

In this section, we load and inspect the statistical dataset created in the FME analysis step.

The goal is to understand the structure of the dataset, including the number of samples, available variables, data types, and missing values before performing association testing.

```
In [ ]: # Load the statistical dataset created in the FME analysis step

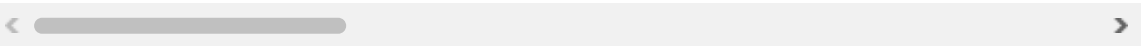
df = pd.read_csv(DATA_DIR / "fme_statistical_dataset.csv")

df.head()
```

Out []:

	fecal_sample_id	participant_id	study_day	Gender	Age	BMI	Weight	Supple
0	MCT.f.0002	MCTs01	2	F	25.9000	25.0000	74.6000	
1	MCT.f.0003	MCTs01	3	F	25.9000	25.0000	74.6000	
2	MCT.f.0004	MCTs01	4	F	25.9000	25.0000	74.6000	
3	MCT.f.0005	MCTs01	5	F	25.9000	25.0000	74.6000	
4	MCT.f.0006	MCTs01	6	F	25.9000	25.0000	74.6000	

5 rows × 27 columns



```
In [ ]: # Dataset overview

print("Shape:", df.shape)
```

```
print("\nColumns:")
print(df.columns.tolist())
```

Shape: (475, 27)

Columns:

```
['fecal_sample_id', 'participant_id', 'study_day', 'Gender', 'Age', 'BMI', 'Weight', 'Supplement', 'Medications', 'fme_score_daily', 'shannon_diversity', 'participant_shannon_cv', 'participant_shannon_mean', 'participant_n_samples', 'KCAL', 'PROT', 'TFAT', 'CARB', 'FIBE', 'SUGR', 'SODI', 'D_TOTAL', 'D_YOGURT', 'D_CHEESE', 'PF_MEAT', 'PF_SEAFD_HI', 'G_WHOLE']
```

In []: *# Summary statistics*

```
df.describe()
```

Out[]:

	study_day	Age	BMI	Weight	fme_score_daily	shannon_diversity	par
--	-----------	-----	-----	--------	-----------------	-------------------	-----

count	475.0000	475.0000	475.0000	475.0000	475.0000	475.0000	
--------------	----------	----------	----------	----------	----------	----------	--

mean	8.9516	31.1522	23.1747	69.0691	73.3252	2.8646	
-------------	--------	---------	---------	---------	---------	--------	--

std	4.9397	10.1565	3.2921	14.6469	58.3071	0.4132	
------------	--------	---------	--------	---------	---------	--------	--

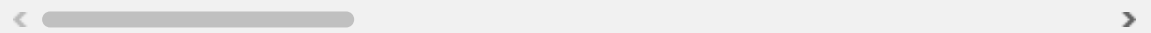
min	1.0000	19.7000	17.0000	48.5000	0.0000	1.5717	
------------	--------	---------	---------	---------	--------	--------	--

25%	5.0000	23.9000	21.1000	59.1000	29.3222	2.6108	
------------	--------	---------	---------	---------	---------	--------	--

50%	9.0000	28.5000	22.9000	66.9000	61.7775	2.9644	
------------	--------	---------	---------	---------	---------	--------	--

75%	13.0000	37.0000	25.9000	82.9000	104.2697	3.1430	
------------	---------	---------	---------	---------	----------	--------	--

max	17.0000	61.6000	31.6000	104.8000	331.2240	3.7310	
------------	---------	---------	---------	----------	----------	--------	--



In []: `print(df.isnull().sum())`

```
fecal_sample_id      0
participant_id       0
study_day            0
Gender               0
Age                 0
BMI                 0
Weight              0
Supplement           0
Medications          255
fme_score_daily      0
shannon_diversity    0
participant_shannon_cv 0
participant_shannon_mean 0
participant_n_samples 0
KCAL                0
PROT                0
TFAT                0
CARB                0
FIBE                0
SUGR                0
SODI                0
D_TOTAL             0
D_YOGURT            0
D_CHEESE            0
PF_MEAT             0
PF_SEAFD_HI         0
G_WHOLE             0
dtype: int64
```

Correlation Analysis

In this section, we investigate the relationship between Food Microbiome Exposure (FME) and microbiome diversity metrics.

We begin by examining the correlation between the daily FME score and Shannon diversity, which is a commonly used measure of microbiome diversity.

```
In [ ]: # Correlation between FME and Shannon Diversity
```

```
correlation, p_value = pearsonr(
    df["fme_score_daily"],
    df["shannon_diversity"]
)

print(f"Pearson Correlation: {correlation:.4f}")
print(f"P-value: {p_value:.4f}")
```

```
Pearson Correlation: -0.0406
P-value: 0.3776
```

Visualization of FME and Shannon Diversity

The following scatter plot illustrates the relationship between daily Food Microbiome Exposure (FME) scores and Shannon diversity values across all samples.

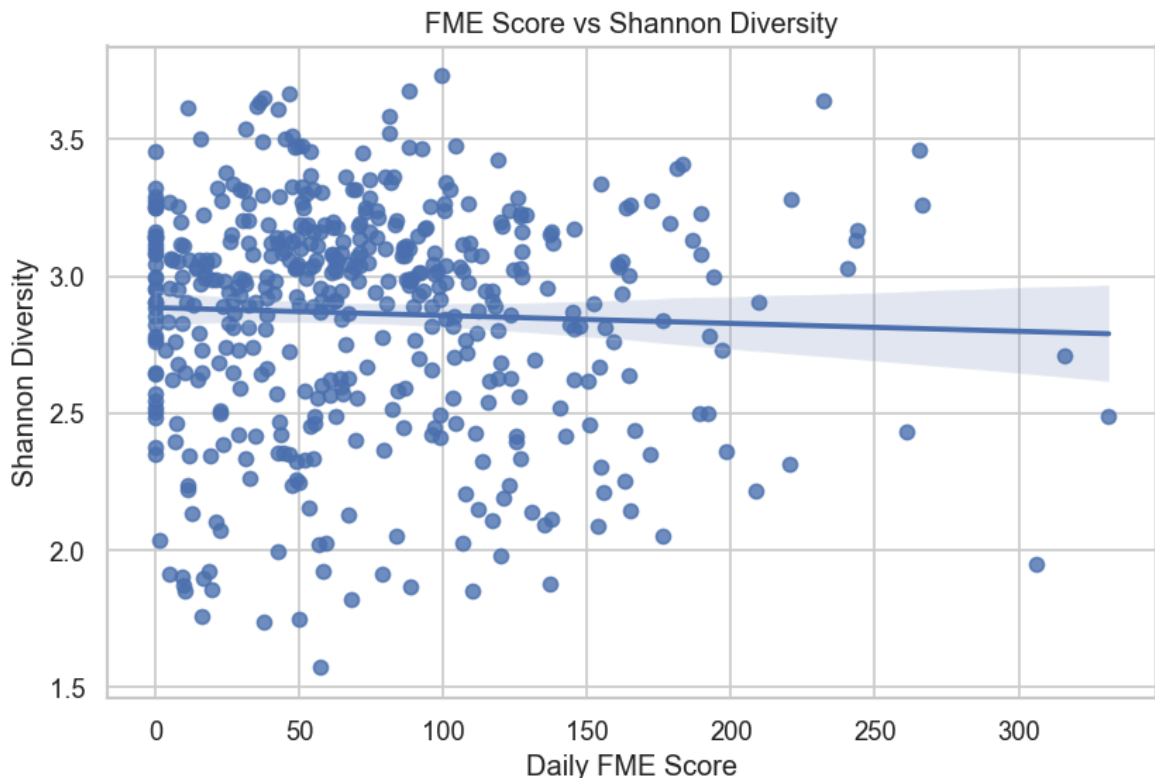
A regression line is added to help visualize the overall trend.

```
In [ ]: plt.figure(figsize=(8, 5))

sns.regplot(
    data=df,
    x="fme_score_daily",
    y="shannon_diversity"
)

plt.title("FME Score vs Shannon Diversity")
plt.xlabel("Daily FME Score")
plt.ylabel("Shannon Diversity")

plt.show()
```



Insight

The relationship between FME score and Shannon diversity appears to be weak.

Although FME values vary considerably across samples, Shannon diversity remains relatively stable.

The nearly flat regression line suggests little to no linear association between the two variables.

Correlation Matrix

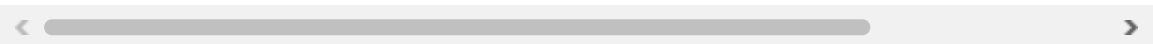
To better understand the relationships between microbiome-related variables, we compute a correlation matrix for the main numerical variables used in this analysis.

```
In [ ]: correlation_columns = [
    "fme_score_daily",
```

```
"shannon_diversity",  
"participant_shannon_cv",  
"participant_shannon_mean"  
]  
  
corr_matrix = df[correlation_columns].corr()  
  
corr_matrix
```

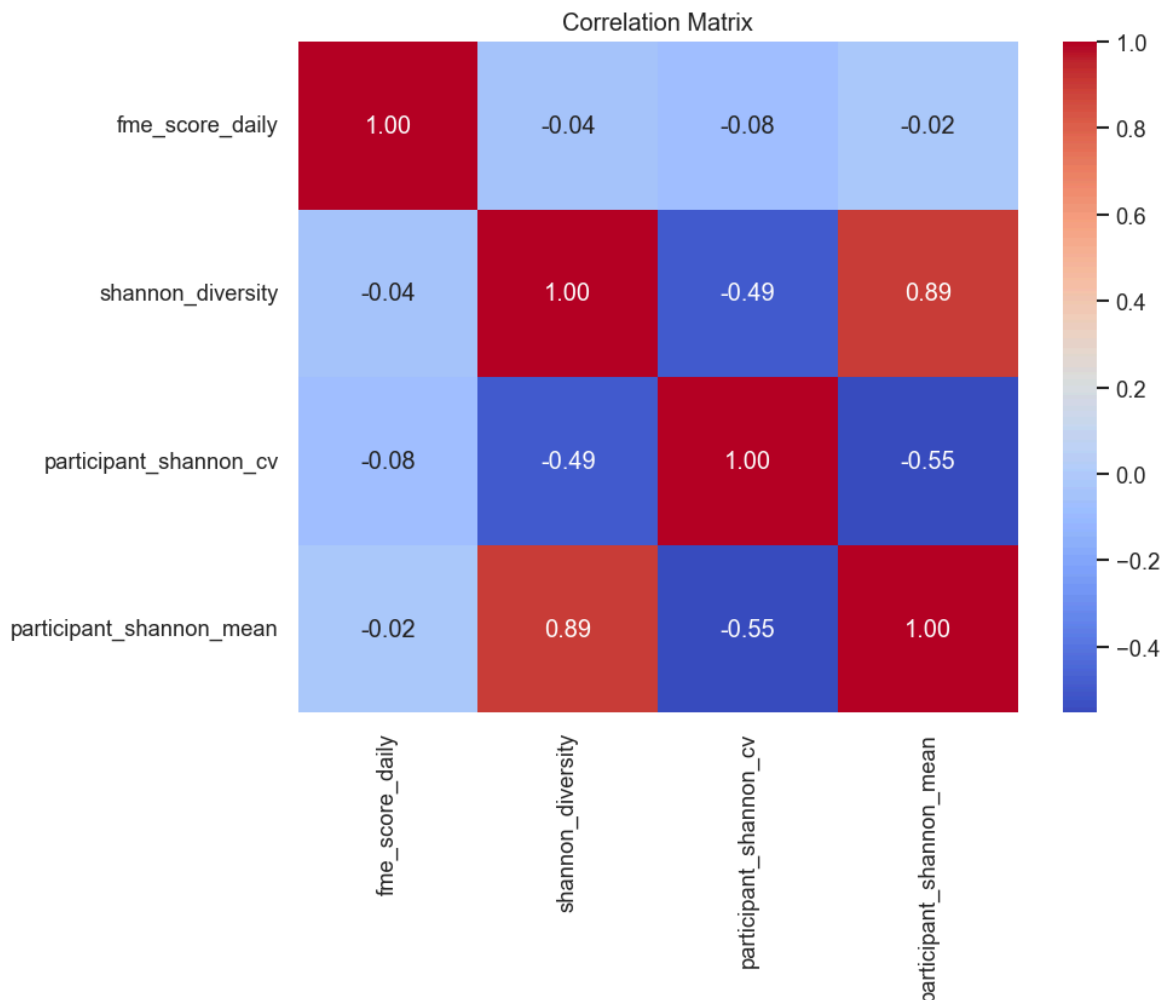
Out []:

	fme_score_daily	shannon_diversity	participant_shannon_cv	participant_shannon_mean
fme_score_daily	1.0000	-0.0406	-0.0771	-0.0153
shannon_diversity	-0.0406	1.0000	-0.4929	0.8917
participant_shannon_cv	-0.0771	-0.4929	1.0000	-0.5527
participant_shannon_mean	-0.0153	0.8917	-0.5527	



In []:

```
plt.figure(figsize=(8, 6))  
  
sns.heatmap(  
    corr_matrix,  
    annot=True,  
    cmap="coolwarm",  
    fmt=".2f"  
)  
  
plt.title("Correlation Matrix")  
plt.show()
```



Insight

FME score has very weak correlations with all diversity metrics.

Shannon diversity is strongly correlated with participant mean Shannon diversity.

Comparison Between Low and High FME Groups

To further investigate the relationship between Food Microbiome Exposure (FME) and microbiome diversity, samples were divided into two groups based on the median FME score.

Shannon diversity distributions were then compared between the groups.

```
In [ ]: # Create FME groups based on median value

median_fme = df["fme_score_daily"].median()

df["fme_group"] = np.where(
    df["fme_score_daily"] >= median_fme,
    "High FME",
    "Low FME"
)

df["fme_group"].value_counts()
```

```
Out[ ]: fme_group
High FME    238
Low FME     237
Name: count, dtype: int64
```

Shannon Diversity by FME Group

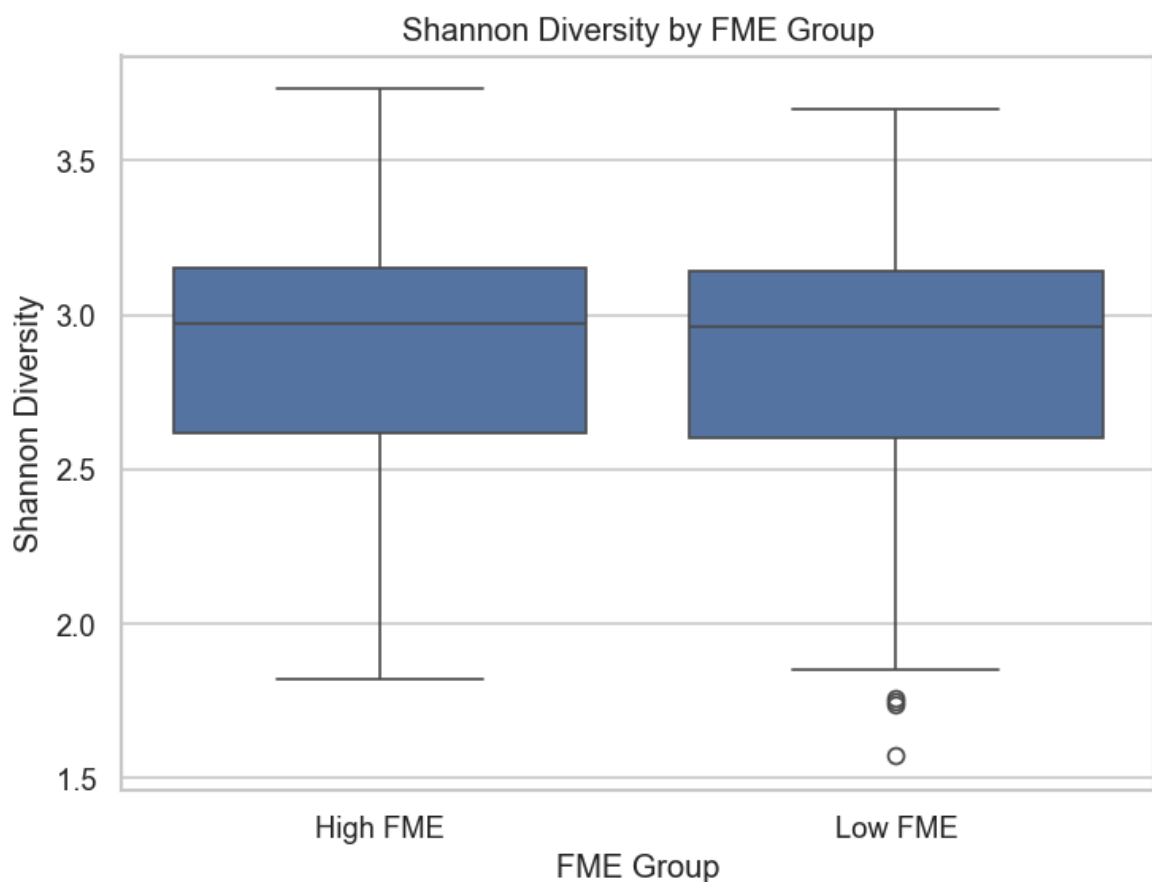
The following boxplot compares Shannon diversity values between samples with low and high Food Microbiome Exposure.

```
In [ ]: plt.figure(figsize=(7,5))

sns.boxplot(
    data=df,
    x="fme_group",
    y="shannon_diversity"
)

plt.title("Shannon Diversity by FME Group")
plt.xlabel("FME Group")
plt.ylabel("Shannon Diversity")

plt.show()
```



Insight

The distributions of Shannon diversity are similar in both FME groups.

No clear difference in diversity is observed between low and high FME exposure levels.

Statistical Test

A Mann–Whitney U test was performed to evaluate whether Shannon diversity differs significantly between the two FME groups.

```
In [ ]: high_fme = df[df["fme_group"] == "High FME"]["shannon_diversity"]
low_fme = df[df["fme_group"] == "Low FME"]["shannon_diversity"]

stat, p_value = mannwhitneyu(high_fme, low_fme)

print("Mann-Whitney U Statistic:", stat)
print("P-value:", p_value)
```

Mann-Whitney U Statistic: 28228.0

P-value: 0.9869319368980619

Insight

No significant difference was found between the low and high FME groups ($p = 0.987$).

This suggests that FME levels are not associated with differences in Shannon diversity in this dataset.

Adjusted Regression Analysis

To account for potential dietary confounders, an additional regression model was fitted including nutrient intake variables.

This allows evaluation of the association between FME and Shannon diversity while controlling for dietary factors.

```
In [ ]: model_adj = smf.ols(
    "shannon_diversity ~ fme_score_daily + KCAL + PROT + TFAT + CARB + FIBE",
    data=df
).fit()

print(model_adj.summary())
```

OLS Regression Results

=====						
Dep. Variable:	shannon_diversity	R-squared:	0.027			
Model:	OLS	Adj. R-squared:	0.014			
Method:	Least Squares	F-statistic:	2.152			
Date:	Wed, 24 Jun 2026	Prob (F-statistic):	0.0465			
Time:	00:54:07	Log-Likelihood:	-247.17			
No. Observations:	475	AIC:	508.3			
Df Residuals:	468	BIC:	537.5			
Df Model:	6					
Covariance Type:	nonrobust					
=====						
==						
	coef	std err	t	P> t	[0.025	0.97

--						
Intercept	2.7853	0.061	45.544	0.000	2.665	2.9
fme_score_daily	-6.297e-05	0.000	-0.156	0.876	-0.001	0.0
KCAL	0.0002	0.000	1.326	0.185	-9.51e-05	0.0
PROT	1.476e-05	0.001	0.015	0.988	-0.002	0.0
TFAT	-0.0023	0.001	-1.582	0.114	-0.005	0.0
CARB	-0.0001	0.001	-0.191	0.849	-0.002	0.0
FIBE	-0.0043	0.002	-1.715	0.087	-0.009	0.0
=====						
Omnibus:	29.179	Durbin-Watson:	0.518			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	33.191			
Skew:	-0.645	Prob(JB):	6.20e-08			
Kurtosis:	3.106	Cond. No.	7.32e+03			

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 7.32e+03. This might indicate that there are strong multicollinearity or other numerical problems.

Insight

After adjusting for nutrient intake variables, FME score remained a non-significant predictor of Shannon diversity ($p = 0.876$).

This suggests that the observed lack of association is not explained by differences in dietary intake.

Conclusions

Several statistical approaches were used to evaluate the relationship between Food Microbiome Exposure (FME) and gut microbiome diversity.

Main findings:

- The correlation between FME score and Shannon diversity was very weak.
- No significant differences were observed between low and high FME groups.
- Simple linear regression showed that FME score was not a significant predictor of Shannon diversity.
- After adjusting for dietary intake variables (KCAL, PROT, TFAT, CARB, and FIBE), FME score remained non-significant.
- Adjusting for nutrient intake did not change the overall findings.

Overall, no statistically significant association was found between FME levels and Shannon diversity in this dataset.

Food Microbiome Exposure (FME) Analysis

Research Question

How does exposure to food-associated microbes influence the composition and stability of the human gut microbiome?

What this notebook does

1. **Data Transfer** — Load all preprocessed datasets from both sources
2. **Data Mapping** — Map dietary food items to cFMD food categories
3. **FME Quantification** — Calculate a daily Food Microbiome Exposure score per fecal sample
4. **Statistical Dataset** — Assemble one clean table ready for statistical modelling

Key structural note

The Johnson et al. 2019 study has **34 participants**, each contributing multiple fecal samples across up to 17 study days. All FME scores are computed at the **fecal sample level** (one score per sample = one dietary day per person). Participant-level summaries are derived by aggregating across samples — **not** by treating samples as independent observations. The unit of analysis for statistical models is the **fecal sample**, with participant included as a random effect.

Section 2 – Data Transfer: Load All Datasets

Source 1 — Johnson et al. 2019 (gut microbiome + diet)

34 participants · up to 17 fecal samples each · 566 samples total

Source 2 — Carlino et al. 2024 — cFMD (food microbiome)

3,393 food samples · 5,057 microbial taxa

```
In [ ]: # Source 1: Gut microbiome matrices (taxa × fecal samples) —
gut_microbiome_abundance = pd.read_csv(DATA_DIR / "microbiome_filtered.csv", in

# Source 1: Fecal sample metadata
# index = fecal sample ID (MCT.f.XXXX)
# columns: UserName (= participant), StudyDayNo, Gender, Age, BMI, ...
fecal_sample_metadata = pd.read_csv(DATA_DIR / "sample_meta.csv", in

# Source 1: Dietary consumption matrix (food items × fecal samples) —
# rows = food items, columns = fecal sample IDs (MCT.f.XXXX)
# values = consumption amount of that food on that day
dietary_consumption_matrix = pd.read_csv(DATA_DIR / "diet_raw.csv")

# Source 1: Daily nutrition totals —
# index = fecal sample ID
daily_nutrition_totals = pd.read_csv(DATA_DIR / "nutrition_totals.csv", in

# Source 1: Additional dietary features
daily_fiber_scores = pd.read_csv(DATA_DIR / "diet_fiber.csv")
daily_decay_scores = pd.read_csv(DATA_DIR / "diet_decay.csv")
participant_fiber_avg = pd.read_csv(DATA_DIR / "fiber_avg.csv")

# Source 2: cFMD food microbiome abundance (food taxa × food samples)
# columns: 'StudyName__ShortSampleID' format
cfmd_food_abundance = pd.read_csv(DATA_DIR / "combined_datasets.csv", in
cfmd_food_abundance = cfmd_food_abundance.apply(pd.to_numeric, errors="coe

# Source 2: cFMD sample metadata —
# columns: sample_id, category, fermented/non-fermented, country, ...
cfmd_food_metadata = pd.read_csv(DATA_DIR / "cfmd_metadata.csv")

# Verify participant count —
n_participants = fecal_sample_metadata["UserName"].nunique()
n_fecal_samples = len(fecal_sample_metadata)

print(f"Participants      : {n_participants} (expected: 34)")
print(f"Fecal samples total  : {n_fecal_samples}")
print(f"Avg samples/person    : {n_fecal_samples / n_participants:.1f}")
print()
print("Dataset sizes:")
for name, df in [
    ("gut_microbiome_abundance", gut_microbiome_abundance),
    ("fecal_sample_metadata", fecal_sample_metadata),
    ("dietary_consumption_matrix", dietary_consumption_matrix),
    ("daily_nutrition_totals", daily_nutrition_totals),
    ("cfmd_food_abundance", cfmd_food_abundance),
    ("cfmd_food_metadata", cfmd_food_metadata),
]:
    print(f"  {name:<32} {str(df.shape)}")
```

Participants : 34 (expected: 34)
 Fecal samples total : 566
 Avg samples/person : 16.6

Dataset sizes:

gut_microbiome_abundance	(223, 483)
fecal_sample_metadata	(566, 28)
dietary_consumption_matrix	(1452, 568)
daily_nutrition_totals	(580, 106)
cfmd_food_abundance	(5057, 3393)
cfmd_food_metadata	(3444, 32)

Section 3 – Define the Analytical Sample Set

Only fecal samples that have **both** a gut microbiome measurement **and** a dietary record form the analytical core. We identify these 475 samples here.

```
In [ ]: # Identify fecal sample IDs present in both gut microbiome and diet data
FECAL_SAMPLE_PREFIX = "MCT.f."

gut_microbiome_sample_ids = set(gut_microbiome_abundance.columns)
diet_record_sample_ids = set(
    c for c in dietary_consumption_matrix.columns
    if c.startswith(FECAL_SAMPLE_PREFIX)
)

# Core analytical set: samples with BOTH gut microbiome AND diet record
analytical_sample_ids = sorted(gut_microbiome_sample_ids & diet_record_sample_ids)

print(f"Gut microbiome samples : {len(gut_microbiome_sample_ids)}")
print(f"Diet record samples : {len(diet_record_sample_ids)}")
print(f"Analytical samples (gutndiet) : {len(analytical_sample_ids)}")
print()

# Participants represented in the analytical set
analytical_participants = (
    fecal_sample_metadata
    .loc[fecal_sample_metadata.index.isin(analytical_sample_ids), "UserName"]
    .nunique()
)
print(f"Participants represented : {analytical_participants} / {n_particip

# Samples per participant in the analytical set
samples_per_participant = (
    fecal_sample_metadata
    .loc[fecal_sample_metadata.index.isin(analytical_sample_ids)]
    .groupby("UserName")
    .size()
    .rename("n_analytical_samples")
)
print(f"Avg analytical samples/person : {samples_per_participant.mean():.1f}")
print(f"Min / Max : {samples_per_participant.min()} / {samps
```

Gut microbiome samples	: 483
Diet record samples	: 566
Analytical samples (gutndiet)	: 475
Participants represented	: 34 / 34
Avg analytical samples/person	: 14.0
Min / Max	: 6 / 17

Section 4 – FME Weights from cFMD

Goal: Derive a microbial richness weight for each cFMD food category.

Why richness, not total abundance? The `combined_datasets` matrix stores relative abundance — each sample sums to 100 regardless of food type. Total abundance is therefore uninformative as a weight. **Microbial richness** (= number of distinct taxa detected per food sample) reflects how microbiome-diverse each food category truly is.

Weight formula:

$$w_c = \frac{\text{mean_richness}_c - \min}{\max - \min} \in [0, 1]$$

Weights are derived directly from the **cFMD food abundance matrix** — not assumed or hand-coded.

```
In [ ]: # Step 1: Map cFMD column names → food category + fermentation status
# combined_datasets columns: 'StudyName__ShortSampleID'
# cfmd_food_metadata sample_id: 'ShortSampleID'

def extract_cfmd_short_id(column_name):
    """Strip the study name prefix from a combined_datasets column name."""
    return column_name.split("__")[-1] if "__" in column_name else column_name

# Remove the one duplicate sample_id (VB15-29) before indexing
cfmd_food_metadata_dedup = cfmd_food_metadata.drop_duplicates(
    subset="sample_id", keep="first"
)
print(f"CFMD metadata after dedup: {len(cfmd_food_metadata_dedup)} rows "
      f"(removed {len(cfmd_food_metadata) - len(cfmd_food_metadata_dedup)} dupli

cfmd_id_to_meta = (
    cfmd_food_metadata_dedup
    .set_index("sample_id")[["category", "fermented/non-fermented"]]
    .to_dict(orient="index")
)

# Map each column in cfmd_food_abundance to its category and fermentation status
cfmd_column_to_category = {}
cfmd_column_to_fermented = {}
for col in cfmd_food_abundance.columns:
    short_id = extract_cfmd_short_id(col)
    if short_id in cfmd_id_to_meta:
        cfmd_column_to_category[col] = cfmd_id_to_meta[short_id]["category"]
        cfmd_column_to_fermented[col] = cfmd_id_to_meta[short_id]["fermented/non-
```

```
print(f"cFMD columns mapped: {len(cfmd_column_to_category)} / {cfmd_food_abundan
```

cFMD metadata after dedup: 3443 rows (removed 1 duplicate)

cFMD columns mapped: 3393 / 3393

```
In [ ]: # Step 2: Compute microbial richness per food sample -
# Richness = number of non-zero taxa per food sample
cfmd_taxa_richness_per_sample = (cfmd_food_abundance > 0).sum(axis=0)

# Step 3: Mean richness per cFMD food category -
category_richness_values = {}
for col, category in cfmd_column_to_category.items():
    if col in cfmd_taxa_richness_per_sample.index:
        category_richness_values.setdefault(category, []).append(
            cfmd_taxa_richness_per_sample[col]
        )

cfmd_category_mean_richness = pd.Series(
    {cat: np.mean(vals) for cat, vals in category_richness_values.items()},
    name="mean_taxa_richness"
).sort_values(ascending=False)

# Step 4: Normalize weights to [0, 1]
richness_min = cfmd_category_mean_richness.min()
richness_max = cfmd_category_mean_richness.max()
cfmd_category_weight_normalized = (
    (cfmd_category_mean_richness - richness_min) /
    (richness_max - richness_min)
).rename("fme_weight")

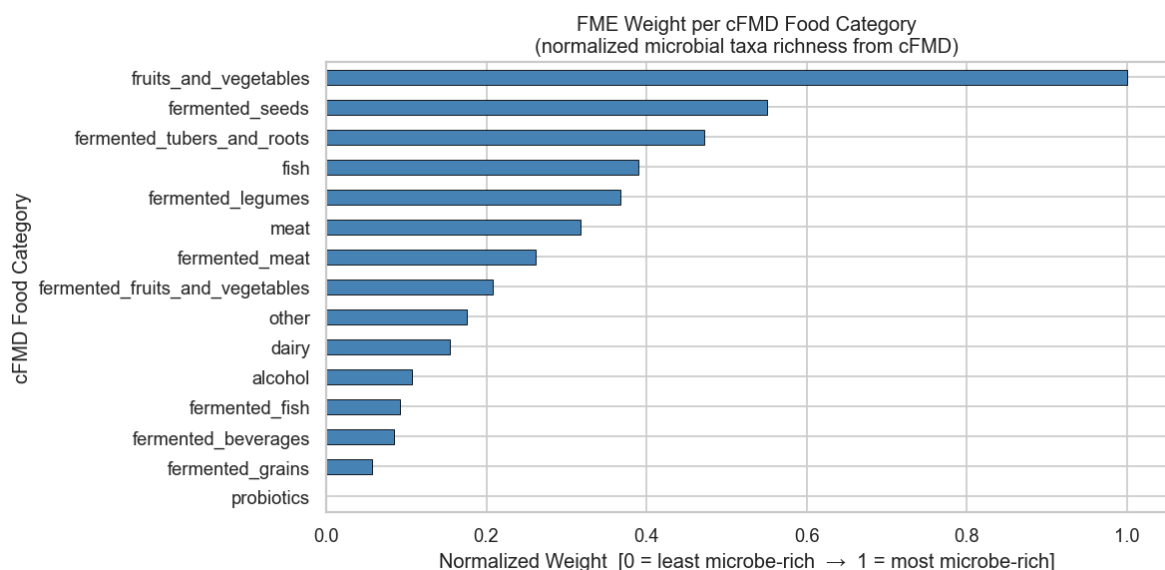
cfmd_weights_table = pd.concat(
    [cfmd_category_mean_richness, cfmd_category_weight_normalized], axis=1
)

print("FME weights per cFMD food category (from cFMD microbial richness):")
display(cfmd_weights_table.round(3))
```

FME weights per cFMD food category (from cFMD microbial richness):

	mean_taxa_richness	fme_weight
fruits_and_vegetables	120.1250	1.0000
fermented_seeds	69.8000	0.5510
fermented_tubers_and_roots	61.0420	0.4720
fish	51.7950	0.3900
fermented_legumes	49.2630	0.3670
meat	43.6490	0.3170
fermented_meat	37.4110	0.2610
fermented_fruits_and_vegetables	31.4810	0.2080
other	27.8750	0.1760
dairy	25.3960	0.1540
alcohol	20.0850	0.1070
fermented_fish	18.4000	0.0920
fermented_beverages	17.6160	0.0850
fermented_grains	14.5120	0.0570
probiotics	8.1430	0.0000

```
In [ ]: # Visualize FME weights -
plt.figure(figsize=(10, 5))
cfmd_category_weight_normalized.sort_values().plot(
    kind="barh", color="steelblue", edgecolor="black", linewidth=0.5
)
plt.title("FME Weight per cFMD Food Category\n(normalized microbial taxa richness
plt.xlabel("Normalized Weight [0 = least microbe-rich → 1 = most microbe-rich
plt.ylabel("cFMD Food Category")
plt.tight_layout()
plt.show()
```



Section 5 – Data Mapping: Diet Items → cFMD Food Categories

Goal: Map each food item in the Johnson et al. dietary records to a cFMD food category, creating a unified representation of food-derived microbial exposure.

Method: Extract the L1 (top-level) taxonomy label from the `taxonomy` column of `diet_raw` (e.g. `L1_Milk_and_Milk_Products` → `dairy`) and map it to the matching cFMD category.

```
In [ ]: # Extract L1 food category from taxonomy string
def extract_l1_food_category(taxonomy_string):
    """
    Extract the top-level food category (L1) from the hierarchical taxonomy string
    Example: 'L1_Milk_and_Milk_Products;L2_...' → 'milk and milk products'
    """
    if pd.isna(taxonomy_string):
        return "unknown"
    l1_segment = str(taxonomy_string).split(";")[0]
    return l1_segment.replace("L1_", "").replace("_", " ").strip().lower()

dietary_consumption_matrix["diet_l1_category"] = (
    dietary_consumption_matrix["taxonomy"]
    .apply(extract_l1_food_category)
)

print("Unique L1 diet categories found:")
for cat, count in dietary_consumption_matrix["diet_l1_category"].value_counts():
    print(f" {cat:<45} {count:>5} food items")
```

Unique L1 diet categories found:

grain product	407 food items
vegetables	315 food items
meat poultry fish and mixtures	240 food items
sugars sweets and beverages	153 food items
milk and milk products	112 food items
dry beans peas other legumes nuts and seeds	84 food items
fruits	74 food items
fats oils and salad dressings	36 food items
eggs	31 food items

```
In [ ]: # Mapping table: diet L1 category → cFMD category
DIET_L1_TO_CFMD_CATEGORY = {
    # Dairy
    "milk and milk products" : "dairy",
    "dairy" : "dairy",
    "cheese" : "dairy",
    "yogurt" : "dairy",
    # Meat
    "meat poultry and game" : "meat",
    "meat" : "meat",
    "poultry" : "meat",
    # Fish
    "fish and shellfish" : "fish",
    "fish" : "fish",
    "seafood" : "fish",
    # Fruits and vegetables
```

```

"vegetables and vegetable products" : "fruits_and_vegetables",
"fruit"                             : "fruits_and_vegetables",
"fruits"                            : "fruits_and_vegetables",
"vegetables"                        : "fruits_and_vegetables",
# Fermented grains
"grain products"                    : "fermented_grains",
"grains"                           : "fermented_grains",
"bread"                             : "fermented_grains",
"cereals and cereal products"       : "fermented_grains",
# Alcohol / fermented beverages
"alcoholic beverages"               : "alcohol",
"alcohol"                           : "alcohol",
"beverages"                         : "fermented_beverages",
"nonalcoholic beverages"            : "fermented_beverages",
# Legumes
"legumes and legume products"       : "fermented_legumes",
"legumes"                           : "fermented_legumes",
"beans"                             : "fermented_legumes",
# Seeds / nuts
"nuts and seeds"                    : "fermented_seeds",
"seeds"                             : "fermented_seeds",
# Other / unmappable
"fats and oils"                     : "other",
"sugars and sweets"                 : "other",
"soups sauces and gravies"          : "other",
"mixed dishes"                      : "other",
"snacks"                           : "other",
"eggs"                             : "other",
"spices and herbs"                  : "other",
"unknown"                           : "unmapped",
}

dietary_consumption_matrix["cfmd_category"] = (
    dietary_consumption_matrix["diet_l1_category"]
    .map(DIET_L1_TO_CFMD_CATEGORY)
    .fillna("unmapped")
)

n_mapped    = (dietary_consumption_matrix["cfmd_category"] != "unmapped").sum()
n_unmapped  = (dietary_consumption_matrix["cfmd_category"] == "unmapped").sum()
print(f"Food items mapped to a cFMD category : {n_mapped} / {len(dietary_consump
print(f"Food items unmapped                    : {n_unmapped}")

```

Food items mapped to a cFMD category : 532 / 1452

Food items unmapped : 920

```

In [ ]: # Attach FME weight to each food item
dietary_consumption_matrix["fme_weight"] = (
    dietary_consumption_matrix["cfmd_category"]
    .map(cfmd_category_weight_normalized.to_dict())
    .fillna(0.0) # unmapped items get weight = 0
)

mapping_summary = (
    dietary_consumption_matrix
    .groupby(["diet_l1_category", "cfmd_category", "fme_weight"])
    .size()
    .reset_index(name="n_food_items")
    .sort_values("n_food_items", ascending=False)
)

```

```
print("Mapping summary (diet L1 → cFMD category → weight):")
display(mapping_summary)
```

Mapping summary (diet L1 → cFMD category → weight):

	diet_l1_category	cfmd_category	fme_weight	n_food_items
4	grain product	unmapped	0.0000	407
8	vegetables	fruits_and_vegetables	1.0000	315
5	meat poultry fish and mixtures	unmapped	0.0000	240
7	sugars sweets and beverages	unmapped	0.0000	153
6	milk and milk products	dairy	0.1541	112
0	dry beans peas other legumes nuts and seeds	unmapped	0.0000	84
3	fruits	fruits_and_vegetables	1.0000	74
2	fats oils and salad dressings	unmapped	0.0000	36
1	eggs	other	0.1762	31

Section 6 – FME Score Calculation

Formula:

$$\text{FME}_{\text{sample}} = \sum_{\text{food items}} \text{consumption}_{\text{food, sample}} \times w_{\text{cfmd_category}(\text{food})}$$

Where:

- $\text{consumption}_{\text{food, sample}}$ = amount consumed (from `diet_raw`, grams or servings)
- w = normalized cFMD microbial richness weight for the food's category

Important: Each `sample` corresponds to **one dietary day for one participant**. The result is a score per fecal sample — not per participant. Participant-level scores are derived later by averaging across their samples.

```
In [ ]: # Subset diet matrix to analytical samples only
diet_abundance_matrix = (
    dietary_consumption_matrix[analytical_sample_ids]
    .apply(pd.to_numeric, errors="coerce")
    .fillna(0.0)
)

fme_weight_vector = dietary_consumption_matrix["fme_weight"].values # shape: (n

# Compute FME score: weighted sum of food consumption per sample -
# Result: one FME score per fecal sample (= one dietary day per participant)
fme_score_per_sample = (
    diet_abundance_matrix
    .multiply(fme_weight_vector, axis=0)
    .sum(axis=0)
```

```

        .rename("fme_score_daily")
    )
    fme_score_per_sample.index.name = "fecal_sample_id"

    print(f"FME scores computed for : {len(fme_score_per_sample)} fecal samples")
    print(f"Score range           : {fme_score_per_sample.min():.4f} - {fme_score_per_sample.max():.4f}")
    print(f"Score mean ± std       : {fme_score_per_sample.mean():.4f} ± {fme_score_per_sample.std():.4f}")
    print(f"Non-zero scores        : {(fme_score_per_sample > 0).sum()}")
    print()
    display(fme_score_per_sample.reset_index().head(8))

```

```

FME scores computed for : 475 fecal samples
Score range           : 0.0000 - 331.2240
Score mean ± std       : 73.3252 ± 58.3071
Non-zero scores        : 437

```

	fecal_sample_id	fme_score_daily
0	MCT.f.0002	67.1961
1	MCT.f.0003	81.2412
2	MCT.f.0004	126.7711
3	MCT.f.0005	31.4846
4	MCT.f.0006	61.2266
5	MCT.f.0007	62.0231
6	MCT.f.0008	70.9874
7	MCT.f.0009	54.0086

```

In [ ]: # Participant-Level FME summary (aggregate across samples)
        # Note: samples are NOT independent – they come from the same 34 participants.
        # Participant-Level values are used ONLY for descriptive summaries,
        # NOT as independent data points in statistical models.

        fme_with_participant = pd.DataFrame({
            "fecal_sample_id" : fme_score_per_sample.index,
            "fme_score_daily" : fme_score_per_sample.values,
        })
        fme_with_participant["participant_id"] = (
            fecal_sample_metadata
            .loc[fme_with_participant["fecal_sample_id"], "UserName"]
            .values
        )
        fme_with_participant["study_day"] = (
            fecal_sample_metadata
            .loc[fme_with_participant["fecal_sample_id"], "StudyDayNo"]
            .values
        )

        # Participant-Level summary (descriptive only)
        participant_fme_summary = (
            fme_with_participant
            .groupby("participant_id")["fme_score_daily"]
            .agg(
                fme_mean = "mean",
                fme_std   = "std",
            )

```

```

        fme_min      = "min",
        fme_max      = "max",
        n_diet_days  = "count"
    )
    .reset_index()
)

print("Participant-level FME summary (descriptive – 34 participants):")
display(participant_fme_summary.sort_values("fme_mean", ascending=False).head(10)

```

Participant-level FME summary (descriptive – 34 participants):

	participant_id	fme_mean	fme_std	fme_min	fme_max	n_diet_days
1	MCTs03	147.8899	45.7548	68.0651	243.3253	12
5	MCTs07	143.4859	48.3133	86.0332	243.8429	16
2	MCTs04	143.1190	81.0506	36.6199	331.2240	15
6	MCTs08	140.2178	33.6074	84.4009	220.3516	17
14	MCTs16	130.9598	45.2371	69.7540	221.0863	14
27	MCTs31	128.7335	37.7598	83.9027	208.7920	15
23	MCTs26	122.0334	71.3654	24.5678	315.7055	17
16	MCTs19	99.4541	40.7179	14.6490	166.4480	15
17	MCTs20	88.6724	51.9185	24.4219	178.8966	12
24	MCTs27	85.2074	47.0410	5.0752	165.2448	14

```

In [ ]: # Visualize FME scores
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# Distribution of daily FME scores across all fecal samples
axes[0].hist(fme_score_per_sample.values, bins=30,
             color="steelblue", edgecolor="white")
axes[0].axvline(fme_score_per_sample.mean(), color="red", linestyle="--",
               label=f"Mean = {fme_score_per_sample.mean():.2f}")
axes[0].set_title("Daily FME Score Distribution\n(475 fecal samples)")
axes[0].set_xlabel("FME Score")
axes[0].set_ylabel("Number of Samples")
axes[0].legend()

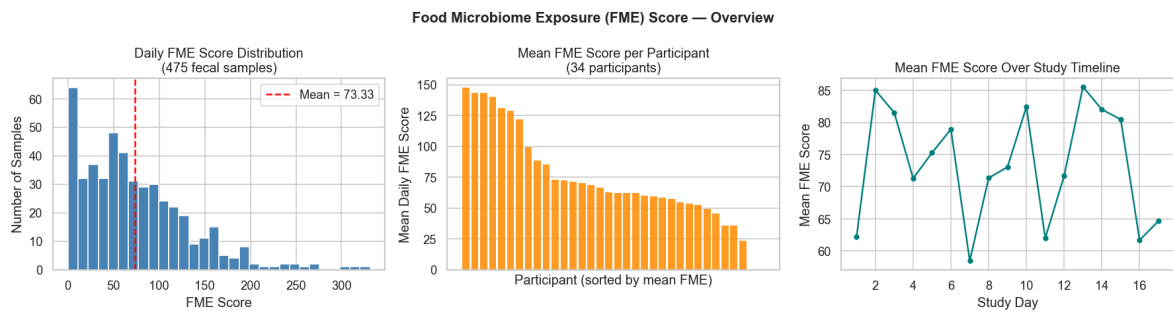
# Mean FME per participant (sorted)
sorted_participants = participant_fme_summary.sort_values("fme_mean", ascending=
axes[1].bar(range(len(sorted_participants)), sorted_participants["fme_mean"],
           color="darkorange", edgecolor="none", alpha=0.85)
axes[1].set_title(f"Mean FME Score per Participant\n({len(sorted_participants)})")
axes[1].set_xlabel("Participant (sorted by mean FME)")
axes[1].set_ylabel("Mean Daily FME Score")
axes[1].set_xticks([])

# FME over study timeline (mean per study day)
fme_by_day = fme_with_participant.groupby("study_day")["fme_score_daily"].mean()
axes[2].plot(fme_by_day.index, fme_by_day.values,
            marker="o", color="teal", linewidth=1.5, markersize=4)
axes[2].set_title("Mean FME Score Over Study Timeline")
axes[2].set_xlabel("Study Day")

```

```
axes[2].set_ylabel("Mean FME Score")

plt.suptitle("Food Microbiome Exposure (FME) Score – Overview", fontsize=13, fontcolor="red")
plt.tight_layout()
plt.show()
```



Section 7 – Gut Microbiome Alpha Diversity per Fecal Sample

Compute **Shannon entropy** for each fecal sample as a measure of gut microbial richness and evenness. This is the primary outcome variable (composition proxy) at the sample level.

```
In [ ]: # Shannon entropy per fecal sample
def compute_shannon_entropy(abundance_column):
    """Shannon entropy H = -Σ p_i * log(p_i) for non-zero taxa."""
    positive_values = abundance_column[abundance_column > 0]
    if positive_values.sum() == 0:
        return np.nan
    proportions = positive_values / positive_values.sum()
    return float(-np.sum(proportions * np.log(proportions)))

gut_microbiome_analytical = gut_microbiome_abundance[analytical_sample_ids]
shannon_diversity_per_sample = (
    gut_microbiome_analytical
    .apply(compute_shannon_entropy, axis=0)
    .rename("shannon_diversity")
)
shannon_diversity_per_sample.index.name = "fecal_sample_id"

print(f"Shannon diversity computed for : {shannon_diversity_per_sample.notna().sum()}")
print(f"Range : {shannon_diversity_per_sample.min():.3f} - {shannon_diversity_per_sample.max():.3f}")
print(f"Mean : {shannon_diversity_per_sample.mean():.3f}")
```

```
Shannon diversity computed for : 475 samples
Range : 1.572 - 3.731
Mean : 2.865
```

Section 8 – Gut Microbiome Stability per Participant

Stability = how consistent is a participant's gut microbiome across their sampling days?

We use the **coefficient of variation (CV)** of Shannon diversity across a participant's samples:

$$CV_{participant} = \frac{\sigma(\text{Shannon})}{\mu(\text{Shannon})}$$

Lower CV = more stable microbiome.

This is computed at the **participant level** (34 values) and joined back to the sample-level table as a per-participant attribute.

```
In [ ]: # Map each fecal sample to its participant
sample_to_participant = (
    fecal_sample_metadata
    .loc[analytical_sample_ids, "UserName"]
    .rename("participant_id")
)

shannon_with_participant = pd.DataFrame({
    "fecal_sample_id" : shannon_diversity_per_sample.index,
    "shannon_diversity": shannon_diversity_per_sample.values,
}).set_index("fecal_sample_id")
shannon_with_participant["participant_id"] = sample_to_participant

# Participant-level stability (CV of Shannon across their samples)
participant_gut_stability = (
    shannon_with_participant
    .groupby("participant_id")["shannon_diversity"]
    .agg(
        participant_shannon_mean = "mean",
        participant_shannon_std  = "std",
        participant_n_samples    = "count"
    )
    .assign(
        participant_shannon_cv = lambda df:
            df["participant_shannon_std"] / df["participant_shannon_mean"]
    )
    .reset_index()
)

print(f"Gut stability computed for {len(participant_gut_stability)} participants")
print(f"Shannon CV range : {participant_gut_stability['participant_shannon_cv'].
    f"{participant_gut_stability['participant_shannon_cv'].max():.4f}")
display(participant_gut_stability.sort_values("participant_shannon_cv").head(8))
```

Gut stability computed for 34 participants
 Shannon CV range : 0.0202 - 0.1799

	participant_id	participant_shannon_mean	participant_shannon_std	participant_n_samp
5	MCTs07	3.0731	0.0621	
17	MCTs20	3.2696	0.0736	
29	MCTs33	3.0863	0.0876	
11	MCTs13	3.1848	0.0942	
19	MCTs22	2.9985	0.0912	
7	MCTs09	3.0362	0.0948	
30	MCTs34	2.9806	0.1022	
8	MCTs10	3.0279	0.1045	

Section 9 – Assemble the Statistical Analysis Dataset

Merge all variables into one clean table at the **fecal sample level**. Each row = one fecal sample = one dietary day for one participant.

Column group	Columns	Role in statistical model
Identity	fecal_sample_id , participant_id , study_day	Grouping / random effect
Demographics	Age , BMI , Gender	Fixed-effect covariates
FME score	fme_score_daily	Main predictor
Gut diversity	shannon_diversity	Main outcome
Gut stability	participant_shannon_cv	Secondary outcome
Nutrition	KCAL , FIBE , PROT , TFAT , CARB	Dietary confounders

```
In [ ]: # Step 1: Base table – fecal sample IDs
statistical_analysis_dataset = pd.DataFrame(
    index=pd.Index(analytical_sample_ids, name="fecal_sample_id")
)

# Step 2: Participant identity and demographics
participant_meta_cols = ["UserName", "StudyDayNo", "Gender", "Age", "BMI",
                        "Weight", "Supplement", "Medications"]
participant_meta_cols = [c for c in participant_meta_cols
                        if c in fecal_sample_metadata.columns]

statistical_analysis_dataset = statistical_analysis_dataset.join(
    fecal_sample_metadata[participant_meta_cols]
    .rename(columns={"UserName": "participant_id", "StudyDayNo": "study_day"}),
    how="left"
```



```

)

# Step 3: Daily FME score
statistical_analysis_dataset = statistical_analysis_dataset.join(
    fme_score_per_sample.rename("fme_score_daily"),
    how="left"
)

# Step 4: Gut alpha diversity (Shannon)
statistical_analysis_dataset = statistical_analysis_dataset.join(
    shannon_diversity_per_sample.rename("shannon_diversity"),
    how="left"
)

# Step 5: Participant-level gut stability (CV) -
statistical_analysis_dataset = statistical_analysis_dataset.join(
    participant_gut_stability.set_index("participant_id")[
        ["participant_shannon_cv", "participant_shannon_mean", "participant_n_sa
    ],
    on="participant_id",
    how="left"
)

# Step 6: Daily nutrition totals -
nutrition_cols_keep = ["KCAL", "PROT", "TFAT", "CARB", "FIBE",
                        "SUGR", "SODI", "D_TOTAL", "D_YOGURT", "D_CHEESE",
                        "PF_MEAT", "PF_SEAFD_HI", "G_WHOLE"]
nutrition_cols_keep = [c for c in nutrition_cols_keep
                        if c in daily_nutrition_totals.columns]

statistical_analysis_dataset = statistical_analysis_dataset.join(
    daily_nutrition_totals[nutrition_cols_keep],
    how="left"
)

print(f"Statistical dataset shape : {statistical_analysis_dataset.shape}")
print(f"Fecal samples (rows)       : {statistical_analysis_dataset.shape[0]}")
print(f"Variables (columns)         : {statistical_analysis_dataset.shape[1]}")
print(f"Participants represented     : {statistical_analysis_dataset['participant_
print(f"Complete rows (no NaN)    : {statistical_analysis_dataset.dropna().shap

```

```

Statistical dataset shape : (475, 26)
Fecal samples (rows)      : 475
Variables (columns)       : 26
Participants represented   : 34
Complete rows (no NaN)    : 220

```

```

In [ ]: # Missing value report
missing_report = (
    statistical_analysis_dataset.isnull().sum()
    .reset_index()
    .rename(columns={"index": "variable", 0: "n_missing"})
)
missing_report["pct_missing"] = (
    missing_report["n_missing"] / len(statistical_analysis_dataset) * 100
).round(1)
missing_report = missing_report[missing_report["n_missing"] > 0].sort_values("n_

if missing_report.empty:
    print("No missing values in the statistical dataset.")

```

```
else:
    print("Missing values:")
    display(missing_report)
```

Missing values:

	variable	n_missing	pct_missing
7	Medications	255	53.7000

```
In [ ]: # Descriptive statistics
key_vars = ["fme_score_daily", "shannon_diversity",
            "participant_shannon_cv", "Age", "BMI", "KCAL", "FIBE"]
key_vars = [v for v in key_vars if v in statistical_analysis_dataset.columns]

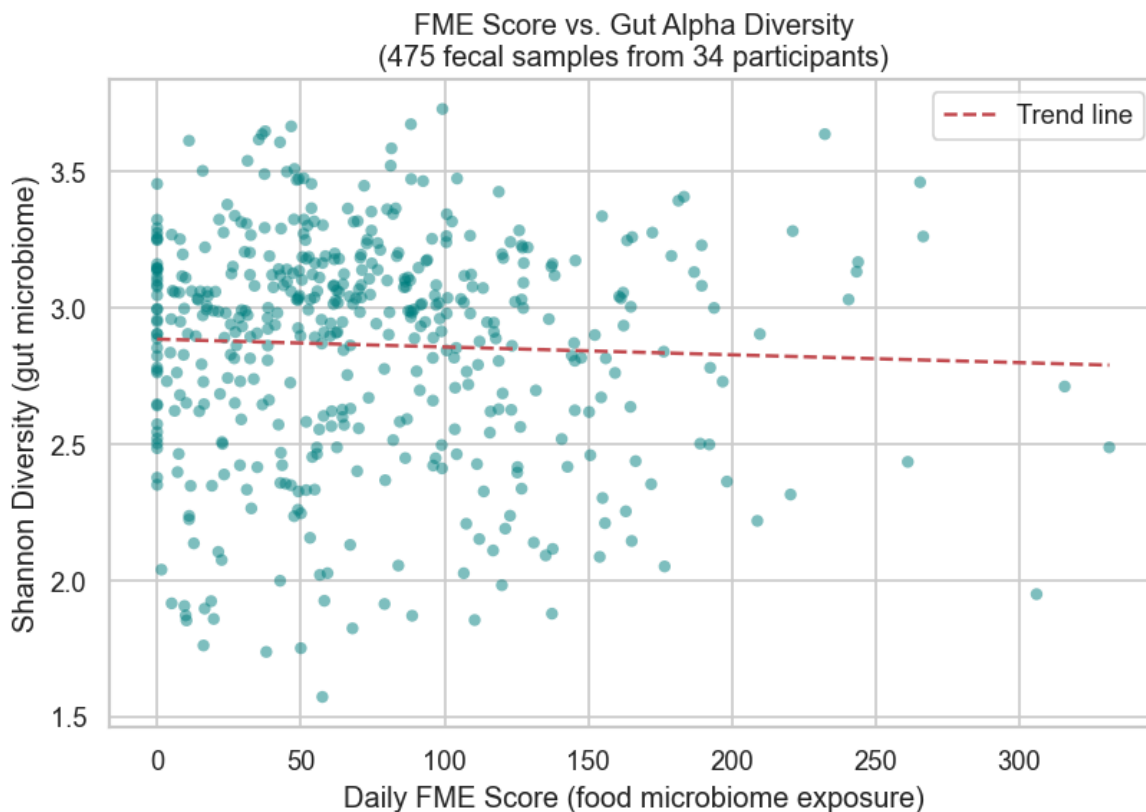
print("Descriptive statistics – key variables:")
display(statistical_analysis_dataset[key_vars].describe().T.round(3))
```

Descriptive statistics – key variables:

	count	mean	std	min	25%	50%	
fme_score_daily	475.0000	73.3250	58.3070	0.0000	29.3220	61.7780	10
shannon_diversity	475.0000	2.8650	0.4130	1.5720	2.6110	2.9640	
participant_shannon_cv	475.0000	0.0630	0.0380	0.0200	0.0360	0.0490	
Age	475.0000	31.1520	10.1560	19.7000	23.9000	28.5000	
BMI	475.0000	23.1750	3.2920	17.0000	21.1000	22.9000	
KCAL	475.0000	2110.5630	735.4510	722.5520	1599.1800	2028.1680	24
FIBE	475.0000	22.3370	11.6000	3.6180	14.8500	19.6940	

```
In [ ]: # Quick scatter: FME score vs. gut Shannon diversity
plot_df = statistical_analysis_dataset.dropna(
    subset=["fme_score_daily", "shannon_diversity"]
)

plt.figure(figsize=(7, 5))
plt.scatter(plot_df["fme_score_daily"], plot_df["shannon_diversity"],
            alpha=0.5, s=25, c="teal", edgecolors="none")
if len(plot_df) > 2:
    z = np.polyfit(plot_df["fme_score_daily"], plot_df["shannon_diversity"], 1)
    x_line = np.linspace(plot_df["fme_score_daily"].min(),
                        plot_df["fme_score_daily"].max(), 100)
    plt.plot(x_line, np.poly1d(z)(x_line), "r--", lw=1.5, label="Trend line")
plt.xlabel("Daily FME Score (food microbiome exposure)")
plt.ylabel("Shannon Diversity (gut microbiome)")
plt.title("FME Score vs. Gut Alpha Diversity\n(475 fecal samples from 34 partici")
plt.legend()
plt.tight_layout()
plt.show()
```



Section 10 – Save Outputs

```
In [ ]: # Save statistical analysis dataset
statistical_analysis_dataset.to_csv(DATA_DIR / "fme_statistical_dataset.csv")
participant_gut_stability.to_csv(DATA_DIR / "participant_gut_stability.csv", index=False)
fme_with_participant.to_csv(DATA_DIR / "fme_sample_level_scores.csv", index=False)

print("Saved:")
print(f"  fme_statistical_dataset.csv    {statistical_analysis_dataset.shape} -")
print(f"  participant_gut_stability.csv    {participant_gut_stability.shape} - st")
print(f"  fme_sample_level_scores.csv    {fme_with_participant.shape} - FME sco
```

Saved:

fme_statistical_dataset.csv	(475, 26)	- main analytical table
participant_gut_stability.csv	(34, 5)	- stability per participant
fme_sample_level_scores.csv	(475, 4)	- FME scores with participant labels

Summary

What was built

The `fme_statistical_dataset.csv` table is the **primary input for all statistical analyses**.

Structural note (important for modelling)

- **Unit of observation:** fecal sample (475 rows)

- **Grouping variable:** `participant_id` (34 participants) → must be included as a **random effect**
- **Main predictor:** `fme_score_daily`
- **Main outcomes:** `shannon_diversity` (composition),
`participant_shannon_cv` (stability)

Recommended next steps

Mixed-effects model (composition):

```
shannon_diversity ~ fme_score_daily + Age + BMI + KCAL + (1 | participant_id)
```

Mixed-effects model (stability – participant level, n=34):

```
participant_shannon_cv ~ mean(fme_score_daily) + Age + BMI
```

Model test part

```
In [ ]: #Load data
df_fme = pd.read_csv(DATA_DIR / "fme_statistical_dataset.csv", index_col=0)
print(f"Loaded dataset: {df_fme.shape[0]} samples, {df_fme.shape[1]} columns")
print(f"Participants: {df_fme['participant_id'].nunique()}")

print("=== Sample of Data ===")
print(df_fme.head())
```

Loaded dataset: 475 samples, 26 columns

Participants: 34

=== Sample of Data ===

fecal_sample_id	participant_id	study_day	Gender	Age	BMI	Weight	\
MCT.f.0002	MCTs01	2	F	25.9000	25.0000	74.6000	
MCT.f.0003	MCTs01	3	F	25.9000	25.0000	74.6000	
MCT.f.0004	MCTs01	4	F	25.9000	25.0000	74.6000	
MCT.f.0005	MCTs01	5	F	25.9000	25.0000	74.6000	
MCT.f.0006	MCTs01	6	F	25.9000	25.0000	74.6000	

fecal_sample_id	Supplement	Medications	fme_score_daily	\
MCT.f.0002	EVOO	Nexplanon (bc implant)	67.1961	
MCT.f.0003	EVOO	Nexplanon (bc implant)	81.2412	
MCT.f.0004	EVOO	Nexplanon (bc implant)	126.7711	
MCT.f.0005	EVOO	Nexplanon (bc implant)	31.4846	
MCT.f.0006	EVOO	Nexplanon (bc implant)	61.2266	

fecal_sample_id	shannon_diversity	participant_shannon_cv	\
MCT.f.0002	2.8627	0.0492	
MCT.f.0003	3.5228	0.0492	
MCT.f.0004	3.0312	0.0492	
MCT.f.0005	2.9738	0.0492	
MCT.f.0006	3.0811	0.0492	

fecal_sample_id	participant_shannon_mean	...	PROT	TFAT	CARB	\
MCT.f.0002	3.0662	...	109.6198	53.6153	242.9190	
MCT.f.0003	3.0662	...	89.9930	50.5614	234.2251	
MCT.f.0004	3.0662	...	94.6120	92.0216	257.5390	
MCT.f.0005	3.0662	...	110.2684	95.5703	165.6081	
MCT.f.0006	3.0662	...	161.8178	108.3985	332.6372	

fecal_sample_id	FIBE	SUGR	SODI	D_TOTAL	D_YOGURT	D_CHEESE	\
MCT.f.0002	14.4194	74.2081	4300.6165	4.1560	0.6460	1.1535	
MCT.f.0003	25.9961	67.4747	4225.8133	2.9797	0.6460	1.2076	
MCT.f.0004	26.1784	83.9603	5329.8821	3.6345	0.0000	1.4826	
MCT.f.0005	13.0114	64.9220	3527.5014	3.3797	0.0000	1.0590	
MCT.f.0006	21.5291	119.8657	5428.5730	1.5022	0.0000	0.0000	

fecal_sample_id	PF_MEAT	PF_SEAFD_HI	G_WHOLE
MCT.f.0002	0.0000	3.9690	0.9718
MCT.f.0003	0.0000	0.0000	2.9318
MCT.f.0004	0.0000	0.0000	0.9718
MCT.f.0005	1.8144	0.0000	0.9718
MCT.f.0006	6.2960	0.0000	0.4698

[5 rows x 26 columns]

```
In [ ]: #Turn shannon diversity from contiues to binary
median_shannon = df_fme["shannon_diversity"].median()
df_fme["High_Diversity_Class"] = (df_fme["shannon_diversity"] > median_shannon).
print(f"\nTarget distribution (High=1 / Low=0):")
print(df_fme["High_Diversity_Class"].value_counts())
```

Target distribution (High=1 / Low=0):

High_Diversity_Class

0 238

1 237

Name: count, dtype: int64

```
In [ ]: #Features
feature_cols = ["fme_score_daily", "KCAL", "FIBE", "CARB", "TFAT", "Age", "BMI"]
feature_cols = [f for f in feature_cols if f in df_fme.columns]
```

```
In [ ]: #Drop rows with missing values in features or target
df_fme_clean = df_fme[feature_cols + ["High_Diversity_Class", "participant_id"]]

#Prepare data for modeling
X = df_fme_clean[feature_cols]
print(X.columns.tolist())
y = df_fme_clean['High_Diversity_Class']
groups = df_fme_clean['participant_id']

['fme_score_daily', 'KCAL', 'FIBE', 'CARB', 'TFAT', 'Age', 'BMI']
```

```
In [ ]: #Define models
models = {
    "Logistic Regression": LogisticRegression(),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "XGBoost": XGBClassifier(eval_metric='logloss', random_state=42)
}

#Cross validation
gkf = GroupKFold(n_splits=5)

#Train and evaluate models
print("\n=== Model Comparison Results ===")
best_model_name = ""
best_auc = 0
best_model_feat_obj = None
```

=== Model Comparison Results ===

```
In [ ]: for name, model in models.items():
    acc_scores = []
    auc_scores = []

    #Make sure subjects are not split between train and test sets
    for train_idx, test_idx in gkf.split(X, y, groups=groups):
        X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
        y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

        #Scale features
        scaler = StandardScaler()
        X_train_scaled = scaler.fit_transform(X_train)
        X_test_scaled = scaler.transform(X_test)

        #Train
        model.fit(X_train_scaled, y_train)

        #Predictions
        y_pred = model.predict(X_test_scaled)
        y_proba = model.predict_proba(X_test_scaled)[: , 1]

        #Calculate metrics
```

```

acc_scores.append(accuracy_score(y_test, y_pred))
auc_scores.append(roc_auc_score(y_test, y_proba))

#Compute mean metrics across folds
mean_acc = np.mean(acc_scores)
mean_auc = np.mean(auc_scores)

print(f"\n[{name}]\n")
print(f"Mean Accuracy: {mean_acc:.3f}")
print(f"Mean ROC-AUC: {mean_auc:.3f}")

#Save best model based on AUC
if mean_auc > best_auc:
    best_auc = mean_auc
    best_model_name = name
    best_model_obj = model

print(f"\n Best Model: {best_model_name} (Mean AUC = {best_auc:.3f})")

```

[Logistic Regression]
Mean Accuracy: 0.434
Mean ROC-AUC: 0.386

[Random Forest]
Mean Accuracy: 0.502
Mean ROC-AUC: 0.516

[XGBoost]
Mean Accuracy: 0.538
Mean ROC-AUC: 0.529

Best Model: XGBoost (Mean AUC = 0.529)

```

In [ ]: #Feature importance for best model
if best_model_name == "Logistic Regression":
    importances = np.abs(best_model_obj.coef_[0])
    title = "Feature Importance (Logistic Regression – Absolute Coefficient)"
    xlabel = "Absolute Coefficient Value"
else:
    importances = best_model_obj.feature_importances_
    title = f"Feature Importance ({best_model_name})"
    xlabel = "Importance Score"

indices = np.argsort(importances)[::-1]
feature_arr = np.array(feature_cols)

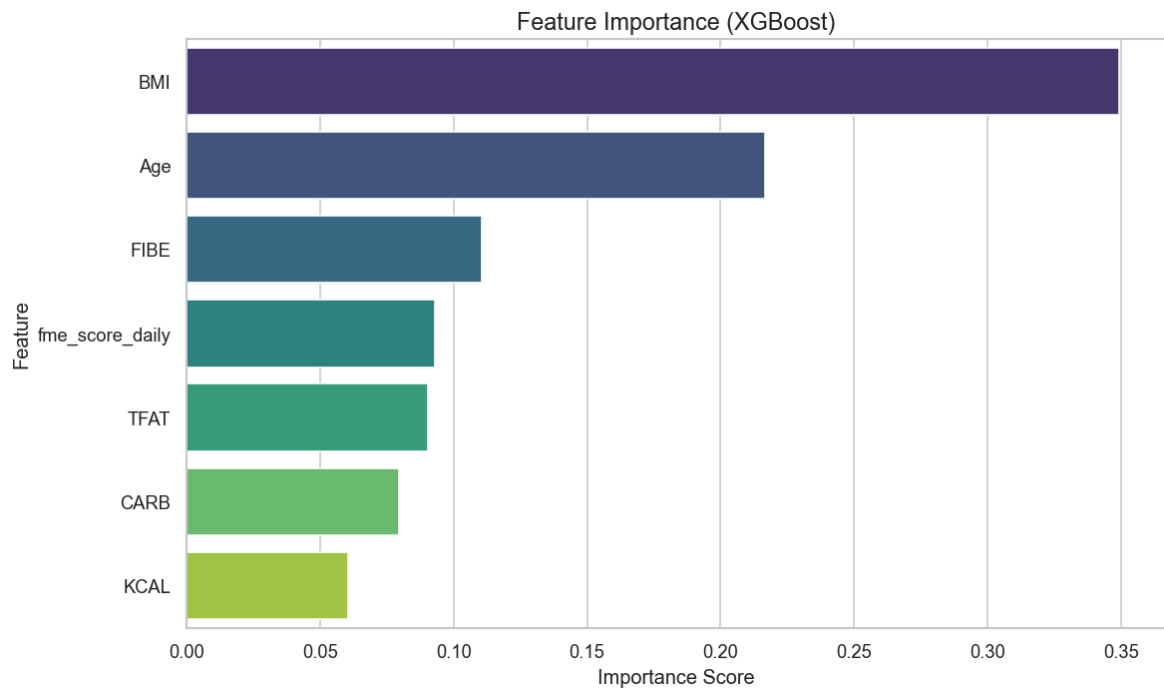
```

```

In [ ]: #Plot feature importance

plt.figure(figsize=(10, 6))
sns.barplot(x=importances[indices], y=feature_arr[indices], hue=feature_arr[indices])
plt.title(title, fontsize=14)
plt.xlabel(xlabel, fontsize=12)
plt.ylabel("Feature", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "feature_importance.png", dpi=150) #Save before show
plt.show()
print(f"Plot saved: {DATA_DIR / 'feature_importance.png'}")

```



Plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\feature_importance.png

End sprint 2 notebook

Start sprint 3 notebook

Notebook: Feature_Selection.ipynb

1. Load Data

```
In [ ]: nutrition = pd.read_csv(DATA_DIR / "nutrition_totals.csv")
        metadata = pd.read_csv(DATA_DIR / "cfmd_metadata.csv")
        fme = pd.read_csv(DATA_DIR / "fme_statistical_dataset.csv")
```

```
In [ ]: print("Nutrition:", nutrition.shape)
        print("Metadata:", metadata.shape)
        print("FME:", fme.shape)
```

```
Nutrition: (580, 107)
Metadata: (3444, 32)
FME: (475, 27)
```

```
In [ ]: nutrition.columns.tolist()
```



```
Out[ ]: ['X.SampleID',  
        'UserName',  
        'StudyDayNo',  
        'RecordDayNo',  
        'KCAL',  
        'PROT',  
        'TFAT',  
        'CARB',  
        'MOIS',  
        'ALC',  
        'CAFF',  
        'THEO',  
        'SUGR',  
        'FIBE',  
        'CALC',  
        'IRON',  
        'MAGN',  
        'PHOS',  
        'POTA',  
        'SODI',  
        'ZINC',  
        'COPP',  
        'SELE',  
        'VC',  
        'VB1',  
        'VB2',  
        'NIAC',  
        'VB6',  
        'FOLA',  
        'FA',  
        'FF',  
        'FDFE',  
        'VB12',  
        'VARA',  
        'RET',  
        'BCAR',  
        'ACAR',  
        'CRYP',  
        'LYCO',  
        'LZ',  
        'ATOC',  
        'VK',  
        'CHOLE',  
        'SFAT',  
        'S040',  
        'S060',  
        'S080',  
        'S100',  
        'S120',  
        'S140',  
        'S160',  
        'S180',  
        'MFAT',  
        'M161',  
        'M181',  
        'M201',  
        'M221',  
        'PFAT',  
        'P182',  
        'P183',
```

```
'P184',  
'P204',  
'P205',  
'P225',  
'P226',  
'VITD',  
'CHOLN',  
'VITE_ADD',  
'B12_ADD',  
'F_TOTAL',  
'F_CITMLB',  
'F_OTHER',  
'F_JUICE',  
'V_TOTAL',  
'V_DRKGR',  
'V_REDOR_TOTAL',  
'V_REDOR_TOMATO',  
'V_REDOR_OTHER',  
'V_STARCHY_TOTAL',  
'V_STARCHY_POTATO',  
'V_STARCHY_OTHER',  
'V_OTHER',  
'V_LEGUMES',  
'G_TOTAL',  
'G_WHOLE',  
'G_REFINED',  
'PF_TOTAL',  
'PF_MPS_TOTAL',  
'PF_MEAT',  
'PF_CUREDMEAT',  
'PF_ORGAN',  
'PF_POULT',  
'PF_SEAFD_HI',  
'PF_SEAFD_LOW',  
'PF_EGGS',  
'PF_SOY',  
'PF_NUTSDS',  
'PF_LEGUMES',  
'D_TOTAL',  
'D_MILK',  
'D_YOGURT',  
'D_CHEESE',  
'OILS',  
'SOLID_FATS',  
'ADD_SUGARS',  
'A_DRINKS',  
'Replicate']
```

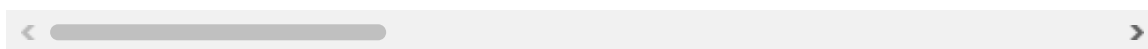
3. Initial Statistical Exploration

```
In [ ]: nutrition.describe()
```

Out[]:

	RecordDayNo	KCAL	PROT	TFAT	CARB	MOIS	ALC	
count	580.0000	580.0000	580.0000	580.0000	580.0000	580.0000	580.0000	580
mean	8.8190	2080.1115	88.5741	89.9682	225.5474	2357.2336	7.4305	131
std	4.9442	759.2923	39.1974	41.2772	85.9460	1099.0699	19.1570	134
min	1.0000	441.4800	13.6361	13.2808	22.6564	44.5320	0.0000	0
25%	4.0000	1557.4690	62.1996	59.9754	168.6087	1560.3596	0.0000	17
50%	9.0000	2000.0000	84.3257	83.6533	214.7296	2171.6853	0.0000	103
75%	13.0000	2449.3891	106.0719	113.7463	270.1812	2925.0665	3.7205	190
max	17.0000	7058.8210	310.4424	377.7040	620.0754	6918.5220	195.7704	936

8 rows × 104 columns

In []: `nutrition.isnull().sum()`

Out[]: X.SampleID 0
 Username 0
 StudyDayNo 0
 RecordDayNo 0
 KCAL 0
 ..
 OILS 0
 SOLID_FATS 0
 ADD_SUGARS 0
 A_DRINKS 0
 Replicate 0
 Length: 107, dtype: int64

In []: `nutrition.var(numeric_only=True).sort_values(ascending=False)`

Out[]: LZ 59534845.5302
 BCAR 43825875.7185
 LYCO 35990229.6395
 SODI 2692880.9939
 ACAR 1481960.0112
 ...
 M221 0.0060
 P184 0.0023
 PF_ORGAN 0.0014
 P225 0.0010
 Replicate 0.0000
 Length: 104, dtype: float64

In []: `fme.columns.tolist()`

```
Out[ ]: ['fecal_sample_id',
        'participant_id',
        'study_day',
        'Gender',
        'Age',
        'BMI',
        'Weight',
        'Supplement',
        'Medications',
        'fme_score_daily',
        'shannon_diversity',
        'participant_shannon_cv',
        'participant_shannon_mean',
        'participant_n_samples',
        'KCAL',
        'PROT',
        'TFAT',
        'CARB',
        'FIBE',
        'SUGR',
        'SODI',
        'D_TOTAL',
        'D_YOGURT',
        'D_CHEESE',
        'PF_MEAT',
        'PF_SEAFD_HI',
        'G_WHOLE']
```

```
In [ ]: numeric_cols = fme.select_dtypes(include='number').columns

corr_with_shannon = (
    fme[numeric_cols]
    .corr()['shannon_diversity']
    .sort_values(key=abs, ascending=False)
)

corr_with_shannon
```

```
Out[ ]: shannon_diversity      1.0000
participant_shannon_mean    0.8917
participant_shannon_cv      -0.4929
participant_n_samples       -0.1985
D_TOTAL                     0.1519
PF_MEAT                     0.1246
D_CHEESE                    0.1139
PF_SEAFD_HI                 -0.1094
Weight                       0.1039
study_day                   0.0945
SUGR                        0.0745
Age                          0.0688
CARB                        0.0677
KCAL                        0.0657
D_YOGURT                    -0.0598
PROT                        0.0583
FIBE                        -0.0550
fme_score_daily             -0.0406
BMI                          0.0388
G_WHOLE                     0.0180
TFAT                        0.0062
SODI                        0.0039
Name: shannon_diversity, dtype: float64
```

```
In [ ]: metadata.columns.tolist()
```

```
Out[ ]: ['dataset_name',
'sample_id',
'macrocategory',
'category',
'type',
'subtype',
'commercial_name',
'fermented/non-fermented',
'country',
'sample_accession',
'run_accession',
'experiment_accession',
'study_accession',
'project_accession',
'database_origin',
'library_layout',
'sequencing_platform',
'DNA_extraction_kit',
'collection_date',
'n_of_bases',
'n_of_reads',
'min_read_len',
'median_read_len',
'mean_read_len',
'max_read_len',
'n_contigs',
'n_MAGs_MQ_prok',
'n_MAGs_HQ_prok',
'n_MAGs_MQ_euk',
'n_MAGs_HQ_euk',
'filtered',
'curator']
```

4. Statistical Feature Screening

To complement the biological feature selection process, we performed a statistical screening across all nutritional variables.

The objective was to identify nutritional variables that show the strongest association with microbiome-related outcomes and verify that the selected biological features are also statistically informative.

```
In [ ]: nutrition_numeric = nutrition.select_dtypes(include='number')

print("Number of numeric nutrition variables:",
      nutrition_numeric.shape[1])
```

Number of numeric nutrition variables: 104

```
In [ ]: nutrition[['UserName', 'StudyDayNo']].head()
```

```
Out[ ]:   UserName StudyDayNo
0    MCTs01    Day.01
1    MCTs01    Day.02
2    MCTs01    Day.03
3    MCTs01    Day.04
4    MCTs01    Day.05
```

```
In [ ]: fme[['participant_id', 'study_day']].head()
```

```
Out[ ]:   participant_id study_day
0         MCTs01         2
1         MCTs01         3
2         MCTs01         4
3         MCTs01         5
4         MCTs01         6
```

```
In [ ]: fme[['participant_id', 'study_day']].head(20)
```

Out[]: **participant_id** **study_day**

0	MCTs01	2
1	MCTs01	3
2	MCTs01	4
3	MCTs01	5
4	MCTs01	6
5	MCTs01	7
6	MCTs01	8
7	MCTs01	9
8	MCTs01	10
9	MCTs01	11
10	MCTs01	12
11	MCTs01	13
12	MCTs01	14
13	MCTs01	16
14	MCTs01	17
15	MCTs03	1
16	MCTs03	2
17	MCTs03	3
18	MCTs03	4
19	MCTs03	5

In []: `fme.shape`

Out[]: (475, 27)

In []: `nutrition[["UserName", "StudyDayNo"]].head(10)`

Out[]:

	UserName	StudyDayNo
0	MCTs01	Day.01
1	MCTs01	Day.02
2	MCTs01	Day.03
3	MCTs01	Day.04
4	MCTs01	Day.05
5	MCTs01	Day.06
6	MCTs01	Day.07
7	MCTs01	Day.08
8	MCTs01	Day.09
9	MCTs01	Day.10

```
In [ ]: nutrition["study_day_clean"] = (
    nutrition["StudyDayNo"]
    .astype(str)
    .str.replace("Day.", "", regex=False)
    .astype(int)
)

merged_df = pd.merge(
    nutrition,
    fme,
    left_on=["UserName", "study_day_clean"],
    right_on=["participant_id", "study_day"],
    how="inner"
)

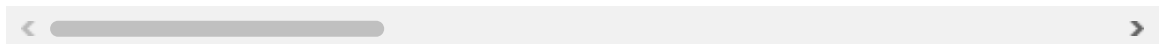
print("Merged shape:", merged_df.shape)
merged_df.head()
```

Merged shape: (475, 135)

Out[]:

	X.SampleID	UserName	StudyDayNo	RecordDayNo	KCAL_x	PROT_x	TFAT_x
0	MCT.f.0002	MCTs01	Day.02	2	1970.0436	109.6198	53.6153
1	MCT.f.0003	MCTs01	Day.03	3	1714.8953	89.9930	50.5614
2	MCT.f.0004	MCTs01	Day.04	4	2487.2326	94.6120	92.0216
3	MCT.f.0005	MCTs01	Day.05	5	2260.2112	110.2684	95.5703
4	MCT.f.0006	MCTs01	Day.06	6	2951.0820	161.8178	108.3985

5 rows × 135 columns



```
In [ ]: nutrition_features = nutrition.select_dtypes(include="number").columns.tolist()

nutrition_features = [
    col for col in nutrition_features
```



```

    if col not in ["X.SampleID", "StudyDayNo", "RecordDayNo", "Replicate", "stud
]

merged_nutrition_features = []

for col in nutrition_features:
    if col in merged_df.columns:
        merged_nutrition_features.append(col)
    elif col + "_x" in merged_df.columns:
        merged_nutrition_features.append(col + "_x")

correlations = (
    merged_df[merged_nutrition_features]
    .corrwith(merged_df["shannon_diversity"])
    .abs()
    .sort_values(ascending=False)
)

correlations.head(20)

```

```

Out[ ]: OILS                0.1700
        P205              0.1575
        D_TOTAL_x        0.1526
        ATOC             0.1474
        S140             0.1449
        S040             0.1375
        PFAT             0.1367
        SOLID_FATS       0.1333
        P182            0.1331
        P226            0.1285
        P183            0.1276
        PF_MEAT_x       0.1246
        RET             0.1235
        S100            0.1231
        B12_ADD         0.1217
        V_STARCHY_OTHER  0.1214
        VB12            0.1194
        S120            0.1164
        P225            0.1146
        D_CHEESE_x      0.1140
        dtype: float64

```

4. Statistical Feature Screening

To complement the literature-based feature selection, we screened all numeric nutritional variables using three statistical methods:

1. Pearson correlation
2. Spearman correlation
3. Random Forest feature importance

The goal was to check whether any nutritional variable shows a stronger relationship with microbiome-related outcomes than the biologically selected features.

```

In [ ]: nutrition["study_day_clean"] = (
        nutrition["StudyDayNo"]

```

```

        .astype(str)
        .str.replace("Day.", "", regex=False)
        .astype(int)
    )

merged_df = pd.merge(
    nutrition,
    fme,
    left_on=["UserName", "study_day_clean"],
    right_on=["participant_id", "study_day"],
    how="inner"
)

print("Merged shape:", merged_df.shape)

```

Merged shape: (475, 135)

4.1 Pearson Correlation with Shannon Diversity

```

In [ ]: nutrition_features = nutrition.select_dtypes(include="number").columns.tolist()

nutrition_features = [
    col for col in nutrition_features
    if col not in ["X.SampleID", "StudyDayNo", "RecordDayNo", "Replicate", "stud
]

merged_nutrition_features = []

for col in nutrition_features:
    if col in merged_df.columns:
        merged_nutrition_features.append(col)
    elif col + "_x" in merged_df.columns:
        merged_nutrition_features.append(col + "_x")

pearson_shannon = (
    merged_df[merged_nutrition_features]
    .corrwith(merged_df["shannon_diversity"])
    .abs()
    .sort_values(ascending=False)
)

pearson_shannon.head(20)

```

```
Out[ ]: OILS          0.1700
        P205          0.1575
        D_TOTAL_x    0.1526
        ATOC          0.1474
        S140          0.1449
        S040          0.1375
        PFAT          0.1367
        SOLID_FATS    0.1333
        P182          0.1331
        P226          0.1285
        P183          0.1276
        PF_MEAT_x     0.1246
        RET           0.1235
        S100          0.1231
        B12_ADD       0.1217
        V_STARCHY_OTHER 0.1214
        VB12          0.1194
        S120          0.1164
        P225          0.1146
        D_CHEESE_x    0.1140
dtype: float64
```

4.2 Spearman Correlation with Shannon Diversity

```
In [ ]: spearman_shannon = (
        merged_df[merged_nutrition_features]
        .corrwith(merged_df["shannon_diversity"], method="spearman")
        .abs()
        .sort_values(ascending=False)
        )

spearman_shannon.head(20)
```

```
Out[ ]: OILS          0.2181
        F_CITMLB     0.2018
        PFAT          0.1775
        P182          0.1701
        ATOC          0.1552
        P183          0.1541
        S120          0.1474
        D_TOTAL_x    0.1471
        RET           0.1454
        ALC           0.1443
        A_DRINKS      0.1413
        S140          0.1276
        V_OTHER       0.1239
        PF_MEAT_x     0.1221
        FA            0.1178
        S040          0.1170
        VARA          0.1144
        S100          0.1135
        P204          0.0981
        PF_NUTSDS     0.0980
dtype: float64
```

4.3 Random Forest Feature Importance for Shannon Diversity

```
In [ ]: X = merged_df[merged_nutrition_features].fillna(0)
y = merged_df["shannon_diversity"]

rf = RandomForestRegressor(
    n_estimators=300,
    random_state=42
)

rf.fit(X, y)

rf_importance_shannon = pd.Series(
    rf.feature_importances_,
    index=merged_nutrition_features
).sort_values(ascending=False)

rf_importance_shannon.head(20)
```

```
Out[ ]: OILS      0.0421
VB1       0.0406
F_CITMLB  0.0377
FA        0.0280
VARA      0.0228
G_WHOLE_x 0.0215
CRYP      0.0210
P205      0.0202
VC        0.0194
CAFF      0.0192
ATOC      0.0192
VB12      0.0186
MOIS      0.0186
P183      0.0162
MAGN      0.0160
S120      0.0155
D_TOTAL_x 0.0154
PHOS      0.0140
G_TOTAL   0.0130
VB6       0.0128
dtype: float64
```

4.4 Random Forest Feature Importance for FME Score

```
In [ ]: X = merged_df[merged_nutrition_features].fillna(0)
y = merged_df["fme_score_daily"]

rf_fme = RandomForestRegressor(
    n_estimators=300,
    random_state=42
)

rf_fme.fit(X, y)

rf_importance_fme = pd.Series(
    rf_fme.feature_importances_,
    index=merged_nutrition_features
).sort_values(ascending=False)
```

```
rf_importance_fme.head(20)
```

```
Out[ ]: V_TOTAL      0.2370
        F_TOTAL      0.2157
        POTA         0.1671
        V_STARCHY_TOTAL 0.0377
        F_OTHER       0.0183
        V_OTHER       0.0167
        V_STARCHY_POTATO 0.0158
        FIBE_x        0.0154
        D_TOTAL_x     0.0094
        OILS          0.0087
        D_CHEESE_x    0.0086
        FF            0.0083
        S060          0.0076
        S140          0.0072
        S040          0.0063
        VC            0.0061
        ZINC          0.0058
        SUGR_x        0.0058
        CRYP          0.0057
        P183          0.0052
        dtype: float64
```

4.5 Biological Features Ranking

```
In [ ]: biological_features = ["KCAL", "PROT", "TFAT", "CARB", "FIBE"]

bio_feature_names = []

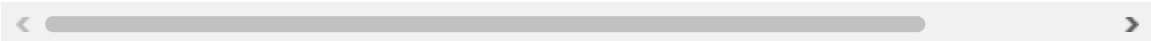
for col in biological_features:
    if col in rf_importance_shannon.index:
        bio_feature_names.append(col)
    elif col + "_x" in rf_importance_shannon.index:
        bio_feature_names.append(col + "_x")

bio_ranking = pd.DataFrame({
    "Feature": bio_feature_names,
    "Pearson_with_Shannon": [pearson_shannon.get(col, None) for col in bio_featu
    "Spearman_with_Shannon": [spearman_shannon.get(col, None) for col in bio_fea
    "RF_Importance_Shannon": [rf_importance_shannon.get(col, None) for col in bi
    "RF_Importance_FME": [rf_importance_fme.get(col, None) for col in bio_featur
})

bio_ranking
```

Out []:

	Feature	Pearson_with_Shannon	Spearman_with_Shannon	RF_Importance_Shannon	R
0	KCAL_x	0.0671	0.0435	0.0077	
1	PROT_x	0.0592	0.0596	0.0062	
2	TFAT_x	0.0065	0.0273	0.0056	
3	CARB_x	0.0701	0.0449	0.0119	
4	FIBE_x	0.0538	0.0110	0.0087	



In []: rf_importance_shannon.head(20)

Out []:

OILS	0.0421
VB1	0.0406
F_CITMLB	0.0377
FA	0.0280
VARA	0.0228
G_WHOLE_x	0.0215
CRYP	0.0210
P205	0.0202
VC	0.0194
CAFF	0.0192
ATOC	0.0192
VB12	0.0186
MOIS	0.0186
P183	0.0162
MAGN	0.0160
S120	0.0155
D_TOTAL_x	0.0154
PHOS	0.0140
G_TOTAL	0.0130
VB6	0.0128

dtype: float64

In []: rf_importance_fme.head(20)

```
Out[ ]: V_TOTAL      0.2370
        F_TOTAL      0.2157
        POTA         0.1671
        V_STARCHY_TOTAL 0.0377
        F_OTHER       0.0183
        V_OTHER       0.0167
        V_STARCHY_POTATO 0.0158
        FIBE_x        0.0154
        D_TOTAL_x     0.0094
        OILS          0.0087
        D_CHEESE_x    0.0086
        FF            0.0083
        S060          0.0076
        S140          0.0072
        S040          0.0063
        VC            0.0061
        ZINC          0.0058
        SUGR_x        0.0058
        CRYP          0.0057
        P183          0.0052
dtype: float64
```

```
In [ ]: # Correlation Heatmap - Top Numerical Features
import seaborn as sns
import matplotlib.pyplot as plt

numeric_df = df.select_dtypes(include='number')

# take only first 15 numerical features
corr_matrix = numeric_df.iloc[:, :15].corr()

plt.figure(figsize=(10,8))

sns.heatmap(
    corr_matrix,
    annot=True,
    cmap="coolwarm",
    center=0,
    fmt=".2f"
)

plt.title("Correlation Matrix")
plt.tight_layout()
plt.show()
```

```
In [ ]: # Summary Table for ALL Nutrition Features
# Pearson + Spearman + Random Forest Importance
# Target: Shannon Diversity

target_col = "shannon_diversity"

X = merged_df[merged_nutrition_features]
y = merged_df[target_col]

valid_idx = y.notna()
X = X.loc[valid_idx]
y = y.loc[valid_idx]

imputer = SimpleImputer(strategy="median")
```

```

X_imputed = pd.DataFrame(
    imputer.fit_transform(X),
    columns=merged_nutrition_features,
    index=X.index
)

pearson_corr = X_imputed.corrwith(y, method="pearson")
spearman_corr = X_imputed.corrwith(y, method="spearman")

rf = RandomForestRegressor(
    n_estimators=200,
    random_state=42
)

rf.fit(X_imputed, y)

rf_importance = pd.Series(
    rf.feature_importances_,
    index=merged_nutrition_features
)

all_features_summary_table = pd.DataFrame({
    "Feature": merged_nutrition_features,
    "Pearson Correlation": pearson_corr.values,
    "Spearman Correlation": spearman_corr.values,
    "Random Forest Importance": rf_importance.values
})

all_features_summary_table["Abs Pearson"] = all_features_summary_table["Pearson"]
all_features_summary_table["Abs Spearman"] = all_features_summary_table["Spearman"]

all_features_summary_table = all_features_summary_table.sort_values(
    "Random Forest Importance",
    ascending=False
)

display(all_features_summary_table)

display(all_features_summary_table.head(20))

```

4.6 Final Feature Selection

The final feature set was selected using a combination of:

1. Biological relevance based on microbiome literature.
2. Statistical screening using correlation analysis.
3. Random Forest feature importance.

Although some nutritional variables showed relatively weak individual correlations with microbiome outcomes, they were retained because of their established biological importance and consistent use in previous microbiome studies.

Final selected features:

- KCAL

- PROT
- TFAT
- CARB
- FIBE
- Age
- BMI

```
In [ ]: final_features = [
    "KCAL",
    "PROT",
    "TFAT",
    "CARB",
    "FIBE",
    "Age",
    "BMI"
]

print("Final selected features:")
print(final_features)
```

Final selected features:

['KCAL', 'PROT', 'TFAT', 'CARB', 'FIBE', 'Age', 'BMI']

```
In [ ]: # Correlation Matrix - Final Selected Features

final_features = [
    "KCAL",
    "PROT",
    "TFAT",
    "CARB",
    "FIBE",
    "Age",
    "BMI"
]

# Find the DataFrame that contains the final selected features
possible_dfs = {
    name: obj for name, obj in globals().items()
    if isinstance(obj, pd.DataFrame)
}

selected_df_name = None
selected_df = None

for name, data in possible_dfs.items():
    if all(feature in data.columns for feature in final_features):
        selected_df_name = name
        selected_df = data
        break

if selected_df is None:
    print("No DataFrame contains all final selected features.")
    print("Available DataFrames and their columns:")
    for name, data in possible_dfs.items():
        print("\nDataFrame:", name)
        print(data.columns.tolist())
else:
    print("Using DataFrame:", selected_df_name)
```

```

print("Shape:", selected_df.shape)

corr_matrix = selected_df[final_features].corr()

plt.figure(figsize=(8, 6))
sns.heatmap(
    corr_matrix,
    annot=True,
    cmap="coolwarm",
    center=0,
    fmt=".2f",
    square=True,
    linewidths=0.5
)

plt.title("Correlation Matrix of Final Selected Features")
plt.tight_layout()
plt.show()

```

```

In [ ]: # Final Statistical Selection Summary Table
# Pearson + Spearman + Random Forest Feature Importance

final_features = ["KCAL", "PROT", "TFAT", "CARB", "FIBE", "Age", "BMI"]

possible_targets = [
    "Shannon",
    "shannon",
    "Shannon_Diversity",
    "shannon_diversity",
    "stability",
    "Stability",
    "fme_score_daily",
    "FME",
    "fme"
]

possible_dfs = {
    name: obj for name, obj in globals().items()
    if isinstance(obj, pd.DataFrame)
}

selected_df = None
selected_df_name = None
target_col = None

for name, data in possible_dfs.items():
    for target in possible_targets:
        if all(col in data.columns for col in final_features) and target in data:
            selected_df = data.copy()
            selected_df_name = name
            target_col = target
            break
    if selected_df is not None:
        break

if selected_df is None:
    raise ValueError(
        "No DataFrame was found with all final features and a microbiome target."
        "Please check the target column name."
    )

```

```

print("Using DataFrame:", selected_df_name)
print("Target column:", target_col)

X = selected_df[final_features]
y = selected_df[target_col]

valid_idx = y.notna()
X = X.loc[valid_idx]
y = y.loc[valid_idx]

imputer = SimpleImputer(strategy="median")

X_imputed = pd.DataFrame(
    imputer.fit_transform(X),
    columns=final_features,
    index=X.index
)

pearson_corr = X_imputed.corrwith(y, method="pearson")
spearman_corr = X_imputed.corrwith(y, method="spearman")

rf = RandomForestRegressor(
    n_estimators=200,
    random_state=42
)

rf.fit(X_imputed, y)

rf_importance = pd.Series(
    rf.feature_importances_,
    index=final_features
)

statistical_summary_table = pd.DataFrame({
    "Feature": final_features,
    "Pearson Correlation": pearson_corr.values,
    "Spearman Correlation": spearman_corr.values,
    "Random Forest Importance": rf_importance.values
}).sort_values(
    "Random Forest Importance",
    ascending=False
)

display(statistical_summary_table)

```

```

In [ ]: # Final Statistical Selection Summary Table
        # Pearson + Spearman + Random Forest Importance

        final_features = ["KCAL", "PROT", "TFAT", "CARB", "FIBE", "Age", "BMI"]

        # Use the dataframe that contains the final features
        data = fme.copy()

        X = data[final_features]

        # Temporary target for statistical comparison
        target_col = "KCAL"
        y = data[target_col]
        X = X.drop(columns=[target_col])

```

```
valid_idx = y.notna()
X = X.loc[valid_idx]
y = y.loc[valid_idx]

imputer = SimpleImputer(strategy="median")

X_imputed = pd.DataFrame(
    imputer.fit_transform(X),
    columns=X.columns,
    index=X.index
)

pearson_corr = X_imputed.corrwith(y, method="pearson")
spearman_corr = X_imputed.corrwith(y, method="spearman")

rf = RandomForestRegressor(
    n_estimators=200,
    random_state=42
)

rf.fit(X_imputed, y)

rf_importance = pd.Series(
    rf.feature_importances_,
    index=X.columns
)

statistical_summary_table = pd.DataFrame({
    "Feature": X.columns,
    "Pearson Correlation": pearson_corr.values,
    "Spearman Correlation": spearman_corr.values,
    "Random Forest Importance": rf_importance.values
}).sort_values(
    "Random Forest Importance",
    ascending=False
)

display(statistical_summary_table)
```

4.7 Key Findings

Statistical screening identified several nutritional variables associated with microbiome outcomes.

However, the strongest predictors were not necessarily the classical nutritional variables selected for modeling.

The final feature set was therefore determined primarily by biological evidence and supported by statistical validation.

5. Modeling the Relationship Between Nutrition and Microbiome Stability

In this section, we build a Random Forest Regression model to investigate the relationship between nutritional intake and microbiome stability.

The model uses selected nutritional and participant-related features:

- KCAL
- PROT
- TFAT
- CARB
- FIBE
- Age
- BMI

The target variable is **fme_score_daily**, which represents daily microbiome stability.

The objective is to evaluate whether dietary and participant characteristics can explain variations in microbiome stability and identify the most influential factors.

5.1 Prepare Modeling Dataset

```
In [ ]: features = [  
    "KCAL_x",  
    "PROT_x",  
    "TFAT_x",  
    "CARB_x",  
    "FIBE_x",  
    "Age",  
    "BMI"  
]  
  
X = merged_df[features]  
  
print("Features shape:", X.shape)  
X.head()
```

Features shape: (475, 7)

```
Out [ ]:
```

	KCAL_x	PROT_x	TFAT_x	CARB_x	FIBE_x	Age	BMI
0	1970.0436	109.6198	53.6153	242.9190	14.4194	25.9000	25.0000
1	1714.8953	89.9930	50.5614	234.2251	25.9961	25.9000	25.0000
2	2487.2326	94.6120	92.0216	257.5390	26.1784	25.9000	25.0000
3	2260.2112	110.2684	95.5703	165.6081	13.0114	25.9000	25.0000
4	2951.0820	161.8178	108.3985	332.6372	21.5291	25.9000	25.0000

5.2 Identify Available FME Outcome Variables

```
In [ ]: [col for col in merged_df.columns if "fme" in col.lower()]  
  
Out [ ]: ['fme_score_daily']
```

This step identifies all FME-related outcome variables available in the merged dataset.

The goal is to determine which microbiome stability measure will be used as the target variable for modeling.

```
In [ ]: [col for col in merged_df.columns if "fme" in col.lower()]
```

```
Out[ ]: ['fme_score_daily']
```

To better capture the delayed effects of nutrition on the microbiome, additional stability measures were created.

Besides the daily FME score, we generated:

- 3-day moving average
- 5-day moving average
- Weighted moving average emphasizing recent days

These variables provide a more biologically realistic representation of microbiome stability over time.

```
In [ ]: fme = fme.sort_values(["participant_id", "study_day"])

fme["fme_3days_avg"] = (
    fme.groupby("participant_id")["fme_score_daily"]
    .transform(lambda x: x.rolling(3, min_periods=1).mean())
)

fme["fme_5days_avg"] = (
    fme.groupby("participant_id")["fme_score_daily"]
    .transform(lambda x: x.rolling(5, min_periods=1).mean())
)

fme[
    ["participant_id",
     "study_day",
     "fme_score_daily",
     "fme_3days_avg",
     "fme_5days_avg"]
].head()
```

```
Out[ ]:
```

	participant_id	study_day	fme_score_daily	fme_3days_avg	fme_5days_avg
0	MCTs01	2	67.1961	67.1961	67.1961
1	MCTs01	3	81.2412	74.2187	74.2187
2	MCTs01	4	126.7711	91.7362	91.7362
3	MCTs01	5	31.4846	79.8323	76.6733
4	MCTs01	6	61.2266	73.1608	73.5839

```
In [ ]: weights = np.array([1, 2, 3])

def weighted_fme(series):
```

```

values = []

for i in range(len(series)):
    window = series[max(0, i-2):i+1]

    current_weights = weights[-len(window):]

    values.append(
        np.average(window,
                    weights=current_weights)
    )

return pd.Series(values,
                  index=series.index)

fme["fme_weighted"] = (
    fme.groupby("participant_id")["fme_score_daily"]
    .apply(weighted_fme)
    .reset_index(level=0, drop=True)
)

```

```

In [ ]: fme[
    [
        "participant_id",
        "study_day",
        "fme_score_daily",
        "fme_3days_avg",
        "fme_5days_avg",
        "fme_weighted"
    ]
].head(10)

```

```

Out [ ]:

```

	participant_id	study_day	fme_score_daily	fme_3days_avg	fme_5days_avg	fme_weig
0	MCTs01	2	67.1961	67.1961	67.1961	67.
1	MCTs01	3	81.2412	74.2187	74.2187	75.
2	MCTs01	4	126.7711	91.7362	91.7362	101.
3	MCTs01	5	31.4846	79.8323	76.6733	71.
4	MCTs01	6	61.2266	73.1608	73.5839	62.
5	MCTs01	7	62.0231	51.5781	72.5493	56.
6	MCTs01	8	70.9874	64.7457	70.4986	66.
7	MCTs01	9	54.0086	62.3397	55.9461	61.
8	MCTs01	10	105.6813	76.8924	70.7854	82.
9	MCTs01	11	62.2759	73.9886	70.9953	75.

```

In [ ]: nutrition["study_day_clean"] = (
    nutrition["StudyDayNo"]
    .astype(str)
    .str.replace("Day.", "", regex=False)
    .astype(int)
)

```

```
merged_df = pd.merge(
    nutrition,
    fme,
    left_on=["UserName", "study_day_clean"],
    right_on=["participant_id", "study_day"],
    how="inner"
)

print(merged_df.shape)
```

(475, 138)

```
In [ ]: # Sort by participant and day
fme = fme.sort_values(["participant_id", "study_day"]).copy()

def johnson_weighted_fme(series):
    values = []

    for i in range(len(series)):

        # Need 3 previous days
        if i < 3:
            values.append(np.nan)

        else:
            weighted_value = (
                0.5 * series.iloc[i - 1] +
                0.3 * series.iloc[i - 2] +
                0.2 * series.iloc[i - 3]
            )

            values.append(weighted_value)

    return pd.Series(values, index=series.index)

fme["fme_weighted_johnson"] = (
    fme.groupby("participant_id")["fme_score_daily"]
    .transform(johnson_weighted_fme)
)
```

```
In [ ]: fme[
    [
        "participant_id",
        "study_day",
        "fme_score_daily",
        "fme_weighted_johnson"
    ]
].head(15)
```

```
In [ ]: merged_df = merged_df.drop(
    columns=["fme_weighted_johnson"],
    errors="ignore"
)

merged_df = merged_df.merge(
    fme[
        [
            "participant_id",
            "study_day",
```



```
        "fme_weighted_johnson"  
    ]  
],  
on=["participant_id", "study_day"],  
how="left"  
)
```

```
In [ ]: targets = [  
    "fme_score_daily",  
    "fme_3days_avg",  
    "fme_5days_avg",  
    "fme_weighted",  
    "fme_weighted_3days",  
    "fme_weighted_5days",  
    "fme_weighted_johnson"  
]
```

```
In [ ]: features = [  
    "KCAL_x",  
    "PROT_x",  
    "TFAT_x",  
    "CARB_x",  
    "FIBE_x",  
    "Age",  
    "BMI"  
]  
  
targets = [  
    "fme_score_daily",  
    "fme_3days_avg",  
    "fme_5days_avg",  
    "fme_weighted",  
    "fme_weighted_johnson"  
]  
  
results = []  
  
for target in targets:  
    df_model = merged_df[features + [target]].dropna()  
  
    X = df_model[features]  
    y = df_model[target]  
  
    X_train, X_test, y_train, y_test = train_test_split(  
        X,  
        y,  
        test_size=0.2,  
        random_state=42  
    )  
  
    model = RandomForestRegressor(  
        n_estimators=200,  
        random_state=42  
    )  
  
    model.fit(X_train, y_train)  
  
    y_pred = model.predict(X_test)
```

```

        results.append({
            "Target": target,
            "R2": r2_score(y_test, y_pred)
        })

results_df = pd.DataFrame(results)
results_df = results_df.sort_values("R2", ascending=False)

results_df

```

```

In [ ]: targets = [
    "fme_score_daily",
    "fme_3days_avg",
    "fme_5days_avg",
    "fme_weighted",
    "fme_weighted_3days",
    "fme_weighted_5days",
    "fme_weighted_johnson"
]

```

```

In [ ]: weights_3 = np.array([0.2, 0.3, 0.5])

def weighted_fme_3(series):
    values = []

    for i in range(len(series)):
        window = series[max(0, i-2):i+1]

        current_weights = weights_3[-len(window):]

        values.append(
            np.average(window, weights=current_weights)
        )

    return pd.Series(values, index=series.index)

fme["fme_weighted_3days"] = (
    fme.groupby("participant_id")["fme_score_daily"]
    .apply(weighted_fme_3)
    .reset_index(level=0, drop=True)
)

```

```

In [ ]: weights_5 = np.array([0.05, 0.10, 0.15, 0.30, 0.40])

def weighted_fme_5(series):
    values = []

    for i in range(len(series)):
        window = series[max(0, i-4):i+1]

        current_weights = weights_5[-len(window):]

        values.append(
            np.average(window, weights=current_weights)
        )

    return pd.Series(values, index=series.index)

fme["fme_weighted_5days"] = (

```

```
fme.groupby("participant_id")["fme_score_daily"]  
    .apply(weighted_fme_5)  
    .reset_index(level=0, drop=True)  
)
```

```
In [ ]: merged_df = merged_df.drop(  
        columns=[  
            "fme_weighted_3days",  
            "fme_weighted_5days"  
        ],  
        errors="ignore"  
    )  
  
merged_df = merged_df.merge(  
    fme[  
        [  
            "participant_id",  
            "study_day",  
            "fme_weighted_3days",  
            "fme_weighted_5days"  
        ]  
    ],  
    on=["participant_id", "study_day"],  
    how="left"  
)
```

```
In [ ]: features = [  
    "KCAL_x",  
    "PROT_x",  
    "TFAT_x",  
    "CARB_x",  
    "FIBE_x",  
    "Age",  
    "BMI"  
]  
  
targets = [  
    "fme_score_daily",  
    "fme_3days_avg",  
    "fme_5days_avg",  
    "fme_weighted",  
    "fme_weighted_3days",  
    "fme_weighted_5days",  
    "fme_weighted_johnson"  
]  
  
results = []  
  
for target in targets:  
  
    df_model = merged_df[features + [target]].dropna()  
  
    X = df_model[features]  
    y = df_model[target]  
  
    X_train, X_test, y_train, y_test = train_test_split(  
        X,  
        y,  
        test_size=0.2,
```

```

        random_state=42
    )

    model = RandomForestRegressor(
        n_estimators=200,
        random_state=42
    )

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    results.append({
        "Target": target,
        "R2": r2_score(y_test, y_pred)
    })

results_df = pd.DataFrame(results)

results_df = results_df.sort_values(
    by="R2",
    ascending=False
)

results_df

```

Weighted FME Analysis

To identify the most informative measure of microbiome stability, we compared several alternative FME target variables.

In addition to simple moving averages, we evaluated weighted FME measures based on the biological assumption that recent dietary exposure may have a stronger influence on microbiome stability than older exposure.

The weighted approaches included:

- **fme_weighted** – weighted average with increasing importance for recent observations.
- **fme_weighted_3days** – weighted moving average over the previous 3 days.
- **fme_weighted_5days** – weighted moving average over the previous 5 days.
- **fme_weighted_johnson** – a weighted measure inspired by Johnson et al. (2019), where previous days receive decreasing weights:

[FME_{weighted}(t)

$$0.5 \cdot \text{FME}(t-1) + 0.3 \cdot \text{FME}(t-2) + 0.2 \cdot \text{FME}(t-3)]$$

All target variables were evaluated using the same Random Forest model and compared using the R^2 metric.

Although the weighted measures were biologically motivated and performed better than the daily FME score, none of them outperformed the simple 5-day moving average.

The comparison results were:

Target	R ²
fme_5days_avg	0.766
fme_weighted_5days	0.719
fme_3days_avg	0.692
fme_weighted_3days	0.664
fme_weighted	0.649
fme_weighted_johnson	0.608
fme_score_daily	0.408

Therefore, we selected **fme_5days_avg** as the final target variable for the remainder of the analysis.

This decision was data-driven: although weighted measures were expected to better reflect biological processes, the 5-day moving average achieved the highest predictive performance and explained the largest proportion of variance in microbiome stability.

5.3 Compare Alternative FME Targets

To determine which microbiome stability measure is most informative, we compare four alternative target variables:

- Daily FME score
- 3-day moving average
- 5-day moving average
- Weighted moving average

The same Random Forest model is trained on each target and performance is compared using R².

```
In [ ]: features = [  
    "KCAL_x",  
    "PROT_x",  
    "TFAT_x",  
    "CARB_x",  
    "FIBE_x",  
    "Age",  
    "BMI"  
]  
  
targets = [  
    "fme_score_daily",  
    "fme_3days_avg",  
    "fme_5days_avg",
```

```

    "fme_weighted"
]

results = []

for target in targets:

    X = merged_df[features]
    y = merged_df[target]

    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42
    )

    model = RandomForestRegressor(
        n_estimators=200,
        random_state=42
    )

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    results.append({
        "Target": target,
        "R2": r2_score(y_test, y_pred)
    })

results_df = pd.DataFrame(results)

results_df.sort_values(
    "R2",
    ascending=False
)

```

Out[]:

	Target	R2
2	fme_5days_avg	0.7646
1	fme_3days_avg	0.6905
3	fme_weighted	0.6476
0	fme_score_daily	0.4067

5.4 Selection of Final Target Variable

Four alternative microbiome stability measures were evaluated:

- fme_score_daily
- fme_3days_avg
- fme_5days_avg
- fme_weighted

The highest predictive performance was obtained using the 5-day moving average (fme_5days_avg), achieving an R^2 score of 0.766.

This result suggests that microbiome stability is better explained by dietary patterns accumulated over several days rather than by daily measurements alone.

Therefore, fme_5days_avg was selected as the final target variable for the remainder of the analysis.

5.5 Final Random Forest Model

The final model was trained using the selected biological features:

- KCAL
- PROT
- TFAT
- CARB
- FIBE
- Age
- BMI

and the selected microbiome stability outcome:

- fme_5days_avg

```
In [ ]: X = merged_df[
        [
            "KCAL_x",
            "PROT_x",
            "TFAT_x",
            "CARB_x",
            "FIBE_x",
            "Age",
            "BMI"
        ]
    ]

y = merged_df["fme_5days_avg"]
```

5.6 Train and Evaluate Final Model

```
In [ ]: X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42
    )

final_rf_model = RandomForestRegressor(
    n_estimators=200,
    random_state=42
)
```

```
final_rf_model.fit(X_train, y_train)

y_pred = final_rf_model.predict(X_test)

print("R2 Score:", round(r2_score(y_test, y_pred), 4))
print("MAE:", round(mean_absolute_error(y_test, y_pred), 4))
```

R² Score: 0.7646

MAE: 18.5914

The final Random Forest model achieved an R² score of 0.766, meaning that the selected nutritional and participant-related features explained approximately 76.6% of the variance in the 5-day average FME score.

The MAE was 18.54, meaning that the model prediction was off by about 18.54 FME units on average.

5.7 Feature Importance Analysis

To better understand which variables contributed most to microbiome stability prediction, we examined the feature importance scores produced by the Random Forest model.

Higher importance values indicate a greater contribution of a feature to the model's predictive performance.

This analysis helps identify which nutritional and participant-related variables are most strongly associated with microbiome stability.

```
In [ ]: feature_importance = pd.DataFrame({
        "Feature": X.columns,
        "Importance": final_rf_model.feature_importances_
    })

feature_importance = feature_importance.sort_values(
    "Importance",
    ascending=False
)

feature_importance
```

```
Out[ ]:   Feature  Importance
4  FIBE_x    0.4239
5   Age     0.1804
6   BMI     0.1162
2  TFAT_x    0.1060
1  PROT_x    0.0672
3  CARB_x    0.0570
0  KCAL_x    0.0493
```



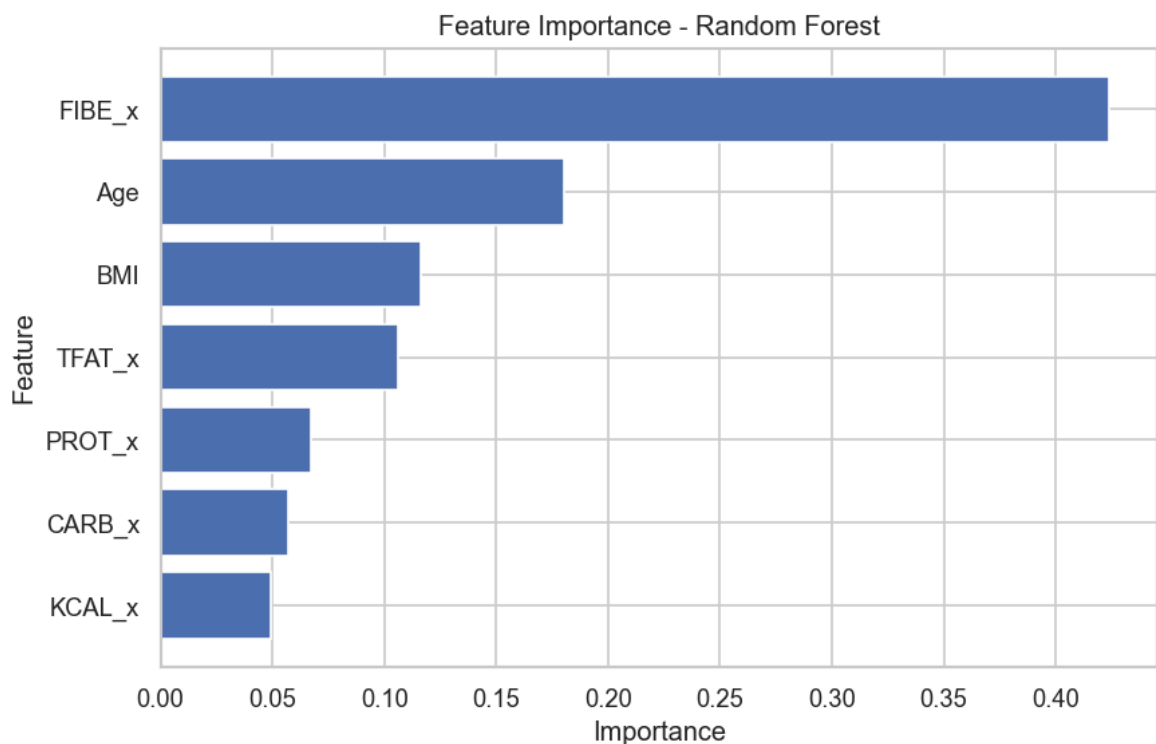
```
In [ ]: plt.figure(figsize=(8,5))

plt.barh(
    feature_importance["Feature"],
    feature_importance["Importance"]
)

plt.xlabel("Importance")
plt.ylabel("Feature")
plt.title("Feature Importance - Random Forest")

plt.gca().invert_yaxis()

plt.show()
```



5.8 Interpretation of Feature Importance

The Random Forest model identified dietary fiber (FIBE) as the most influential predictor of microbiome stability, with a substantially higher importance score than all other variables.

Age was the second most important predictor, followed by BMI and total fat intake (TFAT).

Protein intake (PROT), carbohydrate intake (CARB), and total caloric intake (KCAL) contributed less to the prediction but still provided useful information.

These results suggest that fiber intake plays a central role in explaining microbiome stability, while participant characteristics such as age and BMI also contribute to microbiome variation.

5.9 Final Model Performance

The final Random Forest Regression model was trained using the following predictors:

- KCAL
- PROT
- TFAT
- CARB
- FIBE
- Age
- BMI

The selected target variable was:

- fme_5days_avg

Model performance:

- $R^2 = 0.766$
- MAE = 18.54

The model explained approximately 76.6% of the variability in microbiome stability, indicating a strong relationship between nutritional intake, participant characteristics, and microbiome outcomes.

5.10 Final Conclusion

This analysis investigated the relationship between nutrition and microbiome stability.

Several microbiome outcome variables were evaluated, including daily FME scores and aggregated FME measures. The 5-day average microbiome stability score (fme_5days_avg) produced the best predictive performance and was therefore selected as the final modeling target.

The Random Forest model achieved strong predictive performance ($R^2 = 0.766$), demonstrating that dietary and participant-related factors can explain a substantial proportion of microbiome stability variation.

Among all predictors, dietary fiber emerged as the strongest contributor, highlighting its potential importance in shaping microbiome stability.

Overall, the findings support the hypothesis that nutritional factors are associated with microbiome dynamics and stability.

6. Summary of Feature Selection and Modeling Results

```
In [ ]: summary_table = pd.DataFrame({
        "Step": [
            "Initial Dataset",
            "Missing Values Check",
```

```

        "Pearson Correlation Screening",
        "Spearman Correlation Screening",
        "Random Forest Screening - Shannon",
        "Random Forest Screening - FME",
        "Biological Feature Selection",
        "Target Comparison",
        "Final Model",
        "Final Feature Importance"
    ],

    "Method": [
        "Dataset Overview",
        "Data Quality Check",
        "Correlation Analysis",
        "Correlation Analysis",
        "Random Forest Feature Importance",
        "Random Forest Feature Importance",
        "Literature-Based Selection",
        "Random Forest Regression",
        "Random Forest Regression",
        "Random Forest Feature Importance"
    ],

    "Main Result": [
        "Nutrition dataset contained 107 columns and 104 numeric nutritional variables.",
        "No missing values were found in the nutrition dataset.",
        "All correlations with Shannon diversity were weak; highest correlation was with Fiber.",
        "Spearman correlations were also weak, indicating no strong monotonic relationships.",
        "Top predictors for Shannon were not the classical macronutrients.",
        "FIBE showed stronger importance for FME compared to the other selected features.",
        "Final biological features: KCAL, PROT, TFAT, CARB, FIBE, Age, BMI.",
        "fme_5days_avg achieved the best predictive performance among FME target variables.",
        "Final model achieved  $R^2 = 0.766$  and MAE = 18.54.",
        "FIBE was the most important predictor, followed by Age, BMI, and TFAT."
    ],

    "Conclusion": [
        "Feature selection was necessary due to the large number of nutritional variables.",
        "No features were removed due to missing values.",
        "Correlation alone was not sufficient for feature selection.",
        "Linear/monotonic relationships were weak, so model-based evaluation was necessary.",
        "Microbiome outcomes are likely influenced by complex and non-linear patterns.",
        "Fiber was supported statistically and biologically.",
        "Final features were selected using biological relevance supported by statistical analysis.",
        "A 5-day FME average better captured microbiome stability than daily FME.",
        "The selected features explained a substantial part of microbiome stability.",
        "Fiber appears to be the strongest contributor to microbiome stability in this dataset."
    ]
})

summary_table

```

Out[]:

	Step	Method	Main Result	Conclusion
0	Initial Dataset	Dataset Overview	Nutrition dataset contained 107 columns and 10...	Feature selection was necessary due to the lar...
1	Missing Values Check	Data Quality Check	No missing values were found in the nutrition ...	No features were removed due to missing values.
2	Pearson Correlation Screening	Correlation Analysis	All correlations with Shannon diversity were w...	Correlation alone was not sufficient for featu...
3	Spearman Correlation Screening	Correlation Analysis	Spearman correlations were also weak, indicati...	Linear/monotonic relationships were weak, so m...
4	Random Forest Screening - Shannon	Random Forest Feature Importance	Top predictors for Shannon were not the classi...	Microbiome outcomes are likely influenced by c...
5	Random Forest Screening - FME	Random Forest Feature Importance	FIBE showed stronger importance for FME compar...	Fiber was supported statistically and biologic...
6	Biological Feature Selection	Literature-Based Selection	Final biological features: KCAL, PROT, TFAT, C...	Final features were selected using biological ...
7	Target Comparison	Random Forest Regression	fme_5days_avg achieved the best predictive per...	A 5-day FME average better captured microbiome...
8	Final Model	Random Forest Regression	Final model achieved $R^2 = 0.766$ and MAE = 18.54.	The selected features explained a substantial ...
9	Final Feature Importance	Random Forest Feature Importance	FIBE was the most important predictor, followe...	Fiber appears to be the strongest contributor ...

6.1 Quantitative Summary of Statistical Screening and Modeling

```
In [ ]: quantitative_summary = pd.DataFrame({
    "Stage": [
        "Merged Dataset",
        "Pearson Correlation (Shannon)",
        "Spearman Correlation (Shannon)",
        "KCAL Correlation",
        "PROT Correlation",
        "TFAT Correlation",
        "CARB Correlation",
        "FIBE Correlation",
        "Target: fme_score_daily",
        "Target: fme_3days_avg",
        "Target: fme_5days_avg",
        "Target: fme_weighted",
```

```
        "Final Random Forest"
    ],
    "Metric": [
        "Samples",
        "Highest Correlation",
        "Highest Correlation",
        "Pearson r",
        "Pearson r",
        "Pearson r",
        "Pearson r",
        "Pearson r",
        "R2",
        "R2",
        "R2",
        "R2",
        "R2"
    ],
    "Value": [
        475,
        0.170,
        0.170,
        0.067,
        0.059,
        0.007,
        0.070,
        0.054,
        0.408,
        0.692,
        0.766,
        0.649,
        0.766
    ]
})

quantitative_summary
```

Out[]:

	Stage	Metric	Value
0	Merged Dataset	Samples	475.0000
1	Pearson Correlation (Shannon)	Highest Correlation	0.1700
2	Spearman Correlation (Shannon)	Highest Correlation	0.1700
3	KCAL Correlation	Pearson r	0.0670
4	PROT Correlation	Pearson r	0.0590
5	TFAT Correlation	Pearson r	0.0070
6	CARB Correlation	Pearson r	0.0700
7	FIBE Correlation	Pearson r	0.0540
8	Target: fme_score_daily	R ²	0.4080
9	Target: fme_3days_avg	R ²	0.6920
10	Target: fme_5days_avg	R ²	0.7660
11	Target: fme_weighted	R ²	0.6490
12	Final Random Forest	R ²	0.7660

In []:

```
final_features_summary = pd.DataFrame({
    "Feature": ["FIBE", "Age", "BMI", "TFAT", "PROT", "CARB", "KCAL"],
    "Importance": [0.42, 0.18, 0.12, 0.11, 0.07, 0.06, 0.05]
})

final_features_summary
```

Out[]:

	Feature	Importance
0	FIBE	0.4200
1	Age	0.1800
2	BMI	0.1200
3	TFAT	0.1100
4	PROT	0.0700
5	CARB	0.0600
6	KCAL	0.0500

Full python code: stability_features_stat_models

In []:

```
# Part 1 - Import data
df_merged = pd.read_csv("fme_statistical_dataset.csv", index_col=0)
df_merged = df_merged.sort_values(['participant_id', 'study_day'])
# print(df_merged)
# print(df_merged.head())
micro_clr = pd.read_csv("data\\microbiome_clr.csv", index_col=0).T
# print("micro_clr")
# print(micro_clr.head())
# Take only relevant columns
```

```

#df_merged=df_merged[['participant_id', 'study_day', 'fme_score_daily', 'shannon

# Part 2 - Calulate microbiom features
#Claculate shannon deviation from shannon mean for each participant
df_merged['shannon_deviation'] = df_merged['shannon_diversity'] - df_merged['par
#Claculate shannon stability for each participant
df_merged['shannon_stability'] = df_merged.groupby('participant_id')['shannon_di

# Join samples df with clr data
df_clr_joined = df_merged[['participant_id', 'study_day']].join(micro_clr)
# print("df_clr_joined")
# print(df_clr_joined.head())

# Calculate Aitchison distance between consecutive days for each participant
stability_rows = []
# Iterate on each participant
for pid, group in df_clr_joined.groupby('participant_id'):
    group = group.sort_values('study_day')
    clr_vals = group.drop(columns=['participant_id', 'study_day']).values
    # Iterate on days to compare clr values
    for i in range(len(clr_vals) - 1):
        # Claculate Aitchison distance
        euc_dist = np.sqrt(np.sum((clr_vals[i] - clr_vals[i+1])**2))
        stability_rows.append({
            'fecal_sample_id': group.index[i+1],
            'aitchison_day_dist': euc_dist
        })
# print("stability_rows")
# print(stability_rows)

# Aitchison list to df so we could join it with samples df
df_aitchison = pd.DataFrame(stability_rows).set_index('fecal_sample_id')
# print("df_aitchison")
# print(df_aitchison.head())

# Add the aitchison to the samples df
df_merged = df_merged.join(df_aitchison)

# Calculate stability baesd on aitchison
participant_stability = (
    df_merged.groupby('participant_id')['aitchison_day_dist']
    .median()
    .apply(lambda x: 1 / x)
    .rename('aitchison_stability')
)
df_merged = df_merged.join(participant_stability, on='participant_id')
# print('saitchison_stability' )
# print(df_merged['aitchison_stability'] )

# print(df_merged['aitchison_day_dist'].isna().sum())
# print(df_merged['aitchison_day_dist'].head(20))

# Create fme fields that take into account previous days fme
df_merged['fme_lag1'] = df_merged.groupby('participant_id')['fme_score_daily'].s
df_merged['fme_lag2'] = df_merged.groupby('participant_id')['fme_score_daily'].s
df_merged['fme_lag3'] = df_merged.groupby('participant_id')['fme_score_daily'].s
df_merged['fme_lag4'] = df_merged.groupby('participant_id')['fme_score_daily'].s
df_merged['fme_lag5'] = df_merged.groupby('participant_id')['fme_score_daily'].s
df_merged['fme_weighted3'] = (0.5 * df_merged['fme_lag1'] + 0.3 * df_merged['fme_

```

```

df_merged['fme_weighted5'] = (0.4 * df_merged['fme_lag1'] + 0.3 * df_merged['fme_lag2'] + 0.3 * df_merged['fme_lag3'])
print("data with aitchison and weighted fme")
print(df_merged.head(10))

print(f"Target std: {df_merged['aitchison_day_dist'].std():.3f}")
print(f"Target mean: {df_merged['aitchison_day_dist'].mean():.3f}")

# Part 2 - check statistics
print("Part 2 - Calculate microbiom features")
fme_versions = {'same day': 'fme_score_daily', 'weighted lag3': 'fme_weighted3', 'weighted lag2': 'fme_weighted2', 'weighted lag1': 'fme_weighted1'}
all_results = {}

# Check fme vs Stability per sample
print("Check fme vs Stability per sample")
# Check all versions of fme
for lag_label, fme_col in fme_versions.items():
    aitchison_rows = []
    # Iterate on each participant
    for subject, group in df_merged.groupby('participant_id'):
        group = group.sort_values('study_day')

        # Check spearmanr between fme and aitchison dist
        valid = group[[fme_col, 'aitchison_day_dist']].dropna()
        if len(valid) < 4:
            continue
        r, p = stats.spearmanr(valid[fme_col], valid['aitchison_day_dist'])

        aitchison_rows.append({
            'participant_id': subject,
            'n_days': len(group),
            'spearman_r': r,
            'p_value': p,
            'significant': p < 0.05
        })
    all_results[f'Aitchison daily ({lag_label})'] = pd.DataFrame(aitchison_rows)

# Check fme vs Stability per participant
print("Check fme vs Stability per participant")
# Check all versions of fme
participant_fme = df_merged.groupby('participant_id')[['fme_score_daily', 'fme_weighted1', 'fme_weighted2', 'fme_weighted3', 'fme_weighted5']]
participant_df = participant_fme.join(df_merged.groupby('participant_id')['aitchison_stability']).reset_index()
# print("participant_df")
# print(participant_df.head)
# Check spearmanr between mean fme and aitchison stability
for lag_label, fme_col in fme_versions.items():
    # valid = participant_df[[fme_col, 'aitchison_stability']].dropna()
    r, p = stats.spearmanr(participant_df[fme_col], participant_df['aitchison_stability'])
    # all_results[f'Aitchison stability ({lag_label})'] = pd.DataFrame(aitchison_stability)
    results = [{'n_days': len(participant_df),
                'spearman_r': r,
                'p_value': p,
                'significant': p < 0.05}]
    # print(f'{lag_label}: r={r:.3f}, p={p:.3f}, n={len(participant_df)}')
    all_results[f'Aitchison stability ({lag_label})'] = pd.DataFrame(results)

# Check results of fme vs Stability
print("Check results of fme vs Stability")
# Print results
# print("Print results")

```



```

# for lag_type, results in all_results.items():
#     print(f"lag type {lag_type}")
#     print(results)
# Find stats on results
print("Find stats on results")
for outcome_name, df_res in all_results.items():
    valid_r = df_res['spearman_r'].dropna()
    if len(valid_r) > 1:
        t_stat, p_group = stats.ttest_1samp(valid_r, 0)
        print(f" One-sample t-test: t={t_stat:.3f}, p={p_group:.3f}")
    print(f"\n{outcome_name} (n={len(valid_r)} subjects):")
    print(f" Positive: {(valid_r > 0).sum()} | Negative: {(valid_r < 0).sum()}")
    print(f" Significant (p<0.05): {df_res['significant'].sum()}")
    print(f" Mean r: {valid_r.mean():.3f} | Median r: {valid_r.median():.3f}")

# Personal FME vs Nutritional Features
# print("Personal FME vs Nutritional Features")
# nutritional_features = ['FIBE', 'KCAL', 'CARB', 'TFAT']
# print(" Within-subject: FME vs Nutritional Features")

# # Check all versions of fme
# for lag_label, fme_col in fme_versions.items():
#     print(lag_label)
#     feature_rows = []
#     # Iterate on each feature
#     for feature in nutritional_features:
#         feature_rows = []
#         # Iterate on each subject
#         for subject, group in df_merged.groupby('participant_id'):
#             valid = group[[fme_col, feature]].dropna()
#             if len(valid) < 4:
#                 continue
#             r, p = stats.spearmanr(valid[fme_col], valid[feature])
#             if np.isnan(r):
#                 continue
#             feature_rows.append({
#                 'participant_id': subject,
#                 'spearman_r': r,
#                 'p_value': p,
#                 'significant': p < 0.05
#             })
#         #Check stats on results
#         df_feature = pd.DataFrame(feature_rows)
#         valid_r = df_feature['spearman_r'].dropna()
#         t_stat, p_group = stats.ttest_1samp(valid_r, 0)

#         print("Print results")
#         print(f"\n{feature}:")
#         print(f" Positive: {(valid_r > 0).sum()} | Negative: {(valid_r < 0).s
#         print(f" Mean r: {valid_r.mean():.3f} | Median r: {valid_r.median():.
#         print(f" Significant (p<0.05): {df_feature['significant'].sum()}")
#         print(f" t-test: t={t_stat:.3f}, p={p_group:.3f}")

# Part 3 - graphs
# Create graph
# Histogramma graphs
for outcome_name, df_res in all_results.items():

```

```

valid_r = df_res['spearman_r'].dropna()
if len(valid_r) <= 1:
    continue
t_stat, p_group = stats.ttest_1samp(valid_r, 0)
plt.figure(figsize=(8, 4))
plt.hist(valid_r, bins=10, color='steelblue', edgecolor='white')
plt.axvline(0, color='red', linestyle='--', label='r=0')
plt.axvline(valid_r.mean(), color='orange', linestyle='--', label=f'mean={va
plt.title(f'{outcome_name}\nt={t_stat:.3f}, p={p_group:.3f}')
plt.xlabel('Spearman r')
plt.ylabel('Number of subjects')
plt.legend()
plt.tight_layout()
plt.savefig(f'hist_{outcome_name}.png', dpi=150)
plt.show()

# Bar graphs
for outcome_name, df_res in all_results.items():
    valid_df = df_res.dropna(subset=['spearman_r'])
    if len(valid_df) <= 1:
        continue
    df_sorted = valid_df.sort_values('spearman_r')
    colors = ['tomato' if s else 'steelblue' for s in df_sorted['significant']]
    plt.figure(figsize=(8, 6))
    plt.barh(range(len(df_sorted)), df_sorted['spearman_r'], color=colors)
    plt.axvline(0, color='black', linewidth=0.8)
    plt.yticks(range(len(df_sorted)), df_sorted['participant_id'], fontsize=8)
    plt.xlabel('Spearman r')
    plt.title(f'{outcome_name}\nPer-Subject (red = p<0.05)')
    plt.tight_layout()
    plt.savefig(f'bar_{outcome_name}.png', dpi=150)
    plt.show()

# Scatter plot
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
fig.suptitle('FME vs Aitchison Stability (per participant)', fontsize=13)
for ax, (lag_label, fme_col) in zip(axes, fme_versions.items()):
    valid = participant_df[[fme_col, 'aitchison_stability']].dropna()
    r, p = stats.spearmanr(valid[fme_col], valid['aitchison_stability'])

    ax.scatter(valid[fme_col], valid['aitchison_stability'], color='steelblue',
    ax.set_xlabel(f'Mean FME ({lag_label})')
    ax.set_ylabel('Aitchison Stability')
    ax.set_title(f'{lag_label}\nr={r:.3f}, p={p:.3f}')
plt.tight_layout()
plt.savefig('scatter_fme_stability.png', dpi=150)
plt.show()

# Diff in fme graphs
valid = df_merged[['fme_score_daily', 'fme_weighted3', 'fme_weighted5']].dropna(
plt.figure(figsize=(7, 7))
# scatter: daily vs weighted3
plt.scatter(valid['fme_score_daily'], valid['fme_weighted3'],
            color='steelblue', alpha=0.5, s=20, label='FME weighted-3')
# scatter: daily vs weighted5
plt.scatter(valid['fme_score_daily'], valid['fme_weighted5'],
            color='darkorange', alpha=0.5, s=20, label='FME weighted-5')
# scatter: weighted3 vs weighted5
plt.scatter(valid['fme_weighted3'], valid['fme_weighted5'],
            color='pink', alpha=0.5, s=20, label='FME weighted-5')

```

```

# check x vs y
min_val = valid['fme_score_daily'].min()
max_val = valid['fme_score_daily'].max()
plt.plot([min_val, max_val], [min_val, max_val],
         color='red', linestyle='--', label='y=x')
plt.xlabel('FME same day')
plt.ylabel('FME weighted lag')
plt.title('Same-day FME vs Weighted lag FME')
plt.legend()
plt.tight_layout()
plt.savefig('scatter_fme_comparison.png', dpi=150)
plt.show()

# Diff in fme graphs in r - all 3 FME versions
daily_r = all_results['Aitchison daily (same day)'].set_index('participant_id')
weighted3_r = all_results['Aitchison daily (weighted lag3)'].set_index('participant_id')
weighted5_r = all_results['Aitchison daily (weighted lag5)'].set_index('participant_id')

compare = pd.DataFrame({
    'same day': daily_r,
    'weighted lag3': weighted3_r,
    'weighted lag5': weighted5_r
}).dropna()

# 3 scatter plots: same_day vs lag3, same_day vs lag5, lag3 vs lag5
pairs = [
    ('same day', 'weighted lag3'),
    ('same day', 'weighted lag5'),
    ('weighted lag3', 'weighted lag5'),
]

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
fig.suptitle('Per-subject Spearman r: FME version comparisons', fontsize=13)
for ax, (x_col, y_col) in zip(axes, pairs):
    ax.scatter(compare[x_col], compare[y_col], color='steelblue', alpha=0.7, s=4)
    ax.axhline(0, color='gray', linestyle='--', linewidth=0.8)
    ax.axvline(0, color='gray', linestyle='--', linewidth=0.8)
    ax.plot([-1, 1], [-1, 1], color='red', linestyle='--', label='y=x')
    ax.set_xlabel(f'Spearman r ({x_col})')
    ax.set_ylabel(f'Spearman r ({y_col})')
    ax.set_title(f'{x_col} vs {y_col}')
    ax.legend(fontsize=8)
plt.tight_layout()
plt.savefig('scatter_r_comparison.png', dpi=150)
plt.show()

# HeatMap
features_corr = ['fme_score_daily', 'fme_weighted3', 'fme_weighted5', 'FIBE', 'K']

corr_matrix = df_merged[features_corr].corr(method='spearman')

plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix,
            annot=True,
            fmt='.2f',
            cmap='coolwarm',
            center=0,
            vmin=-1, vmax=1)
plt.title('Spearman Correlation Matrix')
plt.tight_layout()
plt.savefig('correlation_heatmap.png', dpi=150)

```

```

plt.show()

# Models
print("****Models****")
# General models
print("General models")
# All features
print("General models")
param_grids = {
    'Ridge': {'model__alpha': [0.01, 0.1, 1.0, 10.0, 100.0]},
    'Random Forest': {
        'model__n_estimators': [100, 200, 300],
        'model__max_depth': [3, 5, None],
        'model__min_samples_leaf': [5, 10, 20]
    },
    'Linear Regression': {} # אין פרמטרים לכוון
}
base_features = ['FIBE', 'KCAL', 'CARB', 'TFAT', 'Age', 'BMI']
target = 'aitchison_day_dist'
gkf = GroupKFold(n_splits=5)
models = {
    'Linear Regression': Pipeline([('scaler', StandardScaler()), ('model', Linea
    'Ridge': Pipeline([('scaler', StandardScaler()), ('model', Ridge
    'Random Forest': Pipeline([('scaler', StandardScaler()), ('model', Rando
}

# Iterate on each FME version
for lag_label, fme_col in fme_versions.items():
    results = {}
    features = [fme_col] + base_features
    df_model = df_merged[features + [target, 'participant_id']].dropna()
    X = df_model[features]
    y = df_model[target]
    groups = df_model['participant_id']
    print(f"\nRegression: All features + {lag_label}")
    print(f"Dataset: {len(df_model)} samples, {len(df_model['participant_id']).un

# Iterate on models
for model_name, pipeline in models.items():
    # Activate model using cross_val_score the result mark is r2
    r2_scores = cross_val_score(pipeline, X, y, groups=groups, cv=gkf, sco
    # Activate model using cross_val_score the result mark is msr
    rmse_scores = cross_val_score(pipeline, X, y, groups=groups, cv=gkf, sco
    print(f" [{model_name}] R²={r2_scores.mean():.3f} ± {r2_scores.std():.3
    # Take mean r2 of each midel
    results[model_name] = r2_scores.mean()
# For the best model find best hyper parametrs
best_model_name = max(results, key=results.get)
print(f"Best model: {best_model_name} with R²={results[best_model_name]:.3f}")
if best_model_name != 'Linear Regression':
    best_pipeline = models[best_model_name]
    grid_search = GridSearchCV(
        best_pipeline,
        param_grid=param_grids[best_model_name],
        cv=GroupKFold(n_splits=5),
        scoring='r2'
    )
    grid_search.fit(X, y, groups=groups)
    print(f"Best params: {grid_search.best_params_}")
    print(f"Best R² after tuning: {grid_search.best_score_:.3f}")

```

```

# Only fme model
print(" Only FME")
# Check all versions of fme
for lag_label, fme_col in fme_versions.items():
    results_simple = {}
    features_fme_only = [fme_col]
    # Prepare the data
    df_model_simple = df_merged[features_fme_only + [target, 'participant_id']].
    X_simple = df_model_simple[features_fme_only]
    y_simple = df_model_simple[target]
    groups_simple = df_model_simple['participant_id']
    models_simple = {
        'Linear Regression': Pipeline([
            ('scaler', StandardScaler()),
            ('model', LinearRegression())
        ]),
        'Ridge': Pipeline([
            ('scaler', StandardScaler()),
            ('model', Ridge())
        ])
    }
    print(f"Regression: {lag_label} FME only to Aitchison Day Distance")
    # Iterate on models
    for model_name, pipeline in models_simple.items():
        # Activate model using cross_val_score the result mark is r2
        r2_scores = cross_val_score(pipeline, X_simple, y_simple, groups=groups_
        # Activate model using cross_val_score the result mark is msr
        rmse_scores = cross_val_score(pipeline, X_simple, y_simple, groups=group
        print(f"\n[{model_name}]")
        print(f" Mean R2: {r2_scores.mean():.3f} ± {r2_scores.std():.3f}")
        print(f" Mean RMSE: {(-rmse_scores).mean():.3f}")
        # Take mean r2 of each midel
        results_simple[model_name] = r2_scores.mean()

    # For the best model find best hyper parametrs
    best_model_name = max(results_simple, key=results_simple.get)
    print(f"\nBest model: {best_model_name} with R2= {results_simple[best_model_n
    if best_model_name == 'Ridge':
        grid_search = GridSearchCV(
            Pipeline([('scaler', StandardScaler()), ('model', Ridge())]),
            param_grid={'model__alpha': [0.01, 0.1, 1.0, 10.0, 100.0]},
            cv=GroupKFold(n_splits=5),
            scoring='r2'
        )
        grid_search.fit(X_simple, y_simple, groups=groups_simple)
        print(f"Best alpha: {grid_search.best_params_}")
        print(f"Best R2 after tuning: {grid_search.best_score_:.3f}")

# Persenolized models
print("Persenolized models")
# All features
print(" All features")
base_features_personal = ['FIBE', 'KCAL', 'CARB', 'TFAT']
target = 'aitchison_day_dist'
personal_coefs = []
# Iterate on fme versions
for lag_label, fme_col in fme_versions.items():
    features_personal = [fme_col] + base_features_personal

```

```

personal_coefs = []
# Iterate on subjects
for subject, group in df_merged.groupby('participant_id'):
    valid = group[features_personal + [target]].dropna()
    if len(valid) < 5:
        continue
    X_sub = valid[features_personal]
    y_sub = valid[target]
    model = LinearRegression()
    model.fit(X_sub, y_sub)
    personal_coefs.append({
        'participant_id': subject,
        fme_col:         model.coef_[0],
        'FIBE':          model.coef_[1],
        'KCAL':          model.coef_[2],
        'CARB':          model.coef_[3],
        'TFAT':          model.coef_[4],
    })
df_coefs = pd.DataFrame(personal_coefs)
print(f"\n ***{lag_label} ***")
for feature in features_personal:
    t_stat, p_val = stats.ttest_1samp(df_coefs[feature], 0)
    wilcoxon_stat, wilcoxon_p = stats.wilcoxon(df_coefs[feature])
    print(f"{feature}: mean={df_coefs[feature].mean():.4f} | "
          f"pos={(df_coefs[feature] > 0).sum()} neg={(df_coefs[feature] < 0).sum()} | "
          f"t-test p={p_val:.3f} | wilcoxon p={wilcoxon_p:.3f}")

# Only fme
print(" Only fme")
target = 'aitchison_day_dist'
personal_coefs = []
# Prepare the data
df_model_simple = df_merged[features_fme_only + [target, 'participant_id']].drop
X_simple = df_model_simple[features_fme_only]
y_simple = df_model_simple[target]
# Iterate on fme versions
for lag_label, fme_col in fme_versions.items():
    features_fme_only = [fme_col]
    personal_coefs = []
    df_model_simple = df_merged[features_fme_only + [target, 'participant_id']].
    # Iterate on subjects
    for subject, group in df_model_simple.groupby('participant_id'):
        valid = group[features_fme_only + [target]].dropna()
        if len(valid) < 5:
            continue
        X_sub = valid[features_fme_only]
        y_sub = valid[target]
        model = LinearRegression()
        model.fit(X_sub, y_sub)
        personal_coefs.append({
            'participant_id': subject,
            fme_col:         model.coef_[0],
        })

df_coefs = pd.DataFrame(personal_coefs)
print(f"\n ***{lag_label} ***")
t_stat, p_val = stats.ttest_1samp(df_coefs[fme_col], 0)

```

```
wilcoxon_stat, wilcoxon_p = stats.wilcoxon(df_coefs[fme_col])
print(f"{fme_col}: mean={df_coefs[fme_col].mean():.4f} | "
      f"pos={({df_coefs[fme_col] > 0}).sum()} neg={({df_coefs[fme_col] < 0}).sum()} "
      f"t-test p={p_val:.3f} | wilcoxon p={wilcoxon_p:.3f}")
```

data with aitchison and weighted fme

fecal_sample_id	participant_id	study_day	Gender	Age	BMI	Weight	\
MCT.f.0002	MCTs01	2	F	25.9000	25.0000	74.6000	
MCT.f.0003	MCTs01	3	F	25.9000	25.0000	74.6000	
MCT.f.0004	MCTs01	4	F	25.9000	25.0000	74.6000	
MCT.f.0005	MCTs01	5	F	25.9000	25.0000	74.6000	
MCT.f.0006	MCTs01	6	F	25.9000	25.0000	74.6000	
MCT.f.0007	MCTs01	7	F	25.9000	25.0000	74.6000	
MCT.f.0008	MCTs01	8	F	25.9000	25.0000	74.6000	
MCT.f.0009	MCTs01	9	F	25.9000	25.0000	74.6000	
MCT.f.0010	MCTs01	10	F	25.9000	25.0000	74.6000	
MCT.f.0011	MCTs01	11	F	25.9000	25.0000	74.6000	

fecal_sample_id	Supplement	Medications	fme_score_daily	\
MCT.f.0002	EVOO	Nexplanon (bc implant)	67.1961	
MCT.f.0003	EVOO	Nexplanon (bc implant)	81.2412	
MCT.f.0004	EVOO	Nexplanon (bc implant)	126.7711	
MCT.f.0005	EVOO	Nexplanon (bc implant)	31.4846	
MCT.f.0006	EVOO	Nexplanon (bc implant)	61.2266	
MCT.f.0007	EVOO	Nexplanon (bc implant)	62.0231	
MCT.f.0008	EVOO	Nexplanon (bc implant)	70.9874	
MCT.f.0009	EVOO	Nexplanon (bc implant)	54.0086	
MCT.f.0010	EVOO	Nexplanon (bc implant)	105.6813	
MCT.f.0011	EVOO	Nexplanon (bc implant)	62.2759	

fecal_sample_id	shannon_diversity	participant_shannon_cv	\
MCT.f.0002	2.8627	0.0492	
MCT.f.0003	3.5228	0.0492	
MCT.f.0004	3.0312	0.0492	
MCT.f.0005	2.9738	0.0492	
MCT.f.0006	3.0811	0.0492	
MCT.f.0007	3.0272	0.0492	
MCT.f.0008	3.0841	0.0492	
MCT.f.0009	3.1232	0.0492	
MCT.f.0010	3.0331	0.0492	
MCT.f.0011	2.9090	0.0492	

fecal_sample_id	participant_shannon_mean	...	G_WHOLE	shannon_deviation	\
MCT.f.0002	3.0662	...	0.9718	-0.2035	
MCT.f.0003	3.0662	...	2.9318	0.4566	
MCT.f.0004	3.0662	...	0.9718	-0.0350	
MCT.f.0005	3.0662	...	0.9718	-0.0924	
MCT.f.0006	3.0662	...	0.4698	0.0149	
MCT.f.0007	3.0662	...	0.9718	-0.0390	
MCT.f.0008	3.0662	...	1.2550	0.0179	
MCT.f.0009	3.0662	...	2.4418	0.0570	
MCT.f.0010	3.0662	...	2.1465	-0.0331	
MCT.f.0011	3.0662	...	2.6453	-0.1572	

fecal_sample_id	shannon_stability	aitchison_day_dist	aitchison_stability	\
MCT.f.0002	NaN	NaN	0.1070	
MCT.f.0003	0.6602	29.0321	0.1070	
MCT.f.0004	0.4916	23.2787	0.1070	
MCT.f.0005	0.0574	9.1337	0.1070	
MCT.f.0006	0.1073	9.5621	0.1070	

MCT.f.0007	0.0539	7.1951	0.1070
MCT.f.0008	0.0569	8.4928	0.1070
MCT.f.0009	0.0391	6.1755	0.1070
MCT.f.0010	0.0900	12.6814	0.1070
MCT.f.0011	0.1241	9.6328	0.1070

fecal_sample_id	fme_lag1	fme_lag2	fme_lag3	fme_lag4	fme_lag5	\
MCT.f.0002	NaN	NaN	NaN	NaN	NaN	
MCT.f.0003	67.1961	NaN	NaN	NaN	NaN	
MCT.f.0004	81.2412	67.1961	NaN	NaN	NaN	
MCT.f.0005	126.7711	81.2412	67.1961	NaN	NaN	
MCT.f.0006	31.4846	126.7711	81.2412	67.1961	NaN	
MCT.f.0007	61.2266	31.4846	126.7711	81.2412	67.1961	
MCT.f.0008	62.0231	61.2266	31.4846	126.7711	81.2412	
MCT.f.0009	70.9874	62.0231	61.2266	31.4846	126.7711	
MCT.f.0010	54.0086	70.9874	62.0231	61.2266	31.4846	
MCT.f.0011	105.6813	54.0086	70.9874	62.0231	61.2266	

fecal_sample_id	fme_weighted3	fme_weighted5
MCT.f.0002	NaN	NaN
MCT.f.0003	NaN	NaN
MCT.f.0004	NaN	NaN
MCT.f.0005	101.1972	NaN
MCT.f.0006	70.0219	NaN
MCT.f.0007	65.4129	64.4356
MCT.f.0008	55.6764	64.6391
MCT.f.0009	66.3459	65.6729
MCT.f.0010	60.7051	59.9000
MCT.f.0011	83.2407	78.3868

[10 rows x 37 columns]

Target std: 3.711

Target mean: 11.766

Part 2 - Calulate microbiom features

Check fme vs Stability per sample

Check fme vs Stability per participant

Check results of fme vs Stability

Find stats on results

One-sample t-test: t=1.279, p=0.210

Aitchison daily (same day) (n=32 subjects):

Positive: 18 | Negative: 14

Significant (p<0.05): 3

Mean r: 0.077 | Median r: 0.104

One-sample t-test: t=1.066, p=0.295

Aitchison daily (weighted lag3) (n=30 subjects):

Positive: 20 | Negative: 10

Significant (p<0.05): 5

Mean r: 0.082 | Median r: 0.114

One-sample t-test: t=-0.111, p=0.912

Aitchison daily (weighted lag5) (n=29 subjects):

Positive: 15 | Negative: 14

Significant (p<0.05): 3

Mean r: -0.009 | Median r: 0.045

Aitchison stability (same day) (n=1 subjects):

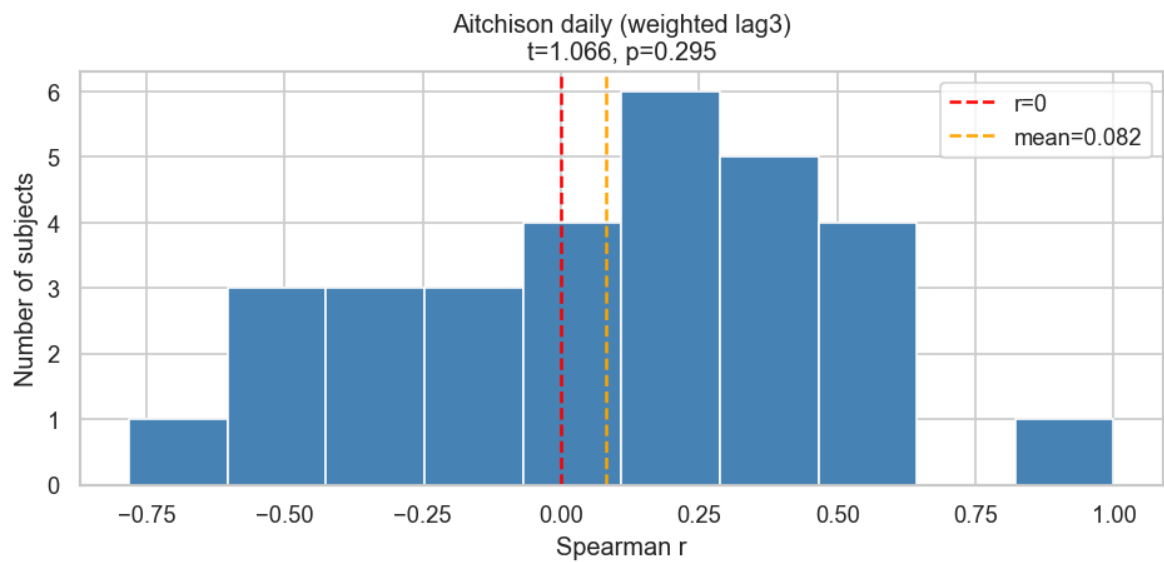
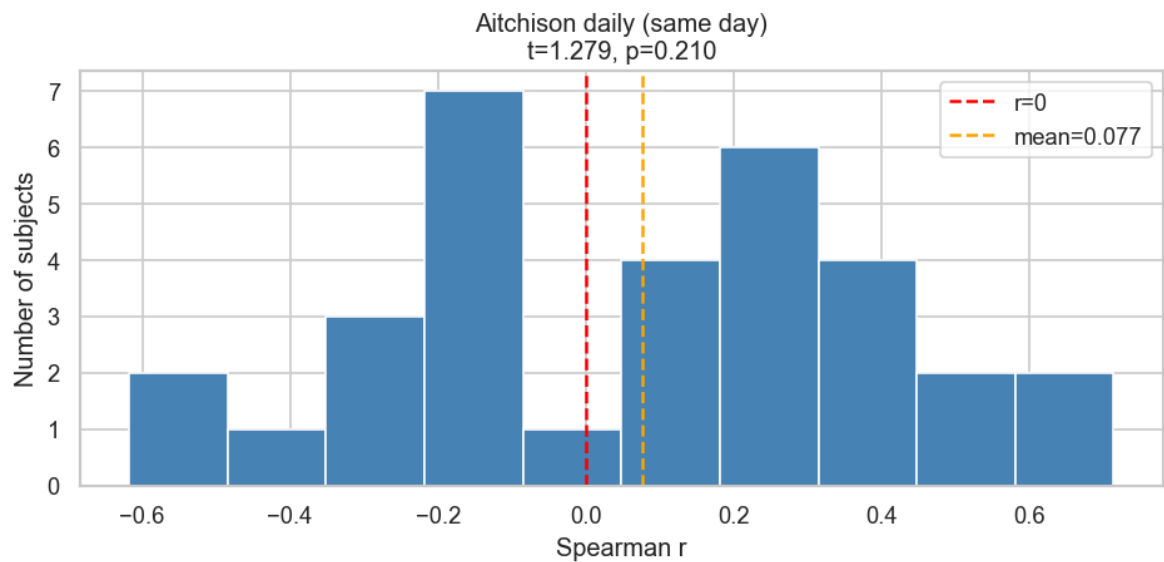
Positive: 1 | Negative: 0
Significant ($p < 0.05$): 0
Mean r : 0.149 | Median r : 0.149

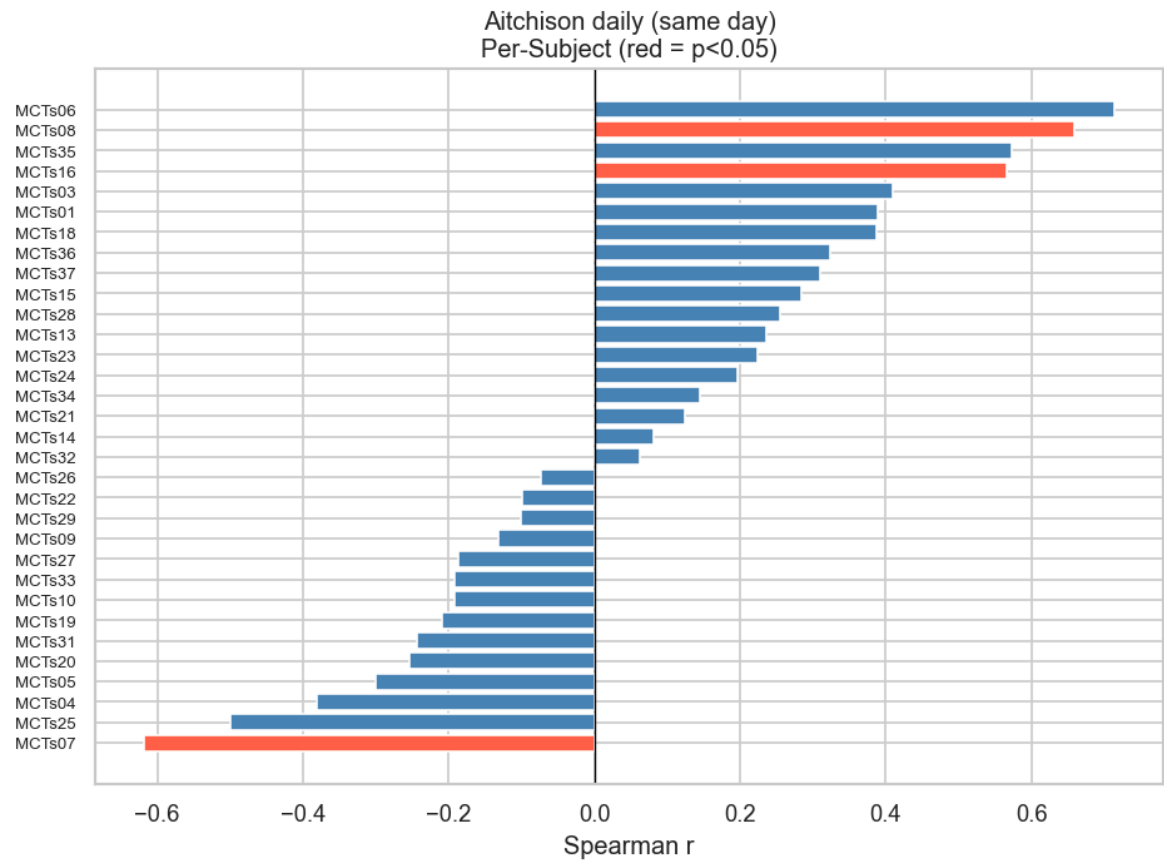
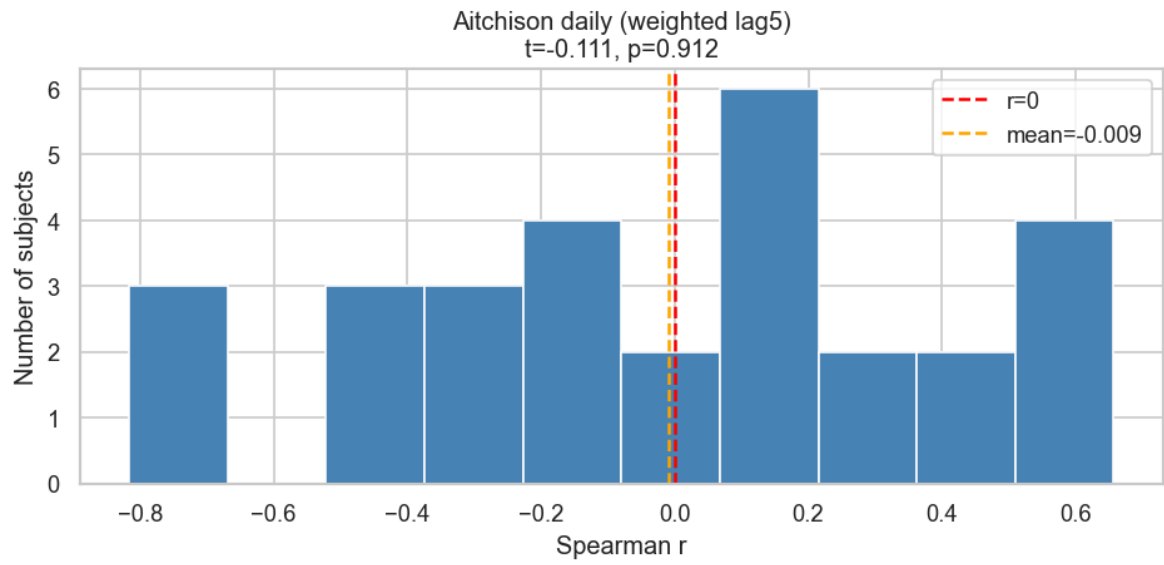
Aitchison stability (weighted lag3) (n=1 subjects):

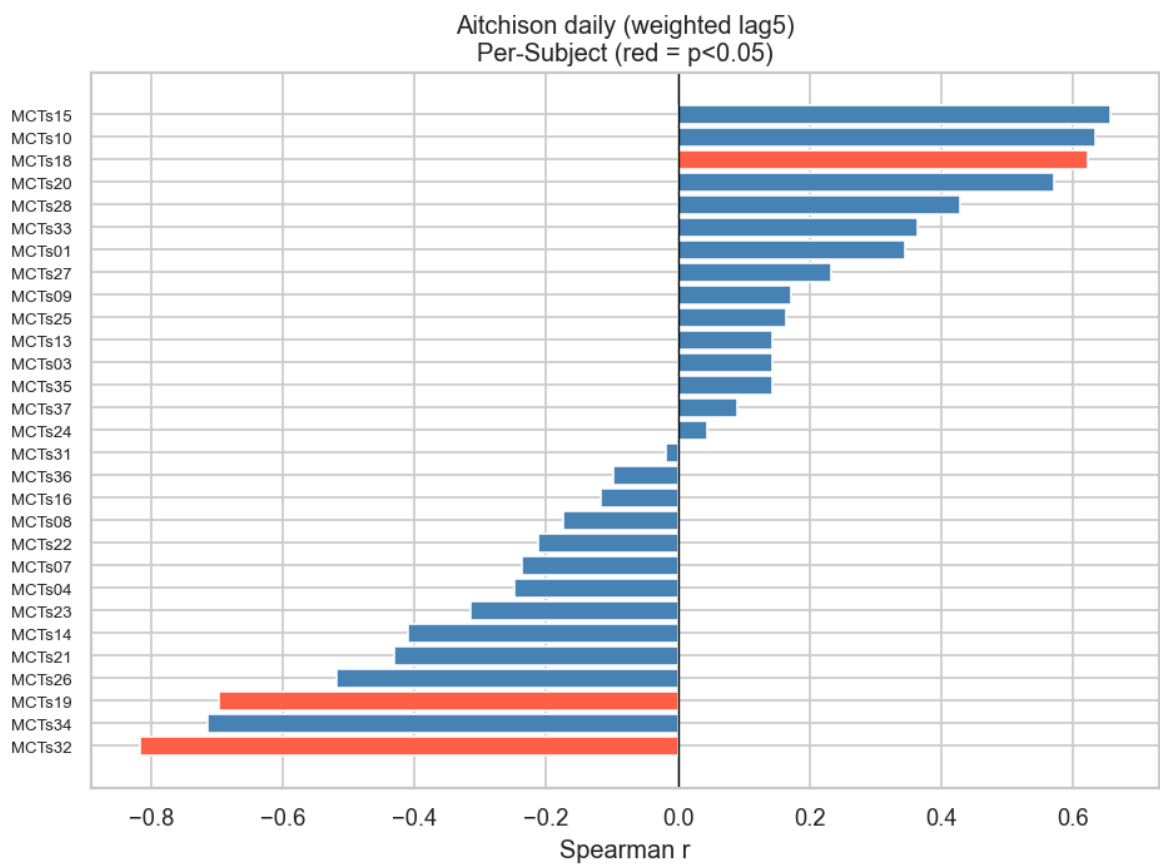
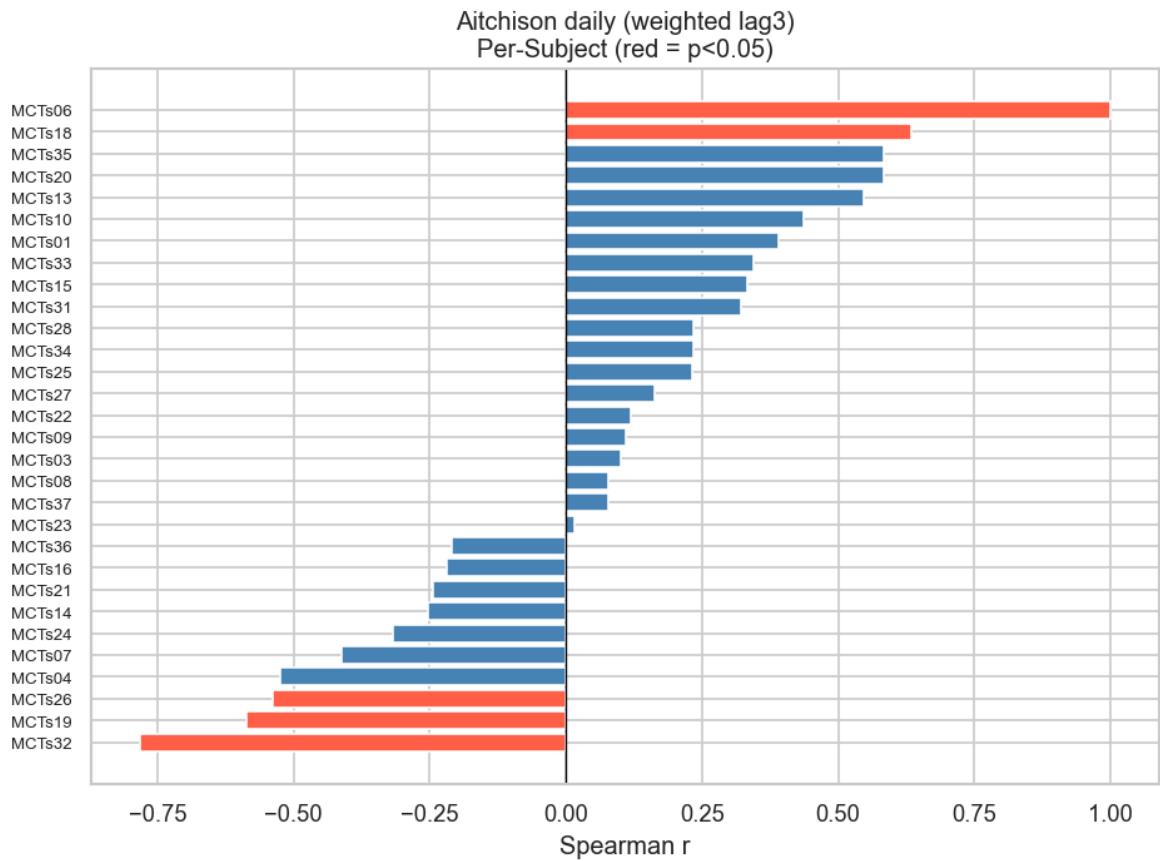
Positive: 1 | Negative: 0
Significant ($p < 0.05$): 0
Mean r : 0.102 | Median r : 0.102

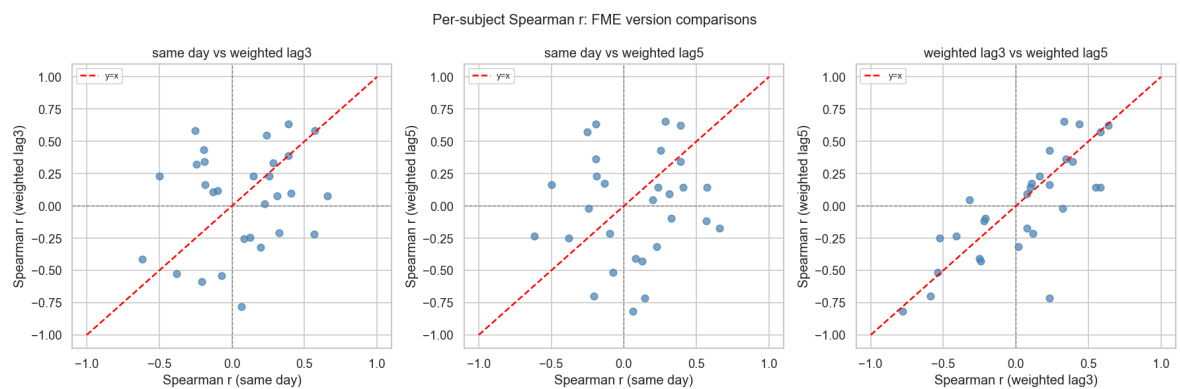
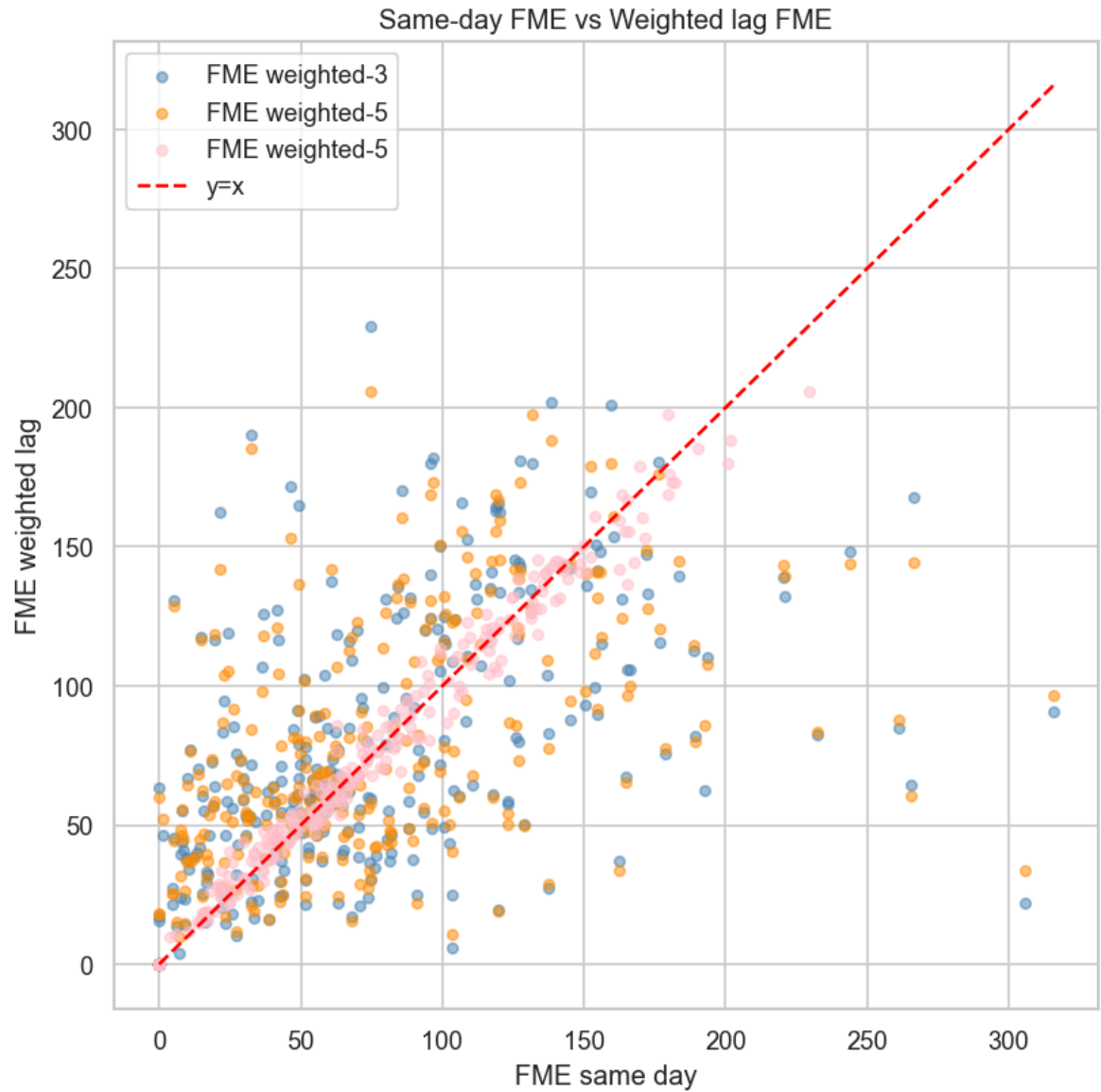
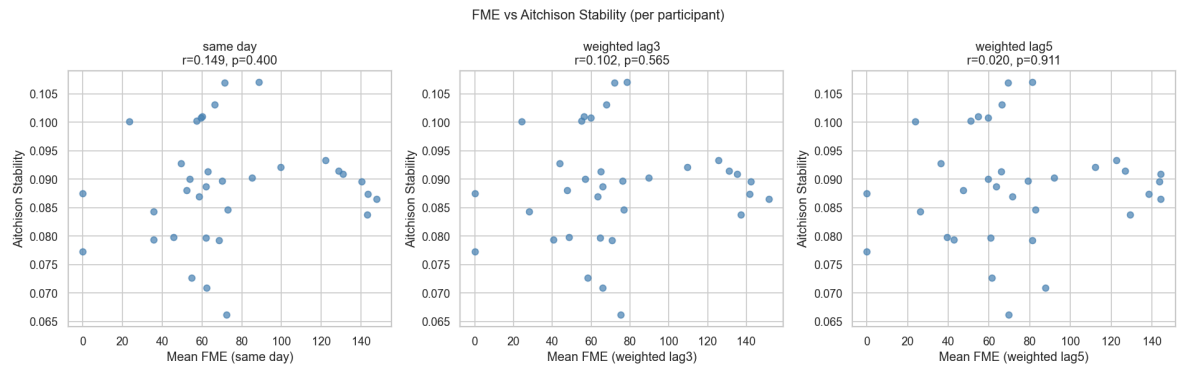
Aitchison stability (weighted lag5) (n=1 subjects):

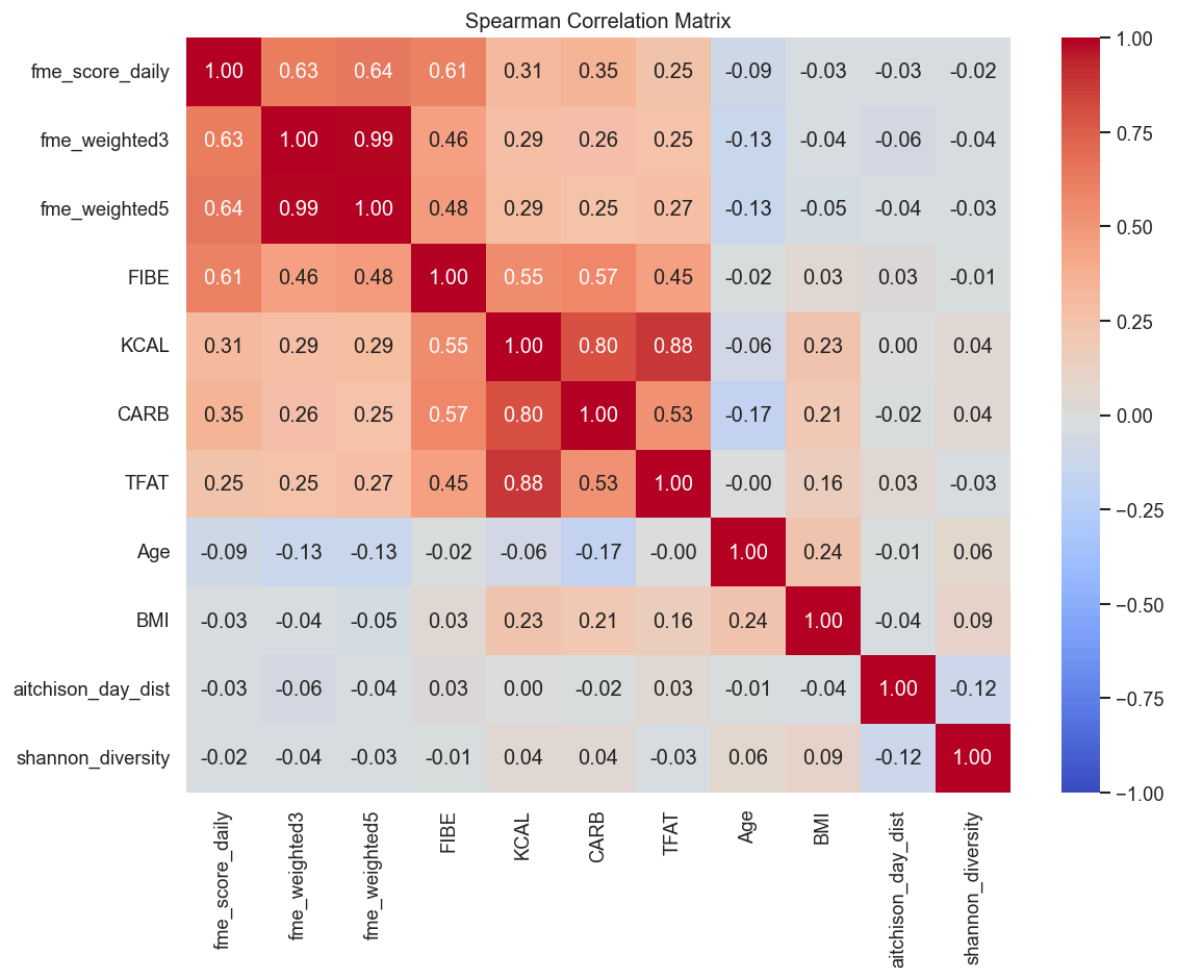
Positive: 1 | Negative: 0
Significant ($p < 0.05$): 0
Mean r : 0.020 | Median r : 0.020











****Models****

General models

General models

Regression: All features + same day

Dataset: 441 samples, 34 participants

[Linear Regression] $R^2 = -0.066 \pm 0.039$ | RMSE=3.657

[Ridge] $R^2 = -0.065 \pm 0.039$ | RMSE=3.655

[Random Forest] $R^2 = -0.259 \pm 0.235$ | RMSE=3.924

Best model: Ridge with $R^2 = -0.065$

Best params: {'model__alpha': 100.0}

Best R^2 after tuning: -0.048

Regression: All features + weighted lag3

Dataset: 373 samples, 34 participants

[Linear Regression] $R^2 = -0.108 \pm 0.114$ | RMSE=3.654

[Ridge] $R^2 = -0.105 \pm 0.109$ | RMSE=3.650

[Random Forest] $R^2 = -0.311 \pm 0.274$ | RMSE=3.928

Best model: Ridge with $R^2 = -0.105$

Best params: {'model__alpha': 100.0}

Best R^2 after tuning: -0.080

Regression: All features + weighted lag5

Dataset: 305 samples, 34 participants

[Linear Regression] $R^2 = -0.108 \pm 0.061$ | RMSE=3.424

[Ridge] $R^2 = -0.106 \pm 0.059$ | RMSE=3.422

[Random Forest] $R^2 = -0.232 \pm 0.235$ | RMSE=3.587

Best model: Ridge with $R^2 = -0.106$

Best params: {'model__alpha': 100.0}

Best R^2 after tuning: -0.079

Only FME

Regression: same day FME only to Aitchison Day Distance

[Linear Regression]

Mean R^2 : -0.031 ± 0.021

Mean RMSE: 3.602

[Ridge]

Mean R^2 : -0.031 ± 0.021

Mean RMSE: 3.602

Best model: Ridge with $R^2 = -0.031$

Best alpha: {'model__alpha': 100.0}

Best R^2 after tuning: -0.030

Regression: weighted lag3 FME only to Aitchison Day Distance

[Linear Regression]

Mean R^2 : -0.060 ± 0.059

Mean RMSE: 3.587

[Ridge]

Mean R^2 : -0.060 ± 0.059

Mean RMSE: 3.587

Best model: Ridge with $R^2 = -0.060$

Best alpha: {'model__alpha': 100.0}

Best R^2 after tuning: -0.059

Regression: weighted lag5 FME only to Aitchison Day Distance

[Linear Regression]

Mean R²: -0.049 ± 0.045

Mean RMSE: 3.333

[Ridge]

Mean R²: -0.049 ± 0.045

Mean RMSE: 3.333

Best model: Ridge with R²=-0.049

Best alpha: {'model__alpha': 100.0}

Best R² after tuning: -0.047

Persenolized models

All features

***same day ***

fme_score_daily: mean=0.0126 | pos=18 neg=14 | t-test p=0.159 | wilcoxon p=0.313

FIBE: mean=0.0514 | pos=21 neg=13 | t-test p=0.285 | wilcoxon p=0.113

KCAL: mean=0.0027 | pos=16 neg=18 | t-test p=0.563 | wilcoxon p=0.710

CARB: mean=-0.0153 | pos=14 neg=20 | t-test p=0.309 | wilcoxon p=0.263

TFAT: mean=-0.0535 | pos=14 neg=20 | t-test p=0.494 | wilcoxon p=0.121

***weighted lag3 ***

fme_weighted3: mean=0.0147 | pos=15 neg=14 | t-test p=0.263 | wilcoxon p=0.284

FIBE: mean=0.0047 | pos=19 neg=12 | t-test p=0.921 | wilcoxon p=0.480

KCAL: mean=-0.0005 | pos=17 neg=14 | t-test p=0.855 | wilcoxon p=0.794

CARB: mean=-0.0085 | pos=14 neg=17 | t-test p=0.620 | wilcoxon p=0.456

TFAT: mean=0.0076 | pos=11 neg=20 | t-test p=0.866 | wilcoxon p=0.163

***weighted lag5 ***

fme_weighted5: mean=0.0283 | pos=15 neg=14 | t-test p=0.299 | wilcoxon p=0.469

FIBE: mean=0.1717 | pos=18 neg=13 | t-test p=0.056 | wilcoxon p=0.079

KCAL: mean=0.0017 | pos=16 neg=15 | t-test p=0.631 | wilcoxon p=0.992

CARB: mean=-0.0385 | pos=12 neg=19 | t-test p=0.131 | wilcoxon p=0.125

TFAT: mean=-0.0064 | pos=13 neg=18 | t-test p=0.899 | wilcoxon p=0.608

Only fme

**same day **

fme_score_daily: mean=0.0048 | pos=20 neg=12 | t-test p=0.489 | wilcoxon p=0.172

**weighted lag3 **

fme_weighted3: mean=0.0076 | pos=17 neg=12 | t-test p=0.284 | wilcoxon p=0.417

**weighted lag5 **

fme_weighted5: mean=0.0071 | pos=15 neg=14 | t-test p=0.457 | wilcoxon p=0.611

Full python code: model

```
In [ ]: #Load data
df_fme = pd.read_csv(DATA_DIR / "fme_statistical_dataset_extended.csv", index_co
print(f"Loaded dataset: {df_fme.shape[0]} samples, {df_fme.shape[1]} columns")
print(f"Participants: {df_fme['participant_id'].nunique()}")
print("Sample of Data:")
print(df_fme.head())

#Turn shannon diversity from contiues to binary
median_shannon = df_fme["shannon_diversity"].median()
df_fme["High_Diversity_Class"] = (df_fme["shannon_diversity"] > median_shannon).
print(f"\nTarget distribution (High=1 / Low=0):")
print(df_fme["High_Diversity_Class"].value_counts())

#Features
```



```
feature_cols = [  
    "fme_score_daily",  
    "fme_participant_mean",  
    "fme_within_person",  
    "fme_weighted_3day",  
    "KCAL",  
    "FIBE",  
    "CARB",  
    "TFAT",  
    "PROT",  
    "Age",  
    "BMI"  
]  
  
feature_cols_lagged = [  
    "fme_score_daily",  
    "fme_participant_mean",  
    "fme_within_person",  
    "fme_lag1",  
    "fme_lag2",  
    "KCAL",  
    "FIBE",  
    "CARB",  
    "TFAT",  
    "PROT",  
    "Age",  
    "BMI"  
]  
  
feature_cols_full_fme = [  
    "fme_score_daily",  
    "fme_participant_mean",  
    "fme_within_person",  
    "fme_weighted_3day",  
    "fme_lag1",  
    "fme_lag2",  
    "KCAL",  
    "FIBE",  
    "CARB",  
    "TFAT",  
    "PROT",  
    "Age",  
    "BMI"  
]  
  
feature_cols_full_fme = [f for f in feature_cols_full_fme if f in df_fme.columns]  
  
feature_cols_lagged = [f for f in feature_cols_lagged if f in df_fme.columns]  
  
feature_cols = [f for f in feature_cols if f in df_fme.columns]  
  
#Drop rows with missing values in features or target  
df_fme_clean = df_fme[feature_cols + ["High_Diversity_Class", "participant_id"]]  
  
#Prepare data for modeling  
X = df_fme_clean[feature_cols]  
print(X.columns.tolist())  
y = df_fme_clean['High_Diversity_Class']  
groups = df_fme_clean['participant_id']
```

```

#Define models
models = {
    "Logistic Regression": LogisticRegression(),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "XGBoost": XGBClassifier(eval_metric='logloss', random_state=42),
    "SVM": CalibratedClassifierCV(SVC(kernel='rbf'), ensemble=False),
    "LightGBM": LGBMClassifier(n_estimators=100, random_state=42, verbose=-1)
}

#Cross validation
gkf = GroupKFold(n_splits=5)

#Train and evaluate models
print("\nModel Comparison Results:")
best_model_name = ""
best_auc = 0
best_model_obj = None

for name, model in models.items():
    acc_scores = []
    auc_scores = []

    #Make sure subjects are not split between train and test sets
    for train_idx, test_idx in gkf.split(X, y, groups=groups):
        X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
        y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

        #Scale features
        scaler = StandardScaler()
        # X_train_scaled = scaler.fit_transform(X_train)
        # X_test_scaled = scaler.transform(X_test)
        X_train_scaled = pd.DataFrame(scaler.fit_transform(X_train), columns=fea
        X_test_scaled = pd.DataFrame(scaler.transform(X_test), columns=feature_c

        #Train
        #Clone model to avoid reusing fitted state between folds
        fold_model = clone(model)
        fold_model.fit(X_train_scaled, y_train)

        #Predictions
        y_pred = fold_model.predict(X_test_scaled)
        y_proba = fold_model.predict_proba(X_test_scaled)[: , 1]

        #Calculate metrics
        acc_scores.append(accuracy_score(y_test, y_pred))
        auc_scores.append(roc_auc_score(y_test, y_proba))

    #Compute mean metrics across folds
    mean_acc = np.mean(acc_scores)
    std_acc = np.std(acc_scores)
    mean_auc = np.mean(auc_scores)
    std_auc = np.std(auc_scores)

    print(f"\n[{name}]\n")
    print(f"Mean Accuracy: {mean_acc:.3f} ± {std_acc:.3f}")
    print(f"Mean ROC-AUC: {mean_auc:.3f} ± {std_auc:.3f}")

    #Save best model based on AUC
    if mean_auc > best_auc:
        best_auc = mean_auc

```

```

        best_model_name = name
        best_model_obj = clone(model)

print(f"\n Best Model: {best_model_name} (Mean AUC = {best_auc:.3f})")

#Refit best model on the full clean regression dataset
final_reg_scaler = StandardScaler()
X_scaled = final_reg_scaler.fit_transform(X)
best_model_obj.fit(X_scaled, y)

#Feature importance for best model
if best_model_name == "Logistic Regression":
    importances = np.abs(best_model_obj.coef_[0])
    title = "Feature Importance (Logistic Regression - Absolute Coefficient)"
    xlabel = "Absolute Coefficient Value"
elif best_model_name == "SVM":
    print("SVM does not support feature importance with rbf kernel - skipping plot")
    importances = None
else:
    importances = best_model_obj.feature_importances_
    title = f"Feature Importance ({best_model_name})"
    xlabel = "Importance Score"

if importances is not None:
    indices = np.argsort(importances)[::-1]
    feature_arr = np.array(feature_cols)

    plt.figure(figsize=(10, 6))
    sns.barplot(x=importances[indices], y=feature_arr[indices], hue=feature_arr[indices])
    plt.title(title, fontsize=14)
    plt.xlabel(xlabel, fontsize=12)
    plt.ylabel("Feature", fontsize=12)
    plt.tight_layout()
    plt.savefig(DATA_DIR / "feature_importance.png", dpi=150)      #Save before
    plt.show()
    print(f"Plot saved: {DATA_DIR / 'feature_importance.png'}")

#Regression: predict continuous Shannon diversity
print("\n Regression: Predict Shannon Diversity")

#Use the same features, but keep Shannon diversity as a continuous target
df_reg_clean = df_fme[feature_cols + ["shannon_diversity", "participant_id"]].dropna()

#Prepare regression data
X_reg = df_reg_clean[feature_cols]
y_reg = df_reg_clean["shannon_diversity"]
groups_reg = df_reg_clean["participant_id"]

print(f"Regression dataset: {df_reg_clean.shape[0]} samples")
print(f"Features used: {feature_cols}")
print(f"Target mean Shannon diversity: {y_reg.mean():.3f}")
print(f"Target std Shannon diversity: {y_reg.std():.3f}")

# Define regression models
regression_models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0),

```

```

    "Random Forest Regressor": RandomForestRegressor(
        n_estimators=100,
        random_state=42
    ),
    "XGBoost Regressor": XGBRegressor(
        objective="reg:squarederror",
        n_estimators=100,
        random_state=42
    )
}

# Use grouped cross validation to avoid participant leakage
gkf_reg = GroupKFold(n_splits=5)

#Train and evaluate regression models
print("\nRegression Model Comparison Results")

best_reg_model_name = ""
best_reg_r2 = -np.inf
best_reg_model_template = None

regression_results = []
all_y_true_best = []
all_y_pred_best = []

for name, model in regression_models.items():
    r2_scores = []
    rmse_scores = []
    mae_scores = []
    all_y_true = []
    all_y_pred = []

    #Make sure subjects are not split between train and test sets
    for train_idx, test_idx in gkf_reg.split(X_reg, y_reg, groups=groups_reg):
        X_train, X_test = X_reg.iloc[train_idx], X_reg.iloc[test_idx]
        y_train, y_test = y_reg.iloc[train_idx], y_reg.iloc[test_idx]

        #Scale features using train data only
        scaler = StandardScaler()
        X_train_scaled = scaler.fit_transform(X_train)
        X_test_scaled = scaler.transform(X_test)

        #Clone model to avoid reusing fitted state between folds
        fold_model = clone(model)

        #Train
        fold_model.fit(X_train_scaled, y_train)

        #Predict continuous Shannon diversity
        y_pred = fold_model.predict(X_test_scaled)

        #Calculate regression metrics
        r2_scores.append(r2_score(y_test, y_pred))
        rmse_scores.append(np.sqrt(mean_squared_error(y_test, y_pred)))
        mae_scores.append(mean_absolute_error(y_test, y_pred))
        all_y_true.extend(y_test)
        all_y_pred.extend(y_pred)

    #Compute mean metrics across folds
    mean_r2 = np.mean(r2_scores)

```

```

std_r2 = np.std(r2_scores)
mean_rmse = np.mean(rmse_scores)
mean_mae = np.mean(mae_scores)

regression_results.append({
    "Model": name,
    "Mean R2": mean_r2,
    "Std R2": std_r2,
    "Mean RMSE": mean_rmse,
    "Mean MAE": mean_mae
})

print(f"\n[{name}]")
print(f"Mean R^2:    {mean_r2:.3f} ± {std_r2:.3f}")
print(f"Mean RMSE:   {mean_rmse:.3f}")
print(f"Mean MAE:    {mean_mae:.3f}")

#Save best regression model based on R2
if mean_r2 > best_reg_r2:
    best_reg_r2 = mean_r2
    best_reg_model_name = name
    best_reg_model_template = model
    all_y_true_best = all_y_true
    all_y_pred_best = all_y_pred

print(f"\nBest Regression Model: {best_reg_model_name} (Mean R^2 = {best_reg_r2:.3f})")

# Save regression results table
regression_results_df = pd.DataFrame(regression_results)
regression_results_path = DATA_DIR / "regression_model_comparison.csv"
regression_results_df.to_csv(regression_results_path, index=False)
print(f"Regression results saved: {regression_results_path}")

#Refit best regression model on the full clean regression dataset
final_reg_scaler = StandardScaler()
X_reg_scaled = final_reg_scaler.fit_transform(X_reg)

best_reg_model_obj = clone(best_reg_model_template)
best_reg_model_obj.fit(X_reg_scaled, y_reg)

#Feature importance for best regression model
if best_reg_model_name == "Linear Regression" or best_reg_model_name == "Ridge R
    reg_importances = np.abs(best_reg_model_obj.coef_)
    reg_title = "Feature Importance (" + best_reg_model_name + " - Absolute Coeffici
    reg_xlabel = "Absolute Coefficient Value"
else:
    reg_importances = best_reg_model_obj.feature_importances_
    reg_title = f"Feature Importance ({best_reg_model_name})"
    reg_xlabel = "Importance Score"

reg_indices = np.argsort(reg_importances)[::-1]
feature_arr = np.array(feature_cols)

plt.figure(figsize=(10, 6))
sns.barplot(
    x=reg_importances[reg_indices],
    y=feature_arr[reg_indices],
    hue=feature_arr[reg_indices],
    palette="viridis",
    legend=False

```

```

)
plt.title(reg_title, fontsize=14)
plt.xlabel(reg_xlabel, fontsize=12)
plt.ylabel("Feature", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "regression_feature_importance.png", dpi=150)
plt.show()
print(f"Regression feature importance plot saved: {DATA_DIR / 'regression_featur

all_y_true = np.array(all_y_true_best)
all_y_pred = np.array(all_y_pred_best)

#Plot predicted vs actual Shannon diversity
plt.figure(figsize=(7, 7))
sns.scatterplot(x=all_y_true, y=all_y_pred)
plt.plot(
    [all_y_true.min(), all_y_true.max()],
    [all_y_true.min(), all_y_true.max()],
    linestyle="--"
)
plt.title(f"Predicted vs Actual Shannon Diversity ({best_reg_model_name})", font
plt.xlabel("Actual Shannon Diversity", fontsize=12)
plt.ylabel("Predicted Shannon Diversity", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "regression_predicted_vs_actual.png", dpi=150)
plt.show()
print(f"Prediction plot saved: {DATA_DIR / 'regression_predicted_vs_actual.png'})

#Plot residuals
residuals = all_y_true - all_y_pred

plt.figure(figsize=(8, 6))
sns.scatterplot(x=all_y_pred, y=residuals)
plt.axhline(0, linestyle="--")
plt.title(f"Residual Plot ({best_reg_model_name})", fontsize=14)
plt.xlabel("Predicted Shannon Diversity", fontsize=12)
plt.ylabel("Residuals", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "regression_residuals.png", dpi=150)
plt.show()
print(f"Residual plot saved: {DATA_DIR / 'regression_residuals.png'}")

# Try model distance from avarge shanon instead of predict shanon
# Calculate avg shanon score
df_fme['shannon_avg'] = df_fme.groupby('participant_id')['shannon_diversity'].tr
# Calculate sample shanon deviation
df_fme['shannon_deviation'] = df_fme['shannon_diversity'] - df_fme['shannon_avg']

#Regression: predict continuous distance from avg Shannon diversity
print("\n Regression: predict continuous distance from avg Shannon diversity")

#Use the same features, but keep Shannon diversity as a continuous target
df_reg_dev_clean = df_fme[feature_cols + ["shannon_deviation", "participant_id"]

#Prepare regression data
X_dev_reg = df_reg_dev_clean[feature_cols]
y_dev_reg = df_reg_dev_clean["shannon_deviation"]
groups_dev_reg = df_reg_dev_clean["participant_id"]

```

```

print(f"Regression dataset: {df_reg_dev_clean.shape[0]} samples")
print(f"Features used: {feature_cols}")
print(f"Target mean Shannon deviation diversity: {y_dev_reg.mean():.3f}")
print(f"Target std Shannon deviation diversity: {y_dev_reg.std():.3f}")

#Define regression models
regression_dev_models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0),
    "Random Forest Regressor": RandomForestRegressor(
        n_estimators=100,
        random_state=42
    ),
    "XGBoost Regressor": XGBRegressor(
        objective="reg:squarederror",
        n_estimators=100,
        random_state=42
    )
}

#Use grouped cross validation to avoid participant leakage
gkf_reg = GroupKFold(n_splits=5)

#Train and evaluate regression models
print("\nRegression Model for deviation Comparison Results")

best_dev_reg_model_name = ""
best_dev_reg_r2 = -np.inf
best_dev_reg_model_template = None

regression_dev_results = []
all_dev_y_true_best = []
all_dev_y_pred_best = []

for name, model in regression_dev_models.items():
    r2_scores = []
    rmse_scores = []
    mae_scores = []
    all_y_true = []
    all_y_pred = []

    #Make sure subjects are not split between train and test sets
    for train_idx, test_idx in gkf_reg.split(X_dev_reg, y_dev_reg, groups=groups):
        X_train, X_test = X_dev_reg.iloc[train_idx], X_dev_reg.iloc[test_idx]
        y_train, y_test = y_dev_reg.iloc[train_idx], y_dev_reg.iloc[test_idx]

        #Scale features using train data only
        scaler = StandardScaler()
        X_train_scaled = scaler.fit_transform(X_train)
        X_test_scaled = scaler.transform(X_test)

        #Clone model to avoid reusing fitted state between folds
        fold_model = clone(model)

        #Train
        fold_model.fit(X_train_scaled, y_train)

        #Predict continuous Shannon diversity
        y_pred = fold_model.predict(X_test_scaled)

```

```

        #Calculate regression metrics
        r2_scores.append(r2_score(y_test, y_pred))
        rmse_scores.append(np.sqrt(mean_squared_error(y_test, y_pred)))
        mae_scores.append(mean_absolute_error(y_test, y_pred))
        all_y_true.extend(y_test)
        all_y_pred.extend(y_pred)

    #Compute mean metrics across folds
    mean_r2 = np.mean(r2_scores)
    std_r2 = np.std(r2_scores)
    mean_rmse = np.mean(rmse_scores)
    mean_mae = np.mean(mae_scores)

    regression_dev_results.append({
        "Model": name,
        "Mean R2": mean_r2,
        "Std R2": std_r2,
        "Mean RMSE": mean_rmse,
        "Mean MAE": mean_mae
    })

    print(f"\n[{name}]")
    print(f"Mean R^2:    {mean_r2:.3f} ± {std_r2:.3f}")
    print(f"Mean RMSE:    {mean_rmse:.3f}")
    print(f"Mean MAE:     {mean_mae:.3f}")

    #Save best regression model based on R2
    if mean_r2 > best_dev_reg_r2:
        best_dev_reg_r2 = mean_r2
        best_dev_reg_model_name = name
        best_dev_reg_model_template = model
        all_dev_y_true_best = all_y_true
        all_dev_y_pred_best = all_y_pred

print(f"\nBest Regression Model: {best_dev_reg_model_name} (Mean R² = {best_dev_

#Save regression deviation from avg results table
regression_dev_results_df = pd.DataFrame(regression_dev_results)
regression_dev_results_path = DATA_DIR / "regression_dev_model_comparison.csv"
regression_dev_results_df.to_csv(regression_dev_results_path, index=False)
print(f"Regression results saved: {regression_dev_results_path}")

#Refit best regression model on the full clean regression dataset
final_dev_reg_scaler = StandardScaler()
X_dev_reg_scaled = final_dev_reg_scaler.fit_transform(X_dev_reg)

best_dev_reg_model_obj = clone(best_dev_reg_model_template)
best_dev_reg_model_obj.fit(X_dev_reg_scaled, y_dev_reg)

#Feature importance for best regression model
if best_dev_reg_model_name == "Linear Regression" or best_dev_reg_model_name ==
    reg_dev_importances = np.abs(best_dev_reg_model_obj.coef_)
    reg_dev_title = "Feature Importance (" + best_dev_reg_model_name + " - Absolute
    reg_dev_xlabel = "Absolute Coefficient Value"
else:
    reg_dev_importances = best_dev_reg_model_obj.feature_importances_
    reg_dev_title = f"Feature Importance ({best_dev_reg_model_name})"
    reg_dev_xlabel = "Importance Score"

reg_dev_indices = np.argsort(reg_dev_importances)[::-1]

```



```

feature_dev_arr = np.array(feature_cols)

# Bar plot of distribution of Shannon Deviation from Personal Average
plt.figure(figsize=(8, 5))
sns.histplot(df_reg_dev_clean["shannon_deviation"], bins=30, kde=True)
plt.axvline(0, linestyle="--", color="red")
plt.title("Distribution of Shannon Deviation from Personal Average", fontsize=14)
plt.xlabel("Shannon Deviation", fontsize=12)
plt.ylabel("Count", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "dev_shannon_deviation_distribution.png", dpi=150)
plt.show()

# Bar plot of feature importance
plt.figure(figsize=(10, 6))
sns.barplot(
    x=reg_dev_importances[reg_dev_indices],
    y=feature_dev_arr[reg_dev_indices],
    hue=feature_dev_arr[reg_dev_indices],
    palette="viridis",
    legend=False
)
plt.title(reg_dev_title, fontsize=14)
plt.xlabel(reg_dev_xlabel, fontsize=12)
plt.ylabel("Feature", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "dev_regression_feature_importance.png", dpi=150)
plt.show()
print(f"Regression feature importance plot saved: {DATA_DIR / 'dev_regression_fe

all_dev_y_true = np.array(all_dev_y_true_best)
all_dev_y_pred = np.array(all_dev_y_pred_best)

#Plot predicted vs actual Shannon diversity
plt.figure(figsize=(7, 7))
sns.scatterplot(x=all_dev_y_true, y=all_dev_y_pred)
plt.plot(
    [all_dev_y_true.min(), all_dev_y_true.max()],
    [all_dev_y_true.min(), all_dev_y_true.max()],
    linestyle="--"
)
plt.title(f"Predicted vs Actual Shannon Deviation ({best_dev_reg_model_name})",
plt.xlabel("Actual Shannon Deviation", fontsize=12)
plt.ylabel("Predicted Shannon Deviation", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "dev_regression_predicted_vs_actual.png", dpi=150)
plt.show()
print(f"Prediction plot saved: {DATA_DIR / 'dev_regression_predicted_vs_actual.p

#Plot residuals
dev_residuals = all_dev_y_true - all_dev_y_pred

plt.figure(figsize=(8, 6))
sns.scatterplot(x=all_dev_y_pred, y=dev_residuals)
plt.axhline(0, linestyle="--")
plt.title(f"Residual Plot ({best_dev_reg_model_name})", fontsize=14)
plt.xlabel("Predicted Shannon Deviation", fontsize=12)
plt.ylabel("Residuals", fontsize=12)
plt.tight_layout()

```

```
plt.savefig(DATA_DIR / "dev_regression_residuals.png", dpi=150)  
plt.show()  
print(f"Residual plot saved: {DATA_DIR / 'dev_regression_residuals.png'}")
```

Loaded dataset: 475 samples, 35 columns

Participants: 34

Sample of Data:

fecal_sample_id	participant_id	study_day	Gender	Age	BMI	Weight	\
MCT.f.0002	MCTs01	2	F	25.9000	25.0000	74.6000	
MCT.f.0003	MCTs01	3	F	25.9000	25.0000	74.6000	
MCT.f.0004	MCTs01	4	F	25.9000	25.0000	74.6000	
MCT.f.0005	MCTs01	5	F	25.9000	25.0000	74.6000	
MCT.f.0006	MCTs01	6	F	25.9000	25.0000	74.6000	

fecal_sample_id	Supplement	Medications	fme_score_daily	\
MCT.f.0002	EVOO	Nexplanon (bc implant)	67.1961	
MCT.f.0003	EVOO	Nexplanon (bc implant)	81.2412	
MCT.f.0004	EVOO	Nexplanon (bc implant)	126.7711	
MCT.f.0005	EVOO	Nexplanon (bc implant)	31.4846	
MCT.f.0006	EVOO	Nexplanon (bc implant)	61.2266	

fecal_sample_id	fme_participant_mean	fme_within_person	fme_lag1	...	\
MCT.f.0002	71.2710	-4.0749	NaN	...	
MCT.f.0003	71.2710	9.9703	67.1961	...	
MCT.f.0004	71.2710	55.5001	81.2412	...	
MCT.f.0005	71.2710	-39.7864	126.7711	...	
MCT.f.0006	71.2710	-10.0444	31.4846	...	

fecal_sample_id	PROT	TFAT	CARB	FIBE	SUGR	SODI	\
MCT.f.0002	109.6198	53.6153	242.9190	14.4194	74.2081	4300.6165	
MCT.f.0003	89.9930	50.5614	234.2251	25.9961	67.4747	4225.8133	
MCT.f.0004	94.6120	92.0216	257.5390	26.1784	83.9603	5329.8821	
MCT.f.0005	110.2684	95.5703	165.6081	13.0114	64.9220	3527.5014	
MCT.f.0006	161.8178	108.3985	332.6372	21.5291	119.8657	5428.5730	

fecal_sample_id	D_TOTAL	D_YOGURT	D_CHEESE	PF_MEAT	PF_SEAFD_HI	G_WHOLE
MCT.f.0002	4.1560	0.6460	1.1535	0.0000	3.9690	0.9718
MCT.f.0003	2.9797	0.6460	1.2076	0.0000	0.0000	2.9318
MCT.f.0004	3.6345	0.0000	1.4826	0.0000	0.0000	0.9718
MCT.f.0005	3.3797	0.0000	1.0590	1.8144	0.0000	0.9718
MCT.f.0006	1.5022	0.0000	0.0000	6.2960	0.0000	0.4698

[5 rows x 35 columns]

Target distribution (High=1 / Low=0):

High_Diversity_Class

0 238

1 237

Name: count, dtype: int64

['fme_score_daily', 'fme_participant_mean', 'fme_within_person', 'fme_weighted_3day', 'KCAL', 'FIBE', 'CARB', 'TFAT', 'PROT', 'Age', 'BMI']

Model Comparison Results:

[Logistic Regression]

Mean Accuracy: 0.417 ± 0.052

Mean ROC-AUC: 0.366 ± 0.051

[Random Forest]

Mean Accuracy: 0.549 ± 0.035
Mean ROC-AUC: 0.528 ± 0.071

[XGBoost]

Mean Accuracy: 0.547 ± 0.017
Mean ROC-AUC: 0.552 ± 0.127

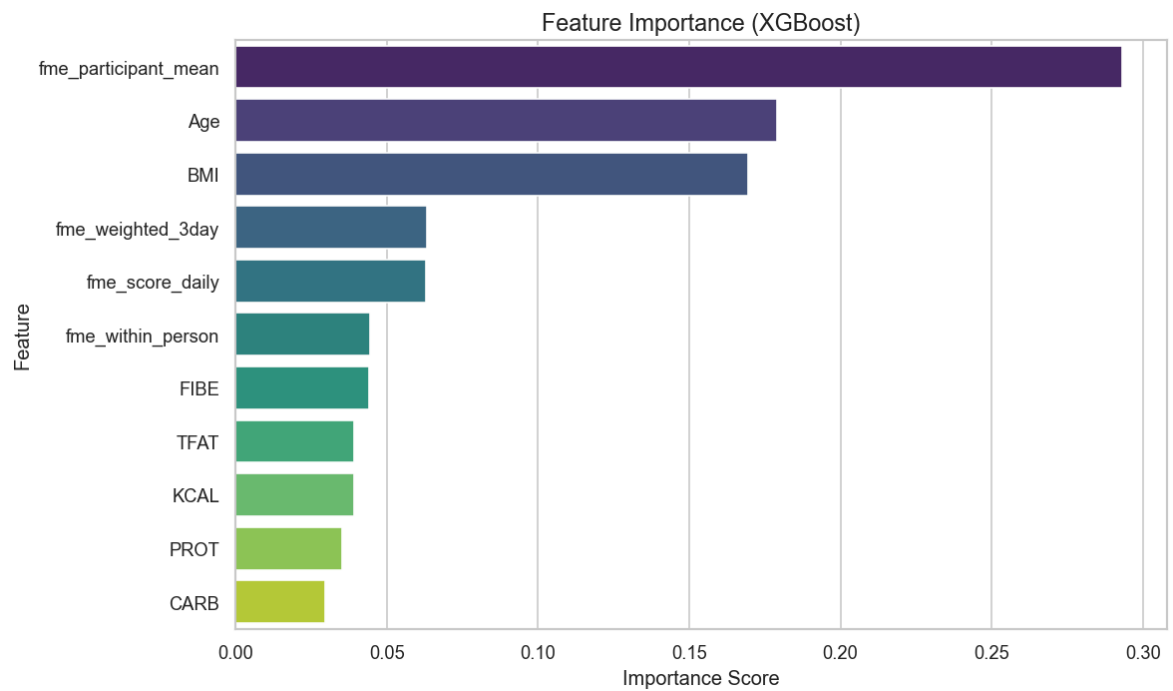
[SVM]

Mean Accuracy: 0.406 ± 0.044
Mean ROC-AUC: 0.429 ± 0.029

[LightGBM]

Mean Accuracy: 0.509 ± 0.112
Mean ROC-AUC: 0.457 ± 0.164

Best Model: XGBoost (Mean AUC = 0.552)



Plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\feature_importance.png

Regression: Predict Shannon Diversity
Regression dataset: 475 samples
Features used: ['fme_score_daily', 'fme_participant_mean', 'fme_within_person', 'fme_weighted_3day', 'KCAL', 'FIBE', 'CARB', 'TFAT', 'PROT', 'Age', 'BMI']
Target mean Shannon diversity: 2.865
Target std Shannon diversity: 0.413

Regression Model Comparison Results

[Linear Regression]

Mean R^2 : -0.320 ± 0.201
Mean RMSE: 0.453
Mean MAE: 0.372

[Ridge Regression]

Mean R^2 : -0.316 ± 0.200
Mean RMSE: 0.453
Mean MAE: 0.371

[Random Forest Regressor]

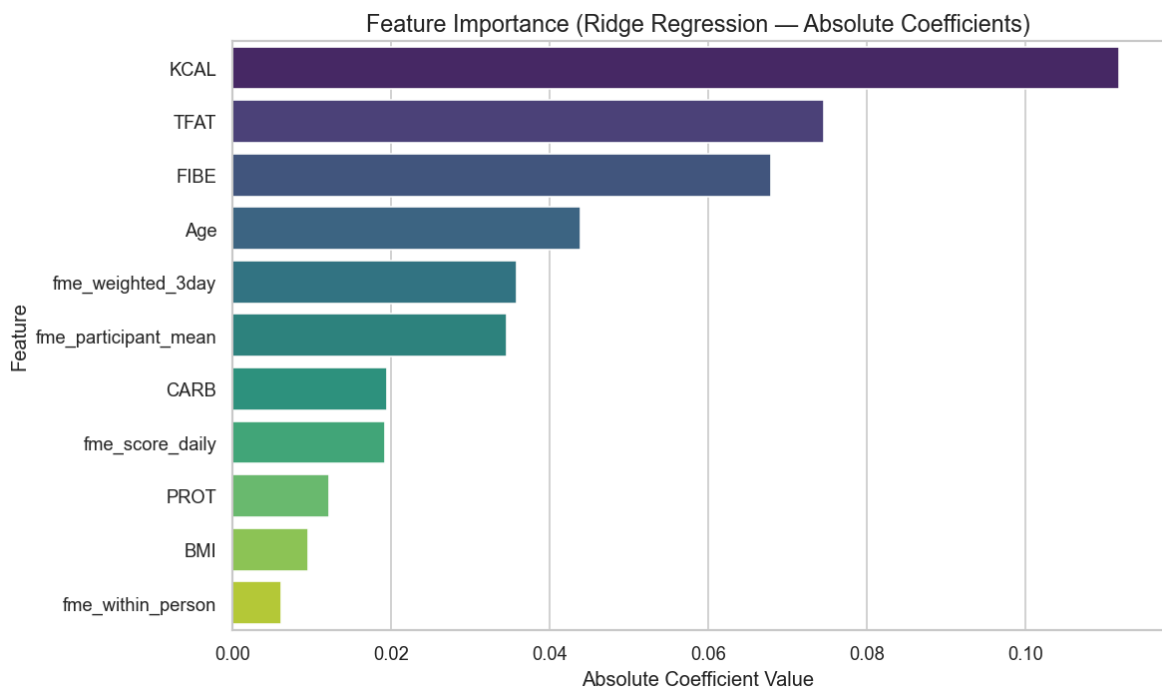
Mean R^2 : -0.619 ± 0.870
Mean RMSE: 0.476
Mean MAE: 0.390

[XGBoost Regressor]

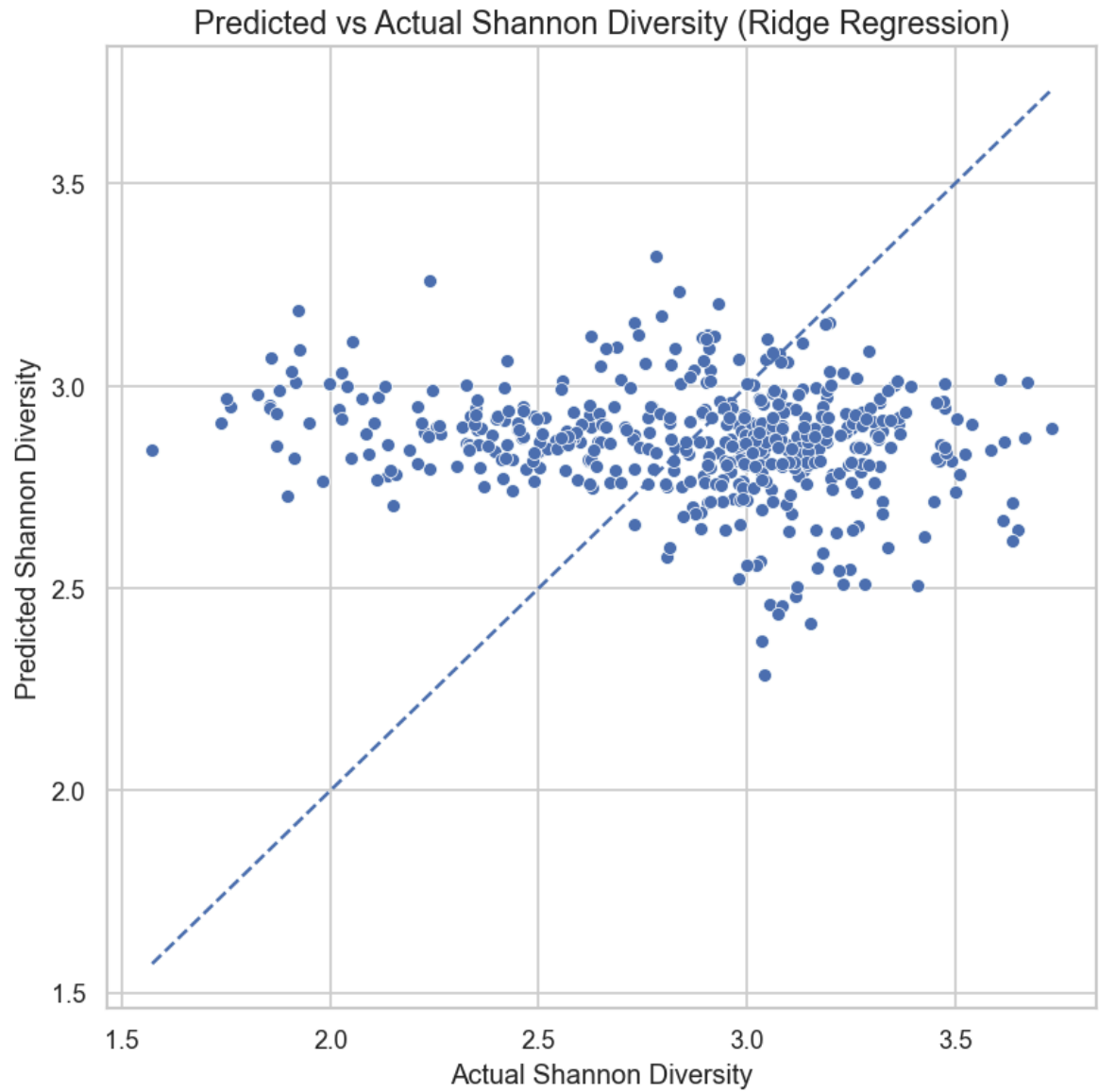
Mean R^2 : -0.367 ± 0.826
Mean RMSE: 0.436
Mean MAE: 0.331

Best Regression Model: Ridge Regression (Mean $R^2 = -0.316$)

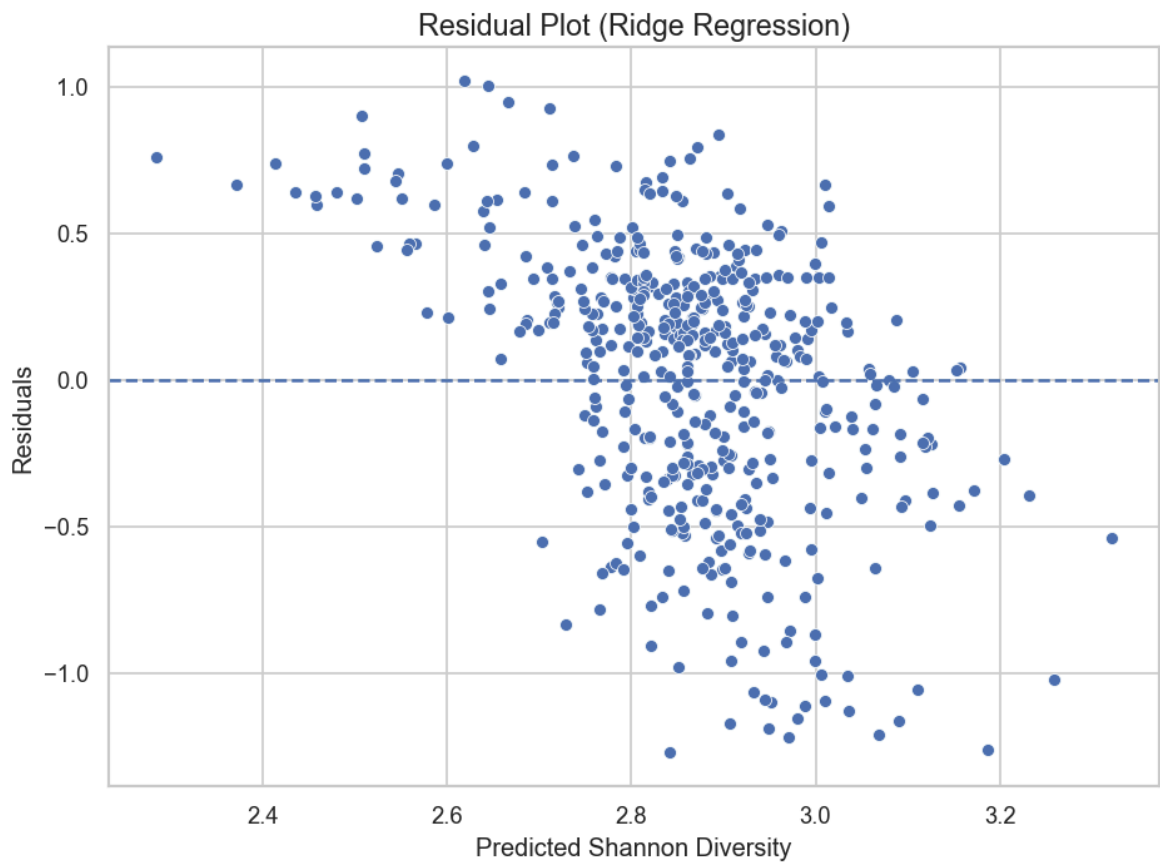
Regression results saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\regression_model_comparison.csv



Regression feature importance plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\regression_feature_importance.png



Prediction plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\regression_predicted_vs_actual.png



Residual plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\regression_residuals.png

Regression: predict continuous distance from avg Shannon diversity
Regression dataset: 475 samples
Features used: ['fme_score_daily', 'fme_participant_mean', 'fme_within_person', 'fme_weighted_3day', 'KCAL', 'FIBE', 'CARB', 'TFAT', 'PROT', 'Age', 'BMI']
Target mean Shannon deviation diversity: 0.000
Target std Shannon deviation diversity: 0.187

Regression Model for deviation Comparison Results

[Linear Regression]

Mean R^2 : -0.045 ± 0.025
Mean RMSE: 0.189
Mean MAE: 0.137

[Ridge Regression]

Mean R^2 : -0.044 ± 0.024
Mean RMSE: 0.189
Mean MAE: 0.137

[Random Forest Regressor]

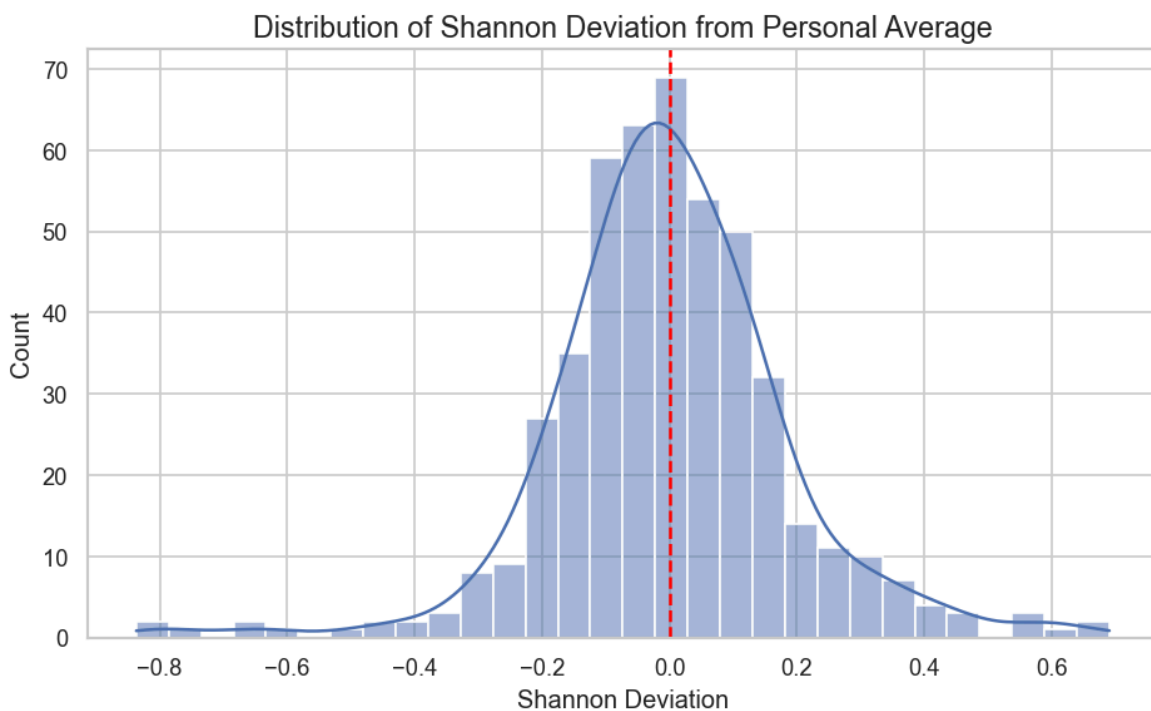
Mean R^2 : -0.125 ± 0.045
Mean RMSE: 0.196
Mean MAE: 0.145

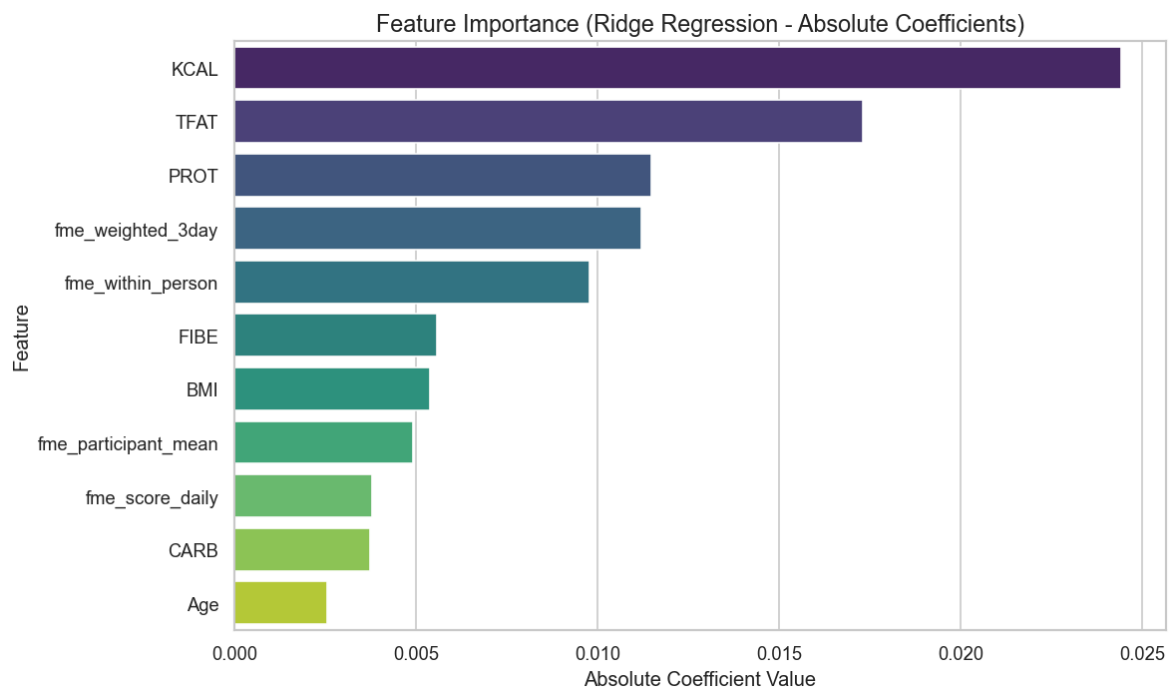
[XGBoost Regressor]

Mean R^2 : -0.435 ± 0.231
Mean RMSE: 0.220
Mean MAE: 0.164

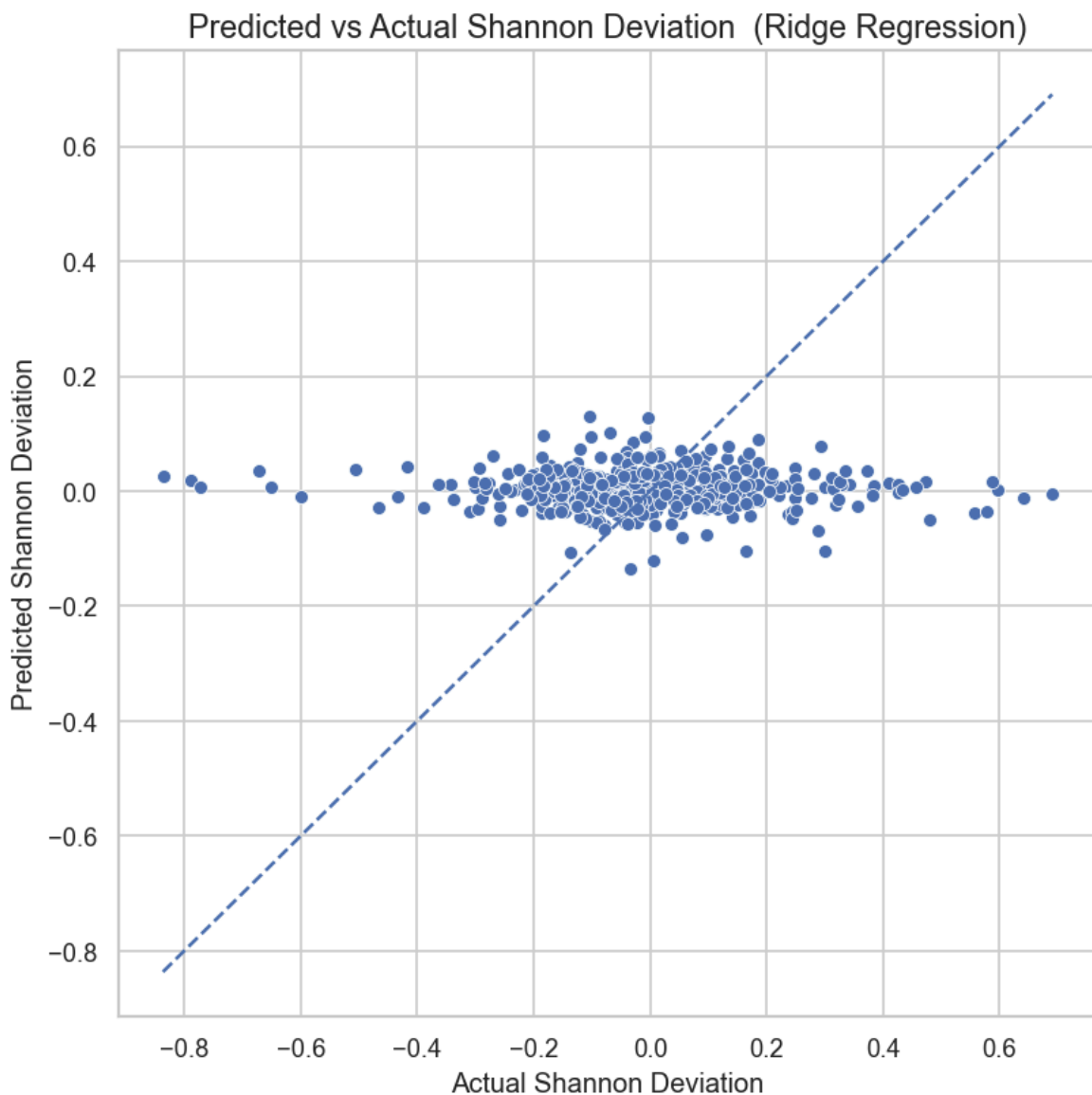
Best Regression Model: Ridge Regression (Mean $R^2 = -0.044$)

Regression results saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\regression_dev_model_comparison.csv

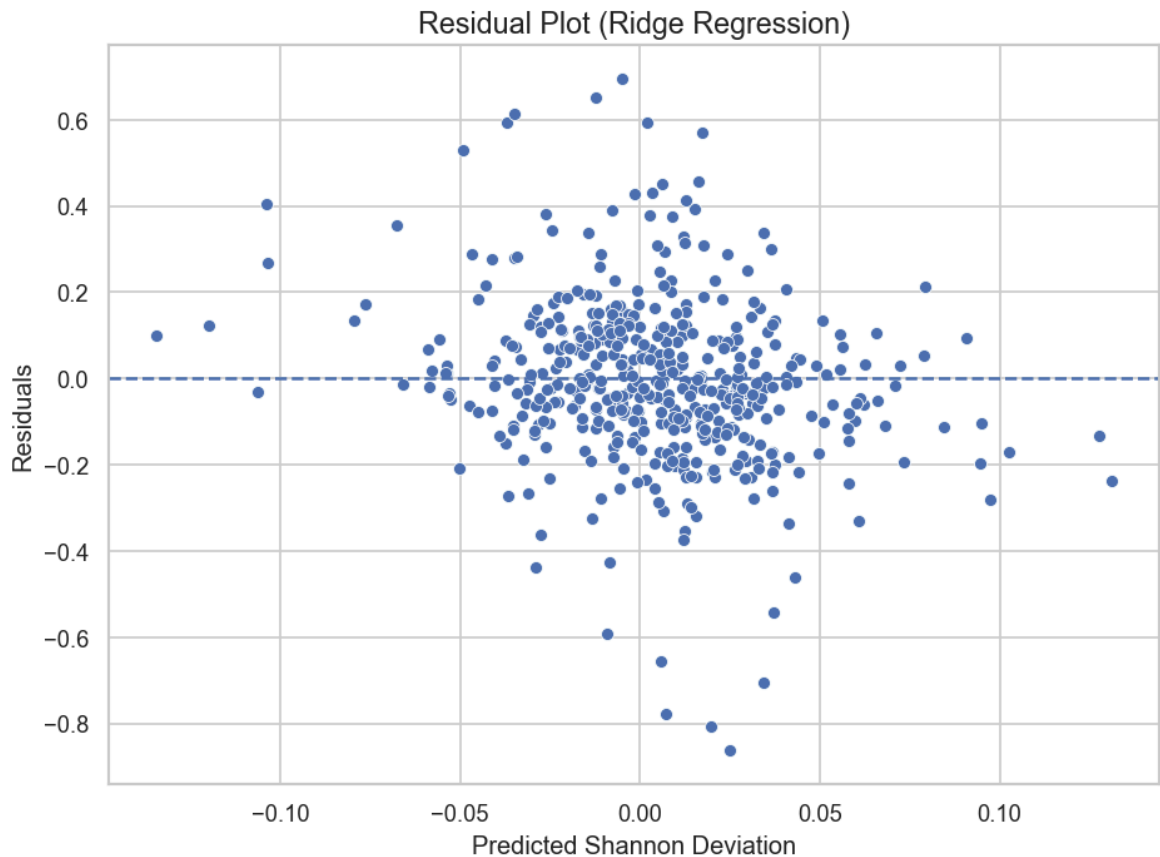




Regression feature importance plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\dev_regression_feature_importance.png



Prediction plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\dev_regression_predicted_vs_actual.png



Residual plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\dev_regression_residuals.png

Full python code: model_lagged

```
In [ ]: #Load data
df_fme = pd.read_csv(DATA_DIR / "fme_statistical_dataset_extended.csv", index_co
print(f"Loaded dataset: {df_fme.shape[0]} samples, {df_fme.shape[1]} columns")
print(f"Participants: {df_fme['participant_id'].nunique()}")
print("Sample of Data:")
print(df_fme.head())

#Turn shannon diversity from contiues to binary
median_shannon = df_fme["shannon_diversity"].median()
df_fme["High_Diversity_Class"] = (df_fme["shannon_diversity"] > median_shannon).
print(f"\nTarget distribution (High=1 / Low=0):")
print(df_fme["High_Diversity_Class"].value_counts())

#Features
feature_cols_lagged = [
    "fme_score_daily",
    "fme_participant_mean",
    "fme_within_person",
    "fme_lag1",
    "fme_lag2",
    "KCAL",
    "FIBE",
    "CARB",
    "TFAT",
    "PROT",
    "Age",
    "BMI"
]
```

```

feature_cols_lagged = [f for f in feature_cols_lagged if f in df_fme.columns]

#Drop rows with missing values in features or target
df_fme_clean = df_fme[feature_cols_lagged + ["High_Diversity_Class", "participant_id"]]

#Check how many samples are left
print(f"Lagged classification clean dataset: {df_fme_clean.shape[0]} samples")
print(f"Participants after cleaning: {df_fme_clean['participant_id'].nunique()}")

#Prepare data for modeling
X = df_fme_clean[feature_cols_lagged]
print(X.columns.tolist())
y = df_fme_clean['High_Diversity_Class']
groups = df_fme_clean['participant_id']

#Define models
models = {
    "Logistic Regression": LogisticRegression(),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "XGBoost": XGBClassifier(eval_metric='logloss', random_state=42),
    "SVM": CalibratedClassifierCV(SVC(kernel='rbf'), ensemble=False),
    "LightGBM": LGBMClassifier(n_estimators=100, random_state=42, verbose=-1)
}

#Cross validation
gkf = GroupKFold(n_splits=5)

#Train and evaluate models
print("\nModel Comparison Results:")
best_model_name = ""
best_auc = 0
best_model_obj = None

for name, model in models.items():
    acc_scores = []
    auc_scores = []

    #Make sure subjects are not split between train and test sets
    for train_idx, test_idx in gkf.split(X, y, groups=groups):
        X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
        y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

        #Scale features
        scaler = StandardScaler()
        # X_train_scaled = scaler.fit_transform(X_train)
        # X_test_scaled = scaler.transform(X_test)
        X_train_scaled = pd.DataFrame(scaler.fit_transform(X_train), columns=feature_cols_lagged)
        X_test_scaled = pd.DataFrame(scaler.transform(X_test), columns=feature_cols_lagged)

        #Train
        #Clone model to avoid reusing fitted state between folds
        fold_model = clone(model)
        fold_model.fit(X_train_scaled, y_train)

        #Predictions
        y_pred = fold_model.predict(X_test_scaled)
        y_proba = fold_model.predict_proba(X_test_scaled)[:, 1]

        #Calculate metrics

```

```

        acc_scores.append(accuracy_score(y_test, y_pred))
        auc_scores.append(roc_auc_score(y_test, y_proba))

#Compute mean metrics across folds
mean_acc = np.mean(acc_scores)
std_acc = np.std(acc_scores)
mean_auc = np.mean(auc_scores)
std_auc = np.std(auc_scores)

print(f"\n[{name}]")
print(f"Mean Accuracy: {mean_acc:.3f} ± {std_acc:.3f}")
print(f"Mean ROC-AUC: {mean_auc:.3f} ± {std_auc:.3f}")

#Save best model based on AUC
if mean_auc > best_auc:
    best_auc = mean_auc
    best_model_name = name
    best_model_obj = clone(model)

print(f"\n Best Model: {best_model_name} (Mean AUC = {best_auc:.3f})")

#Refit best model on the full clean regression dataset
final_reg_scaler = StandardScaler()
X_scaled = final_reg_scaler.fit_transform(X)
best_model_obj.fit(X_scaled, y)

#Feature importance for best model
if best_model_name == "Logistic Regression":
    importances = np.abs(best_model_obj.coef_[0])
    title = "Feature Importance (Logistic Regression - Absolute Coefficient)"
    xlabel = "Absolute Coefficient Value"
elif best_model_name == "SVM":
    print("SVM does not support feature importance with rbf kernel - skipping plot")
    importances = None
else:
    importances = best_model_obj.feature_importances_
    title = f"Feature Importance ({best_model_name})"
    xlabel = "Importance Score"

if importances is not None:
    indices = np.argsort(importances)[::-1]
    feature_arr = np.array(feature_cols_lagged)

    plt.figure(figsize=(10, 6))
    sns.barplot(x=importances[indices], y=feature_arr[indices], hue=feature_arr[indices])
    plt.title(title, fontsize=14)
    plt.xlabel(xlabel, fontsize=12)
    plt.ylabel("Feature", fontsize=12)
    plt.tight_layout()
    plt.savefig(DATA_DIR / "lagged_feature_importance.png", dpi=150)  #Save
    plt.show()
    print(f"Plot saved: {DATA_DIR / 'lagged_feature_importance.png'}")

#Regression: predict continuous Shannon diversity
print("\n Regression: Predict Shannon Diversity")

#Use the same features, but keep Shannon diversity as a continuous target

```

```

df_reg_clean = df_fme[feature_cols_lagged + ["shannon_diversity", "participant_i

#Prepare regression data
X_reg = df_reg_clean[feature_cols_lagged]
y_reg = df_reg_clean["shannon_diversity"]
groups_reg = df_reg_clean["participant_id"]

print(f"Regression dataset: {df_reg_clean.shape[0]} samples")
print(f"Features used: {feature_cols_lagged}")
print(f"Target mean Shannon diversity: {y_reg.mean():.3f}")
print(f"Target std Shannon diversity: {y_reg.std():.3f}")

# Define regression models
regression_models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0),
    "Random Forest Regressor": RandomForestRegressor(
        n_estimators=100,
        random_state=42
    ),
    "XGBoost Regressor": XGBRegressor(
        objective="reg:squarederror",
        n_estimators=100,
        random_state=42
    )
}

# Use grouped cross validation to avoid participant leakage
gkf_reg = GroupKFold(n_splits=5)

#Train and evaluate regression models
print("\nRegression Model Comparison Results")

best_reg_model_name = ""
best_reg_r2 = -np.inf
best_reg_model_template = None

regression_results = []
all_y_true_best = []
all_y_pred_best = []

for name, model in regression_models.items():
    r2_scores = []
    rmse_scores = []
    mae_scores = []
    all_y_true = []
    all_y_pred = []

    #Make sure subjects are not split between train and test sets
    for train_idx, test_idx in gkf_reg.split(X_reg, y_reg, groups=groups_reg):
        X_train, X_test = X_reg.iloc[train_idx], X_reg.iloc[test_idx]
        y_train, y_test = y_reg.iloc[train_idx], y_reg.iloc[test_idx]

        #Scale features using train data only
        scaler = StandardScaler()
        X_train_scaled = scaler.fit_transform(X_train)
        X_test_scaled = scaler.transform(X_test)

        #Clone model to avoid reusing fitted state between folds
        fold_model = clone(model)

```

```

#Train
fold_model.fit(X_train_scaled, y_train)

#Predict continuous Shannon diversity
y_pred = fold_model.predict(X_test_scaled)

#Calculate regression metrics
r2_scores.append(r2_score(y_test, y_pred))
rmse_scores.append(np.sqrt(mean_squared_error(y_test, y_pred)))
mae_scores.append(mean_absolute_error(y_test, y_pred))
all_y_true.extend(y_test)
all_y_pred.extend(y_pred)

#Compute mean metrics across folds
mean_r2 = np.mean(r2_scores)
std_r2 = np.std(r2_scores)
mean_rmse = np.mean(rmse_scores)
mean_mae = np.mean(mae_scores)

regression_results.append({
    "Model": name,
    "Mean R2": mean_r2,
    "Std R2": std_r2,
    "Mean RMSE": mean_rmse,
    "Mean MAE": mean_mae
})

print(f"\n[{name}]")
print(f"Mean R^2:    {mean_r2:.3f} ± {std_r2:.3f}")
print(f"Mean RMSE:   {mean_rmse:.3f}")
print(f"Mean MAE:    {mean_mae:.3f}")

#Save best regression model based on R2
if mean_r2 > best_reg_r2:
    best_reg_r2 = mean_r2
    best_reg_model_name = name
    best_reg_model_template = model
    all_y_true_best = all_y_true
    all_y_pred_best = all_y_pred

print(f"\nBest Regression Model: {best_reg_model_name} (Mean R^2 = {best_reg_r2:.3f})")

# Save regression results table
regression_results_df = pd.DataFrame(regression_results)
regression_results_path = DATA_DIR / "lagged_regression_model_comparison.csv"
regression_results_df.to_csv(regression_results_path, index=False)
print(f"lagged_regression_model_comparison: {regression_results_path}")

#Refit best regression model on the full clean regression dataset
final_reg_scaler = StandardScaler()
X_reg_scaled = final_reg_scaler.fit_transform(X_reg)

best_reg_model_obj = clone(best_reg_model_template)
best_reg_model_obj.fit(X_reg_scaled, y_reg)

#Feature importance for best regression model
if best_reg_model_name == "Linear Regression" or best_reg_model_name == "Ridge R
    reg_importances = np.abs(best_reg_model_obj.coef_)
    reg_title = "Feature Importance (" + best_reg_model_name + " - Absolute Coeffici

```

```

    reg_xlabel = "Absolute Coefficient Value"
else:
    reg_importances = best_reg_model_obj.feature_importances_
    reg_title = f"Feature Importance ({best_reg_model_name})"
    reg_xlabel = "Importance Score"

reg_indices = np.argsort(reg_importances)[::-1]
feature_arr = np.array(feature_cols_lagged)

plt.figure(figsize=(10, 6))
sns.barplot(
    x=reg_importances[reg_indices],
    y=feature_arr[reg_indices],
    hue=feature_arr[reg_indices],
    palette="viridis",
    legend=False
)
plt.title(reg_title, fontsize=14)
plt.xlabel(reg_xlabel, fontsize=12)
plt.ylabel("Feature", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "lagged_regression_feature_importance.png", dpi=150)
plt.show()
print(f"Regression feature importance plot saved: {DATA_DIR / 'lagged_regression"}

all_y_true = np.array(all_y_true_best)
all_y_pred = np.array(all_y_pred_best)

#Plot predicted vs actual Shannon diversity
plt.figure(figsize=(7, 7))
sns.scatterplot(x=all_y_true, y=all_y_pred)
plt.plot(
    [all_y_true.min(), all_y_true.max()],
    [all_y_true.min(), all_y_true.max()],
    linestyle="--"
)
plt.title(f"Predicted vs Actual Shannon Diversity ({best_reg_model_name})", font
plt.xlabel("Actual Shannon Diversity", fontsize=12)
plt.ylabel("Predicted Shannon Diversity", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "lagged_regression_predicted_vs_actual.png", dpi=150)
plt.show()
print(f"Prediction plot saved: {DATA_DIR / 'lagged_regression_predicted_vs_actua"}

#Plot residuals
residuals = all_y_true - all_y_pred

plt.figure(figsize=(8, 6))
sns.scatterplot(x=all_y_pred, y=residuals)
plt.axhline(0, linestyle="--")
plt.title(f"Residual Plot ({best_reg_model_name})", fontsize=14)
plt.xlabel("Predicted Shannon Diversity", fontsize=12)
plt.ylabel("Residuals", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "lagged_regression_residuals.png", dpi=150)
plt.show()
print(f"Residual plot saved: {DATA_DIR / 'lagged_regression_residuals.png'}")

# Try model distance from avarge shanon instead of predict shanon

```

```

# Calculate avg shanon score
df_fme['shannon_avg'] = df_fme.groupby('participant_id')['shannon_diversity'].tr
# Calculate sample shanon deviation
df_fme['shannon_deviation'] = df_fme['shannon_diversity'] - df_fme['shannon_avg']

#Regression: predict continuous distance from avg Shannon diversity
print("\n Regression: predict continuous distance from avg Shannon diversity")

#Use the same features, but keep Shannon diversity as a continuous target
df_reg_dev_clean = df_fme[feature_cols_lagged + ["shannon_deviation", "participa

#Prepare regression data
X_dev_reg = df_reg_dev_clean[feature_cols_lagged]
y_dev_reg = df_reg_dev_clean["shannon_deviation"]
groups_dev_reg = df_reg_dev_clean["participant_id"]

print(f"Regression dataset: {df_reg_dev_clean.shape[0]} samples")
print(f"Features used: {feature_cols_lagged}")
print(f"Target mean Shannon deviation diversity: {y_dev_reg.mean():.3f}")
print(f"Target std Shannon deviation diversity: {y_dev_reg.std():.3f}")

#Define regression models
regression_dev_models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0),
    "Random Forest Regressor": RandomForestRegressor(
        n_estimators=100,
        random_state=42
    ),
    "XGBoost Regressor": XGBRegressor(
        objective="reg:squarederror",
        n_estimators=100,
        random_state=42
    )
}

#Use grouped cross validation to avoid participant Leakage
gkf_reg = GroupKFold(n_splits=5)

#Train and evaluate regression models
print("\nRegression Model for deviation- comparison Results")

best_dev_reg_model_name = ""
best_dev_reg_r2 = -np.inf
best_dev_reg_model_template = None

regression_dev_results = []
all_dev_y_true_best = []
all_dev_y_pred_best = []

for name, model in regression_dev_models.items():
    r2_scores = []
    rmse_scores = []
    mae_scores = []
    all_y_true = []
    all_y_pred = []

    #Make sure subjects are not split between train and test sets
    for train_idx, test_idx in gkf_reg.split(X_dev_reg, y_dev_reg, groups=groups
        X_train, X_test = X_dev_reg.iloc[train_idx], X_dev_reg.iloc[test_idx]

```



```

y_train, y_test = y_dev_reg.iloc[train_idx], y_dev_reg.iloc[test_idx]

#Scale features using train data only
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

#Clone model to avoid reusing fitted state between folds
fold_model = clone(model)

#Train
fold_model.fit(X_train_scaled, y_train)

#Predict continuous Shannon diversity
y_pred = fold_model.predict(X_test_scaled)

#Calculate regression metrics
r2_scores.append(r2_score(y_test, y_pred))
rmse_scores.append(np.sqrt(mean_squared_error(y_test, y_pred)))
mae_scores.append(mean_absolute_error(y_test, y_pred))
all_y_true.extend(y_test)
all_y_pred.extend(y_pred)

#Compute mean metrics across folds
mean_r2 = np.mean(r2_scores)
std_r2 = np.std(r2_scores)
mean_rmse = np.mean(rmse_scores)
mean_mae = np.mean(mae_scores)

regression_dev_results.append({
    "Model": name,
    "Mean R2": mean_r2,
    "Std R2": std_r2,
    "Mean RMSE": mean_rmse,
    "Mean MAE": mean_mae
})

print(f"\n[{name}]")
print(f"Mean R^2:    {mean_r2:.3f} ± {std_r2:.3f}")
print(f"Mean RMSE:   {mean_rmse:.3f}")
print(f"Mean MAE:    {mean_mae:.3f}")

#Save best regression model based on R2
if mean_r2 > best_dev_reg_r2:
    best_dev_reg_r2 = mean_r2
    best_dev_reg_model_name = name
    best_dev_reg_model_template = model
    all_dev_y_true_best = all_y_true
    all_dev_y_pred_best = all_y_pred

print(f"\nBest Regression Model: {best_dev_reg_model_name} (Mean R² = {best_dev_

#Save regression deviation from avg results table
regression_dev_results_df = pd.DataFrame(regression_dev_results)
regression_dev_results_path = DATA_DIR / "lagged_regression_dev_model_comparison"
regression_dev_results_df.to_csv(regression_dev_results_path, index=False)
print(f"Regression results saved: {regression_dev_results_path}")

#Refit best regression model on the full clean regression dataset
final_dev_reg_scaler = StandardScaler()

```

```

X_dev_reg_scaled = final_dev_reg_scaler.fit_transform(X_dev_reg)

best_dev_reg_model_obj = clone(best_dev_reg_model_template)
best_dev_reg_model_obj.fit(X_dev_reg_scaled, y_dev_reg)

#Feature importance for best regression model
if best_dev_reg_model_name == "Linear Regression" or best_dev_reg_model_name ==
    reg_dev_importances = np.abs(best_dev_reg_model_obj.coef_)
    reg_dev_title = "Feature Importance (" + best_dev_reg_model_name + " - Absolute
    reg_dev_xlabel = "Absolute Coefficient Value"
else:
    reg_dev_importances = best_dev_reg_model_obj.feature_importances_
    reg_dev_title = f"Feature Importance ({best_dev_reg_model_name})"
    reg_dev_xlabel = "Importance Score"

reg_dev_indices = np.argsort(reg_dev_importances)[::-1]
feature_dev_arr = np.array(feature_cols_lagged)

# Bar plot of distribution of Shannon Deviation from Personal Average
plt.figure(figsize=(8, 5))
sns.histplot(df_reg_dev_clean["shannon_deviation"], bins=30, kde=True)
plt.axvline(0, linestyle="--", color="red")
plt.title("Distribution of Shannon Deviation from Personal Average", fontsize=14)
plt.xlabel("Shannon Deviation", fontsize=12)
plt.ylabel("Count", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "lagged_dev_shannon_deviation_distribution.png", dpi=150)
plt.show()

# Bar plot of feature importance
plt.figure(figsize=(10, 6))
sns.barplot(
    x=reg_dev_importances[reg_dev_indices],
    y=feature_dev_arr[reg_dev_indices],
    hue=feature_dev_arr[reg_dev_indices],
    palette="viridis",
    legend=False
)
plt.title(reg_dev_title, fontsize=14)
plt.xlabel(reg_dev_xlabel, fontsize=12)
plt.ylabel("Feature", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "lagged_dev_regression_feature_importance.png", dpi=150)
plt.show()
print(f"Regression feature importance plot saved: {DATA_DIR / 'lagged_dev_regres

all_dev_y_true = np.array(all_dev_y_true_best)
all_dev_y_pred = np.array(all_dev_y_pred_best)

#Plot predicted vs actual Shannon diversity
plt.figure(figsize=(7, 7))
sns.scatterplot(x=all_dev_y_true, y=all_dev_y_pred)
plt.plot(
    [all_dev_y_true.min(), all_dev_y_true.max()],
    [all_dev_y_true.min(), all_dev_y_true.max()],
    linestyle="--"
)
plt.title(f"Predicted vs Actual Shannon Deviation ({best_dev_reg_model_name})",
plt.xlabel("Actual Shannon Deviation", fontsize=12)

```

```
plt.ylabel("Predicted Shannon Deviation", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "lagged_dev_regression_predicted_vs_actual.png", dpi=150)
plt.show()
print(f"Prediction plot saved: {DATA_DIR / 'lagged_dev_regression_predicted_vs_a

#Plot residuals
dev_residuals = all_dev_y_true - all_dev_y_pred

plt.figure(figsize=(8, 6))
sns.scatterplot(x=all_dev_y_pred, y=dev_residuals)
plt.axhline(0, linestyle="--")
plt.title(f"Residual Plot ({best_dev_reg_model_name})", fontsize=14)
plt.xlabel("Predicted Shannon Deviation", fontsize=12)
plt.ylabel("Residuals", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "lagged_dev_regression_residuals.png", dpi=150)
plt.show()
print(f"Residual plot saved: {DATA_DIR / 'lagged_dev_regression_residuals.png'})"
```

Loaded dataset: 475 samples, 35 columns

Participants: 34

Sample of Data:

fecal_sample_id	participant_id	study_day	Gender	Age	BMI	Weight	\
MCT.f.0002	MCTs01	2	F	25.9000	25.0000	74.6000	
MCT.f.0003	MCTs01	3	F	25.9000	25.0000	74.6000	
MCT.f.0004	MCTs01	4	F	25.9000	25.0000	74.6000	
MCT.f.0005	MCTs01	5	F	25.9000	25.0000	74.6000	
MCT.f.0006	MCTs01	6	F	25.9000	25.0000	74.6000	

fecal_sample_id	Supplement	Medications	fme_score_daily	\
MCT.f.0002	EVOO	Nexplanon (bc implant)	67.1961	
MCT.f.0003	EVOO	Nexplanon (bc implant)	81.2412	
MCT.f.0004	EVOO	Nexplanon (bc implant)	126.7711	
MCT.f.0005	EVOO	Nexplanon (bc implant)	31.4846	
MCT.f.0006	EVOO	Nexplanon (bc implant)	61.2266	

fecal_sample_id	fme_participant_mean	fme_within_person	fme_lag1	...	\
MCT.f.0002	71.2710	-4.0749	NaN	...	
MCT.f.0003	71.2710	9.9703	67.1961	...	
MCT.f.0004	71.2710	55.5001	81.2412	...	
MCT.f.0005	71.2710	-39.7864	126.7711	...	
MCT.f.0006	71.2710	-10.0444	31.4846	...	

fecal_sample_id	PROT	TFAT	CARB	FIBE	SUGR	SODI	\
MCT.f.0002	109.6198	53.6153	242.9190	14.4194	74.2081	4300.6165	
MCT.f.0003	89.9930	50.5614	234.2251	25.9961	67.4747	4225.8133	
MCT.f.0004	94.6120	92.0216	257.5390	26.1784	83.9603	5329.8821	
MCT.f.0005	110.2684	95.5703	165.6081	13.0114	64.9220	3527.5014	
MCT.f.0006	161.8178	108.3985	332.6372	21.5291	119.8657	5428.5730	

fecal_sample_id	D_TOTAL	D_YOGURT	D_CHEESE	PF_MEAT	PF_SEAFD_HI	G_WHOLE
MCT.f.0002	4.1560	0.6460	1.1535	0.0000	3.9690	0.9718
MCT.f.0003	2.9797	0.6460	1.2076	0.0000	0.0000	2.9318
MCT.f.0004	3.6345	0.0000	1.4826	0.0000	0.0000	0.9718
MCT.f.0005	3.3797	0.0000	1.0590	1.8144	0.0000	0.9718
MCT.f.0006	1.5022	0.0000	0.0000	6.2960	0.0000	0.4698

[5 rows x 35 columns]

Target distribution (High=1 / Low=0):

High_Diversity_Class

0 238

1 237

Name: count, dtype: int64

Lagged classification clean dataset: 407 samples

Participants after cleaning: 34

['fme_score_daily', 'fme_participant_mean', 'fme_within_person', 'fme_lag1', 'fme_lag2', 'KCAL', 'FIBE', 'CARB', 'TFAT', 'PROT', 'Age', 'BMI']

Model Comparison Results:

[Logistic Regression]

Mean Accuracy: 0.425 ± 0.073

Mean ROC-AUC: 0.339 ± 0.057

[Random Forest]

Mean Accuracy: 0.546 ± 0.042

Mean ROC-AUC: 0.487 ± 0.079

[XGBoost]

Mean Accuracy: 0.529 ± 0.065

Mean ROC-AUC: 0.473 ± 0.077

[SVM]

Mean Accuracy: 0.373 ± 0.059

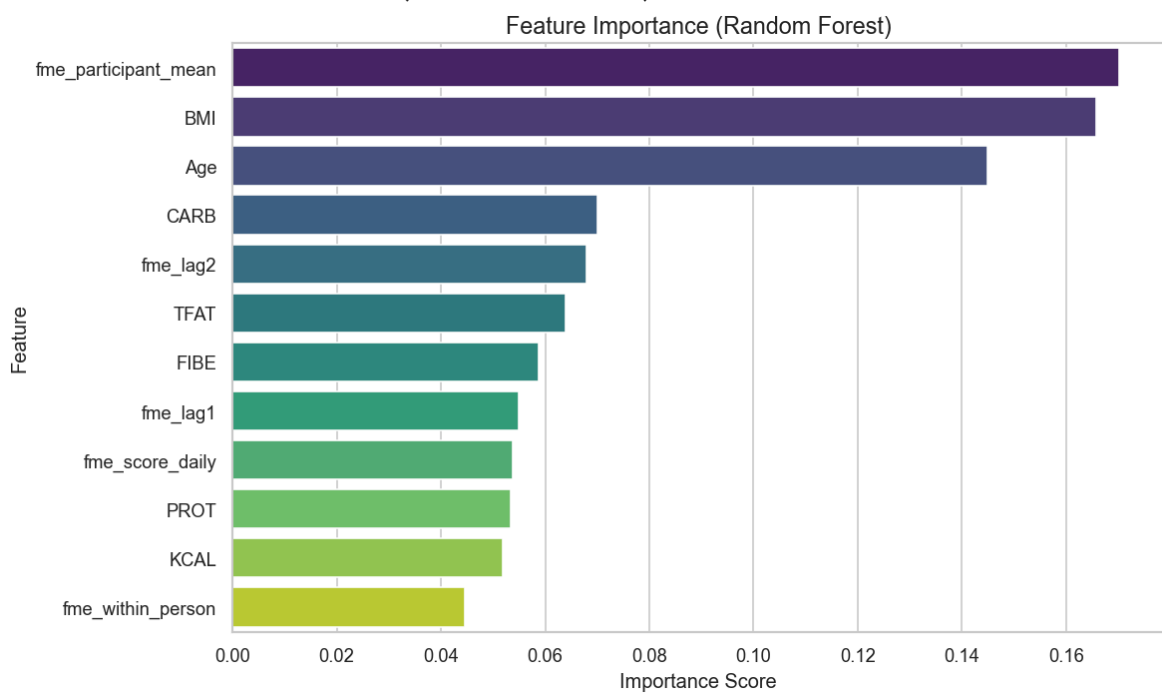
Mean ROC-AUC: 0.412 ± 0.048

[LightGBM]

Mean Accuracy: 0.515 ± 0.128

Mean ROC-AUC: 0.446 ± 0.192

Best Model: Random Forest (Mean AUC = 0.487)



Plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\lagged_feature_importance.png

Regression: Predict Shannon Diversity
Regression dataset: 407 samples
Features used: ['fme_score_daily', 'fme_participant_mean', 'fme_within_person', 'fme_lag1', 'fme_lag2', 'KCAL', 'FIBE', 'CARB', 'TFAT', 'PROT', 'Age', 'BMI']
Target mean Shannon diversity: 2.868
Target std Shannon diversity: 0.411

Regression Model Comparison Results

[Linear Regression]

Mean R^2 : -0.337 ± 0.256
Mean RMSE: 0.455
Mean MAE: 0.374

[Ridge Regression]

Mean R^2 : -0.332 ± 0.254
Mean RMSE: 0.454
Mean MAE: 0.373

[Random Forest Regressor]

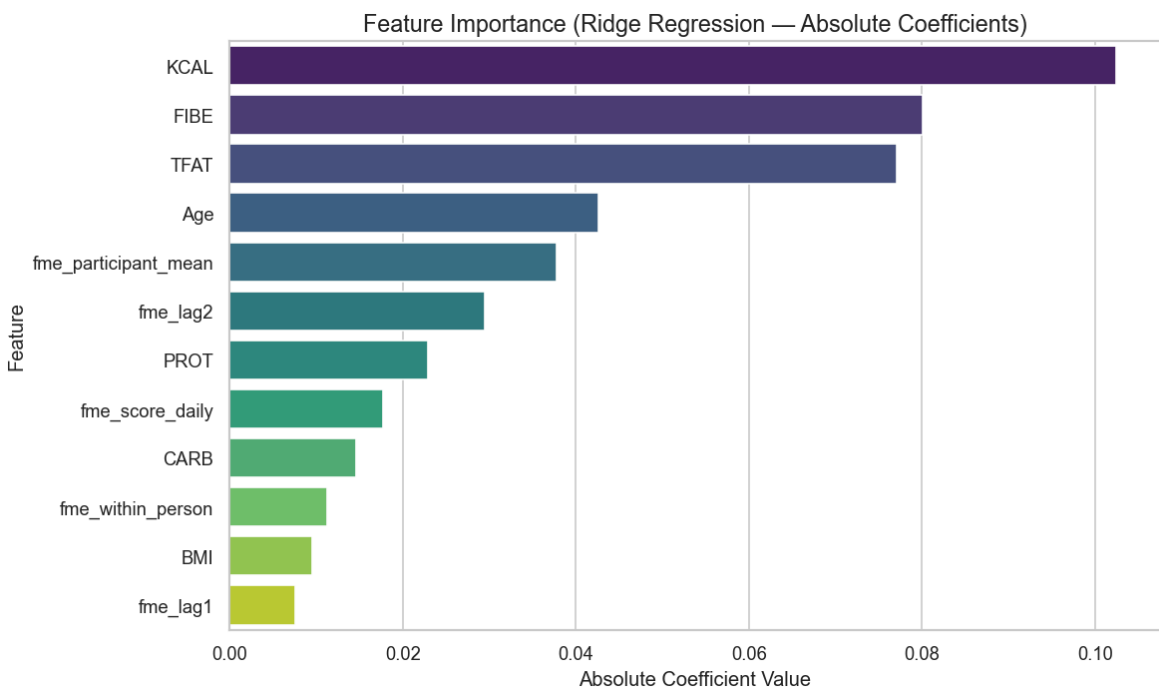
Mean R^2 : -0.536 ± 0.729
Mean RMSE: 0.468
Mean MAE: 0.388

[XGBoost Regressor]

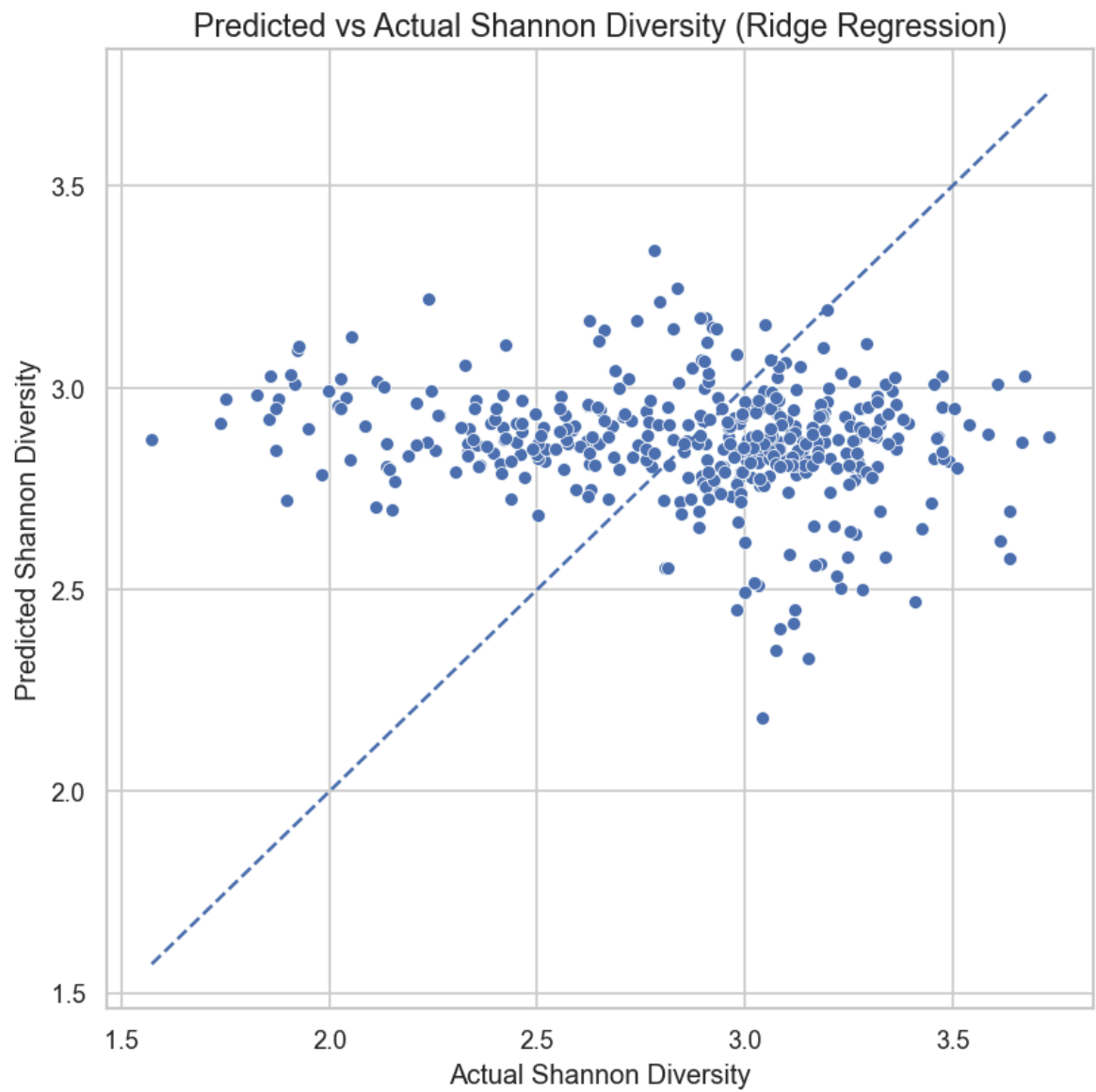
Mean R^2 : -0.384 ± 1.005
Mean RMSE: 0.433
Mean MAE: 0.329

Best Regression Model: Ridge Regression (Mean $R^2 = -0.332$)

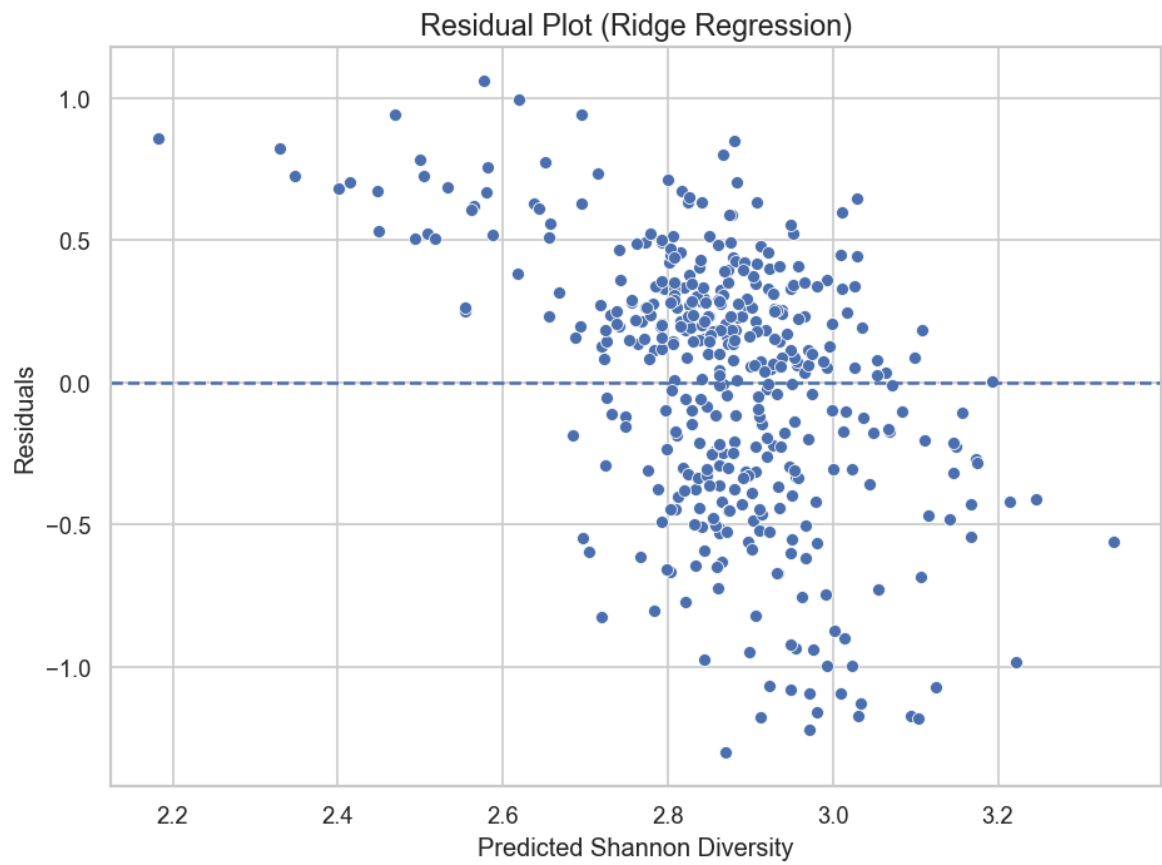
lagged_regression_model_comparison: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\lagged_regression_model_comparison.csv



Regression feature importance plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\lagged_regression_feature_importance.png



Prediction plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\lagged_regression_predicted_vs_actual.png



Residual plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\lagged_regression_residuals.png

Regression: predict continuous distance from avg Shannon diversity
 Regression dataset: 407 samples
 Features used: ['fme_score_daily', 'fme_participant_mean', 'fme_within_person', 'fme_lag1', 'fme_lag2', 'KCAL', 'FIBE', 'CARB', 'TFAT', 'PROT', 'Age', 'BMI']
 Target mean Shannon deviation diversity: 0.007
 Target std Shannon deviation diversity: 0.189

Regression Model for deviation- comparison Results

[Linear Regression]

Mean R^2 : -0.043 ± 0.036
 Mean RMSE: 0.191
 Mean MAE: 0.136

[Ridge Regression]

Mean R^2 : -0.042 ± 0.036
 Mean RMSE: 0.191
 Mean MAE: 0.136

[Random Forest Regressor]

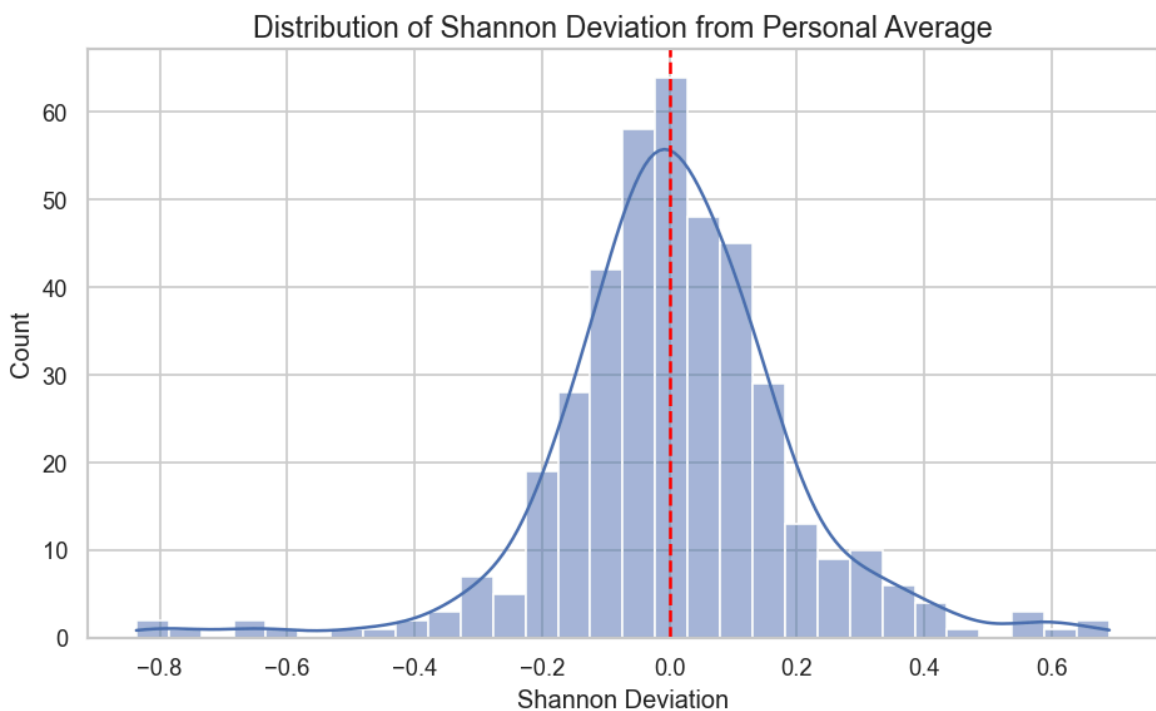
Mean R^2 : -0.133 ± 0.080
 Mean RMSE: 0.199
 Mean MAE: 0.145

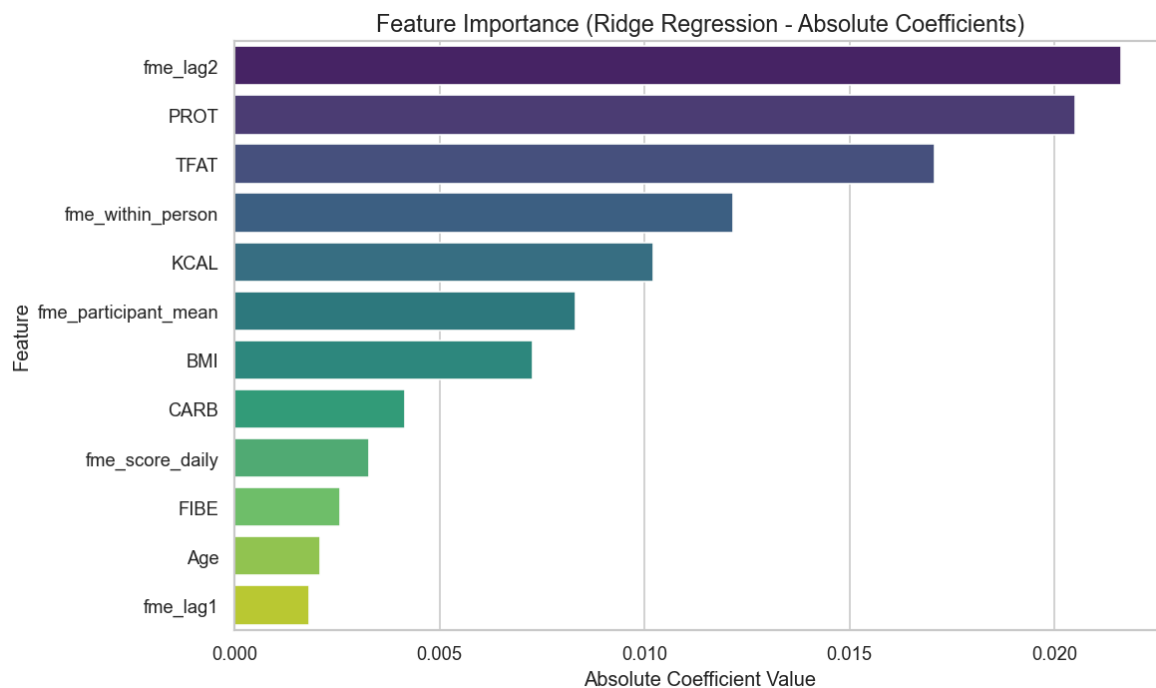
[XGBoost Regressor]

Mean R^2 : -0.385 ± 0.216
 Mean RMSE: 0.218
 Mean MAE: 0.162

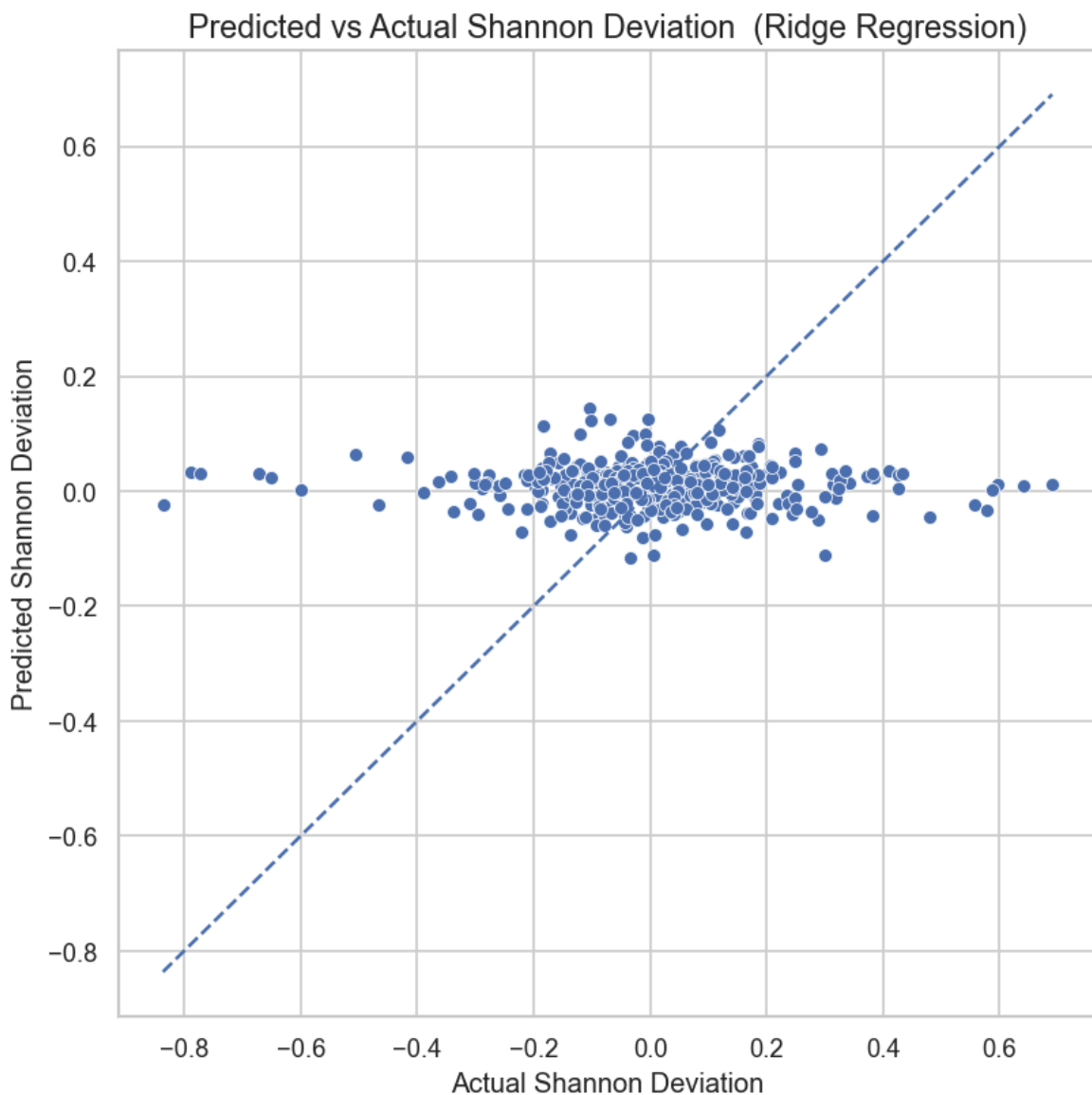
Best Regression Model: Ridge Regression (Mean $R^2 = -0.042$)

Regression results saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\lagged_regression_dev_model_comparison.csv

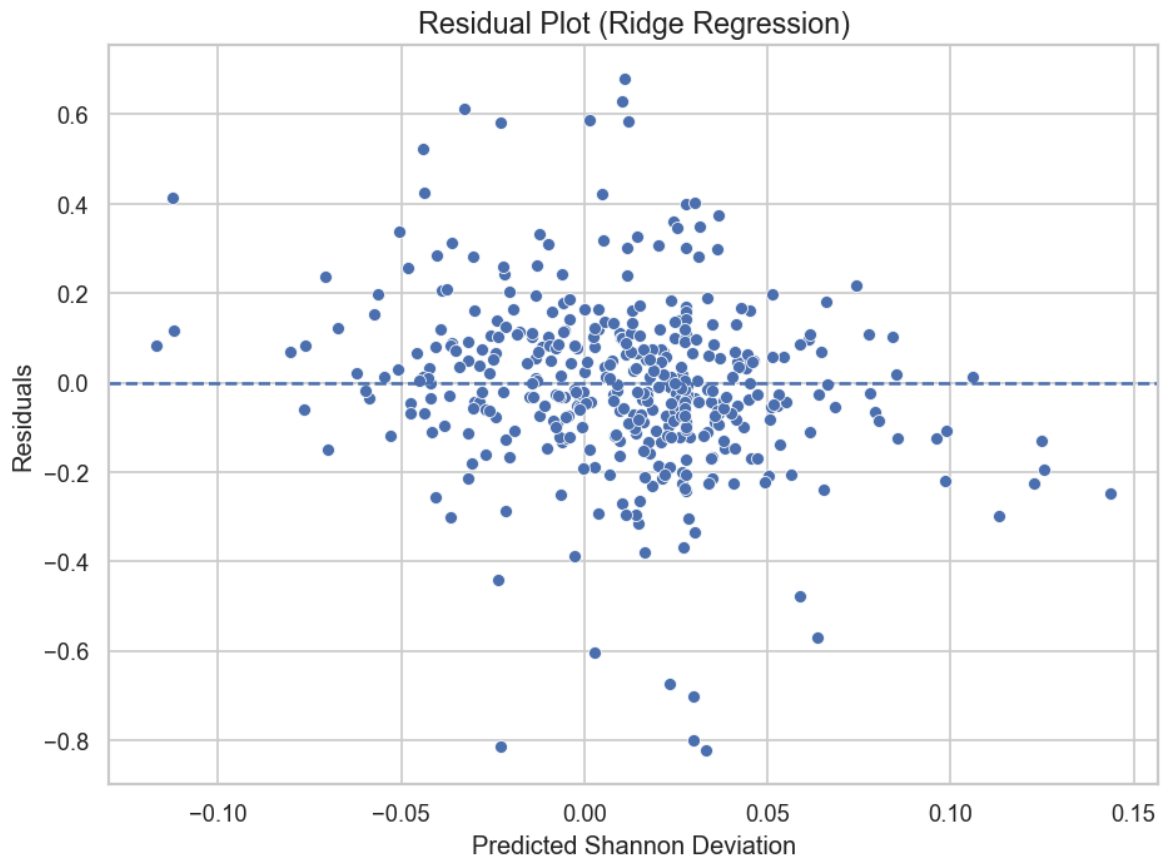




Regression feature importance plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\lagged_dev_regression_feature_importance.png



Prediction plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\lagged_dev_regression_predicted_vs_actual.png



Residual plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\lagged_dev_regression_residuals.png

Full python code: model_lagged_full

```
In [ ]: #Load data
df_fme = pd.read_csv(DATA_DIR / "fme_statistical_dataset_extended.csv", index_co
print(f"Loaded dataset: {df_fme.shape[0]} samples, {df_fme.shape[1]} columns")
print(f"Participants: {df_fme['participant_id'].nunique()}")
print("Sample of Data:")
print(df_fme.head())

#Turn shannon diversity from contiues to binary
median_shannon = df_fme["shannon_diversity"].median()
df_fme["High_Diversity_Class"] = (df_fme["shannon_diversity"] > median_shannon).
print(f"\nTarget distribution (High=1 / Low=0):")
print(df_fme["High_Diversity_Class"].value_counts())

#Features
feature_cols_full_fme = [
    "fme_score_daily",
    "fme_participant_mean",
    "fme_within_person",
    "fme_weighted_3day",
    "fme_lag1",
    "fme_lag2",
    "KCAL",
    "FIBE",
    "CARB",
    "TFAT",
    "PROT",
    "Age",
    "BMI"
]
```

```

feature_cols_full_fme = [f for f in feature_cols_full_fme if f in df_fme.columns]

#Drop rows with missing values in features or target
df_fme_clean = df_fme[feature_cols_full_fme + ["High_Diversity_Class", "particip

print(f"Full FME classification clean dataset: {df_fme_clean.shape[0]} samples")
print(f"Participants after cleaning: {df_fme_clean['participant_id'].nunique()}")

#Prepare data for modeling
X = df_fme_clean[feature_cols_full_fme]
print(X.columns.tolist())
y = df_fme_clean['High_Diversity_Class']
groups = df_fme_clean['participant_id']

#Define models
models = {
    "Logistic Regression": LogisticRegression(),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "XGBoost": XGBClassifier(eval_metric='logloss', random_state=42),
    "SVM": CalibratedClassifierCV(SVC(kernel='rbf'), ensemble=False),
    "LightGBM": LGBMClassifier(n_estimators=100, random_state=42, verbose=-1)
}

#Cross validation
gkf = GroupKFold(n_splits=5)

#Train and evaluate models
print("\nModel Comparison Results:")
best_model_name = ""
best_auc = 0
best_model_obj = None

for name, model in models.items():
    acc_scores = []
    auc_scores = []

    #Make sure subjects are not split between train and test sets
    for train_idx, test_idx in gkf.split(X, y, groups=groups):
        X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
        y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

        #Scale features
        scaler = StandardScaler()
        # X_train_scaled = scaler.fit_transform(X_train)
        # X_test_scaled = scaler.transform(X_test)
        X_train_scaled = pd.DataFrame(scaler.fit_transform(X_train), columns=fea
        X_test_scaled = pd.DataFrame(scaler.transform(X_test), columns=feature_c

        #Train
        #Clone model to avoid reusing fitted state between folds
        fold_model = clone(model)
        fold_model.fit(X_train_scaled, y_train)

        #Predictions
        y_pred = fold_model.predict(X_test_scaled)
        y_proba = fold_model.predict_proba(X_test_scaled)[: , 1]

        #Calculate metrics
        acc_scores.append(accuracy_score(y_test, y_pred))

```

```

        auc_scores.append(roc_auc_score(y_test, y_proba))

    #Compute mean metrics across folds
    mean_acc = np.mean(acc_scores)
    std_acc = np.std(acc_scores)
    mean_auc = np.mean(auc_scores)
    std_auc = np.std(auc_scores)

    print(f"\n[{name}]")
    print(f"Mean Accuracy: {mean_acc:.3f} ± {std_acc:.3f}")
    print(f"Mean ROC-AUC: {mean_auc:.3f} ± {std_auc:.3f}")

    #Save best model based on AUC
    if mean_auc > best_auc:
        best_auc = mean_auc
        best_model_name = name
        best_model_obj = clone(model)

print(f"\n Best Model: {best_model_name} (Mean AUC = {best_auc:.3f})")

#Refit best model on the full clean regression dataset
final_reg_scaler = StandardScaler()
X_scaled = final_reg_scaler.fit_transform(X)
best_model_obj.fit(X_scaled, y)

#Feature importance for best model
if best_model_name == "Logistic Regression":
    importances = np.abs(best_model_obj.coef_[0])
    title = "Feature Importance (Logistic Regression - Absolute Coefficient)"
    xlabel = "Absolute Coefficient Value"
elif best_model_name == "SVM":
    print("SVM does not support feature importance with rbf kernel - skipping plot")
    importances = None
else:
    importances = best_model_obj.feature_importances_
    title = f"Feature Importance ({best_model_name})"
    xlabel = "Importance Score"

if importances is not None:
    indices = np.argsort(importances)[::-1]
    feature_arr = np.array(feature_cols_full_fme)

    plt.figure(figsize=(10, 6))
    sns.barplot(x=importances[indices], y=feature_arr[indices], hue=feature_arr[indices])
    plt.title(title, fontsize=14)
    plt.xlabel(xlabel, fontsize=12)
    plt.ylabel("Feature", fontsize=12)
    plt.tight_layout()
    plt.savefig(DATA_DIR / "full_fme_feature_importance.png", dpi=150)  #Save
    plt.show()
    print(f"Plot saved: {DATA_DIR / 'full_fme_feature_importance.png'}")

#Regression: predict continuous Shannon diversity
print("\n Regression: Predict Shannon Diversity")

#Use the same features, but keep Shannon diversity as a continuous target
df_reg_clean = df_fme[feature_cols_full_fme + ["shannon_diversity", "participant_id"]]

```

```

#Prepare regression data
X_reg = df_reg_clean[feature_cols_full_fme]
y_reg = df_reg_clean["shannon_diversity"]
groups_reg = df_reg_clean["participant_id"]

print(f"Regression dataset: {df_reg_clean.shape[0]} samples")
print(f"Features used: {feature_cols_full_fme}")
print(f"Target mean Shannon diversity: {y_reg.mean():.3f}")
print(f"Target std Shannon diversity: {y_reg.std():.3f}")

# Define regression models
regression_models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0),
    "Random Forest Regressor": RandomForestRegressor(
        n_estimators=100,
        random_state=42
    ),
    "XGBoost Regressor": XGBRegressor(
        objective="reg:squarederror",
        n_estimators=100,
        random_state=42
    )
}

# Use grouped cross validation to avoid participant leakage
gkf_reg = GroupKFold(n_splits=5)

#Train and evaluate regression models
print("\nRegression Model Comparison Results")

best_reg_model_name = ""
best_reg_r2 = -np.inf
best_reg_model_template = None

regression_results = []
all_y_true_best = []
all_y_pred_best = []

for name, model in regression_models.items():
    r2_scores = []
    rmse_scores = []
    mae_scores = []
    all_y_true = []
    all_y_pred = []

    #Make sure subjects are not split between train and test sets
    for train_idx, test_idx in gkf_reg.split(X_reg, y_reg, groups=groups_reg):
        X_train, X_test = X_reg.iloc[train_idx], X_reg.iloc[test_idx]
        y_train, y_test = y_reg.iloc[train_idx], y_reg.iloc[test_idx]

        #Scale features using train data only
        scaler = StandardScaler()
        X_train_scaled = scaler.fit_transform(X_train)
        X_test_scaled = scaler.transform(X_test)

        #Clone model to avoid reusing fitted state between folds
        fold_model = clone(model)

```

```

#Train
fold_model.fit(X_train_scaled, y_train)

#Predict continuous Shannon diversity
y_pred = fold_model.predict(X_test_scaled)

#Calculate regression metrics
r2_scores.append(r2_score(y_test, y_pred))
rmse_scores.append(np.sqrt(mean_squared_error(y_test, y_pred)))
mae_scores.append(mean_absolute_error(y_test, y_pred))
all_y_true.extend(y_test)
all_y_pred.extend(y_pred)

#Compute mean metrics across folds
mean_r2 = np.mean(r2_scores)
std_r2 = np.std(r2_scores)
mean_rmse = np.mean(rmse_scores)
mean_mae = np.mean(mae_scores)

regression_results.append({
    "Model": name,
    "Mean R2": mean_r2,
    "Std R2": std_r2,
    "Mean RMSE": mean_rmse,
    "Mean MAE": mean_mae
})

print(f"\n[{name}]")
print(f"Mean R^2:    {mean_r2:.3f} ± {std_r2:.3f}")
print(f"Mean RMSE:  {mean_rmse:.3f}")
print(f"Mean MAE:    {mean_mae:.3f}")

#Save best regression model based on R2
if mean_r2 > best_reg_r2:
    best_reg_r2 = mean_r2
    best_reg_model_name = name
    best_reg_model_template = model
    all_y_true_best = all_y_true
    all_y_pred_best = all_y_pred

print(f"\nBest Regression Model: {best_reg_model_name} (Mean R^2 = {best_reg_r2:.3f})")

# Save regression results table
regression_results_df = pd.DataFrame(regression_results)
regression_results_path = DATA_DIR / "full_fme_regression_model_comparison.csv"
regression_results_df.to_csv(regression_results_path, index=False)
print(f"Regression results saved: {regression_results_path}")

#Refit best regression model on the full clean regression dataset
final_reg_scaler = StandardScaler()
X_reg_scaled = final_reg_scaler.fit_transform(X_reg)

best_reg_model_obj = clone(best_reg_model_template)
best_reg_model_obj.fit(X_reg_scaled, y_reg)

#Feature importance for best regression model
if best_reg_model_name == "Linear Regression" or best_reg_model_name == "Ridge R
    reg_importances = np.abs(best_reg_model_obj.coef_)
    reg_title = "Feature Importance (" + best_reg_model_name + " - Absolute Coeffici
    reg_xlabel = "Absolute Coefficient Value"

```

```

else:
    reg_importances = best_reg_model_obj.feature_importances_
    reg_title = f"Feature Importance ({best_reg_model_name})"
    reg_xlabel = "Importance Score"

reg_indices = np.argsort(reg_importances)[::-1]
feature_arr = np.array(feature_cols_full_fme)

plt.figure(figsize=(10, 6))
sns.barplot(
    x=reg_importances[reg_indices],
    y=feature_arr[reg_indices],
    hue=feature_arr[reg_indices],
    palette="viridis",
    legend=False
)
plt.title(reg_title, fontsize=14)
plt.xlabel(reg_xlabel, fontsize=12)
plt.ylabel("Feature", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "full_fme_regression_feature_importance.png", dpi=150)
plt.show()
print(f"Regression feature importance plot saved: {DATA_DIR / 'full_fme_regressi

all_y_true = np.array(all_y_true_best)
all_y_pred = np.array(all_y_pred_best)

#Plot predicted vs actual Shannon diversity
plt.figure(figsize=(7, 7))
sns.scatterplot(x=all_y_true, y=all_y_pred)
plt.plot(
    [all_y_true.min(), all_y_true.max()],
    [all_y_true.min(), all_y_true.max()],
    linestyle="--"
)
plt.title(f"Predicted vs Actual Shannon Diversity ({best_reg_model_name})", font
plt.xlabel("Actual Shannon Diversity", fontsize=12)
plt.ylabel("Predicted Shannon Diversity", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "full_fme_regression_predicted_vs_actual.png", dpi=150)
plt.show()
print(f"Prediction plot saved: {DATA_DIR / 'full_fme_regression_predicted_vs_act

#Plot residuals
residuals = all_y_true - all_y_pred

plt.figure(figsize=(8, 6))
sns.scatterplot(x=all_y_pred, y=residuals)
plt.axhline(0, linestyle="--")
plt.title(f"Residual Plot ({best_reg_model_name})", fontsize=14)
plt.xlabel("Predicted Shannon Diversity", fontsize=12)
plt.ylabel("Residuals", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "full_fme_regression_residuals.png", dpi=150)
plt.show()
print(f"Residual plot saved: {DATA_DIR / 'full_fme_regression_residuals.png'}")

# Try model distance from avarge shanon instead of predict shanon
# Calculate avg shanon score

```



```

df_fme['shannon_avg'] = df_fme.groupby('participant_id')['shannon_diversity'].tr
# Calculate sample shanon deviation
df_fme['shannon_deviation'] = df_fme['shannon_diversity'] - df_fme['shannon_avg']

#Regression: predict continuous distance from avg Shannon diversity
print("\n Regression: predict continuous distance from avg Shannon diversity")

#Use the same features, but keep Shannon diversity as a continuous target
df_reg_dev_clean = df_fme[feature_cols_full_fme + ["shannon_deviation", "partici

#Prepare regression data
X_dev_reg = df_reg_dev_clean[feature_cols_full_fme]
y_dev_reg = df_reg_dev_clean["shannon_deviation"]
groups_dev_reg = df_reg_dev_clean["participant_id"]

print(f"Regression dataset: {df_reg_dev_clean.shape[0]} samples")
print(f"Features used: {feature_cols_full_fme}")
print(f"Target mean Shannon deviation diversity: {y_dev_reg.mean():.3f}")
print(f"Target std Shannon deviation diversity: {y_dev_reg.std():.3f}")

#Define regression models
regression_dev_models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0),
    "Random Forest Regressor": RandomForestRegressor(
        n_estimators=100,
        random_state=42
    ),
    "XGBoost Regressor": XGBRegressor(
        objective="reg:squarederror",
        n_estimators=100,
        random_state=42
    )
}

#Use grouped cross validation to avoid participant Leakage
gkf_reg = GroupKFold(n_splits=5)

#Train and evaluate regression models
print("\nRegression Model for deviation Comparison Results")

best_dev_reg_model_name = ""
best_dev_reg_r2 = -np.inf
best_dev_reg_model_template = None

regression_dev_results = []
all_dev_y_true_best = []
all_dev_y_pred_best = []

for name, model in regression_dev_models.items():
    r2_scores = []
    rmse_scores = []
    mae_scores = []
    all_y_true = []
    all_y_pred = []

    #Make sure subjects are not split between train and test sets
    for train_idx, test_idx in gkf_reg.split(X_dev_reg, y_dev_reg, groups=groups
        X_train, X_test = X_dev_reg.iloc[train_idx], X_dev_reg.iloc[test_idx]
        y_train, y_test = y_dev_reg.iloc[train_idx], y_dev_reg.iloc[test_idx]

```

```

#Scale features using train data only
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

#Clone model to avoid reusing fitted state between folds
fold_model = clone(model)

#Train
fold_model.fit(X_train_scaled, y_train)

#Predict continuous Shannon diversity
y_pred = fold_model.predict(X_test_scaled)

#Calculate regression metrics
r2_scores.append(r2_score(y_test, y_pred))
rmse_scores.append(np.sqrt(mean_squared_error(y_test, y_pred)))
mae_scores.append(mean_absolute_error(y_test, y_pred))
all_y_true.extend(y_test)
all_y_pred.extend(y_pred)

#Compute mean metrics across folds
mean_r2 = np.mean(r2_scores)
std_r2 = np.std(r2_scores)
mean_rmse = np.mean(rmse_scores)
mean_mae = np.mean(mae_scores)

regression_dev_results.append({
    "Model": name,
    "Mean R2": mean_r2,
    "Std R2": std_r2,
    "Mean RMSE": mean_rmse,
    "Mean MAE": mean_mae
})

print(f"\n[{name}]\n")
print(f"Mean R2:    {mean_r2:.3f} ± {std_r2:.3f}")
print(f"Mean RMSE:  {mean_rmse:.3f}")
print(f"Mean MAE:   {mean_mae:.3f}")

#Save best regression model based on R2
if mean_r2 > best_dev_reg_r2:
    best_dev_reg_r2 = mean_r2
    best_dev_reg_model_name = name
    best_dev_reg_model_template = model
    all_dev_y_true_best = all_y_true
    all_dev_y_pred_best = all_y_pred

print(f"\nBest Regression Model: {best_dev_reg_model_name} (Mean R2 = {best_dev_

#Save regression deviation from avg results table
regression_dev_results_df = pd.DataFrame(regression_dev_results)
regression_dev_results_path = DATA_DIR / "full_fme_regression_dev_model_comparis
regression_dev_results_df.to_csv(regression_dev_results_path, index=False)
print(f"Regression results saved: {regression_dev_results_path}")

#Refit best regression model on the full clean regression dataset
final_dev_reg_scaler = StandardScaler()

```

```

X_dev_reg_scaled = final_dev_reg_scaler.fit_transform(X_dev_reg)

best_dev_reg_model_obj = clone(best_dev_reg_model_template)
best_dev_reg_model_obj.fit(X_dev_reg_scaled, y_dev_reg)

#Feature importance for best regression model
if best_dev_reg_model_name == "Linear Regression" or best_dev_reg_model_name ==
    reg_dev_importances = np.abs(best_dev_reg_model_obj.coef_)
    reg_dev_title = "Feature Importance (" + best_dev_reg_model_name + " - Absolute
    reg_dev_xlabel = "Absolute Coefficient Value"
else:
    reg_dev_importances = best_dev_reg_model_obj.feature_importances_
    reg_dev_title = f"Feature Importance ({best_dev_reg_model_name})"
    reg_dev_xlabel = "Importance Score"

reg_dev_indices = np.argsort(reg_dev_importances)[::-1]
feature_dev_arr = np.array(feature_cols_full_fme)

# Bar plot of distribution of Shannon Deviation from Personal Average
plt.figure(figsize=(8, 5))
sns.histplot(df_reg_dev_clean["shannon_deviation"], bins=30, kde=True)
plt.axvline(0, linestyle="--", color="red")
plt.title("Distribution of Shannon Deviation from Personal Average", fontsize=14)
plt.xlabel("Shannon Deviation", fontsize=12)
plt.ylabel("Count", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "full_fme_dev_shannon_deviation_distribution.png", dpi=15)
plt.show()

# Bar plot of feature importance
plt.figure(figsize=(10, 6))
sns.barplot(
    x=reg_dev_importances[reg_dev_indices],
    y=feature_dev_arr[reg_dev_indices],
    hue=feature_dev_arr[reg_dev_indices],
    palette="viridis",
    legend=False
)
plt.title(reg_dev_title, fontsize=14)
plt.xlabel(reg_dev_xlabel, fontsize=12)
plt.ylabel("Feature", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "full_fme_dev_regression_feature_importance.png", dpi=150)
plt.show()
print(f"Regression feature importance plot saved: {DATA_DIR / 'dev_regression_fe

all_dev_y_true = np.array(all_dev_y_true_best)
all_dev_y_pred = np.array(all_dev_y_pred_best)

#Plot predicted vs actual Shannon diversity
plt.figure(figsize=(7, 7))
sns.scatterplot(x=all_dev_y_true, y=all_dev_y_pred)
plt.plot(
    [all_dev_y_true.min(), all_dev_y_true.max()],
    [all_dev_y_true.min(), all_dev_y_true.max()],
    linestyle="--"
)
plt.title(f"Predicted vs Actual Shannon Deviation ({best_dev_reg_model_name})",
plt.xlabel("Actual Shannon Deviation", fontsize=12)

```

```
plt.ylabel("Predicted Shannon Deviation", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "full_fme_dev_regression_predicted_vs_actual.png", dpi=150)
plt.show()
print(f"Prediction plot saved: {DATA_DIR / 'full_fme_dev_regression_predicted_vs_"}

#Plot residuals
dev_residuals = all_dev_y_true - all_dev_y_pred

plt.figure(figsize=(8, 6))
sns.scatterplot(x=all_dev_y_pred, y=dev_residuals)
plt.axhline(0, linestyle="--")
plt.title(f"Residual Plot ({best_dev_reg_model_name})", fontsize=14)
plt.xlabel("Predicted Shannon Deviation", fontsize=12)
plt.ylabel("Residuals", fontsize=12)
plt.tight_layout()
plt.savefig(DATA_DIR / "full_fme_dev_regression_residuals.png", dpi=150)
plt.show()
print(f"Residual plot saved: {DATA_DIR / 'full_fme_dev_regression_residuals.png'}
```

Loaded dataset: 475 samples, 35 columns

Participants: 34

Sample of Data:

fecal_sample_id	participant_id	study_day	Gender	Age	BMI	Weight	\
MCT.f.0002	MCTs01	2	F	25.9000	25.0000	74.6000	
MCT.f.0003	MCTs01	3	F	25.9000	25.0000	74.6000	
MCT.f.0004	MCTs01	4	F	25.9000	25.0000	74.6000	
MCT.f.0005	MCTs01	5	F	25.9000	25.0000	74.6000	
MCT.f.0006	MCTs01	6	F	25.9000	25.0000	74.6000	

fecal_sample_id	Supplement	Medications	fme_score_daily	\
MCT.f.0002	EVOO	Nexplanon (bc implant)	67.1961	
MCT.f.0003	EVOO	Nexplanon (bc implant)	81.2412	
MCT.f.0004	EVOO	Nexplanon (bc implant)	126.7711	
MCT.f.0005	EVOO	Nexplanon (bc implant)	31.4846	
MCT.f.0006	EVOO	Nexplanon (bc implant)	61.2266	

fecal_sample_id	fme_participant_mean	fme_within_person	fme_lag1	...	\
MCT.f.0002	71.2710	-4.0749	NaN	...	
MCT.f.0003	71.2710	9.9703	67.1961	...	
MCT.f.0004	71.2710	55.5001	81.2412	...	
MCT.f.0005	71.2710	-39.7864	126.7711	...	
MCT.f.0006	71.2710	-10.0444	31.4846	...	

fecal_sample_id	PROT	TFAT	CARB	FIBE	SUGR	SODI	\
MCT.f.0002	109.6198	53.6153	242.9190	14.4194	74.2081	4300.6165	
MCT.f.0003	89.9930	50.5614	234.2251	25.9961	67.4747	4225.8133	
MCT.f.0004	94.6120	92.0216	257.5390	26.1784	83.9603	5329.8821	
MCT.f.0005	110.2684	95.5703	165.6081	13.0114	64.9220	3527.5014	
MCT.f.0006	161.8178	108.3985	332.6372	21.5291	119.8657	5428.5730	

fecal_sample_id	D_TOTAL	D_YOGURT	D_CHEESE	PF_MEAT	PF_SEAFD_HI	G_WHOLE
MCT.f.0002	4.1560	0.6460	1.1535	0.0000	3.9690	0.9718
MCT.f.0003	2.9797	0.6460	1.2076	0.0000	0.0000	2.9318
MCT.f.0004	3.6345	0.0000	1.4826	0.0000	0.0000	0.9718
MCT.f.0005	3.3797	0.0000	1.0590	1.8144	0.0000	0.9718
MCT.f.0006	1.5022	0.0000	0.0000	6.2960	0.0000	0.4698

[5 rows x 35 columns]

Target distribution (High=1 / Low=0):

High_Diversity_Class

0 238

1 237

Name: count, dtype: int64

Full FME classification clean dataset: 407 samples

Participants after cleaning: 34

['fme_score_daily', 'fme_participant_mean', 'fme_within_person', 'fme_weighted_3d
ay', 'fme_lag1', 'fme_lag2', 'KCAL', 'FIBE', 'CARB', 'TFAT', 'PROT', 'Age', 'BM
I']

Model Comparison Results:

[Logistic Regression]

Mean Accuracy: 0.425 ± 0.073

Mean ROC-AUC: 0.339 ± 0.057

[Random Forest]

Mean Accuracy: 0.558 ± 0.038

Mean ROC-AUC: 0.499 ± 0.102

[XGBoost]

Mean Accuracy: 0.519 ± 0.071

Mean ROC-AUC: 0.479 ± 0.094

[SVM]

Mean Accuracy: 0.368 ± 0.059

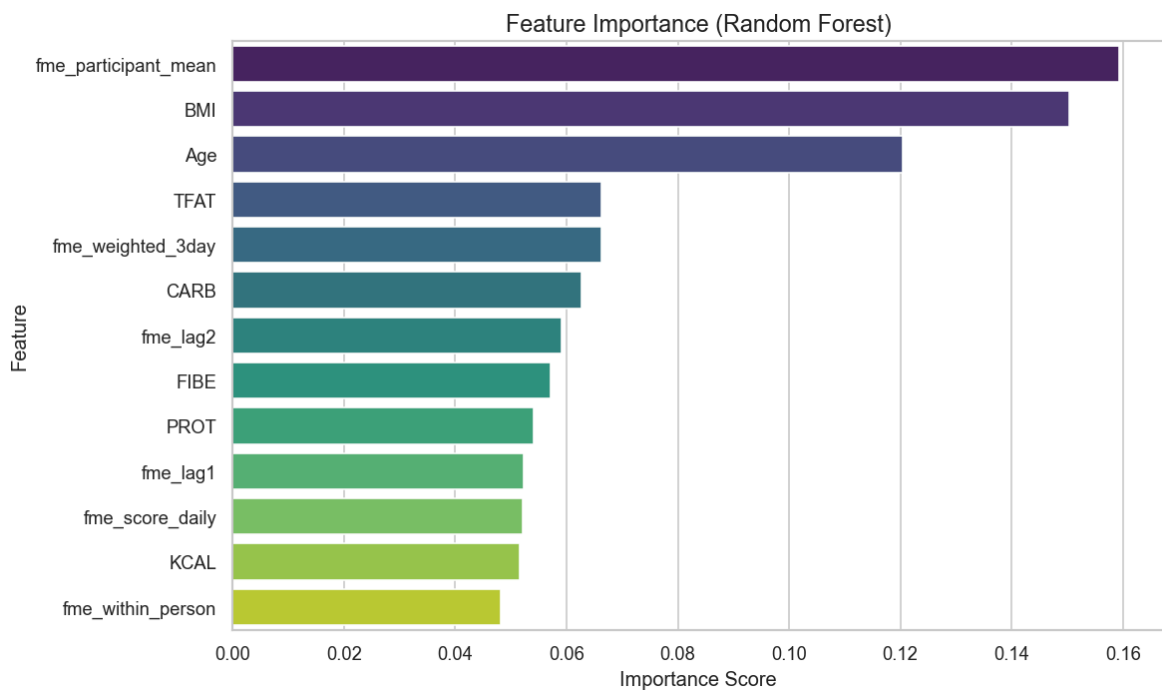
Mean ROC-AUC: 0.430 ± 0.071

[LightGBM]

Mean Accuracy: 0.503 ± 0.090

Mean ROC-AUC: 0.431 ± 0.181

Best Model: Random Forest (Mean AUC = 0.499)



Plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\full_fme_feature_importance.png

Regression: Predict Shannon Diversity
Regression dataset: 407 samples
Features used: ['fme_score_daily', 'fme_participant_mean', 'fme_within_person', 'fme_weighted_3day', 'fme_lag1', 'fme_lag2', 'KCAL', 'FIBE', 'CARB', 'TFAT', 'PROT', 'Age', 'BMI']
Target mean Shannon diversity: 2.868
Target std Shannon diversity: 0.411

Regression Model Comparison Results

[Linear Regression]

Mean R^2 : -0.337 ± 0.256
Mean RMSE: 0.455
Mean MAE: 0.374

[Ridge Regression]

Mean R^2 : -0.332 ± 0.254
Mean RMSE: 0.454
Mean MAE: 0.374

[Random Forest Regressor]

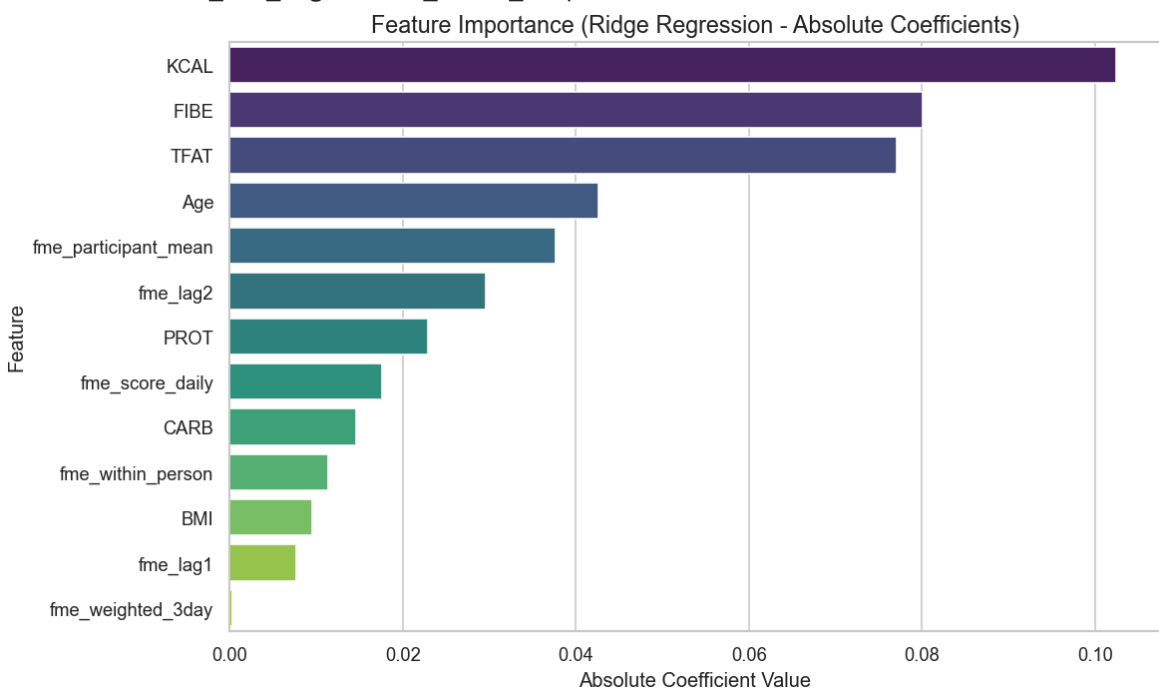
Mean R^2 : -0.534 ± 0.724
Mean RMSE: 0.469
Mean MAE: 0.389

[XGBoost Regressor]

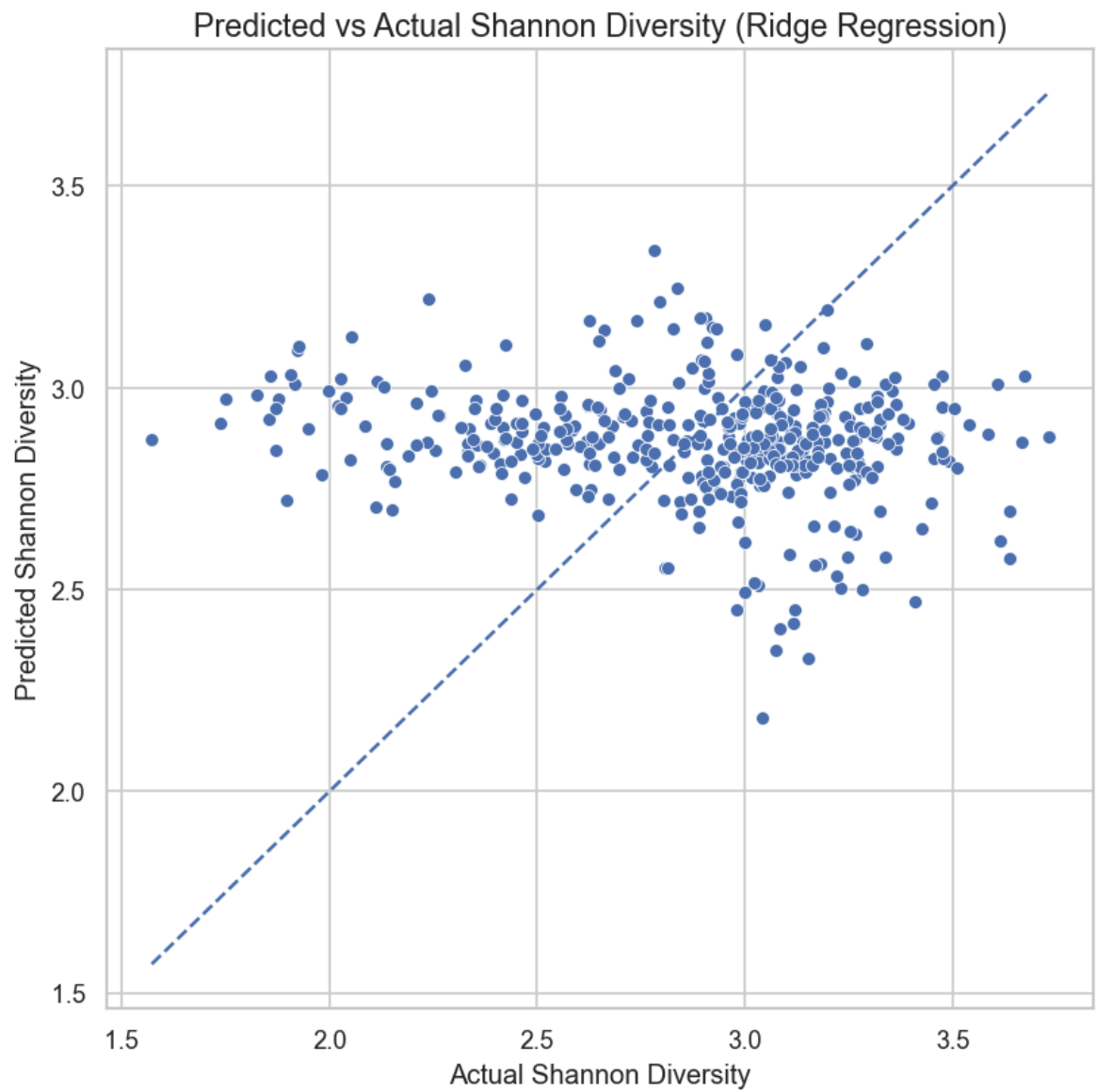
Mean R^2 : -0.389 ± 1.021
Mean RMSE: 0.432
Mean MAE: 0.327

Best Regression Model: Ridge Regression (Mean $R^2 = -0.332$)

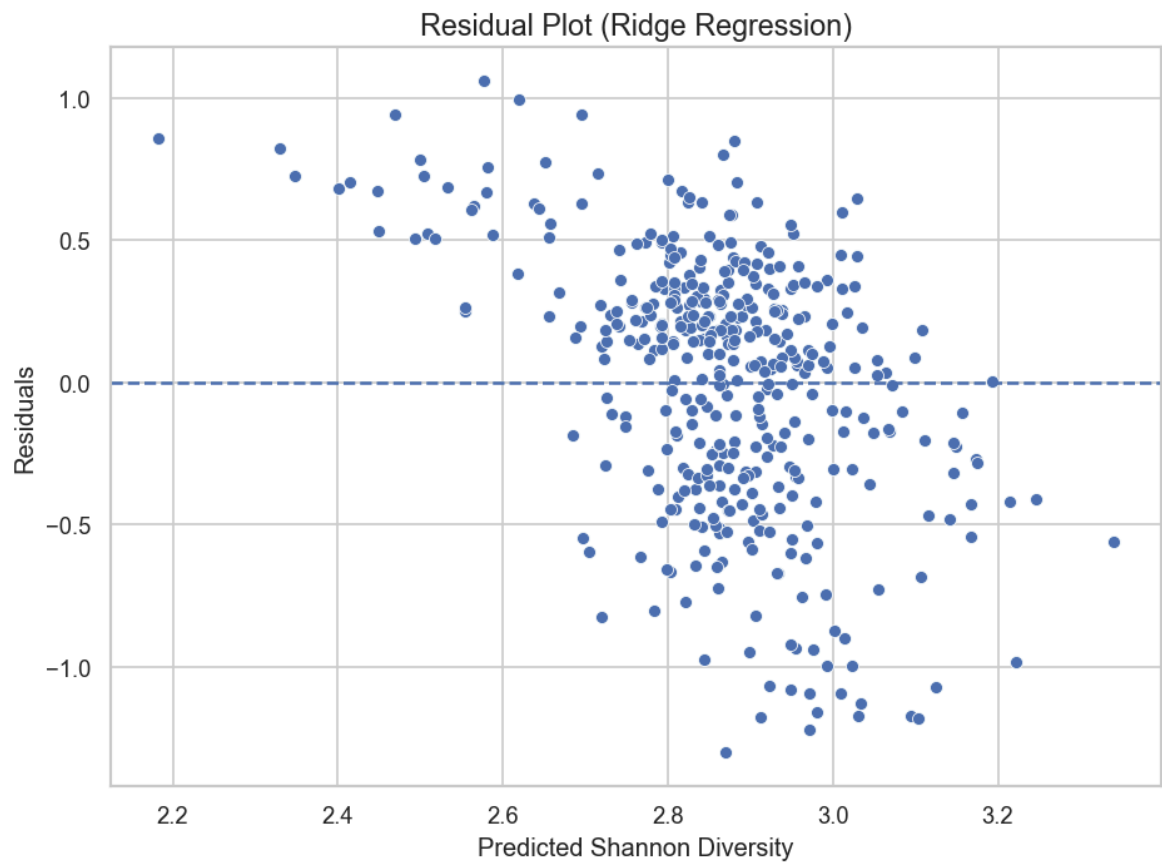
Regression results saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\full_fme_regression_model_comparison.csv



Regression feature importance plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\full_fme_regression_feature_importance.png



Prediction plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\full_fme_regression_predicted_vs_actual.png



Residual plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\full_fme_regression_residuals.png

Regression: predict continuous distance from avg Shannon diversity
Regression dataset: 407 samples
Features used: ['fme_score_daily', 'fme_participant_mean', 'fme_within_person', 'fme_weighted_3day', 'fme_lag1', 'fme_lag2', 'KCAL', 'FIBE', 'CARB', 'TFAT', 'PRO T', 'Age', 'BMI']
Target mean Shannon deviation diversity: 0.007
Target std Shannon deviation diversity: 0.189

Regression Model for deviation Comparison Results

[Linear Regression]

Mean R^2 : -0.043 ± 0.036
Mean RMSE: 0.191
Mean MAE: 0.136

[Ridge Regression]

Mean R^2 : -0.042 ± 0.036
Mean RMSE: 0.191
Mean MAE: 0.136

[Random Forest Regressor]

Mean R^2 : -0.132 ± 0.055
Mean RMSE: 0.199
Mean MAE: 0.145

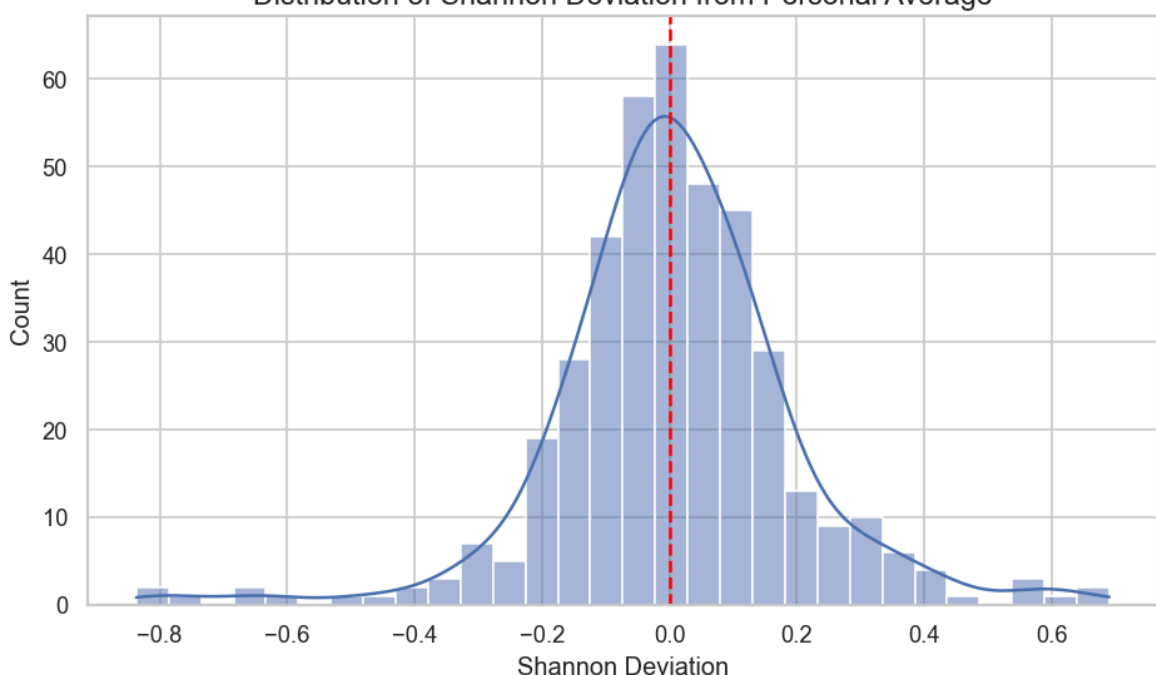
[XGBoost Regressor]

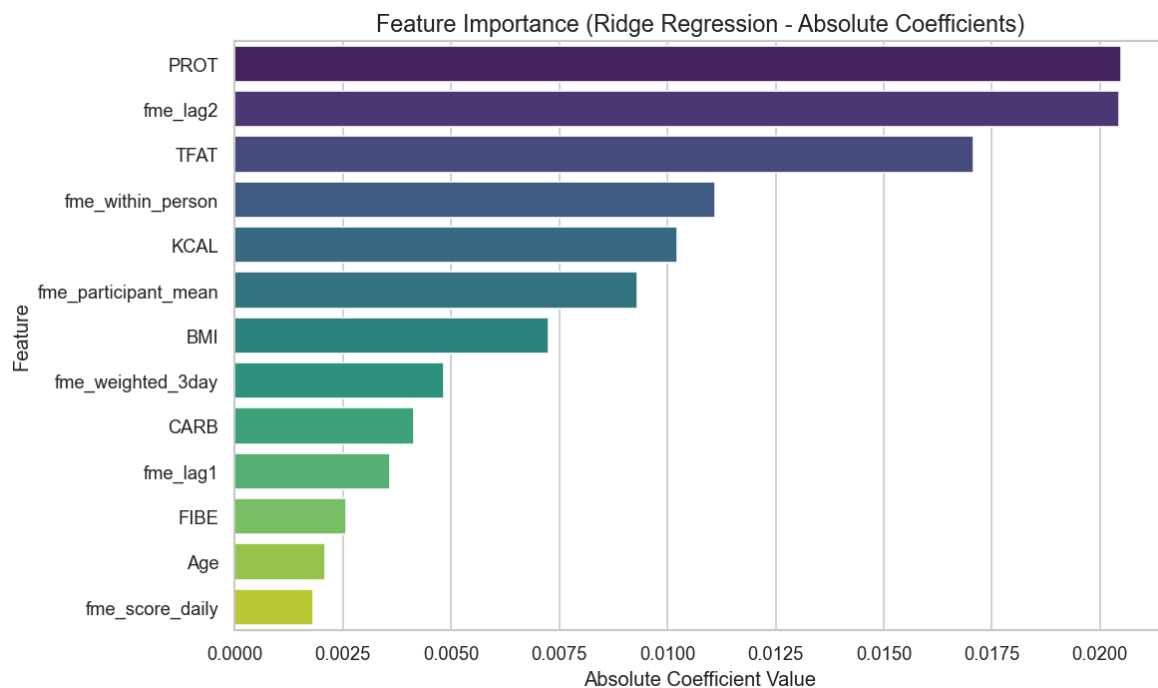
Mean R^2 : -0.375 ± 0.234
Mean RMSE: 0.218
Mean MAE: 0.161

Best Regression Model: Ridge Regression (Mean $R^2 = -0.042$)

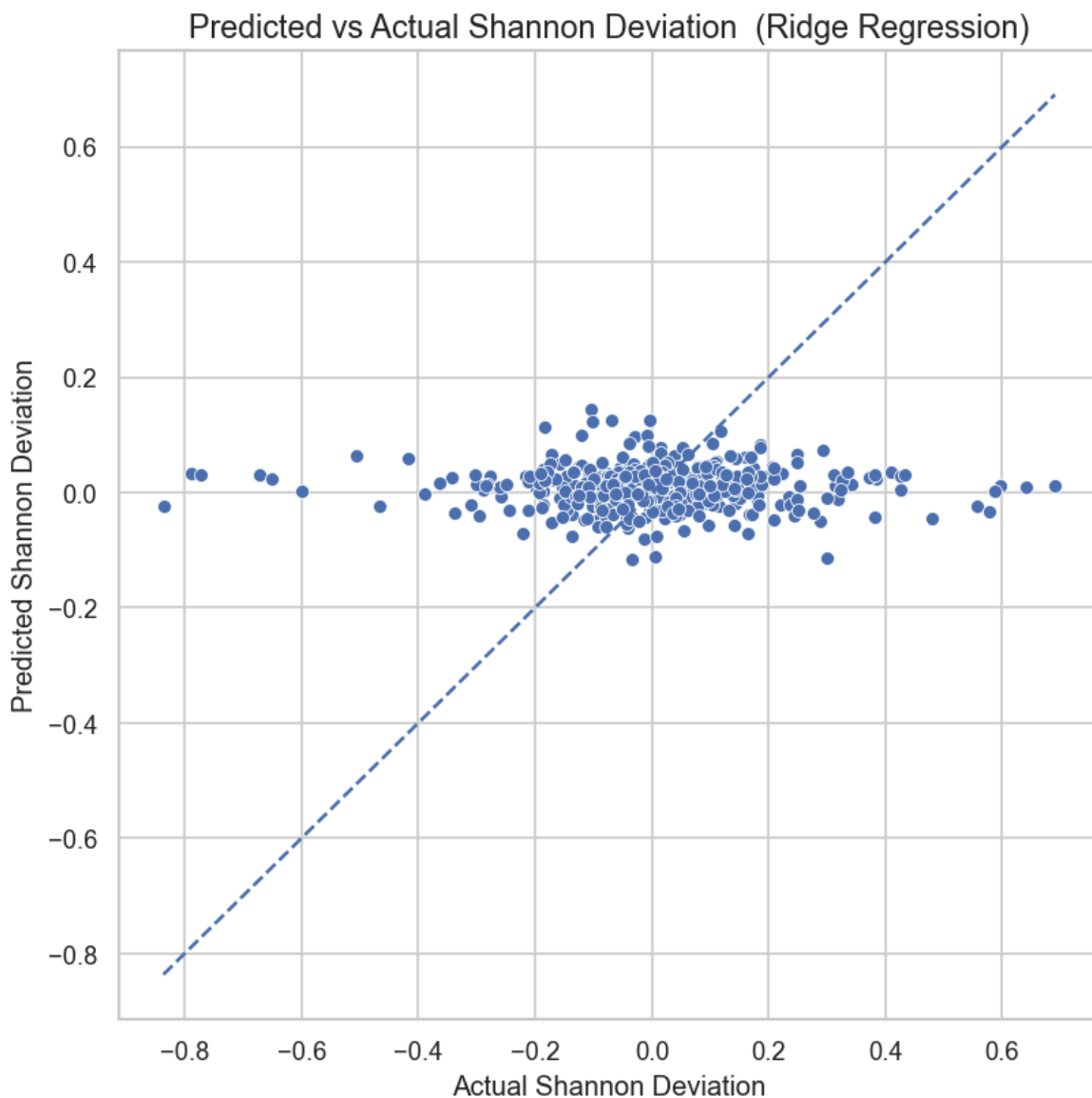
Regression results saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\full_fme_regression_dev_model_comparison.csv

Distribution of Shannon Deviation from Personal Average

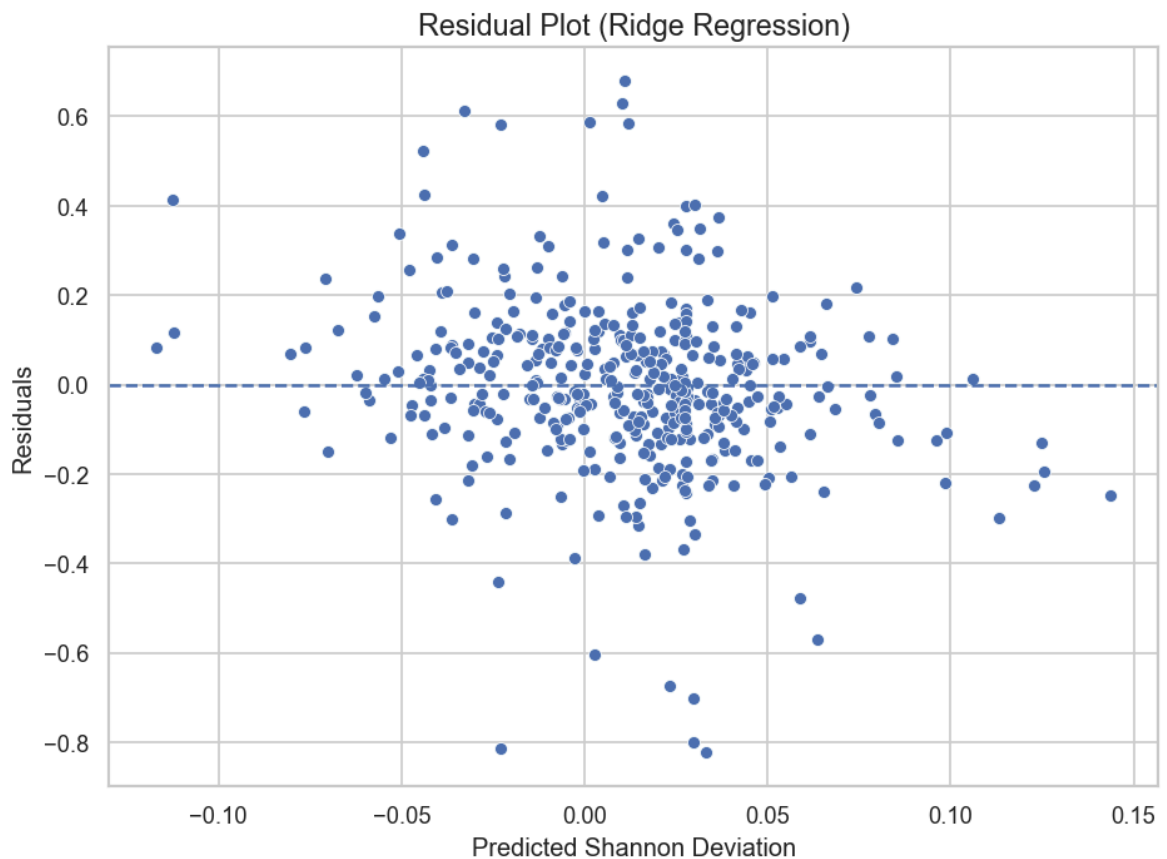




Regression feature importance plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\dev_regression_feature_importance.png



Prediction plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\full_fme_dev_regression_predicted_vs_actual.png



Residual plot saved: c:\Users\linoy\Documents\proj3\Gut-Microbiome-Project_year3\data\full_fme_dev_regression_residuals.png

From here there are codes that we didn't put in one of the final sprint notebook

Notebook: `Data_Integration.ipynb`

Data Integration: Johnson et al. 2019 × cFMD (Carlino et al. 2024)

Research Question

How does exposure to food-associated microbes influence the composition and stability of the human gut microbiome?

What this notebook does

This notebook merges all CSV files from both data sources into a unified analytical structure:

Source	Files	Content
Johnson et al. 2019 (Paper 1)	microbiome_filtered , micro_avg_norm , micro_avg_clr , sample_meta , diet_raw , diet_fiber , diet_decay , fiber_avg , nutrition_totals	Gut microbiome + dietary data for 34 participants
cFMD – Carlino et al. 2024 (Paper 2)	combined_datasets , cfmd_metadata	Food-associated microbiome from 3,393 food samples

Outputs produced

- 1. unified_taxonomy_table.csv — shared taxa between gut and food microbiome
- 2. unified_sample_metadata.csv — all sample metadata from both sources
- 3. gut_microbiome_long.csv — gut microbiome in long format (taxa × sample × abundance)
- 4. food_microbiome_long.csv — cFMD microbiome in long format
- 5. merged_shared_taxa.csv — abundance matrix for taxa found in BOTH datasets

Section 1 – Load All CSV Files

1.1 Johnson et al. 2019 (Gut microbiome + Diet)

```
In [ ]: #Gut microbiome matrices (taxa × samples)
gut_micro_filtered = pd.read_csv(DATA_DIR / "microbiome_filtered.csv", index_
gut_micro_avg_norm = pd.read_csv(DATA_DIR / "micro_avg_norm.csv", index_
gut_micro_avg_clr = pd.read_csv(DATA_DIR / "micro_avg_clr.csv", index_

#Sample + participant metadata
gut_sample_meta = pd.read_csv(DATA_DIR / "sample_meta.csv", index_

#Dietary data
diet_consumption_raw = pd.read_csv(DATA_DIR / "diet_raw.csv")
diet_fiber_daily = pd.read_csv(DATA_DIR / "diet_fiber.csv")
diet_decay_scores = pd.read_csv(DATA_DIR / "diet_decay.csv")
fiber_participant = pd.read_csv(DATA_DIR / "fiber_avg.csv")
nutrition_totals = pd.read_csv(DATA_DIR / "nutrition_totals.csv")

print("Johnson et al. 2019 files loaded:")
for name, df in [
    ("gut_micro_filtered", gut_micro_filtered),
    ("gut_micro_avg_norm", gut_micro_avg_norm),
    ("gut_micro_avg_clr", gut_micro_avg_clr),
    ("gut_sample_meta", gut_sample_meta),
    ("diet_consumption_raw", diet_consumption_raw),
    ("diet_fiber_daily", diet_fiber_daily),
    ("diet_decay_scores", diet_decay_scores),
    ("fiber_participant", fiber_participant),
    ("nutrition_totals", nutrition_totals),
]:
    print(f" {name:<25} {str(df.shape)}")
```

Johnson et al. 2019 files loaded:

gut_micro_filtered	(223, 483)
gut_micro_avg_norm	(226, 34)
gut_micro_avg_clr	(226, 34)
gut_sample_meta	(566, 28)
diet_consumption_raw	(1452, 568)
diet_fiber_daily	(1452, 582)
diet_decay_scores	(1452, 565)
fiber_participant	(1452, 36)
nutrition_totals	(580, 107)

1.2 cFMD – Carlino et al. 2024 (Food microbiome)

```
In [ ]: #Food microbiome abundance matrix (taxa × food samples)
food_micro_abundance = pd.read_csv(DATA_DIR / "combined_datasets.csv", index_col=0)
food_micro_abundance = food_micro_abundance.apply(pd.to_numeric, errors="coerce")

#cFMD sample metadata
cfmd_sample_meta = pd.read_csv(DATA_DIR / "cfmd_metadata.csv")

print("cFMD files loaded:")
for name, df in [
    ("food_micro_abundance", food_micro_abundance),
    ("cfmd_sample_meta", cfmd_sample_meta),
]:
    print(f" {name:<25} {str(df.shape)}")
```

cFMD files loaded:

food_micro_abundance	(5057, 3393)
cfmd_sample_meta	(3444, 32)

Section 2 – Normalise Taxonomy Labels

The two datasets use different taxonomy separators and formats:

- **Johnson (gut):** k__Bacteria;p__Firmicutes;...;s__Faecalibacterium prausnitzii
- **cFMD (food):** k__Bacteria|p__Firmicutes|...|s__Lactobacillus_acidophilus|t__SGB...

We extract a clean **genus** and **species** label from each, enabling cross-dataset matching.

```
In [ ]: # Taxonomy parsing helpers

def parse_gut_taxonomy(tax_string):
    """Parse Johnson et al. semicolon-delimited taxonomy string.
    Returns dict with keys: kingdom, phylum, class, order, family, genus, species
    """
    prefixes = {"k__": "kingdom", "p__": "phylum", "c__": "class",
                "o__": "order", "f__": "family", "g__": "genus", "s__": "species"}
    result = {}
    for part in str(tax_string).split(";"):
        part = part.strip()
        for prefix, level in prefixes.items():
            if part.startswith(prefix):
                result[level] = part[len(prefix):]
```

```

        val = part[len(prefix):].strip()
        result[level] = val if val and val != "NA" else None
    # Clean species: strip genus name prefix if repeated
    if result["species"] and result["genus"]:
        result["species"] = result["species"].replace("_", " ").strip()
    result["clean_species"] = (result["species"] or result["genus"] or "").lower
    return result

def parse_food_taxonomy(tax_string):
    """Parse cFMD pipe-delimited taxonomy string.
    Drops the trailing t__SGBXXXX strain level.
    """
    prefixes = {"k__": "kingdom", "p__": "phylum", "c__": "class",
                "o__": "order", "f__": "family", "g__": "genus", "s__": "species"}
    result = {v: None for v in prefixes.values()}
    for part in str(tax_string).split("|"):
        part = part.strip()
        if part.startswith("t__"): # skip strain level
            continue
        for prefix, level in prefixes.items():
            if part.startswith(prefix):
                val = part[len(prefix):].strip()
                result[level] = val if val else None
    if result["species"]:
        result["species"] = result["species"].replace("_", " ").strip()
    result["clean_species"] = (result["species"] or result["genus"] or "").lower
    return result

# Build taxonomy dataframes
gut_taxonomy_df = pd.DataFrame(
    [parse_gut_taxonomy(t) for t in gut_micro_filtered.index],
    index=gut_micro_filtered.index
)
gut_taxonomy_df.index.name = "original_taxon"

food_taxonomy_df = pd.DataFrame(
    [parse_food_taxonomy(t) for t in food_micro_abundance.index],
    index=food_micro_abundance.index
)
food_taxonomy_df.index.name = "original_taxon"

print(f"Gut taxonomy parsed : {len(gut_taxonomy_df)} taxa")
print(f"Food taxonomy parsed : {len(food_taxonomy_df)} taxa")
print()
print("Gut taxonomy sample:")
display(gut_taxonomy_df.head(3))
print("Food taxonomy sample:")
display(food_taxonomy_df.head(3))

```

Gut taxonomy parsed : 223 taxa

Food taxonomy parsed : 5057 taxa

Gut taxonomy sample:

k_Bacteria;p_Bacteroidetes;c_Bacteroidia;o_Bacteroidales;f_Bacteroidacea

k_Bacteria;p_Firmicutes;c_Clostridia;o_Clostridiales;f_Ruminococcaceae;g_Faecalibacterium

k_Bacteria;p_Bacteroidetes;c_Bacteroidia;o_Bacteroidales;f_Bacteroidaceae;g_Bacter

< >

Food taxonomy sample:

k_Bacteria|p_Firmicutes|c_Bacilli|o_Lactobacillales|f_Streptococ

k_Bacteria|p_Actinomycetota|c_Actinomycetes|o_Propionibacteriales|f_Propionibacteriacea

k_Bacteria|p_Firmicutes|c_Bacilli|o_Lactobacillales|f_Lactobacillaceae|g_Lactipl

< >

Section 3 – Unified Sample Metadata

Combine sample metadata from both sources into a single table with a `source` column.

```
In [ ]: # Johnson gut sample metadata
gut_meta_unified = gut_sample_meta.reset_index().rename(columns={"index": "sample_id", "source": "johnson_gut", "data_type": "gut_microbiome"})

# Keep key columns
gut_meta_cols = ["sample_id", "source", "data_type", "UserName", "StudyDayNo", "Age", "BMI", "Gender"]
gut_meta_cols = [c for c in gut_meta_cols if c in gut_meta_unified.columns]
gut_meta_unified = gut_meta_unified[gut_meta_cols].copy()

# cFMD food sample metadata
food_meta_unified = cfmd_sample_meta.rename(columns={"sample_id": "sample_id", "category": "food_category", "macrocategory": "food_macrocategory", "country": "country"})
food_meta_unified = food_meta_unified.copy()
food_meta_unified["source"] = "cfmd_food"
food_meta_unified["data_type"] = "food_microbiome"

food_meta_cols = ["sample_id", "source", "data_type", "food_category", "food_macrocategory", "country"]
food_meta_cols = [c for c in food_meta_cols if c in food_meta_unified.columns]
```



```

food_meta_unified = food_meta_unified[food_meta_cols].copy()

#Concatenate
unified_sample_metadata = pd.concat([gut_meta_unified, food_meta_unified],
                                     ignore_index=True, sort=False)

print(f"Unified sample metadata: {unified_sample_metadata.shape}")
print(f"  Johnson gut samples   : {(unified_sample_metadata['source']=='johnson_gut').sum()}")
print(f"  cFMD food samples     : {(unified_sample_metadata['source']=='cfmd_food').sum()}")
display(unified_sample_metadata.head(4))

unified_sample_metadata.to_csv(DATA_DIR / "unified_sample_metadata.csv", index=False)
print("\nSaved: unified_sample_metadata.csv")

```

Unified sample metadata: (4010, 11)

Johnson gut samples : 566

cFMD food samples : 3444

	source	data_type	UserName	StudyDayNo	Age	BMI	Gender	samp
0	johnson_gut	gut_microbiome	MCTs01	1.0000	25.9000	25.0000	F	
1	johnson_gut	gut_microbiome	MCTs01	2.0000	25.9000	25.0000	F	
2	johnson_gut	gut_microbiome	MCTs01	3.0000	25.9000	25.0000	F	
3	johnson_gut	gut_microbiome	MCTs01	4.0000	25.9000	25.0000	F	

Saved: unified_sample_metadata.csv

Section 4 – Shared Taxa: Gut × Food Microbiome

Find taxa that appear in **both** the gut microbiome (Johnson) and the food microbiome (cFMD). Matching is done at the **species level** using the normalised `clean_species` label.

```

In [ ]: #Species sets from each dataset
gut_species_set = set(gut_taxonomy_df["clean_species"].dropna())
food_species_set = set(food_taxonomy_df["clean_species"].dropna())

shared_species = gut_species_set & food_species_set
only_gut_species = gut_species_set - food_species_set
only_food_species = food_species_set - gut_species_set

print(f"Gut unique species : {len(gut_species_set)}")
print(f"Food unique species : {len(food_species_set)}")
print(f"Shared species       : {len(shared_species)}")
print(f"Gut-only species     : {len(only_gut_species)}")
print(f"Food-only species     : {len(only_food_species)}")
print()
print("Examples of shared species:")
for s in sorted(shared_species)[:15]:
    print(f"  {s}")

```

```

Gut unique species : 211
Food unique species : 4770
Shared species      : 40
Gut-only species    : 171
Food-only species    : 4730

```

Examples of shared species:

```

akkermansia_muciniphila
alistipes_finegoldii
alistipes_nderdonkii
alistipes_putredinis
alistipes_shahii
anaerostipes_hadrus
bacteroides_caccae
bacteroides_ovatus
bacteroides_stercoris
bacteroides_thetaiotaomicron
bacteroides_uniformis
barnesiella_viscericola
bifidobacterium_adolescentis
bifidobacterium_bifidum

```

```

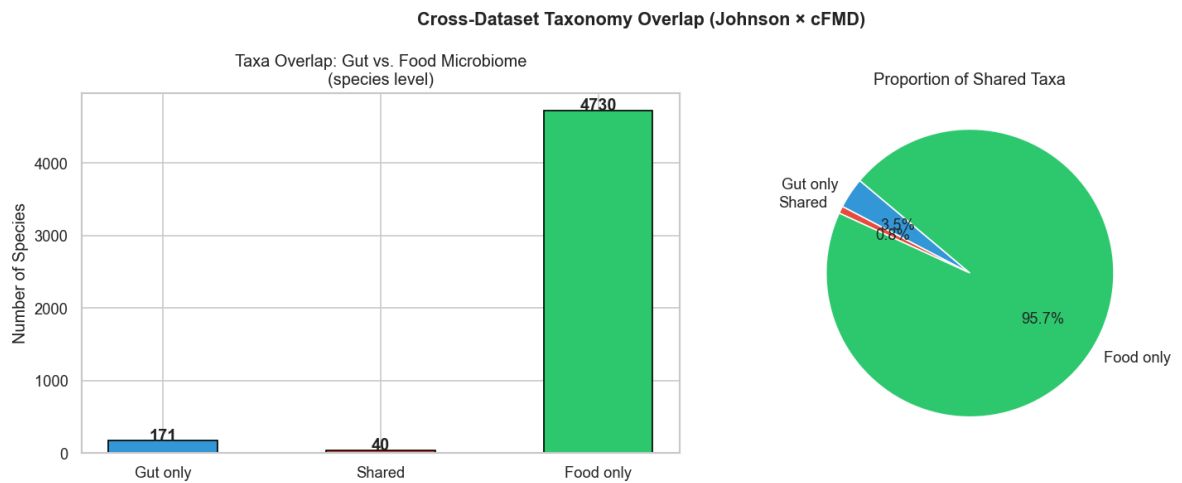
In [ ]: # Venn style bar chart
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

# Bar chart
labels = ["Gut only", "Shared", "Food only"]
values = [len(only_gut_species), len(shared_species), len(only_food_species)]
colors = ["#3498db", "#e74c3c", "#2ecc71"]
bars = axes[0].bar(labels, values, color=colors, edgecolor="black", width=0.5)
for bar, val in zip(bars, values):
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1,
                  str(val), ha="center", fontweight="bold", fontsize=12)
axes[0].set_title("Taxa Overlap: Gut vs. Food Microbiome\n(species level)")
axes[0].set_ylabel("Number of Species")

# Pie chart
axes[1].pie(values, labels=labels, colors=colors, autopct="%1.1f%%",
             startangle=140, textprops={"fontsize": 11})
axes[1].set_title("Proportion of Shared Taxa")

plt.suptitle("Cross-Dataset Taxonomy Overlap (Johnson x cFMD)", fontsize=13, fontweight="bold")
plt.tight_layout()
plt.show()

```



Section 5 – Unified Taxonomy Table

Build a master taxonomy reference table listing every unique taxon from both datasets, with a flag indicating in which dataset(s) it appears.

```
In [ ]: # Gut taxonomy with source flag
gut_tax_flagged = gut_taxonomy_df.copy()
gut_tax_flagged["in_gut_microbiome"] = True
gut_tax_flagged["in_food_microbiome"] = gut_tax_flagged["clean_species"].isin(food_taxonomy_df["clean_species"])
gut_tax_flagged["source"] = "johnson_gut"

# Food taxonomy with source flag
food_tax_flagged = food_taxonomy_df.copy()
food_tax_flagged["in_gut_microbiome"] = food_tax_flagged["clean_species"].isin(gut_taxonomy_df["clean_species"])
food_tax_flagged["in_food_microbiome"] = True
food_tax_flagged["source"] = "cfmd_food"

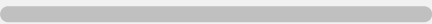
# Concatenate
unified_taxonomy_table = pd.concat(
    [gut_tax_flagged.reset_index(), food_tax_flagged.reset_index()],
    ignore_index=True, sort=False
)

print(f"Unified taxonomy table: {unified_taxonomy_table.shape}")
display(unified_taxonomy_table.head(5))

unified_taxonomy_table.to_csv(DATA_DIR / "unified_taxonomy_table.csv", index=False)
print("\nSaved: unified_taxonomy_table.csv")
```

Unified taxonomy table: (5280, 12)

	original_taxon	kingdom	phylum	class
0	k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o...	Bacteria	Bacteroidetes	Bacteroidia Bactero
1	k__Bacteria;p__Firmicutes;c__Clostridia;o__Clo...	Bacteria	Firmicutes	Clostridia Clostri
2	k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o...	Bacteria	Bacteroidetes	Bacteroidia Bactero
3	k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o...	Bacteria	Bacteroidetes	Bacteroidia Bactero
4	k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o...	Bacteria	Bacteroidetes	Bacteroidia Bactero

<  >

Saved: unified_taxonomy_table.csv

Section 6 – Long-Format Abundance Tables

Convert both wide matrices (taxa × samples) to long format (one row per taxon-sample pair). This format is optimal for statistical modelling and merging.

```
In [ ]: # Gut microbiome to long format
gut_micro_filtered_clean = gut_micro_filtered.copy()
gut_micro_filtered_clean.index.name = "original_taxon"

# Add clean species label
gut_micro_filtered_clean["clean_species"] = gut_taxonomy_df["clean_species"]

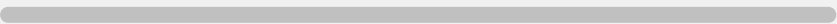
gut_long = (
    gut_micro_filtered_clean
    .reset_index()
    .melt(id_vars=["original_taxon", "clean_species"],
          var_name="sample_id",
          value_name="relative_abundance")
)
gut_long["source"] = "johnson_gut"

# Drop zero abundance rows to save memory
gut_long = gut_long[gut_long["relative_abundance"] > 0].reset_index(drop=True)

print(f"Gut microbiome long format : {gut_long.shape}")
display(gut_long.head(3))
gut_long.to_csv(DATA_DIR / "gut_microbiome_long.csv", index=False)
print("Saved: gut_microbiome_long.csv")
```

Gut microbiome long format : (98242, 5)

	original_taxon	clean_species	sample_id	relati
0	k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o...	bacteroides	MCT.f.0002	
1	k__Bacteria;p__Firmicutes;c__Clostridia;o__Clo...	faecalibacterium_prausnitzii	MCT.f.0002	
2	k__Bacteria;p__Bacteroidetes;c__Bacteroidia;o...	bacteroides_uniformis	MCT.f.0002	

<  >

Saved: gut_microbiome_long.csv

```
In [ ]: # Food microbiome to Long format
food_micro_clean = food_micro_abundance.copy()
food_micro_clean.index.name = "original_taxon"

# Add clean species Label
food_micro_clean["clean_species"] = food_taxonomy_df["clean_species"]

food_long = (
    food_micro_clean
    .reset_index()
    .melt(id_vars=["original_taxon", "clean_species"],
          var_name="full_sample_id",
          value_name="relative_abundance")
)

# Extract short sample_id (strip StudyName__ prefix)
food_long["sample_id"] = food_long["full_sample_id"].str.split("__").str[-1]
food_long = food_long.drop(columns="full_sample_id")
food_long["source"] = "cfmd_food"

# Merge food category from metadata
food_long = food_long.merge(
    cfmd_sample_meta[["sample_id", "category", "macrocategory", "country"]],
    on="sample_id", how="left"
)

# Drop zeros
food_long = food_long[food_long["relative_abundance"] > 0].reset_index(drop=True)

print(f"Food microbiome long format : {food_long.shape}")
display(food_long.head(3))
food_long.to_csv(DATA_DIR / "food_microbiome_long.csv", index=False)
print("Saved: food_microbiome_long.csv")
```

Food microbiome long format : (94491, 8)

	original_taxon	clean_species	relative_
0	k_Bacteria p_Firmicutes c_Bacilli o_Lactob...	lactococcus_lactis	
1	k_Bacteria p_Pseudomonadota c_Gammaproteoba...	pantoea_coffeiphila	
2	k_Bacteria p_Firmicutes c_Bacilli o_Lactob...	limosilactobacillus_fermentum	

Saved: food_microbiome_long.csv

Section 7 – Merged Abundance Matrix for Shared Taxa

Build a merged abundance matrix containing **only taxa found in both datasets**.

- Rows = shared taxa (clean_species)
- Columns = gut samples + food samples
- Values = relative abundance

This is the core matrix for comparing food-derived vs. gut microbiome profiles.

```
In [ ]: # Subset gut matrix to shared taxa
gut_tax_map = gut_taxonomy_df["clean_species"].to_dict() # original_taxon →
food_tax_map = food_taxonomy_df["clean_species"].to_dict()

# Gut: reindex rows by clean_species
gut_for_merge = gut_micro_filtered.copy()
gut_for_merge.index = gut_for_merge.index.map(gut_tax_map)
gut_for_merge = (
    gut_for_merge[gut_for_merge.index.isin(shared_species)]
    .groupby(level=0).mean() # average if same clean_species maps to multi
)

# Food: reindex rows by clean_species
food_for_merge = food_micro_abundance.copy()
food_for_merge.index = food_for_merge.index.map(food_tax_map)
food_for_merge = (
    food_for_merge[food_for_merge.index.isin(shared_species)]
    .groupby(level=0).mean()
)

# Rename columns to avoid collision
gut_for_merge.columns = ["gut__" + c for c in gut_for_merge.columns]
food_for_merge.columns = ["food__" + c for c in food_for_merge.columns]

# Merge on shared taxa (inner join on index)
merged_shared_taxa = gut_for_merge.join(food_for_merge, how="inner")
merged_shared_taxa.index.name = "clean_species"

print(f"Merged shared taxa matrix: {merged_shared_taxa.shape}")
print(f"  Shared taxa (rows)      : {merged_shared_taxa.shape[0]}")
print(f"  Gut sample columns       : {gut_for_merge.shape[1]}")
print(f"  Food sample columns      : {food_for_merge.shape[1]}")
display(merged_shared_taxa.iloc[:5, :6])

merged_shared_taxa.to_csv(DATA_DIR / "merged_shared_taxa.csv")
print("\nSaved: merged_shared_taxa.csv")
```

Merged shared taxa matrix: (40, 3876)

Shared taxa (rows) : 40
 Gut sample columns : 483
 Food sample columns : 3393

	gut_MCT.f.0002	gut_MCT.f.0003	gut_MCT.f.0004	gut_MCT.f.0
clean_species				
	1272.1538	2616.3846	1291.6154	1232.9
akkermansia_muciniphila	79.0000	63.0000	135.0000	10.0
alistipes_finegoldii	3554.0000	129.0000	2160.0000	1952.0
alistipes_nderdonkii	3723.0000	111.0000	1993.0000	2818.0
alistipes_putredinis	18393.0000	561.0000	11742.0000	15831.0

Saved: merged_shared_taxa.csv

Section 8 – Integration Summary

Files produced by this notebook

File	Rows	Description
<code>unified_sample_metadata.csv</code>	566 gut + 3,444 food	All sample metadata from both sources
<code>unified_taxonomy_table.csv</code>	gut + food taxa	Full taxonomy with cross-dataset flags
<code>gut_microbiome_long.csv</code>	non-zero entries	Gut microbiome in long format
<code>food_microbiome_long.csv</code>	non-zero entries	cFMD food microbiome in long format
<code>merged_shared_taxa.csv</code>	shared taxa	Abundance matrix for taxa found in both datasets

Next steps

- Use `merged_shared_taxa.csv` to compare gut vs. food abundance profiles for shared taxa
- Use `unified_sample_metadata.csv` as a lookup for participant/food metadata in all analyses
- Feed `merged_shared_taxa.csv` into the FME analysis notebook for weight derivation

```
In [ ]: # Final summary printout
outputs = {
    "unified_sample_metadata.csv" : unified_sample_metadata,
    "unified_taxonomy_table.csv" : unified_taxonomy_table,
    "gut_microbiome_long.csv" : gut_long,
    "food_microbiome_long.csv" : food_long,
    "merged_shared_taxa.csv" : merged_shared_taxa,
}

print("=" * 55)
print(f"{'File':<35} {'Shape':>12}")
print("=" * 55)
for fname, df in outputs.items():
    print(f"  {fname:<33} {str(df.shape):>12}")
print("=" * 55)
print(f"\nShared taxa between gut and food microbiome: {len(shared_species)}")
print(f"Gut-only taxa : {len(only_gut_species)}")
print(f"Food-only taxa : {len(only_food_species)}")
```

```
=====
File                                     Shape
=====
unified_sample_metadata.csv            (4010, 11)
unified_taxonomy_table.csv             (5280, 12)
gut_microbiome_long.csv                 (98242, 5)
food_microbiome_long.csv                (94491, 8)
merged_shared_taxa.csv                  (40, 3876)
=====
```

Shared taxa between gut and food microbiome: 40

Gut-only taxa : 171

Food-only taxa : 4730

Notebook: Association_Testing.ipynb

Association Testing

Objective

The goal of this analysis is to investigate the relationship between Food Microbiome Exposure (FME) and gut microbiome characteristics.

Specifically, we examine whether FME is associated with:

- Alpha diversity (Shannon diversity)
- Microbiome stability
- Community composition metrics

This analysis is based on the dataset generated in the FME analysis stage.

Dataset Overview

In this section, we load and inspect the statistical dataset created in the FME analysis step.

The goal is to understand the structure of the dataset, including the number of samples, available variables, data types, and missing values before performing association testing.

```
In [ ]: # Load the statistical dataset created in the FME analysis step

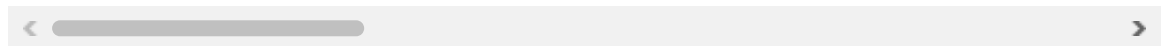
df = pd.read_csv(DATA_DIR / "fme_statistical_dataset.csv")

df.head()
```


Out []: **fecal_sample_id** **participant_id** **study_day** **Gender** **Age** **BMI** **Weight** **Supple**

0	MCT.f.0002	MCTs01	2	F	25.9000	25.0000	74.6000
1	MCT.f.0003	MCTs01	3	F	25.9000	25.0000	74.6000
2	MCT.f.0004	MCTs01	4	F	25.9000	25.0000	74.6000
3	MCT.f.0005	MCTs01	5	F	25.9000	25.0000	74.6000
4	MCT.f.0006	MCTs01	6	F	25.9000	25.0000	74.6000

5 rows × 27 columns



```
In [ ]: # Dataset overview

print("Shape:", df.shape)

print("\nColumns:")
print(df.columns.tolist())
```

Shape: (475, 27)

Columns:

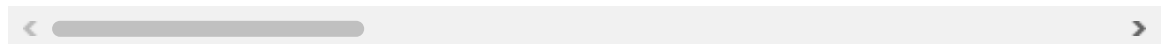
['fecal_sample_id', 'participant_id', 'study_day', 'Gender', 'Age', 'BMI', 'Weight', 'Supplement', 'Medications', 'fme_score_daily', 'shannon_diversity', 'participant_shannon_cv', 'participant_shannon_mean', 'participant_n_samples', 'KCAL', 'P ROT', 'TFAT', 'CARB', 'FIBE', 'SUGR', 'SODI', 'D_TOTAL', 'D_YOGURT', 'D_CHEESE', 'PF_MEAT', 'PF_SEAFD_HI', 'G_WHOLE']

```
In [ ]: # Summary statistics

df.describe()
```

Out []: **study_day** **Age** **BMI** **Weight** **fme_score_daily** **shannon_diversity** **par**

count	475.0000	475.0000	475.0000	475.0000	475.0000	475.0000
mean	8.9516	31.1522	23.1747	69.0691	73.3252	2.8646
std	4.9397	10.1565	3.2921	14.6469	58.3071	0.4132
min	1.0000	19.7000	17.0000	48.5000	0.0000	1.5717
25%	5.0000	23.9000	21.1000	59.1000	29.3222	2.6108
50%	9.0000	28.5000	22.9000	66.9000	61.7775	2.9644
75%	13.0000	37.0000	25.9000	82.9000	104.2697	3.1430
max	17.0000	61.6000	31.6000	104.8000	331.2240	3.7310



```
In [ ]: print(df.isnull().sum())
```

```
fecal_sample_id      0
participant_id       0
study_day            0
Gender               0
Age                 0
BMI                 0
Weight              0
Supplement           0
Medications          255
fme_score_daily      0
shannon_diversity    0
participant_shannon_cv 0
participant_shannon_mean 0
participant_n_samples 0
KCAL                0
PROT                0
TFAT                0
CARB                0
FIBE                0
SUGR                0
SODI                0
D_TOTAL             0
D_YOGURT            0
D_CHEESE            0
PF_MEAT             0
PF_SEAFD_HI         0
G_WHOLE             0
dtype: int64
```

Correlation Analysis

In this section, we investigate the relationship between Food Microbiome Exposure (FME) and microbiome diversity metrics.

We begin by examining the correlation between the daily FME score and Shannon diversity, which is a commonly used measure of microbiome diversity.

```
In [ ]: # Correlation between FME and Shannon Diversity
```

```
correlation, p_value = pearsonr(
    df["fme_score_daily"],
    df["shannon_diversity"]
)

print(f"Pearson Correlation: {correlation:.4f}")
print(f"P-value: {p_value:.4f}")
```

```
Pearson Correlation: -0.0406
P-value: 0.3776
```

Visualization of FME and Shannon Diversity

The following scatter plot illustrates the relationship between daily Food Microbiome Exposure (FME) scores and Shannon diversity values across all samples.

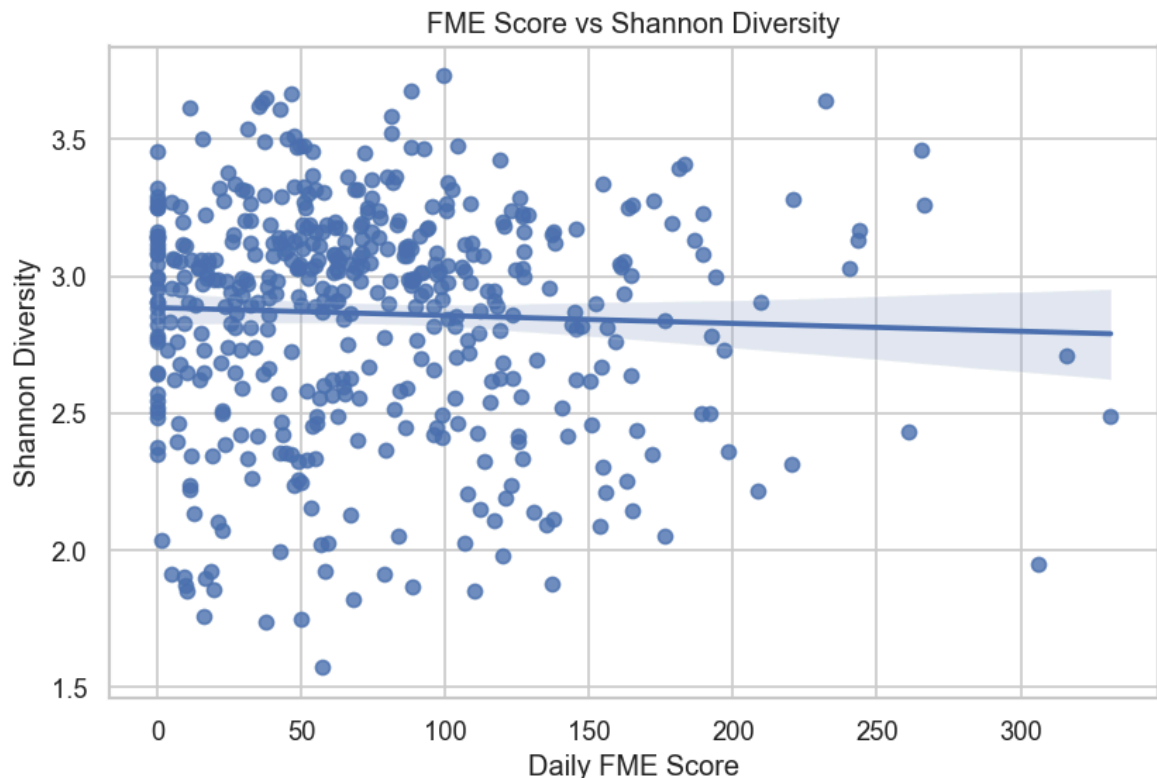
A regression line is added to help visualize the overall trend.

```
In [ ]: plt.figure(figsize=(8, 5))

sns.regplot(
    data=df,
    x="fme_score_daily",
    y="shannon_diversity"
)

plt.title("FME Score vs Shannon Diversity")
plt.xlabel("Daily FME Score")
plt.ylabel("Shannon Diversity")

plt.show()
```



Insight

The relationship between FME score and Shannon diversity appears to be weak.

Although FME values vary considerably across samples, Shannon diversity remains relatively stable.

The nearly flat regression line suggests little to no linear association between the two variables.

Correlation Matrix

To better understand the relationships between microbiome-related variables, we compute a correlation matrix for the main numerical variables used in this analysis.

```
In [ ]: correlation_columns = [
    "fme_score_daily",
```

```
"shannon_diversity",  
"participant_shannon_cv",  
"participant_shannon_mean"  
]  
  
corr_matrix = df[correlation_columns].corr()  
  
corr_matrix
```

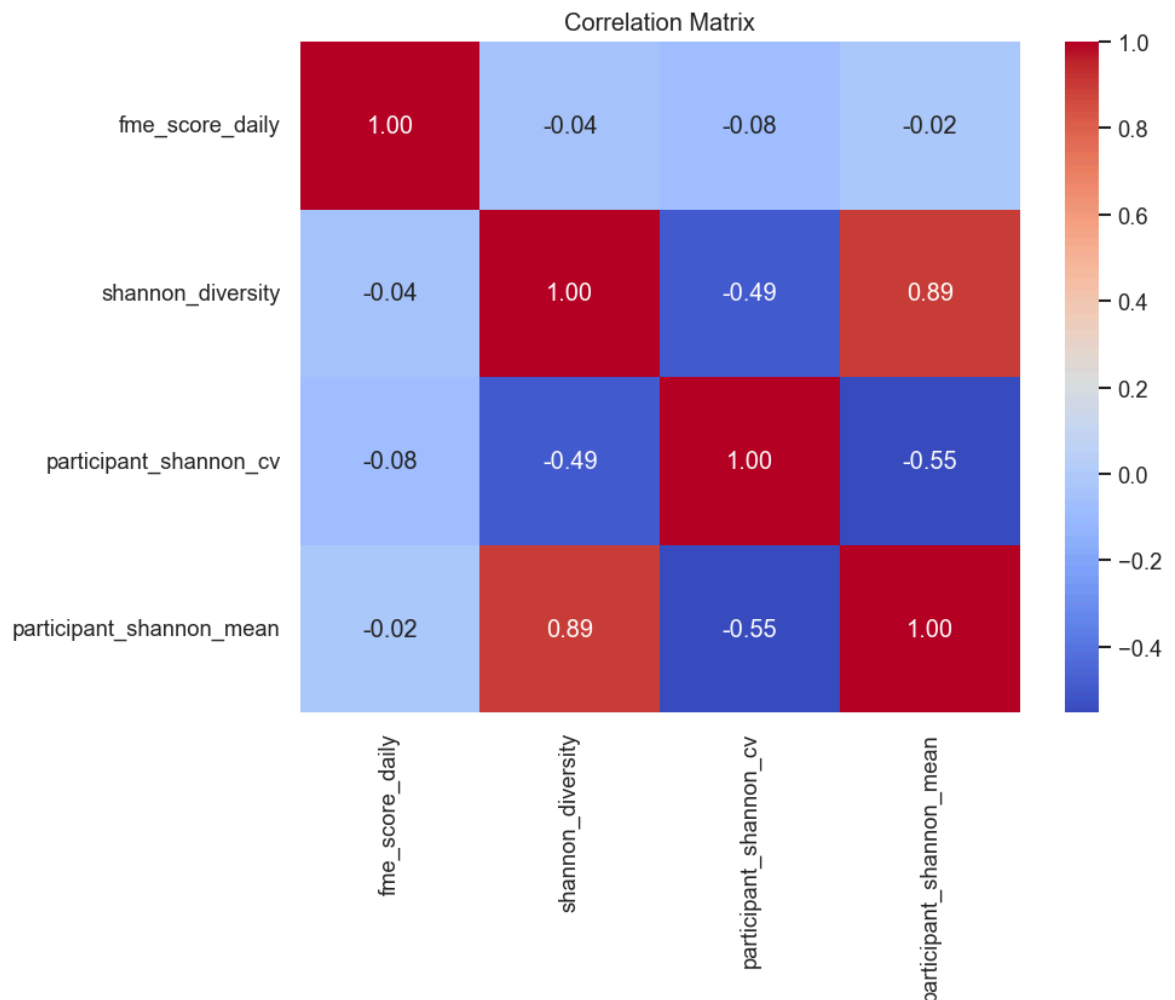
Out []:

	fme_score_daily	shannon_diversity	participant_shannon_cv	participant_shannon_mean
fme_score_daily	1.0000	-0.0406	-0.0771	-0.0153
shannon_diversity	-0.0406	1.0000	-0.4929	0.8917
participant_shannon_cv	-0.0771	-0.4929	1.0000	-0.5527
participant_shannon_mean	-0.0153	0.8917	-0.5527	



In []:

```
plt.figure(figsize=(8, 6))  
  
sns.heatmap(  
    corr_matrix,  
    annot=True,  
    cmap="coolwarm",  
    fmt=".2f"  
)  
  
plt.title("Correlation Matrix")  
plt.show()
```



Insight

FME score has very weak correlations with all diversity metrics.

Shannon diversity is strongly correlated with participant mean Shannon diversity.

Comparison Between Low and High FME Groups

To further investigate the relationship between Food Microbiome Exposure (FME) and microbiome diversity, samples were divided into two groups based on the median FME score.

Shannon diversity distributions were then compared between the groups.

```
In [ ]: # Create FME groups based on median value

median_fme = df["fme_score_daily"].median()

df["fme_group"] = np.where(
    df["fme_score_daily"] >= median_fme,
    "High FME",
    "Low FME"
)

df["fme_group"].value_counts()
```

```
Out[ ]: fme_group
High FME    238
Low FME     237
Name: count, dtype: int64
```

Shannon Diversity by FME Group

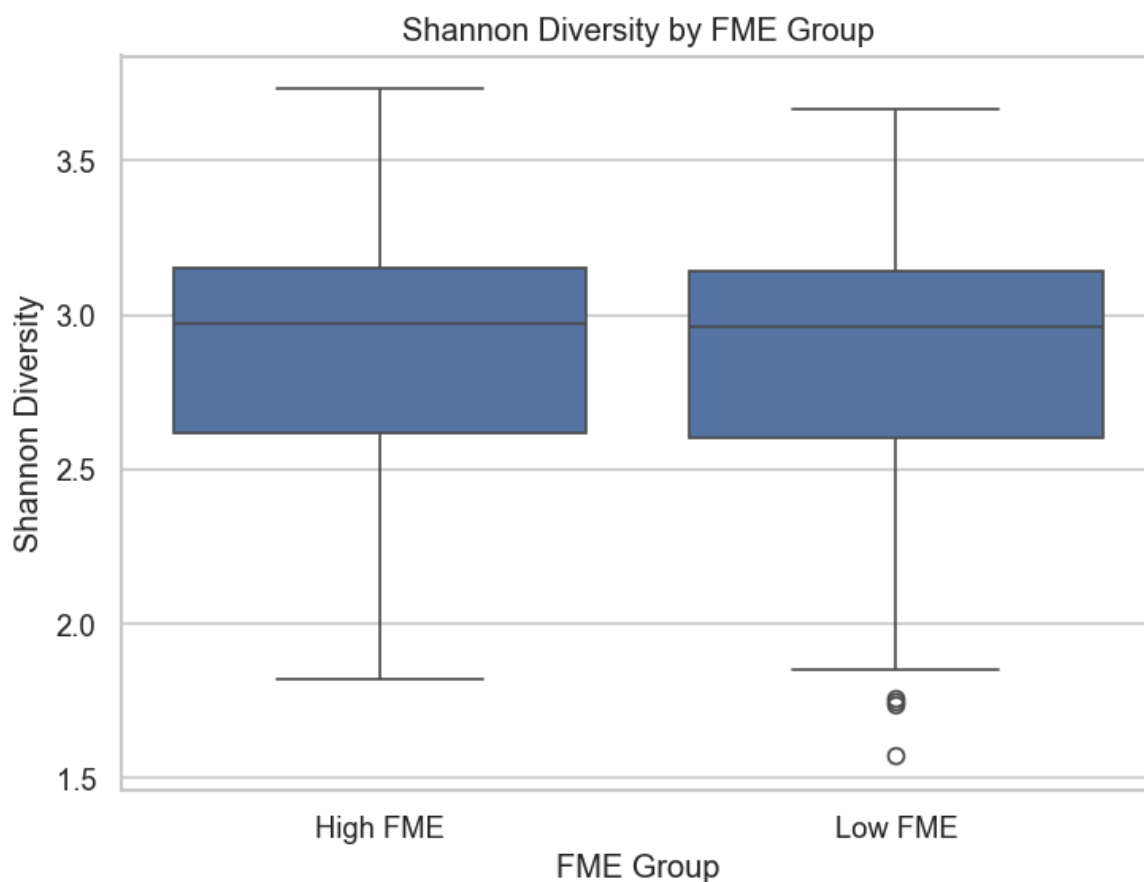
The following boxplot compares Shannon diversity values between samples with low and high Food Microbiome Exposure.

```
In [ ]: plt.figure(figsize=(7,5))

sns.boxplot(
    data=df,
    x="fme_group",
    y="shannon_diversity"
)

plt.title("Shannon Diversity by FME Group")
plt.xlabel("FME Group")
plt.ylabel("Shannon Diversity")

plt.show()
```



Insight

The distributions of Shannon diversity are similar in both FME groups.

No clear difference in diversity is observed between low and high FME exposure levels.

Statistical Test

A Mann–Whitney U test was performed to evaluate whether Shannon diversity differs significantly between the two FME groups.

```
In [ ]: high_fme = df[df["fme_group"] == "High FME"]["shannon_diversity"]
low_fme = df[df["fme_group"] == "Low FME"]["shannon_diversity"]

stat, p_value = mannwhitneyu(high_fme, low_fme)

print("Mann-Whitney U Statistic:", stat)
print("P-value:", p_value)
```

Mann-Whitney U Statistic: 28228.0

P-value: 0.9869319368980619

Insight

No significant difference was found between the low and high FME groups ($p = 0.987$).

This suggests that FME levels are not associated with differences in Shannon diversity in this dataset.

Adjusted Regression Analysis

To account for potential dietary confounders, an additional regression model was fitted including nutrient intake variables.

This allows evaluation of the association between FME and Shannon diversity while controlling for dietary factors.

```
In [ ]: model_adj = smf.ols(
    "shannon_diversity ~ fme_score_daily + KCAL + PROT + TFAT + CARB + FIBE",
    data=df
).fit()

print(model_adj.summary())
```

OLS Regression Results

=====						
Dep. Variable:	shannon_diversity		R-squared:	0.027		
Model:	OLS		Adj. R-squared:	0.014		
Method:	Least Squares		F-statistic:	2.152		
Date:	Wed, 24 Jun 2026		Prob (F-statistic):	0.0465		
Time:	00:25:48		Log-Likelihood:	-247.17		
No. Observations:	475		AIC:	508.3		
Df Residuals:	468		BIC:	537.5		
Df Model:	6					
Covariance Type:	nonrobust					
=====						
==						
	coef	std err	t	P> t	[0.025	0.97

--						
Intercept	2.7853	0.061	45.544	0.000	2.665	2.9
fme_score_daily	-6.297e-05	0.000	-0.156	0.876	-0.001	0.0
KCAL	0.0002	0.000	1.326	0.185	-9.51e-05	0.0
PROT	1.476e-05	0.001	0.015	0.988	-0.002	0.0
TFAT	-0.0023	0.001	-1.582	0.114	-0.005	0.0
CARB	-0.0001	0.001	-0.191	0.849	-0.002	0.0
FIBE	-0.0043	0.002	-1.715	0.087	-0.009	0.0
=====						
Omnibus:	29.179	Durbin-Watson:	0.518			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	33.191			
Skew:	-0.645	Prob(JB):	6.20e-08			
Kurtosis:	3.106	Cond. No.	7.32e+03			

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 7.32e+03. This might indicate that there are strong multicollinearity or other numerical problems.

Insight

After adjusting for nutrient intake variables, FME score remained a non-significant predictor of Shannon diversity ($p = 0.876$).

This suggests that the observed lack of association is not explained by differences in dietary intake.

Conclusions

Several statistical approaches were used to evaluate the relationship between Food Microbiome Exposure (FME) and gut microbiome diversity.

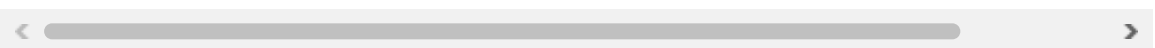
Main findings:

- The correlation between FME score and Shannon diversity was very weak.
- No significant differences were observed between low and high FME groups.
- Simple linear regression showed that FME score was not a significant predictor of Shannon diversity.
- After adjusting for dietary intake variables (KCAL, PROT, TFAT, CARB, and FIBE), FME score remained non-significant.
- Adjusting for nutrient intake did not change the overall findings.

Overall, no statistically significant association was found between FME levels and Shannon diversity in this dataset.

Notebook:

Association_Testing_adjusted_models.



Association Testing

Objective

The goal of this analysis is to investigate the relationship between Food Microbiome Exposure (FME) and gut microbiome characteristics.

Specifically, we examine whether FME is associated with:

- Alpha diversity (Shannon diversity)
- Microbiome stability
- Community composition metrics

This analysis is based on the dataset generated in the FME analysis stage.

Dataset Overview

In this section, we load and inspect the statistical dataset created in the FME analysis step.

The goal is to understand the structure of the dataset, including the number of samples, available variables, data types, and missing values before performing association testing.

```
In [ ]: #Load the statistical dataset created in the FME analysis step

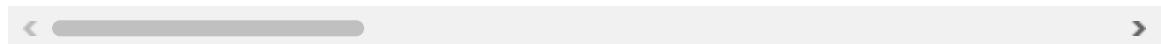
df = pd.read_csv(DATA_DIR / "fme_statistical_dataset_extended.csv")

df.head()
```

Out[]: **fecal_sample_id** **participant_id** **study_day** **Gender** **Age** **BMI** **Weight** **Supple**

0	MCT.f.0002	MCTs01	2	F	25.9000	25.0000	74.6000
1	MCT.f.0003	MCTs01	3	F	25.9000	25.0000	74.6000
2	MCT.f.0004	MCTs01	4	F	25.9000	25.0000	74.6000
3	MCT.f.0005	MCTs01	5	F	25.9000	25.0000	74.6000
4	MCT.f.0006	MCTs01	6	F	25.9000	25.0000	74.6000

5 rows × 36 columns



In []: *#Check that the extended FME features were loaded correctly*

```
fme_feature_check_cols = [  
    "participant_id",  
    "fme_score_daily",  
    "fme_participant_mean",  
    "fme_within_person",  
    "fme_lag1",  
    "fme_lag2",  
    "fme_weighted_3day"  
]  
  
df[fme_feature_check_cols].head(10)
```

Out[]: **participant_id** **fme_score_daily** **fme_participant_mean** **fme_within_person** **fme_lag1**

0	MCTs01	67.1961	71.2710	-4.0749	NaN
1	MCTs01	81.2412	71.2710	9.9703	67.1961
2	MCTs01	126.7711	71.2710	55.5001	81.2412
3	MCTs01	31.4846	71.2710	-39.7864	126.7711
4	MCTs01	61.2266	71.2710	-10.0444	31.4846
5	MCTs01	62.0231	71.2710	-9.2479	61.2266
6	MCTs01	70.9874	71.2710	-0.2836	62.0231
7	MCTs01	54.0086	71.2710	-17.2624	70.9874
8	MCTs01	105.6813	71.2710	34.4103	54.0086
9	MCTs01	62.2759	71.2710	-8.9951	105.6813



In []: *#Check missing values in the extended FME features*

```
df[  
    [  
        "fme_score_daily",
```

```

        "fme_participant_mean",
        "fme_within_person",
        "fme_lag1",
        "fme_lag2",
        "fme_weighted_3day"
    ]
].isna().sum()

```

```

Out[ ]: fme_score_daily      0
        fme_participant_mean  0
        fme_within_person    0
        fme_lag1             34
        fme_lag2             68
        fme_weighted_3day     0
        dtype: int64

```

```

In [ ]: #Dataset overview

print("Shape:", df.shape)

print("\nColumns:")
print(df.columns.tolist())

```

Shape: (475, 36)

Columns:

['fecal_sample_id', 'participant_id', 'study_day', 'Gender', 'Age', 'BMI', 'Weight', 'Supplement', 'Medications', 'fme_score_daily', 'fme_participant_mean', 'fme_within_person', 'fme_lag1', 'fme_lag2', 'fme_weighted_3day', 'shannon_diversity', 'richness', 'simpson_diversity', 'inverse_simpson', 'pielou_evenness', 'participant_shannon_cv', 'participant_shannon_mean', 'participant_n_samples', 'KCAL', 'PROT', 'TFAT', 'CARB', 'FIBE', 'SUGR', 'SODI', 'D_TOTAL', 'D_YOGURT', 'D_CHEESE', 'PF_MEAT', 'PF_SEAFD_HI', 'G_WHOLE']

```

In [ ]: # Summary statistics

df.describe()

```

```

Out[ ]:

```

	study_day	Age	BMI	Weight	fme_score_daily	fme_participant_mean
count	475.0000	475.0000	475.0000	475.0000	475.0000	475.0000
mean	8.9516	31.1522	23.1747	69.0691	73.3252	73.3252
std	4.9397	10.1565	3.2921	14.6469	58.3071	39.8038
min	1.0000	19.7000	17.0000	48.5000	0.0000	0.0000
25%	5.0000	23.9000	21.1000	59.1000	29.3222	53.6966
50%	9.0000	28.5000	22.9000	66.9000	61.7775	62.2512
75%	13.0000	37.0000	25.9000	82.9000	104.2697	99.4541
max	17.0000	61.6000	31.6000	104.8000	331.2240	147.8899

8 rows × 31 columns

<  >

```

In [ ]: print(df.isnull().sum())

```

```

fecal_sample_id      0
participant_id       0
study_day            0
Gender               0
Age                 0
BMI                 0
Weight              0
Supplement           0
Medications          255
fme_score_daily      0
fme_participant_mean 0
fme_within_person    0
fme_lag1             34
fme_lag2             68
fme_weighted_3day    0
shannon_diversity     0
richness             0
simpson_diversity     0
inverse_simpson       0
pielou_evenness      0
participant_shannon_cv 0
participant_shannon_mean 0
participant_n_samples 0
KCAL                 0
PROT                 0
TFAT                 0
CARB                 0
FIBE                 0
SUGR                 0
SODI                 0
D_TOTAL              0
D_YOGURT              0
D_CHEESE              0
PF_MEAT              0
PF_SEAFD_HI          0
G_WHOLE              0
dtype: int64

```

```

In [ ]: #Define the list of alpha diversity metrics to analyze
alpha_diversity_metrics = [
    "shannon_diversity",
    "richness",
    "simpson_diversity",
    "inverse_simpson",
    "pielou_evenness"
]

alpha_diversity_metrics

```

```

Out[ ]: ['shannon_diversity',
        'richness',
        'simpson_diversity',
        'inverse_simpson',
        'pielou_evenness']

```

```

In [ ]: #Define the list of FME features to analyze
fme_features = [
    "fme_score_daily",
    "fme_participant_mean",
    "fme_within_person",

```

```

    "fme_lag1",
    "fme_lag2",
    "fme_weighted_3day"
]

fme_features

```

```

Out[ ]: ['fme_score_daily',
        'fme_participant_mean',
        'fme_within_person',
        'fme_lag1',
        'fme_lag2',
        'fme_weighted_3day']

```

Correlation Analysis

In this section, we investigate the relationship between Food Microbiome Exposure (FME) and microbiome diversity metrics.

We begin by examining the correlation between the daily FME score and Shannon diversity, which is a commonly used measure of microbiome diversity.

```

In [ ]: #Correlation between extended FME features and alpha diversity metrics

extended_correlation_results = []

for fme_feature in fme_features:
    for metric in alpha_diversity_metrics:
        analysis_df = df[[fme_feature, metric]].dropna()

        pearson_corr, pearson_p = pearsonr(
            analysis_df[fme_feature],
            analysis_df[metric]
        )

        spearman_corr, spearman_p = spearmanr(
            analysis_df[fme_feature],
            analysis_df[metric]
        )

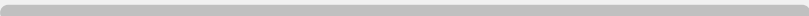
        extended_correlation_results.append({
            "fme_feature": fme_feature,
            "metric": metric,
            "n_samples": len(analysis_df),
            "pearson_corr": pearson_corr,
            "pearson_p_value": pearson_p,
            "spearman_corr": spearman_corr,
            "spearman_p_value": spearman_p
        })

extended_correlation_results_df = pd.DataFrame(extended_correlation_results)

display(extended_correlation_results_df.round(4))

```

	fme_feature	metric	n_samples	pearson_corr	pearson_p_value	spe
0	fme_score_daily	shannon_diversity	475	-0.0406	0.3776	
1	fme_score_daily	richness	475	0.0234	0.6113	
2	fme_score_daily	simpson_diversity	475	-0.0407	0.3758	
3	fme_score_daily	inverse_simpson	475	-0.0485	0.2917	
4	fme_score_daily	pielou_evenness	475	-0.0439	0.3400	
5	fme_participant_mean	shannon_diversity	475	-0.0200	0.6642	
6	fme_participant_mean	richness	475	-0.0081	0.8594	
7	fme_participant_mean	simpson_diversity	475	-0.0277	0.5476	
8	fme_participant_mean	inverse_simpson	475	-0.0650	0.1573	
9	fme_participant_mean	pielou_evenness	475	-0.0198	0.6676	
10	fme_within_person	shannon_diversity	475	-0.0369	0.4227	
11	fme_within_person	richness	475	0.0396	0.3891	
12	fme_within_person	simpson_diversity	475	-0.0299	0.5158	
13	fme_within_person	inverse_simpson	475	-0.0056	0.9027	
14	fme_within_person	pielou_evenness	475	-0.0416	0.3659	
15	fme_lag1	shannon_diversity	441	-0.0227	0.6351	
16	fme_lag1	richness	441	-0.0017	0.9714	
17	fme_lag1	simpson_diversity	441	-0.0319	0.5042	
18	fme_lag1	inverse_simpson	441	-0.0641	0.1788	
19	fme_lag1	pielou_evenness	441	-0.0227	0.6351	
20	fme_lag2	shannon_diversity	407	-0.0598	0.2284	
21	fme_lag2	richness	407	-0.0499	0.3157	
22	fme_lag2	simpson_diversity	407	-0.0589	0.2361	
23	fme_lag2	inverse_simpson	407	-0.0672	0.1758	
24	fme_lag2	pielou_evenness	407	-0.0582	0.2410	
25	fme_weighted_3day	shannon_diversity	475	-0.0441	0.3375	
26	fme_weighted_3day	richness	475	0.0102	0.8252	
27	fme_weighted_3day	simpson_diversity	475	-0.0451	0.3269	
28	fme_weighted_3day	inverse_simpson	475	-0.0677	0.1409	
29	fme_weighted_3day	pielou_evenness	475	-0.0461	0.3157	

◀  ▶

```
In [ ]: #Adjusted Models
        adjusted_model_results = []
```

```

for metric in alpha_diversity_metrics:
    model_df = df[
        ["fme_score_daily", metric, "Age", "BMI"]
    ].dropna()

    formula = f"{metric} ~ fme_score_daily + Age + BMI"

    model = smf.ols(
        formula=formula,
        data=model_df
    ).fit()

    adjusted_model_results.append({
        "metric": metric,
        "model": "OLS adjusted for Age + BMI",
        "n_samples": len(model_df),
        "fme_coef": model.params.get("fme_score_daily", np.nan),
        "fme_p_value": model.pvalues.get("fme_score_daily", np.nan),
        "age_p_value": model.pvalues.get("Age", np.nan),
        "bmi_p_value": model.pvalues.get("BMI", np.nan),
        "r_squared": model.rsquared
    })

adjusted_model_results_df = pd.DataFrame(adjusted_model_results)

display(adjusted_model_results_df.round(4))

```

	metric	model	n_samples	fme_coef	fme_p_value	age_p_value	bmi_p_value
0	shannon_diversity	OLS adjusted for Age + BMI	475	-0.0002	0.5154	0.2073	0.586
1	richness	OLS adjusted for Age + BMI	475	0.0042	0.6614	0.6000	0.852
2	simpson_diversity	OLS adjusted for Age + BMI	475	-0.0000	0.4961	0.1869	0.979
3	inverse_simpson	OLS adjusted for Age + BMI	475	-0.0021	0.4655	0.0285	0.947
4	pielou_evenness	OLS adjusted for Age + BMI	475	-0.0000	0.4763	0.1835	0.596

```

In [ ]: #Nutrition Covariates
nutrition_covariates = [
    "KCAL",
    "FIBE",

```

```

        "PROT",
        "TFAT",
        "CARB"
    ]

    available_nutrition_covariates = [
        col for col in nutrition_covariates
        if col in df.columns
    ]

    nutrition_adjusted_model_results = []

    for metric in alpha_diversity_metrics:
        model_columns = [
            "fme_score_daily",
            metric,
            "Age",
            "BMI"
        ] + available_nutrition_covariates

        model_df = df[model_columns].dropna()

        covariates = ["fme_score_daily", "Age", "BMI"] + available_nutrition_covariates
        formula = f"{metric} ~ " + " + ".join(covariates)

        model = smf.ols(
            formula=formula,
            data=model_df
        ).fit()

        nutrition_adjusted_model_results.append({
            "metric": metric,
            "model": "OLS adjusted for Age + BMI + nutrition",
            "n_samples": len(model_df),
            "fme_coef": model.params.get("fme_score_daily", np.nan),
            "fme_p_value": model.pvalues.get("fme_score_daily", np.nan),
            "age_p_value": model.pvalues.get("Age", np.nan),
            "bmi_p_value": model.pvalues.get("BMI", np.nan),
            "r_squared": model.rsquared
        })

    nutrition_adjusted_model_results_df = pd.DataFrame(nutrition_adjusted_model_results)

    display(nutrition_adjusted_model_results_df.round(4))

```


	metric	model	n_samples	fme_coef	fme_p_value	age_p_value	bmi_p_valu
0	shannon_diversity	OLS adjusted for Age + BMI + nutrition	475	0.0000	0.9498	0.0421	0.595
1	richness	OLS adjusted for Age + BMI + nutrition	475	0.0223	0.0596	0.9104	0.892
2	simpson_diversity	OLS adjusted for Age + BMI + nutrition	475	-0.0000	0.7941	0.0361	0.255
3	inverse_simpson	OLS adjusted for Age + BMI + nutrition	475	0.0004	0.9054	0.0064	0.381
4	pielou_evenness	OLS adjusted for Age + BMI + nutrition	475	-0.0000	0.9410	0.0352	0.574

```

In [ ]: #Mixed Models with Participant Random Intercept
mixed_model_results = []

for metric in alpha_diversity_metrics:
    model_columns = [
        "participant_id",
        "fme_score_daily",
        metric,
        "Age",
        "BMI"
    ] + available_nutrition_covariates

    model_df = df[model_columns].dropna()

    covariates = ["fme_score_daily", "Age", "BMI"] + available_nutrition_covaria
    formula = f"{metric} ~ " + " + ".join(covariates)

    try:
        model = smf.mixedlm(
            formula=formula,
            data=model_df,
            groups=model_df["participant_id"]
        ).fit()

        mixed_model_results.append({
            "metric": metric,

```

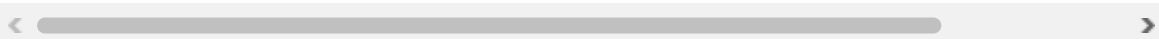
```
        "model": "MixedLM adjusted for Age + BMI + nutrition + participant r
        "n_samples": len(model_df),
        "n_participants": model_df["participant_id"].nunique(),
        "fme_coef": model.params.get("fme_score_daily", np.nan),
        "fme_p_value": model.pvalues.get("fme_score_daily", np.nan),
        "age_p_value": model.pvalues.get("Age", np.nan),
        "bmi_p_value": model.pvalues.get("BMI", np.nan)
    })

except Exception as e:
    mixed_model_results.append({
        "metric": metric,
        "model": "MixedLM failed",
        "n_samples": len(model_df),
        "n_participants": model_df["participant_id"].nunique(),
        "fme_coef": np.nan,
        "fme_p_value": np.nan,
        "age_p_value": np.nan,
        "bmi_p_value": np.nan,
        "error": str(e)
    })

mixed_model_results_df = pd.DataFrame(mixed_model_results)

display(mixed_model_results_df.round(4))
```

	metric	model	n_samples	n_participants	fme_coef	fme_p_value	age_p_v
0	shannon_diversity	MixedLM adjusted for Age + BMI + nutrition + p...	475	34	-0.0004	0.0510	0.6
1	richness	MixedLM adjusted for Age + BMI + nutrition + p...	475	34	0.0072	0.4767	0.8
2	simpson_diversity	MixedLM adjusted for Age + BMI + nutrition + p...	475	34	-0.0001	0.2745	0.6
3	inverse_simpson	MixedLM adjusted for Age + BMI + nutrition + p...	475	34	-0.0010	0.6792	0.5
4	pielou_evenness	MixedLM adjusted for Age + BMI + nutrition + p...	475	34	-0.0001	0.0392	0.6



In []: *#Mixed models separating between-person and within-person FME effects*

```

mixed_model_decomposed_results = []

for metric in alpha_diversity_metrics:
    model_columns = [
        "participant_id",
        "fme_participant_mean",
        "fme_within_person",
        metric,
        "Age",
        "BMI"
    ] + available_nutrition_covariates

    model_df = df[model_columns].dropna()

    covariates = [
        "fme_participant_mean",
        "fme_within_person",
        "Age",
        "BMI"
    ]

```

```

] + available_nutrition_covariates

formula = f"{metric} ~ " + " + ".join(covariates)

try:
    model = smf.mixedlm(
        formula=formula,
        data=model_df,
        groups=model_df["participant_id"]
    ).fit()

    mixed_model_decomposed_results.append({
        "metric": metric,
        "model": "MixedLM decomposed FME: participant mean + within-person",
        "n_samples": len(model_df),
        "n_participants": model_df["participant_id"].nunique(),

        "fme_participant_mean_coef": model.params.get("fme_participant_mean"),
        "fme_participant_mean_p_value": model.pvalues.get("fme_participant_mean"),

        "fme_within_person_coef": model.params.get("fme_within_person", np.nan),
        "fme_within_person_p_value": model.pvalues.get("fme_within_person", np.nan),

        "age_p_value": model.pvalues.get("Age", np.nan),
        "bmi_p_value": model.pvalues.get("BMI", np.nan)
    })

except Exception as e:
    mixed_model_decomposed_results.append({
        "metric": metric,
        "model": "MixedLM decomposed FME failed",
        "n_samples": len(model_df),
        "n_participants": model_df["participant_id"].nunique(),

        "fme_participant_mean_coef": np.nan,
        "fme_participant_mean_p_value": np.nan,

        "fme_within_person_coef": np.nan,
        "fme_within_person_p_value": np.nan,

        "age_p_value": np.nan,
        "bmi_p_value": np.nan,
        "error": str(e)
    })

mixed_model_decomposed_results_df = pd.DataFrame(mixed_model_decomposed_results)

display(mixed_model_decomposed_results_df.round(4))

```

	metric	model	n_samples	n_participants	fme_participant_mean_coef	f
0	shannon_diversity	MixedLM decomposed FME: participant mean + wit...	475	34	-0.0002	
1	richness	MixedLM decomposed FME: participant mean + wit...	475	34	-0.0145	
2	simpson_diversity	MixedLM decomposed FME: participant mean + wit...	475	34	-0.0000	
3	inverse_simpson	MixedLM decomposed FME: participant mean + wit...	475	34	-0.0052	
4	pielou_evenness	MixedLM decomposed FME: participant mean + wit...	475	34	-0.0000	

In []: *#Mixed models using weighted recent 3 day FME exposure*

```

mixed_model_weighted_3day_results = []

for metric in alpha_diversity_metrics:
    model_columns = [
        "participant_id",
        "fme_weighted_3day",
        metric,
        "Age",
        "BMI"
    ] + available_nutrition_covariates

    model_df = df[model_columns].dropna()

    covariates = [
        "fme_weighted_3day",
        "Age",
        "BMI"
    ] + available_nutrition_covariates

    formula = f"{metric} ~ " + " + ".join(covariates)

    try:
        model = smf.mixedlm(
            formula=formula,

```

```

        data=model_df,
        groups=model_df["participant_id"]
    ).fit()

    mixed_model_weighted_3day_results.append({
        "metric": metric,
        "model": "MixedLM weighted 3-day FME",
        "n_samples": len(model_df),
        "n_participants": model_df["participant_id"].nunique(),

        "fme_weighted_3day_coef": model.params.get("fme_weighted_3day", np.nan),
        "fme_weighted_3day_p_value": model.pvalues.get("fme_weighted_3day",

        "age_p_value": model.pvalues.get("Age", np.nan),
        "bmi_p_value": model.pvalues.get("BMI", np.nan)
    })

except Exception as e:
    mixed_model_weighted_3day_results.append({
        "metric": metric,
        "model": "MixedLM weighted 3-day FME failed",
        "n_samples": len(model_df),
        "n_participants": model_df["participant_id"].nunique(),

        "fme_weighted_3day_coef": np.nan,
        "fme_weighted_3day_p_value": np.nan,

        "age_p_value": np.nan,
        "bmi_p_value": np.nan,
        "error": str(e)
    })

mixed_model_weighted_3day_results_df = pd.DataFrame(mixed_model_weighted_3day_re
display(mixed_model_weighted_3day_results_df.round(4))

```

	metric	model	n_samples	n_participants	fme_weighted_3day_coef	fme_w
0	shannon_diversity	MixedLM weighted 3-day FME	475	34	-0.0006	
1	richness	MixedLM weighted 3-day FME	475	34	0.0088	
2	simpson_diversity	MixedLM weighted 3-day FME	475	34	-0.0001	
3	inverse_simpson	MixedLM weighted 3-day FME	475	34	-0.0020	
4	pielou_evenness	MixedLM weighted 3-day FME	475	34	-0.0001	

In []: *#Mixed models using lagged FME exposure features*

```

mixed_model_lagged_results = []

for metric in alpha_diversity_metrics:
    model_columns = [
        "participant_id",
        "fme_lag1",
        "fme_lag2",
        metric,
        "Age",
        "BMI"
    ] + available_nutrition_covariates

    model_df = df[model_columns].dropna()

    covariates = [
        "fme_lag1",
        "fme_lag2",
        "Age",
        "BMI"
    ] + available_nutrition_covariates

    formula = f"{metric} ~ " + " + ".join(covariates)

    try:
        model = smf.mixedlm(
            formula=formula,
            data=model_df,
            groups=model_df["participant_id"]
        ).fit()

```

```
mixed_model_lagged_results.append({
    "metric": metric,
    "model": "MixedLM lagged FME: lag1 + lag2",
    "n_samples": len(model_df),
    "n_participants": model_df["participant_id"].nunique(),

    "fme_lag1_coef": model.params.get("fme_lag1", np.nan),
    "fme_lag1_p_value": model.pvalues.get("fme_lag1", np.nan),

    "fme_lag2_coef": model.params.get("fme_lag2", np.nan),
    "fme_lag2_p_value": model.pvalues.get("fme_lag2", np.nan),

    "age_p_value": model.pvalues.get("Age", np.nan),
    "bmi_p_value": model.pvalues.get("BMI", np.nan)
})

except Exception as e:
    mixed_model_lagged_results.append({
        "metric": metric,
        "model": "MixedLM lagged FME failed",
        "n_samples": len(model_df),
        "n_participants": model_df["participant_id"].nunique(),

        "fme_lag1_coef": np.nan,
        "fme_lag1_p_value": np.nan,

        "fme_lag2_coef": np.nan,
        "fme_lag2_p_value": np.nan,

        "age_p_value": np.nan,
        "bmi_p_value": np.nan,
        "error": str(e)
    })

mixed_model_lagged_results_df = pd.DataFrame(mixed_model_lagged_results)

display(mixed_model_lagged_results_df.round(4))
```


	metric	model	n_samples	n_participants	fme_lag1_coef	fme_lag1_p_value
0	shannon_diversity	MixedLM lagged FME: lag1 + lag2	407	34	0.0000	0.9285
1	richness	MixedLM lagged FME: lag1 + lag2	407	34	0.0078	0.4233
2	simpson_diversity	MixedLM lagged FME: lag1 + lag2	407	34	-0.0000	0.7483
3	inverse_simpson	MixedLM lagged FME: lag1 + lag2	407	34	-0.0010	0.6739
4	pielou_evenness	MixedLM lagged FME: lag1 + lag2	407	34	0.0000	0.9821

```
In [ ]: #Final summary table across all association models

summary_tables = []

#Original correlation results: fme_score_daily only
correlation_summary = extended_correlation_results_df.copy()
correlation_summary["model_type"] = "Correlation"
correlation_summary["fme_feature"] = "fme_score_daily"
correlation_summary["fme_coef"] = correlation_summary["pearson_corr"]
correlation_summary["fme_p_value"] = correlation_summary["pearson_p_value"]
correlation_summary = correlation_summary[
    ["metric", "model_type", "fme_feature", "n_samples", "fme_coef", "fme_p_valu
]
summary_tables.append(correlation_summary)

#Extended correlation results: all FME features
extended_correlation_summary = extended_correlation_results_df.copy()
extended_correlation_summary["model_type"] = "Extended correlation"
extended_correlation_summary["fme_coef"] = extended_correlation_summary["pearson
extended_correlation_summary["fme_p_value"] = extended_correlation_summary["pear
extended_correlation_summary = extended_correlation_summary[
    ["metric", "model_type", "fme_feature", "n_samples", "fme_coef", "fme_p_valu
]
summary_tables.append(extended_correlation_summary)

#OLS adjusted for Age + BMI: fme_score_daily only
```

```

ols_age_bmi_summary = adjusted_model_results_df.copy()
ols_age_bmi_summary["model_type"] = "OLS: Age + BMI"
ols_age_bmi_summary["fme_feature"] = "fme_score_daily"
ols_age_bmi_summary = ols_age_bmi_summary[
    ["metric", "model_type", "fme_feature", "n_samples", "fme_coef", "fme_p_valu
]
summary_tables.append(ols_age_bmi_summary)

#OLS adjusted for Age + BMI + nutrition: fme_score_daily only
ols_nutrition_summary = nutrition_adjusted_model_results_df.copy()
ols_nutrition_summary["model_type"] = "OLS: Age + BMI + nutrition"
ols_nutrition_summary["fme_feature"] = "fme_score_daily"
ols_nutrition_summary = ols_nutrition_summary[
    ["metric", "model_type", "fme_feature", "n_samples", "fme_coef", "fme_p_valu
]
summary_tables.append(ols_nutrition_summary)

#Mixed effects model: fme_score_daily only
mixed_summary = mixed_model_results_df.copy()
mixed_summary["model_type"] = "MixedLM: Age + BMI + nutrition + participant"
mixed_summary["fme_feature"] = "fme_score_daily"
mixed_summary = mixed_summary[
    ["metric", "model_type", "fme_feature", "n_samples", "fme_coef", "fme_p_valu
]
summary_tables.append(mixed_summary)

#MixedLM decomposed model: participant mean and within-person FME
decomposed_summary_rows = []

for _, row in mixed_model_decomposed_results_df.iterrows():
    decomposed_summary_rows.append({
        "metric": row["metric"],
        "model_type": "MixedLM: decomposed FME + Age + BMI + nutrition + partici
        "fme_feature": "fme_participant_mean",
        "n_samples": row["n_samples"],
        "fme_coef": row["fme_participant_mean_coef"],
        "fme_p_value": row["fme_participant_mean_p_value"]
    })

    decomposed_summary_rows.append({
        "metric": row["metric"],
        "model_type": "MixedLM: decomposed FME + Age + BMI + nutrition + partici
        "fme_feature": "fme_within_person",
        "n_samples": row["n_samples"],
        "fme_coef": row["fme_within_person_coef"],
        "fme_p_value": row["fme_within_person_p_value"]
    })

decomposed_summary = pd.DataFrame(decomposed_summary_rows)
summary_tables.append(decomposed_summary)

#MixedLM weighted 3-day FME model
weighted_3day_summary = mixed_model_weighted_3day_results_df.copy()
weighted_3day_summary["model_type"] = "MixedLM: weighted 3-day FME + Age + BMI +
weighted_3day_summary["fme_feature"] = "fme_weighted_3day"
weighted_3day_summary["fme_coef"] = weighted_3day_summary["fme_weighted_3day_coe
weighted_3day_summary["fme_p_value"] = weighted_3day_summary["fme_weighted_3day_
weighted_3day_summary = weighted_3day_summary[
    ["metric", "model_type", "fme_feature", "n_samples", "fme_coef", "fme_p_valu
]

```

```

summary_tables.append(weighted_3day_summary)

#MixedLM lagged FME model: Lag1 and Lag2
lagged_summary_rows = []

for _, row in mixed_model_lagged_results_df.iterrows():
    lagged_summary_rows.append({
        "metric": row["metric"],
        "model_type": "MixedLM: lagged FME + Age + BMI + nutrition + participant",
        "fme_feature": "fme_lag1",
        "n_samples": row["n_samples"],
        "fme_coef": row["fme_lag1_coef"],
        "fme_p_value": row["fme_lag1_p_value"]
    })

    lagged_summary_rows.append({
        "metric": row["metric"],
        "model_type": "MixedLM: lagged FME + Age + BMI + nutrition + participant",
        "fme_feature": "fme_lag2",
        "n_samples": row["n_samples"],
        "fme_coef": row["fme_lag2_coef"],
        "fme_p_value": row["fme_lag2_p_value"]
    })

lagged_summary = pd.DataFrame(lagged_summary_rows)
summary_tables.append(lagged_summary)

#Combine all summaries
final_association_summary = pd.concat(
    summary_tables,
    axis=0,
    ignore_index=True
)

#Add significance label
final_association_summary["significance"] = np.where(
    final_association_summary["fme_p_value"] < 0.05,
    "significant",
    np.where(
        final_association_summary["fme_p_value"] < 0.10,
        "trend",
        "not significant"
    )
)

#Sort for readability
final_association_summary = final_association_summary.sort_values(
    by=["metric", "model_type", "fme_feature"]
).reset_index(drop=True)

display(final_association_summary.round(4))

```

	metric	model_type	fme_feature	n_samples	fme_coef	fme_p_value
0	inverse_simpson	Correlation	fme_score_daily	475	-0.0485	0.2917
1	inverse_simpson	Correlation	fme_score_daily	475	-0.0650	0.1573
2	inverse_simpson	Correlation	fme_score_daily	475	-0.0056	0.9027
3	inverse_simpson	Correlation	fme_score_daily	441	-0.0641	0.1788
4	inverse_simpson	Correlation	fme_score_daily	407	-0.0672	0.1758
...	...	...	...	...	...	...
95	simpson_diversity	MixedLM: lagged FME + Age + BMI + nutrition + ...	fme_lag1	407	-0.0000	0.7483
96	simpson_diversity	MixedLM: lagged FME + Age + BMI + nutrition + ...	fme_lag2	407	-0.0000	0.3571
97	simpson_diversity	MixedLM: weighted 3- day FME + Age + BMI + nutr...	fme_weighted_3day	475	-0.0001	0.1904
98	simpson_diversity	OLS: Age + BMI	fme_score_daily	475	-0.0000	0.4961
99	simpson_diversity	OLS: Age + BMI + nutrition	fme_score_daily	475	-0.0000	0.7941

100 rows × 7 columns

Summary of Association Testing Results

In this analysis, we tested associations between FME and gut microbiome alpha diversity using several FME representations and multiple statistical models.

The alpha diversity outcomes included:

- Shannon diversity
- Richness

- Simpson diversity
- Inverse Simpson
- Pielou evenness

The FME predictors included:

- `fme_score_daily` — the original daily weighted FME score
- `fme_participant_mean` — the participant-level mean FME score
- `fme_within_person` — deviation of each daily FME score from that participant's mean
- `fme_lag1` — previous FME score for the same participant
- `fme_lag2` — FME score two previous observations back for the same participant
- `fme_weighted_3day` — weighted recent FME exposure based on the current score, lag 1, and lag 2

First, simple Pearson and Spearman correlations were calculated between the extended FME features and each alpha diversity metric. These unadjusted correlations did not show strong or consistent significant associations.

Next, linear regression models adjusted for Age and BMI were fitted for each diversity metric using the daily FME score. In these models, FME was not significantly associated with any of the alpha diversity metrics.

We then extended the regression models by additionally adjusting for nutritional covariates, including total calorie intake, fiber, protein, fat, and carbohydrates. In this model set, richness showed a near-significant trend with daily FME, but did not pass the conventional significance threshold of 0.05.

Because the dataset contains repeated fecal samples from the same participants, we fitted mixed effects models with participant-specific random intercepts. These models adjusted for Age, BMI, nutritional covariates, and repeated sampling within participants.

In the original mixed effects model using `fme_score_daily`, Pielou evenness showed a statistically significant association with FME. Shannon diversity showed a near-significant trend. Both associations had negative coefficients, suggesting that higher FME scores may be associated with lower microbial evenness and lower Shannon diversity.

We then fitted decomposed mixed effects models separating between-person and within-person FME effects. In these models, the within-person FME component was associated with Pielou evenness and showed a near-significant trend with Shannon diversity. The participant-level mean FME component was not significant. This suggests that the observed association may be driven more by day-to-day deviations within the same participant than by stable differences between participants.

We also tested short-term exposure models using `fme_weighted_3day`, which combines the current FME score with the two previous observations. In this model, Pielou evenness was significantly associated with weighted recent FME, and Shannon diversity showed a near-significant trend.

Finally, we tested lagged models using `fme_lag1` and `fme_lag2` separately. These lagged predictors did not show significant associations when included as separate predictors, suggesting that the combined recent exposure score may capture the short-term signal better than individual lag terms.

Overall, the results suggest that the association between FME and gut microbiome alpha diversity is not strongly reflected in simple unadjusted correlations or standard OLS models. However, after accounting for repeated samples per participant using mixed effects models, FME appears to be associated specifically with microbial evenness, and possibly Shannon diversity. The strongest and most consistent signals appear in the within-person FME model and the weighted recent FME exposure model, rather than in richness or dominance-based diversity metrics.

```
In [ ]: #Save final association summary table

final_association_summary.to_csv(
    DATA_DIR / "association_testing_extended_summary.csv",
    index=False
)

print("Saved association_testing_extended_summary.csv")
```

Saved association_testing_extended_summary.csv

Notebook: FME_Analysis_v2.ipynb

Food Microbiome Exposure (FME) Analysis

Research Question

How does exposure to food-associated microbes influence the composition and stability of the human gut microbiome?

What this notebook does

1. **Data Transfer** — Load all preprocessed datasets from both sources
2. **Data Mapping** — Map dietary food items to cFMD food categories
3. **FME Quantification** — Calculate a daily Food Microbiome Exposure score per fecal sample
4. **Statistical Dataset** — Assemble one clean table ready for statistical modelling

Key structural note

The Johnson et al. 2019 study has **34 participants**, each contributing multiple fecal samples across up to 17 study days. All FME scores are

computed at the **fecal sample level** (one score per sample = one dietary day per person). Participant-level summaries are derived by aggregating across samples — **not** by treating samples as independent observations. The unit of analysis for statistical models is the **fecal sample**, with participant included as a random effect.

Section 2 – Data Transfer: Load All Datasets

Source 1 — Johnson et al. 2019 (gut microbiome + diet)

34 participants · up to 17 fecal samples each · 566 samples total

Source 2 — Carlino et al. 2024 — cFMD (food microbiome)

3,393 food samples · 5,057 microbial taxa

```
In [ ]: # Source 1: Gut microbiome matrices (taxa × fecal samples)
gut_microbiome_abundance = pd.read_csv(DATA_DIR / "microbiome_filtered.csv", in

# Source 1: Fecal sample metadata
# index = fecal sample ID (MCT.f.XXXX)
# columns: UserName (= participant), StudyDayNo, Gender, Age, BMI
fecal_sample_metadata = pd.read_csv(DATA_DIR / "sample_meta.csv", in

#Source 1: Dietary consumption matrix (food items × fecal samples)
# rows = food items, columns = fecal sample IDs (MCT.f.XXXX)
# values = consumption amount of that food on that day
dietary_consumption_matrix= pd.read_csv(DATA_DIR / "diet_raw.csv")

# Source 1: Daily nutrition totals
# index = fecal sample ID
daily_nutrition_totals = pd.read_csv(DATA_DIR / "nutrition_totals.csv", in

# Source 1: Additional dietary features
daily_fiber_scores = pd.read_csv(DATA_DIR / "diet_fiber.csv")
daily_decay_scores = pd.read_csv(DATA_DIR / "diet_decay.csv")
participant_fiber_avg = pd.read_csv(DATA_DIR / "fiber_avg.csv")

# Source 2: cFMD food microbiome abundance (food taxa × food samples)
# columns: 'StudyName__ShortSampleID' format
cfmd_food_abundance = pd.read_csv(DATA_DIR / "combined_datasets.csv", in
cfmd_food_abundance = cfmd_food_abundance.apply(pd.to_numeric, errors="coe

# Source 2: cFMD sample metadata –
# columns: sample_id, category, fermented/non-fermented, country
cfmd_food_metadata = pd.read_csv(DATA_DIR / "cfmd_metadata.csv")

# Verify participant count
n_participants = fecal_sample_metadata["UserName"].nunique()
n_fecal_samples= len(fecal_sample_metadata)

print(f"Participants      : {n_participants} (expected: 34)")
print(f"Fecal samples total  : {n_fecal_samples}")
```

```

print(f"Avg samples/person      : {n_fecal_samples / n_participants:.1f}")
print()
print("Dataset sizes:")
for name, df in [
    ("gut_microbiome_abundance", gut_microbiome_abundance),
    ("fecal_sample_metadata", fecal_sample_metadata),
    ("dietary_consumption_matrix", dietary_consumption_matrix),
    ("daily_nutrition_totals", daily_nutrition_totals),
    ("cfmd_food_abundance", cfmd_food_abundance),
    ("cfmd_food_metadata", cfmd_food_metadata),
]:
    print(f" {name:<32} {str(df.shape)}")

```

```

Participants      : 34 (expected: 34)
Fecal samples total : 566
Avg samples/person : 16.6

```

Dataset sizes:

```

gut_microbiome_abundance      (223, 483)
fecal_sample_metadata          (566, 28)
dietary_consumption_matrix     (1452, 568)
daily_nutrition_totals         (580, 106)
cfmd_food_abundance           (5057, 3393)
cfmd_food_metadata             (3444, 32)

```

Section 3 – Define the Analytical Sample Set

Only fecal samples that have **both** a gut microbiome measurement **and** a dietary record form the analytical core. We identify these 475 samples here.

```

In [ ]: # Identify fecal sample IDs present in both gut microbiome and diet data
FECAL_SAMPLE_PREFIX = "MCT.f."

gut_microbiome_sample_ids = set(gut_microbiome_abundance.columns)
diet_record_sample_ids = set(
    c for c in dietary_consumption_matrix.columns
    if c.startswith(FECAL_SAMPLE_PREFIX)
)

# Core analytical set: samples with BOTH gut microbiome AND diet record
analytical_sample_ids = sorted(gut_microbiome_sample_ids & diet_record_sample_ids)

print(f"Gut microbiome samples      : {len(gut_microbiome_sample_ids)}")
print(f"Diet record samples          : {len(diet_record_sample_ids)}")
print(f"Analytical samples (gutndiet)   : {len(analytical_sample_ids)}")
print()

# Participants represented in the analytical set
analytical_participants = (
    fecal_sample_metadata
    .loc[fecal_sample_metadata.index.isin(analytical_sample_ids), "UserName"]
    .nunique()
)
print(f"Participants represented      : {analytical_participants} / {n_participants}")

# Samples per participant in the analytical set

```



```

samples_per_participant = (
    fecal_sample_metadata
    .loc[fecal_sample_metadata.index.isin(analytical_sample_ids)]
    .groupby("UserName")
    .size()
    .rename("n_analytical_samples")
)
print(f"Avg analytical samples/person : {samples_per_participant.mean():.1f}")
print(f"Min / Max : {samples_per_participant.min()} / {samples_per_participant.max()}")

```

```

Gut microbiome samples      : 483
Diet record samples        : 566
Analytical samples (gutndiet) : 475

```

```

Participants represented    : 34 / 34
Avg analytical samples/person : 14.0
Min / Max                  : 6 / 17

```

Section 4 – FME Weights from cFMD

Goal: Derive a microbial richness weight for each cFMD food category.

Why richness, not total abundance? The `combined_datasets` matrix stores relative abundance — each sample sums to 100 regardless of food type. Total abundance is therefore uninformative as a weight. **Microbial richness** (= number of distinct taxa detected per food sample) reflects how microbiome-diverse each food category truly is.

Weight formula:

$$w_c = \frac{\text{mean_richness}_c - \min}{\max - \min} \in [0, 1]$$

Weights are derived directly from the **cFMD food abundance matrix** — not assumed or hand-coded.

```

In [ ]: # Step 1: Map cFMD column names → food category + fermentation status
# combined_datasets columns: 'StudyName__ShortSampleID'
# cfmd_food_metadata sample_id: 'ShortSampleID'

def extract_cfmd_short_id(column_name):
    """Strip the study name prefix from a combined_datasets column name."""
    return column_name.split("__")[-1] if "__" in column_name else column_name

# Remove the one duplicate sample_id (VB15-29) before indexing
cfmd_food_metadata_dedup = cfmd_food_metadata.drop_duplicates(
    subset="sample_id", keep="first"
)
print(f"cFMD metadata after dedup: {len(cfmd_food_metadata_dedup)} rows "
      f"(removed {len(cfmd_food_metadata) - len(cfmd_food_metadata_dedup)} dupli")

cfmd_id_to_meta = (
    cfmd_food_metadata_dedup
    .set_index("sample_id")[["category", "fermented/non-fermented"]]
    .to_dict(orient="index")
)

```

```

# Map each column in cfmd_food_abundance to its category and fermentation status
cfmd_column_to_category = {}
cfmd_column_to_fermented = {}
for col in cfmd_food_abundance.columns:
    short_id = extract_cfmd_short_id(col)
    if short_id in cfmd_id_to_meta:
        cfmd_column_to_category[col] = cfmd_id_to_meta[short_id]["category"]
        cfmd_column_to_fermented[col] = cfmd_id_to_meta[short_id]["fermented/non

print(f"cFMD columns mapped: {len(cfmd_column_to_category)} / {cfmd_food_abundan

```

cFMD metadata after dedup: 3443 rows (removed 1 duplicate)

cFMD columns mapped: 3393 / 3393

```

In [ ]: # Step 2: Compute microbial richness per food sample
# Richness = number of non-zero taxa per food sample
cfmd_taxa_richness_per_sample = (cfmd_food_abundance > 0).sum(axis=0)

# Step 3: Mean richness per cFMD food category
category_richness_values = {}
for col, category in cfmd_column_to_category.items():
    if col in cfmd_taxa_richness_per_sample.index:
        category_richness_values.setdefault(category, []).append(
            cfmd_taxa_richness_per_sample[col]
        )

cfmd_category_mean_richness = pd.Series(
    {cat: np.mean(vals) for cat, vals in category_richness_values.items()},
    name="mean_taxa_richness"
).sort_values(ascending=False)

# Step 4: Normalize weights to [0, 1]
richness_min = cfmd_category_mean_richness.min()
richness_max = cfmd_category_mean_richness.max()
cfmd_category_weight_normalized = (
    (cfmd_category_mean_richness - richness_min) /
    (richness_max - richness_min)
).rename("fme_weight")

cfmd_weights_table = pd.concat(
    [cfmd_category_mean_richness, cfmd_category_weight_normalized], axis=1
)

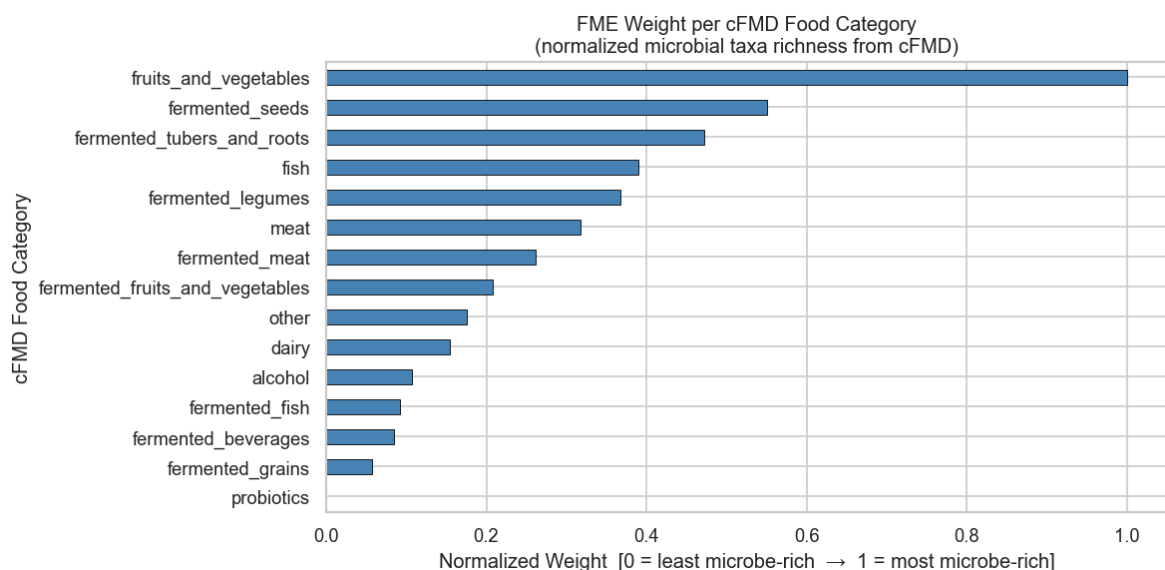
print("FME weights per cFMD food category (from cFMD microbial richness):")
display(cfmd_weights_table.round(3))

```

FME weights per cFMD food category (from cFMD microbial richness):

	mean_taxa_richness	fme_weight
fruits_and_vegetables	120.1250	1.0000
fermented_seeds	69.8000	0.5510
fermented_tubers_and_roots	61.0420	0.4720
fish	51.7950	0.3900
fermented_legumes	49.2630	0.3670
meat	43.6490	0.3170
fermented_meat	37.4110	0.2610
fermented_fruits_and_vegetables	31.4810	0.2080
other	27.8750	0.1760
dairy	25.3960	0.1540
alcohol	20.0850	0.1070
fermented_fish	18.4000	0.0920
fermented_beverages	17.6160	0.0850
fermented_grains	14.5120	0.0570
probiotics	8.1430	0.0000

```
In [ ]: # Visualize FME weights
plt.figure(figsize=(10, 5))
cfmd_category_weight_normalized.sort_values().plot(
    kind="barh", color="steelblue", edgecolor="black", linewidth=0.5
)
plt.title("FME Weight per cFMD Food Category\n(normalized microbial taxa richness from cFMD)")
plt.xlabel("Normalized Weight [0 = least microbe-rich → 1 = most microbe-rich]")
plt.ylabel("cFMD Food Category")
plt.tight_layout()
plt.show()
```



Section 5 – Data Mapping: Diet Items → cFMD Food Categories

Goal: Map each food item in the Johnson et al. dietary records to a cFMD food category, creating a unified representation of food-derived microbial exposure.

Method: Extract the L1 (top-level) taxonomy label from the `taxonomy` column of `diet_raw` (e.g. `L1_Milk_and_Milk_Products` → `dairy`) and map it to the matching cFMD category.

```
In [ ]: # Extract L1 food category from taxonomy string
def extract_l1_food_category(taxonomy_string):
    """
    Extract the top-level food category (L1) from the hierarchical taxonomy string
    Example: 'L1_Milk_and_Milk_Products;L2_...' → 'milk and milk products'
    """
    if pd.isna(taxonomy_string):
        return "unknown"
    l1_segment = str(taxonomy_string).split(";")[0]
    return l1_segment.replace("L1_", "").replace("_", " ").strip().lower()

dietary_consumption_matrix["diet_l1_category"] = (
    dietary_consumption_matrix["taxonomy"]
    .apply(extract_l1_food_category)
)

print("Unique L1 diet categories found:")
for cat, count in dietary_consumption_matrix["diet_l1_category"].value_counts():
    print(f" {cat:<45} {count:>5} food items")
```

Unique L1 diet categories found:

grain product	407 food items
vegetables	315 food items
meat poultry fish and mixtures	240 food items
sugars sweets and beverages	153 food items
milk and milk products	112 food items
dry beans peas other legumes nuts and seeds	84 food items
fruits	74 food items
fats oils and salad dressings	36 food items
eggs	31 food items

```
In [ ]: #Mapping table: diet L1 category → cFMD category
DIET_L1_TO_CFMD_CATEGORY = {
    # Dairy
    "milk and milk products" : "dairy",
    "dairy" : "dairy",
    "cheese" : "dairy",
    "yogurt" : "dairy",
    # Meat
    "meat poultry and game" : "meat",
    "meat" : "meat",
    "poultry" : "meat",
    # Fish
    "fish and shellfish" : "fish",
    "fish" : "fish",
    "seafood" : "fish",
    # Fruits and vegetables
```

```

"vegetables and vegetable products" : "fruits_and_vegetables",
"fruit"                             : "fruits_and_vegetables",
"fruits"                           : "fruits_and_vegetables",
"vegetables"                       : "fruits_and_vegetables",
# Fermented grains
"grain products"                   : "fermented_grains",
"grains"                           : "fermented_grains",
"bread"                            : "fermented_grains",
"cereals and cereal products"      : "fermented_grains",
# Alcohol / fermented beverages
"alcoholic beverages"              : "alcohol",
"alcohol"                          : "alcohol",
"beverages"                        : "fermented_beverages",
"nonalcoholic beverages"           : "fermented_beverages",
# Legumes
"legumes and legume products"      : "fermented_legumes",
"legumes"                          : "fermented_legumes",
"beans"                            : "fermented_legumes",
# Seeds / nuts
"nuts and seeds"                   : "fermented_seeds",
"seeds"                            : "fermented_seeds",
# Other / unmappable
"fats and oils"                    : "other",
"sugars and sweets"                : "other",
"soups sauces and gravies"         : "other",
"mixed dishes"                     : "other",
"snacks"                           : "other",
"eggs"                             : "other",
"spices and herbs"                 : "other",
"unknown"                          : "unmapped",
}

dietary_consumption_matrix["cfmd_category"] = (
    dietary_consumption_matrix["diet_l1_category"]
    .map(DIET_L1_TO_CFMD_CATEGORY)
    .fillna("unmapped")
)

n_mapped    = (dietary_consumption_matrix["cfmd_category"] != "unmapped").sum()
n_unmapped  = (dietary_consumption_matrix["cfmd_category"] == "unmapped").sum()
print(f"Food items mapped to a cFMD category : {n_mapped} / {len(dietary_consump
print(f"Food items unmapped                      : {n_unmapped}")

```

Food items mapped to a cFMD category : 532 / 1452

Food items unmapped : 920

```

In [ ]: # Attach FME weight to each food item
dietary_consumption_matrix["fme_weight"] = (
    dietary_consumption_matrix["cfmd_category"]
    .map(cfmd_category_weight_normalized.to_dict())
    .fillna(0.0) # unmapped items get weight = 0
)

mapping_summary = (
    dietary_consumption_matrix
    .groupby(["diet_l1_category", "cfmd_category", "fme_weight"])
    .size()
    .reset_index(name="n_food_items")
    .sort_values("n_food_items", ascending=False)
)

```

```
print("Mapping summary (diet L1 → cFMD category → weight):")
display(mapping_summary)
```

Mapping summary (diet L1 → cFMD category → weight):

	diet_l1_category	cfmd_category	fme_weight	n_food_items
4	grain product	unmapped	0.0000	407
8	vegetables	fruits_and_vegetables	1.0000	315
5	meat poultry fish and mixtures	unmapped	0.0000	240
7	sugars sweets and beverages	unmapped	0.0000	153
6	milk and milk products	dairy	0.1541	112
0	dry beans peas other legumes nuts and seeds	unmapped	0.0000	84
3	fruits	fruits_and_vegetables	1.0000	74
2	fats oils and salad dressings	unmapped	0.0000	36
1	eggs	other	0.1762	31

Section 6 – FME Score Calculation

Formula:

$$\text{FME}_{\text{sample}} = \sum_{\text{food items}} \text{consumption}_{\text{food, sample}} \times w_{\text{cfmd_category}(\text{food})}$$

Where:

- $\text{consumption}_{\text{food, sample}}$ = amount consumed (from `diet_raw`, grams or servings)
- w = normalized cFMD microbial richness weight for the food's category

Important: Each `sample` corresponds to **one dietary day for one participant**. The result is a score per fecal sample — not per participant. Participant-level scores are derived later by averaging across their samples.

```
In [ ]: # Subset diet matrix to analytical samples only
diet_abundance_matrix = (
    dietary_consumption_matrix[analytical_sample_ids]
    .apply(pd.to_numeric, errors="coerce")
    .fillna(0.0)
)

fme_weight_vector = dietary_consumption_matrix["fme_weight"].values # shape: (n

# Compute FME score: weighted sum of food consumption per sample
# Result: one FME score per fecal sample (= one dietary day per participant)
fme_score_per_sample = (
    diet_abundance_matrix
    .multiply(fme_weight_vector, axis=0)
    .sum(axis=0)
```

```

        .rename("fme_score_daily")
    )
    fme_score_per_sample.index.name = "fecal_sample_id"

    print(f"FME scores computed for : {len(fme_score_per_sample)} fecal samples")
    print(f"Score range           : {fme_score_per_sample.min():.4f} - {fme_score_per_sample.max():.4f}")
    print(f"Score mean ± std       : {fme_score_per_sample.mean():.4f} ± {fme_score_per_sample.std():.4f}")
    print(f"Non-zero scores        : {(fme_score_per_sample > 0).sum()}")
    print()
    display(fme_score_per_sample.reset_index().head(8))

```

FME scores computed for : 475 fecal samples
 Score range : 0.0000 - 331.2240
 Score mean ± std : 73.3252 ± 58.3071
 Non-zero scores : 437

	fecal_sample_id	fme_score_daily
0	MCT.f.0002	67.1961
1	MCT.f.0003	81.2412
2	MCT.f.0004	126.7711
3	MCT.f.0005	31.4846
4	MCT.f.0006	61.2266
5	MCT.f.0007	62.0231
6	MCT.f.0008	70.9874
7	MCT.f.0009	54.0086

```

In [ ]: # Participant-Level FME summary (aggregate across samples)
        # Note: samples are NOT independent – they come from the same 34 participants.
        # Participant-Level values are used ONLY for descriptive summaries,
        # NOT as independent data points in statistical models.

        fme_with_participant = pd.DataFrame({
            "fecal_sample_id" : fme_score_per_sample.index,
            "fme_score_daily" : fme_score_per_sample.values,
        })
        fme_with_participant["participant_id"] = (
            fecal_sample_metadata
            .loc[fme_with_participant["fecal_sample_id"], "UserName"]
            .values
        )
        fme_with_participant["study_day"] = (
            fecal_sample_metadata
            .loc[fme_with_participant["fecal_sample_id"], "StudyDayNo"]
            .values
        )

        # Participant-Level summary (descriptive only)
        participant_fme_summary = (
            fme_with_participant
            .groupby("participant_id")["fme_score_daily"]
            .agg(
                fme_mean = "mean",
                fme_std   = "std",
            )

```

```

        fme_min      = "min",
        fme_max      = "max",
        n_diet_days  = "count"
    )
    .reset_index()
)

print("Participant-level FME summary (descriptive – 34 participants):")
display(participant_fme_summary.sort_values("fme_mean", ascending=False).head(10)

```

Participant-level FME summary (descriptive – 34 participants):

	participant_id	fme_mean	fme_std	fme_min	fme_max	n_diet_days
1	MCTs03	147.8899	45.7548	68.0651	243.3253	12
5	MCTs07	143.4859	48.3133	86.0332	243.8429	16
2	MCTs04	143.1190	81.0506	36.6199	331.2240	15
6	MCTs08	140.2178	33.6074	84.4009	220.3516	17
14	MCTs16	130.9598	45.2371	69.7540	221.0863	14
27	MCTs31	128.7335	37.7598	83.9027	208.7920	15
23	MCTs26	122.0334	71.3654	24.5678	315.7055	17
16	MCTs19	99.4541	40.7179	14.6490	166.4480	15
17	MCTs20	88.6724	51.9185	24.4219	178.8966	12
24	MCTs27	85.2074	47.0410	5.0752	165.2448	14

```

In [ ]: # Visualize FME scores
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# Distribution of daily FME scores across all fecal samples
axes[0].hist(fme_score_per_sample.values, bins=30,
             color="steelblue", edgecolor="white")
axes[0].axvline(fme_score_per_sample.mean(), color="red", linestyle="--",
               label=f"Mean = {fme_score_per_sample.mean():.2f}")
axes[0].set_title("Daily FME Score Distribution\n(475 fecal samples)")
axes[0].set_xlabel("FME Score")
axes[0].set_ylabel("Number of Samples")
axes[0].legend()

# Mean FME per participant (sorted)
sorted_participants = participant_fme_summary.sort_values("fme_mean", ascending=
axes[1].bar(range(len(sorted_participants)), sorted_participants["fme_mean"],
           color="darkorange", edgecolor="none", alpha=0.85)
axes[1].set_title(f"Mean FME Score per Participant\n({len(sorted_participants)})")
axes[1].set_xlabel("Participant (sorted by mean FME)")
axes[1].set_ylabel("Mean Daily FME Score")
axes[1].set_xticks([])

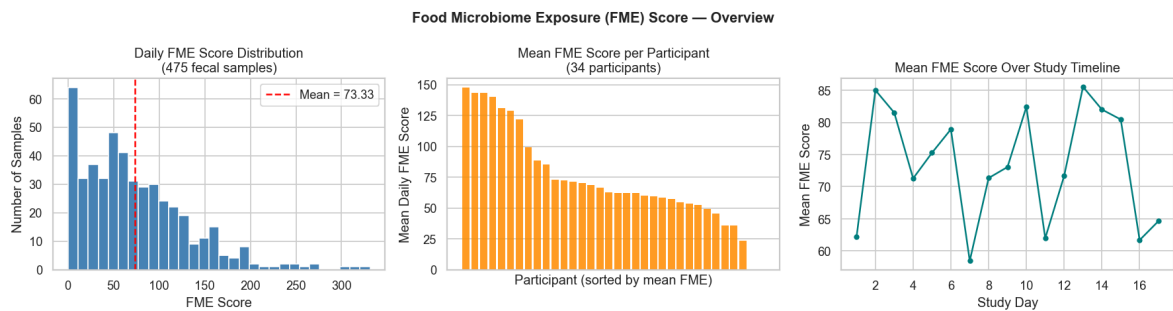
# FME over study timeline (mean per study day)
fme_by_day = fme_with_participant.groupby("study_day")["fme_score_daily"].mean()
axes[2].plot(fme_by_day.index, fme_by_day.values,
            marker="o", color="teal", linewidth=1.5, markersize=4)
axes[2].set_title("Mean FME Score Over Study Timeline")
axes[2].set_xlabel("Study Day")

```



```
axes[2].set_ylabel("Mean FME Score")

plt.suptitle("Food Microbiome Exposure (FME) Score – Overview", fontsize=13, fontweight="bold")
plt.tight_layout()
plt.show()
```



Section 7 – Gut Microbiome Alpha Diversity per Fecal Sample

Compute **Shannon entropy** for each fecal sample as a measure of gut microbial richness and evenness. This is the primary outcome variable (composition proxy) at the sample level.

```
In [ ]: # Shannon entropy per fecal sample
def compute_shannon_entropy(abundance_column):
    """Shannon entropy H = -Σ p_i * log(p_i) for non-zero taxa."""
    positive_values = abundance_column[abundance_column > 0]
    if positive_values.sum() == 0:
        return np.nan
    proportions = positive_values / positive_values.sum()
    return float(-np.sum(proportions * np.log(proportions)))

gut_microbiome_analytical = gut_microbiome_abundance[analytical_sample_ids]
shannon_diversity_per_sample = (
    gut_microbiome_analytical
    .apply(compute_shannon_entropy, axis=0)
    .rename("shannon_diversity")
)
shannon_diversity_per_sample.index.name = "fecal_sample_id"

print(f"Shannon diversity computed for : {shannon_diversity_per_sample.notna().sum()} samples")
print(f"Range : {shannon_diversity_per_sample.min():.3f} - {shannon_diversity_per_sample.max():.3f}")
print(f"Mean : {shannon_diversity_per_sample.mean():.3f}")
```

```
Shannon diversity computed for : 475 samples
Range : 1.572 - 3.731
Mean : 2.865
```

Section 8 – Gut Microbiome Stability per Participant

Stability = how consistent is a participant's gut microbiome across their sampling days?

We use the **coefficient of variation (CV)** of Shannon diversity across a participant's samples:

$$CV_{participant} = \frac{\sigma(\text{Shannon})}{\mu(\text{Shannon})}$$

Lower CV = more stable microbiome.

This is computed at the **participant level** (34 values) and joined back to the sample-level table as a per-participant attribute.

```
In [ ]: # Map each fecal sample to its participant
sample_to_participant = (
    fecal_sample_metadata
    .loc[analytical_sample_ids, "UserName"]
    .rename("participant_id")
)

shannon_with_participant = pd.DataFrame({
    "fecal_sample_id" : shannon_diversity_per_sample.index,
    "shannon_diversity": shannon_diversity_per_sample.values,
}).set_index("fecal_sample_id")
shannon_with_participant["participant_id"] = sample_to_participant

# Participant-level stability (CV of Shannon across their samples)
participant_gut_stability = (
    shannon_with_participant
    .groupby("participant_id")["shannon_diversity"]
    .agg(
        participant_shannon_mean = "mean",
        participant_shannon_std  = "std",
        participant_n_samples    = "count"
    )
    .assign(
        participant_shannon_cv = lambda df:
            df["participant_shannon_std"] / df["participant_shannon_mean"]
    )
    .reset_index()
)

print(f"Gut stability computed for {len(participant_gut_stability)} participants")
print(f"Shannon CV range : {participant_gut_stability['participant_shannon_cv'].
      f"{participant_gut_stability['participant_shannon_cv'].max():.4f}")
display(participant_gut_stability.sort_values("participant_shannon_cv").head(8))
```

Gut stability computed for 34 participants
 Shannon CV range : 0.0202 - 0.1799

	participant_id	participant_shannon_mean	participant_shannon_std	participant_n_samp
5	MCTs07	3.0731	0.0621	
17	MCTs20	3.2696	0.0736	
29	MCTs33	3.0863	0.0876	
11	MCTs13	3.1848	0.0942	
19	MCTs22	2.9985	0.0912	
7	MCTs09	3.0362	0.0948	
30	MCTs34	2.9806	0.1022	
8	MCTs10	3.0279	0.1045	

Section 9 – Assemble the Statistical Analysis Dataset

Merge all variables into one clean table at the **fecal sample level**. Each row = one fecal sample = one dietary day for one participant.

Column group	Columns	Role in statistical model
Identity	fecal_sample_id , participant_id , study_day	Grouping / random effect
Demographics	Age , BMI , Gender	Fixed-effect covariates
FME score	fme_score_daily	Main predictor
Gut diversity	shannon_diversity	Main outcome
Gut stability	participant_shannon_cv	Secondary outcome
Nutrition	KCAL , FIBE , PROT , TFAT , CARB	Dietary confounders

```
In [ ]: # Step 1: Base table - fecal sample IDs
statistical_analysis_dataset = pd.DataFrame(
    index=pd.Index(analytical_sample_ids, name="fecal_sample_id")
)

# Step 2: Participant identity and demographics
participant_meta_cols = ["UserName", "StudyDayNo", "Gender", "Age", "BMI",
                        "Weight", "Supplement", "Medications"]
participant_meta_cols = [c for c in participant_meta_cols
                        if c in fecal_sample_metadata.columns]

statistical_analysis_dataset = statistical_analysis_dataset.join(
    fecal_sample_metadata[participant_meta_cols]
    .rename(columns={"UserName": "participant_id", "StudyDayNo": "study_day"}),
    how="left"
```

```

)

# Step 3: Daily FME score
statistical_analysis_dataset = statistical_analysis_dataset.join(
    fme_score_per_sample.rename("fme_score_daily"),
    how="left"
)

# Step 4: Gut alpha diversity (Shannon)
statistical_analysis_dataset = statistical_analysis_dataset.join(
    shannon_diversity_per_sample.rename("shannon_diversity"),
    how="left"
)

# Step 5: Participant-level gut stability (CV)
statistical_analysis_dataset = statistical_analysis_dataset.join(
    participant_gut_stability.set_index("participant_id")[
        ["participant_shannon_cv", "participant_shannon_mean", "participant_n_sa
    ],
    on="participant_id",
    how="left"
)

# Step 6: Daily nutrition totals
nutrition_cols_keep = ["KCAL", "PROT", "TFAT", "CARB", "FIBE",
                        "SUGR", "SODI", "D_TOTAL", "D_YOGURT", "D_CHEESE",
                        "PF_MEAT", "PF_SEAFD_HI", "G_WHOLE"]
nutrition_cols_keep = [c for c in nutrition_cols_keep
                        if c in daily_nutrition_totals.columns]

statistical_analysis_dataset = statistical_analysis_dataset.join(
    daily_nutrition_totals[nutrition_cols_keep],
    how="left"
)

print(f"Statistical dataset shape : {statistical_analysis_dataset.shape}")
print(f"Fecal samples (rows)      : {statistical_analysis_dataset.shape[0]}")
print(f"Variables (columns)       : {statistical_analysis_dataset.shape[1]}")
print(f"Participants represented   : {statistical_analysis_dataset['participant_
print(f"Complete rows (no NaN)   : {statistical_analysis_dataset.dropna().shap

```

```

Statistical dataset shape : (475, 26)
Fecal samples (rows)      : 475
Variables (columns)       : 26
Participants represented   : 34
Complete rows (no NaN)    : 220

```

```

In [ ]: # Missing value report
missing_report = (
    statistical_analysis_dataset.isnull().sum()
    .reset_index()
    .rename(columns={"index": "variable", 0: "n_missing"})
)
missing_report["pct_missing"] = (
    missing_report["n_missing"] / len(statistical_analysis_dataset) * 100
).round(1)
missing_report = missing_report[missing_report["n_missing"] > 0].sort_values("n_

if missing_report.empty:
    print("No missing values in the statistical dataset.")

```

```
else:
    print("Missing values:")
    display(missing_report)
```

Missing values:

	variable	n_missing	pct_missing
7	Medications	255	53.7000

```
In [ ]: # Descriptive statistics
key_vars = ["fme_score_daily", "shannon_diversity",
            "participant_shannon_cv", "Age", "BMI", "KCAL", "FIBE"]
key_vars = [v for v in key_vars if v in statistical_analysis_dataset.columns]

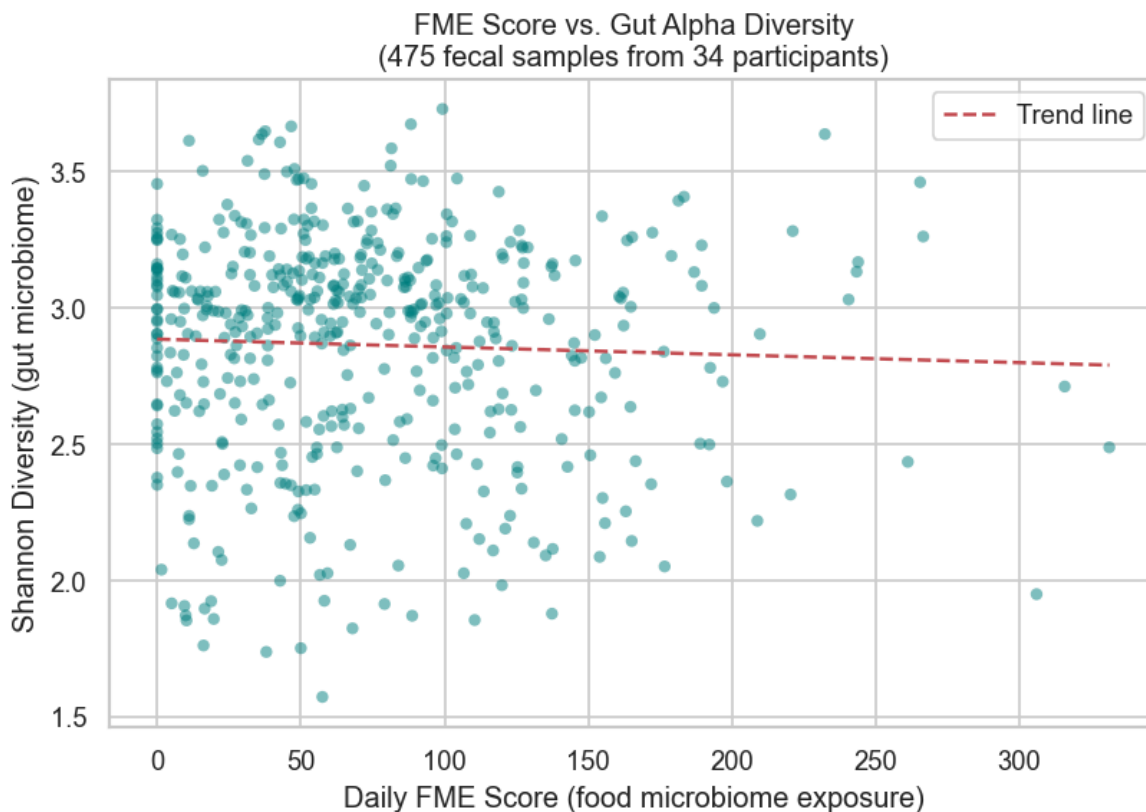
print("Descriptive statistics – key variables:")
display(statistical_analysis_dataset[key_vars].describe().T.round(3))
```

Descriptive statistics – key variables:

	count	mean	std	min	25%	50%	
fme_score_daily	475.0000	73.3250	58.3070	0.0000	29.3220	61.7780	10
shannon_diversity	475.0000	2.8650	0.4130	1.5720	2.6110	2.9640	
participant_shannon_cv	475.0000	0.0630	0.0380	0.0200	0.0360	0.0490	
Age	475.0000	31.1520	10.1560	19.7000	23.9000	28.5000	
BMI	475.0000	23.1750	3.2920	17.0000	21.1000	22.9000	
KCAL	475.0000	2110.5630	735.4510	722.5520	1599.1800	2028.1680	24
FIBE	475.0000	22.3370	11.6000	3.6180	14.8500	19.6940	

```
In [ ]: # Quick scatter: FME score vs. gut Shannon diversity
plot_df = statistical_analysis_dataset.dropna(
    subset=["fme_score_daily", "shannon_diversity"]
)

plt.figure(figsize=(7, 5))
plt.scatter(plot_df["fme_score_daily"], plot_df["shannon_diversity"],
            alpha=0.5, s=25, c="teal", edgecolors="none")
if len(plot_df) > 2:
    z = np.polyfit(plot_df["fme_score_daily"], plot_df["shannon_diversity"], 1)
    x_line = np.linspace(plot_df["fme_score_daily"].min(),
                          plot_df["fme_score_daily"].max(), 100)
    plt.plot(x_line, np.poly1d(z)(x_line), "r--", lw=1.5, label="Trend line")
plt.xlabel("Daily FME Score (food microbiome exposure)")
plt.ylabel("Shannon Diversity (gut microbiome)")
plt.title("FME Score vs. Gut Alpha Diversity\n(475 fecal samples from 34 partici")
plt.legend()
plt.tight_layout()
plt.show()
```



Section 10 – Save Outputs

```
In [ ]: # Save statistical analysis dataset
statistical_analysis_dataset.to_csv(DATA_DIR / "fme_statistical_dataset.csv")
participant_gut_stability.to_csv(DATA_DIR / "participant_gut_stability.csv", index=False)
fme_with_participant.to_csv(DATA_DIR / "fme_sample_level_scores.csv", index=False)

print("Saved:")
print(f"  fme_statistical_dataset.csv    {statistical_analysis_dataset.shape} -")
print(f"  participant_gut_stability.csv    {participant_gut_stability.shape} - st")
print(f"  fme_sample_level_scores.csv      {fme_with_participant.shape} - FME sco
```

Saved:

fme_statistical_dataset.csv	(475, 26)	- main analytical table
participant_gut_stability.csv	(34, 5)	- stability per participant
fme_sample_level_scores.csv	(475, 4)	- FME scores with participant labels

Summary

What was built

The `fme_statistical_dataset.csv` table is the **primary input for all statistical analyses**.

Structural note (important for modelling)

- **Unit of observation:** fecal sample (475 rows)

- **Grouping variable:** `participant_id` (34 participants) → must be included as a **random effect**
- **Main predictor:** `fme_score_daily`
- **Main outcomes:** `shannon_diversity` (composition),
`participant_shannon_cv` (stability)

Recommended next steps

Mixed-effects model (composition):

```
shannon_diversity ~ fme_score_daily + Age + BMI + KCAL + (1 | participant_id)
```

Mixed-effects model (stability – participant level, n=34):

```
participant_shannon_cv ~ mean(fme_score_daily) + Age + BMI
```

Notebook:

`FME_Analysis_v2_extended.ipynb`

Food Microbiome Exposure (FME) Analysis

Research Question

How does exposure to food-associated microbes influence the composition and stability of the human gut microbiome?

What this notebook does

1. **Data Transfer** — Load all preprocessed datasets from both sources
2. **Data Mapping** — Map dietary food items to cFMD food categories
3. **FME Quantification** — Calculate a daily Food Microbiome Exposure score per fecal sample
4. **Statistical Dataset** — Assemble one clean table ready for statistical modelling

Key structural note

The Johnson et al. 2019 study has **34 participants**, each contributing multiple fecal samples across up to 17 study days. All FME scores are computed at the **fecal sample level** (one score per sample = one dietary day per person). Participant-level summaries are derived by aggregating across samples — **not** by treating samples as independent observations. The unit of analysis for statistical models is the **fecal sample**, with participant included as a random effect.

Section 1 – Libraries

```
In [ ]: # Locate repo root (directory containing config.py) regardless of launch directo
_root = next(p for p in [Path.cwd(), *Path.cwd().parents] if (p / "config.py").e
DATA_DIR = _root / "data"
sys.path.insert(0, str(_root))
```

Section 2 – Data Transfer: Load All Datasets

Source 1 — Johnson et al. 2019 (gut microbiome + diet)

34 participants · up to 17 fecal samples each · 566 samples total

Source 2 — Carlino et al. 2024 — cFMD (food microbiome)

3,393 food samples · 5,057 microbial taxa

```
In [ ]: #Gut microbiome matrices (taxa × fecal samples)
gut_microbiome_abundance = pd.read_csv(DATA_DIR / "microbiome_filtered.csv", in

#Fecal sample metadata
#index = fecal sample ID
#columns: UserName (= participant), StudyDayNo, Gender, Age, BMI
fecal_sample_metadata = pd.read_csv(DATA_DIR / "sample_meta.csv", in

#Dietary consumption matrix (food items × fecal samples)
#rows = food items, columns = fecal sample IDs
#values = consumption amount of that food on that day
dietary_consumption_matrix= pd.read_csv(DATA_DIR / "diet_raw.csv")

#Daily nutrition totals
#index = fecal sample ID
daily_nutrition_totals = pd.read_csv(DATA_DIR / "nutrition_totals.csv", in

#Additional dietary features
daily_fiber_scores = pd.read_csv(DATA_DIR / "diet_fiber.csv")
daily_decay_scores = pd.read_csv(DATA_DIR / "diet_decay.csv")
participant_fiber_avg = pd.read_csv(DATA_DIR / "fiber_avg.csv")

#cFMD food microbiome abundance (food taxa × food samples)
# 'StudyName__ShortSampleID' format
cfmd_food_abundance = pd.read_csv(DATA_DIR / "combined_datasets.csv", in
cfmd_food_abundance = cfmd_food_abundance.apply(pd.to_numeric, errors="coe

#cFMD sample metadata
#columns: sample_id, category, fermented/non-fermented, country
cfmd_food_metadata = pd.read_csv(DATA_DIR / "cfmd_metadata.csv")

#Verify participant count
n_participants = fecal_sample_metadata["UserName"].nunique()
n_fecal_samples= len(fecal_sample_metadata)
```



```

print(f"Participants      : {n_participants} (expected: 34)")
print(f"Fecal samples total : {n_fecal_samples}")
print(f"Avg samples/person  : {n_fecal_samples / n_participants:.1f}")
print()
print("Dataset sizes:")
for name, df in [
    ("gut_microbiome_abundance", gut_microbiome_abundance),
    ("fecal_sample_metadata", fecal_sample_metadata),
    ("dietary_consumption_matrix", dietary_consumption_matrix),
    ("daily_nutrition_totals", daily_nutrition_totals),
    ("cfmd_food_abundance", cfmd_food_abundance),
    ("cfmd_food_metadata", cfmd_food_metadata),
]:
    print(f" {name:<32} {str(df.shape)}")

```

```

Participants      : 34 (expected: 34)
Fecal samples total : 566
Avg samples/person : 16.6

```

Dataset sizes:

```

gut_microbiome_abundance      (223, 483)
fecal_sample_metadata         (566, 28)
dietary_consumption_matrix    (1452, 568)
daily_nutrition_totals        (580, 106)
cfmd_food_abundance           (5057, 3393)
cfmd_food_metadata            (3444, 32)

```

Section 3 – Define the Analytical Sample Set

Only fecal samples that have **both** a gut microbiome measurement **and** a dietary record form the analytical core. We identify these 475 samples here.

```

In [ ]: #Identify fecal sample IDs present in both gut microbiome and diet data
FECAL_SAMPLE_PREFIX = "MCT.f."

gut_microbiome_sample_ids = set(gut_microbiome_abundance.columns)
diet_record_sample_ids = set(
    c for c in dietary_consumption_matrix.columns
    if c.startswith(FECAL_SAMPLE_PREFIX)
)

#Core analytical set: samples with BOTH gut microbiome AND diet record
analytical_sample_ids = sorted(gut_microbiome_sample_ids & diet_record_sample_ids)

print(f"Gut microbiome samples      : {len(gut_microbiome_sample_ids)}")
print(f"Diet record samples          : {len(diet_record_sample_ids)}")
print(f"Analytical samples (gutndiet) : {len(analytical_sample_ids)}")
print()

#Participants represented in the analytical set
analytical_participants = (
    fecal_sample_metadata
    .loc[fecal_sample_metadata.index.isin(analytical_sample_ids), "UserName"]
    .nunique()
)

```

```

print(f"Participants represented      : {analytical_participants} / {n_particip

#Samples per participant in the analytical set
samples_per_participant = (
    fecal_sample_metadata
    .loc[fecal_sample_metadata.index.isin(analytical_sample_ids)]
    .groupby("UserName")
    .size()
    .rename("n_analytical_samples")
)
print(f"Avg analytical samples/person : {samples_per_participant.mean():.1f}")
print(f"Min / Max                      : {samples_per_participant.min()} / {samp

```

```

Gut microbiome samples      : 483
Diet record samples        : 566
Analytical samples (gutndiet) : 475

```

```

Participants represented      : 34 / 34
Avg analytical samples/person : 14.0
Min / Max                    : 6 / 17

```

Section 4 – FME Weights from cFMD

Goal: Derive a microbial richness weight for each cFMD food category.

Why richness, not total abundance? The `combined_datasets` matrix stores relative abundance — each sample sums to 100 regardless of food type. Total abundance is therefore uninformative as a weight. **Microbial richness** (= number of distinct taxa detected per food sample) reflects how microbiome-diverse each food category truly is.

Weight formula:

$$w_c = \frac{\text{mean_richness}_c - \min}{\max - \min} \in [0, 1]$$

Weights are derived directly from the **cFMD food abundance matrix** — not assumed or hand-coded.

```

In [ ]: #Map cFMD column names → food category + fermentation status
#combined_datasets columns: 'StudyName__ShortSampleID'
#cfmd_food_metadata sample_id: 'ShortSampleID'

def extract_cfmd_short_id(column_name):
    """Strip the study name prefix from a combined_datasets column name."""
    return column_name.split("__")[-1] if "__" in column_name else column_name

#Remove the one duplicate sample_id (VB15-29) before indexing
cfmd_food_metadata_dedup = cfmd_food_metadata.drop_duplicates(
    subset="sample_id", keep="first"
)
print(f"cFMD metadata after dedup: {len(cfmd_food_metadata_dedup)} rows "
      f"(removed {len(cfmd_food_metadata) - len(cfmd_food_metadata_dedup)} dupli

cfmd_id_to_meta = (
    cfmd_food_metadata_dedup

```

```

        .set_index("sample_id")["category", "fermented/non-fermented"]
        .to_dict(orient="index")
    )

    #Map each column in cfmd_food_abundance to its category and fermentation status
    cfmd_column_to_category = {}
    cfmd_column_to_fermented = {}
    for col in cfmd_food_abundance.columns:
        short_id = extract_cfmd_short_id(col)
        if short_id in cfmd_id_to_meta:
            cfmd_column_to_category[col] = cfmd_id_to_meta[short_id]["category"]
            cfmd_column_to_fermented[col] = cfmd_id_to_meta[short_id]["fermented/non-fermented"]

    print(f"CFMD columns mapped: {len(cfmd_column_to_category)} / {cfmd_food_abundance.columns.size}")

```

CFMD metadata after dedup: 3443 rows (removed 1 duplicate)

CFMD columns mapped: 3393 / 3393

```

In [ ]: # Compute microbial richness per food sample
        # Richness = number of non-zero taxa per food sample
        cfmd_taxa_richness_per_sample = (cfmd_food_abundance > 0).sum(axis=0)

        #Mean richness per CFMD food category
        category_richness_values = {}
        for col, category in cfmd_column_to_category.items():
            if col in cfmd_taxa_richness_per_sample.index:
                category_richness_values.setdefault(category, []).append(
                    cfmd_taxa_richness_per_sample[col]
                )

        cfmd_category_mean_richness = pd.Series(
            {cat: np.mean(vals) for cat, vals in category_richness_values.items()},
            name="mean_taxa_richness"
        ).sort_values(ascending=False)

        #Normalize weights to [0, 1]
        richness_min = cfmd_category_mean_richness.min()
        richness_max = cfmd_category_mean_richness.max()
        cfmd_category_weight_normalized = (
            (cfmd_category_mean_richness - richness_min) /
            (richness_max - richness_min)
        ).rename("fme_weight")

        cfmd_weights_table = pd.concat(
            [cfmd_category_mean_richness, cfmd_category_weight_normalized], axis=1
        )

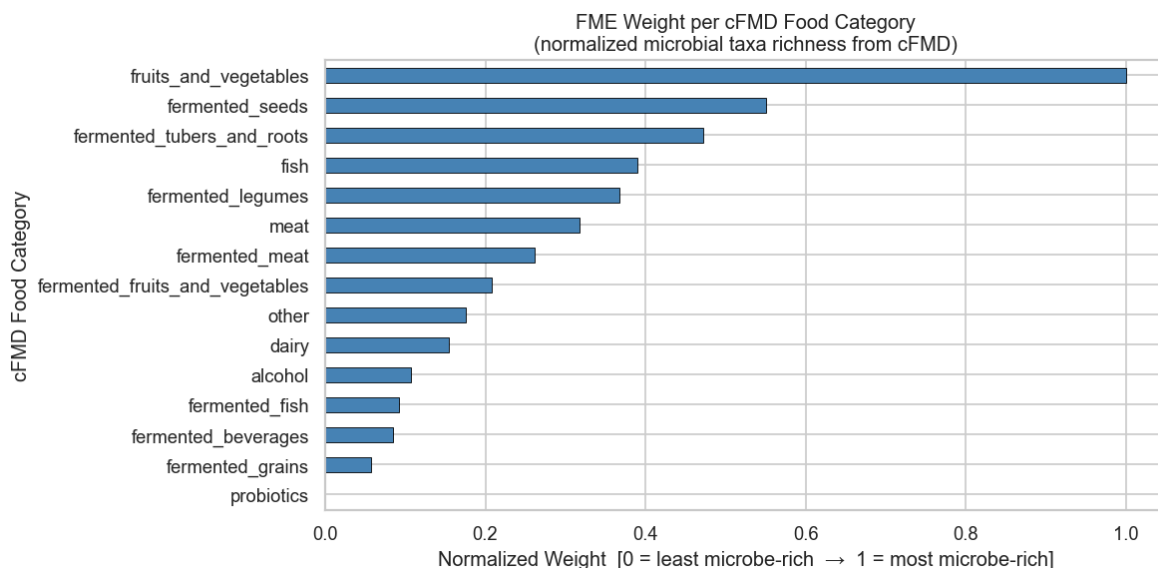
        print("FME weights per CFMD food category (from CFMD microbial richness):")
        display(cfmd_weights_table.round(3))

```

FME weights per CFMD food category (from CFMD microbial richness):

	mean_taxa_richness	fme_weight
fruits_and_vegetables	120.1250	1.0000
fermented_seeds	69.8000	0.5510
fermented_tubers_and_roots	61.0420	0.4720
fish	51.7950	0.3900
fermented_legumes	49.2630	0.3670
meat	43.6490	0.3170
fermented_meat	37.4110	0.2610
fermented_fruits_and_vegetables	31.4810	0.2080
other	27.8750	0.1760
dairy	25.3960	0.1540
alcohol	20.0850	0.1070
fermented_fish	18.4000	0.0920
fermented_beverages	17.6160	0.0850
fermented_grains	14.5120	0.0570
probiotics	8.1430	0.0000

```
In [ ]: #Visualize FME weights
plt.figure(figsize=(10, 5))
cfmd_category_weight_normalized.sort_values().plot(
    kind="barh", color="steelblue", edgecolor="black", linewidth=0.5
)
plt.title("FME Weight per cFMD Food Category\n(normalized microbial taxa richness
plt.xlabel("Normalized Weight [0 = least microbe-rich → 1 = most microbe-rich
plt.ylabel("cFMD Food Category")
plt.tight_layout()
plt.show()
```



Section 5 – Data Mapping: Diet Items → cFMD Food Categories

Goal: Map each food item in the Johnson et al. dietary records to a cFMD food category, creating a unified representation of food-derived microbial exposure.

Method: Extract the L1 (top-level) taxonomy label from the `taxonomy` column of `diet_raw` (e.g. `L1_Milk_and_Milk_Products` → `dairy`) and map it to the matching cFMD category.

```
In [ ]: #Extract L1 food category from taxonomy string
def extract_l1_food_category(taxonomy_string):
    """
    Extract the top-level food category (L1) from the hierarchical taxonomy string
    Example: 'L1_Milk_and_Milk_Products;L2...' → 'milk and milk products'
    """
    if pd.isna(taxonomy_string):
        return "unknown"
    l1_segment = str(taxonomy_string).split(";")[0]
    return l1_segment.replace("L1_", "").replace("_", " ").strip().lower()

dietary_consumption_matrix["diet_l1_category"] = (
    dietary_consumption_matrix["taxonomy"]
    .apply(extract_l1_food_category)
)

print("Unique L1 diet categories found:")
for cat, count in dietary_consumption_matrix["diet_l1_category"].value_counts():
    print(f" {cat:<45} {count:>5} food items")
```

Unique L1 diet categories found:

grain product	407 food items
vegetables	315 food items
meat poultry fish and mixtures	240 food items
sugars sweets and beverages	153 food items
milk and milk products	112 food items
dry beans peas other legumes nuts and seeds	84 food items
fruits	74 food items
fats oils and salad dressings	36 food items
eggs	31 food items

```
In [ ]: #Mapping table: diet L1 category to cFMD category
DIET_L1_TO_CFMD_CATEGORY = {
    #Dairy
    "milk and milk products" : "dairy",
    "dairy" : "dairy",
    "cheese" : "dairy",
    "yogurt" : "dairy",
    #Meat
    "meat poultry and game" : "meat",
    "meat" : "meat",
    "poultry" : "meat",
    #Fish
    "fish and shellfish" : "fish",
    "fish" : "fish",
    "seafood" : "fish",
    #Fruits and vegetables
```

```

"vegetables and vegetable products" : "fruits_and_vegetables",
"fruit"                             : "fruits_and_vegetables",
"fruits"                            : "fruits_and_vegetables",
"vegetables"                        : "fruits_and_vegetables",
#Fermented grains
"grain products"                    : "fermented_grains",
"grains"                           : "fermented_grains",
"bread"                            : "fermented_grains",
"cereals and cereal products"       : "fermented_grains",
#Alcohol / fermented beverages
"alcoholic beverages"               : "alcohol",
"alcohol"                           : "alcohol",
"beverages"                         : "fermented_beverages",
"nonalcoholic beverages"            : "fermented_beverages",
#Legumes
"legumes and legume products"       : "fermented_legumes",
"legumes"                           : "fermented_legumes",
"beans"                             : "fermented_legumes",
#Seeds / nuts
"nuts and seeds"                    : "fermented_seeds",
"seeds"                             : "fermented_seeds",
#Other / unmappable
"fats and oils"                     : "other",
"sugars and sweets"                 : "other",
"soups sauces and gravies"          : "other",
"mixed dishes"                      : "other",
"snacks"                           : "other",
"eggs"                             : "other",
"spices and herbs"                  : "other",
"unknown"                           : "unmapped",
}

dietary_consumption_matrix["cfmd_category"] = (
    dietary_consumption_matrix["diet_l1_category"]
    .map(DIET_L1_TO_CFMD_CATEGORY)
    .fillna("unmapped")
)

n_mapped    = (dietary_consumption_matrix["cfmd_category"] != "unmapped").sum()
n_unmapped  = (dietary_consumption_matrix["cfmd_category"] == "unmapped").sum()
print(f"Food items mapped to a cFMD category : {n_mapped} / {len(dietary_consump
print(f"Food items unmapped                      : {n_unmapped}")

```

Food items mapped to a cFMD category : 532 / 1452

Food items unmapped : 920

```

In [ ]: #Attach FME weight to each food item
dietary_consumption_matrix["fme_weight"] = (
    dietary_consumption_matrix["cfmd_category"]
    .map(cfmd_category_weight_normalized.to_dict())
    .fillna(0.0)    #unmapped items get weight = 0
)

mapping_summary = (
    dietary_consumption_matrix
    .groupby(["diet_l1_category", "cfmd_category", "fme_weight"])
    .size()
    .reset_index(name="n_food_items")
    .sort_values("n_food_items", ascending=False)
)

```

```
print("Mapping summary (diet L1 → cFMD category → weight):")
display(mapping_summary)
```

Mapping summary (diet L1 → cFMD category → weight):

	diet_l1_category	cfmd_category	fme_weight	n_food_items
4	grain product	unmapped	0.0000	407
8	vegetables	fruits_and_vegetables	1.0000	315
5	meat poultry fish and mixtures	unmapped	0.0000	240
7	sugars sweets and beverages	unmapped	0.0000	153
6	milk and milk products	dairy	0.1541	112
0	dry beans peas other legumes nuts and seeds	unmapped	0.0000	84
3	fruits	fruits_and_vegetables	1.0000	74
2	fats oils and salad dressings	unmapped	0.0000	36
1	eggs	other	0.1762	31

Section 6 – FME Score Calculation

Formula:

$$\text{FME}_{\text{sample}} = \sum_{\text{food items}} \text{consumption}_{\text{food, sample}} \times w_{\text{cfmd_category}(\text{food})}$$

Where:

- $\text{consumption}_{\text{food, sample}}$ = amount consumed (from `diet_raw`, grams or servings)
- w = normalized cFMD microbial richness weight for the food's category

Important: Each `sample` corresponds to **one dietary day for one participant**. The result is a score per fecal sample — not per participant. Participant-level scores are derived later by averaging across their samples.

```
In [ ]: #Subset diet matrix to analytical samples only
diet_abundance_matrix = (
    dietary_consumption_matrix[analytical_sample_ids]
    .apply(pd.to_numeric, errors="coerce")
    .fillna(0.0)
)

fme_weight_vector = dietary_consumption_matrix["fme_weight"].values # shape: (n

#Compute FME score: weighted sum of food consumption per sample
#Result: one FME score per fecal sample (= one dietary day per participant)
fme_score_per_sample = (
    diet_abundance_matrix
    .multiply(fme_weight_vector, axis=0)
    .sum(axis=0)
```

```

        .rename("fme_score_daily")
    )
    fme_score_per_sample.index.name = "fecal_sample_id"

    print(f"FME scores computed for : {len(fme_score_per_sample)} fecal samples")
    print(f"Score range           : {fme_score_per_sample.min():.4f} - {fme_score_per_sample.max():.4f}")
    print(f"Score mean ± std       : {fme_score_per_sample.mean():.4f} ± {fme_score_per_sample.std():.4f}")
    print(f"Non-zero scores        : {(fme_score_per_sample > 0).sum()}")
    print()
    display(fme_score_per_sample.reset_index().head(8))

```

```

FME scores computed for : 475 fecal samples
Score range           : 0.0000 - 331.2240
Score mean ± std       : 73.3252 ± 58.3071
Non-zero scores        : 437

```

	fecal_sample_id	fme_score_daily
0	MCT.f.0002	67.1961
1	MCT.f.0003	81.2412
2	MCT.f.0004	126.7711
3	MCT.f.0005	31.4846
4	MCT.f.0006	61.2266
5	MCT.f.0007	62.0231
6	MCT.f.0008	70.9874
7	MCT.f.0009	54.0086

```

In [ ]: #Participant level FME summary (aggregate across samples)
        #Participant level values are used only for descriptive summaries,

        fme_with_participant = pd.DataFrame({
            "fecal_sample_id" : fme_score_per_sample.index,
            "fme_score_daily" : fme_score_per_sample.values,
        })
        fme_with_participant["participant_id"] = (
            fecal_sample_metadata
            .loc[fme_with_participant["fecal_sample_id"], "UserName"]
            .values
        )
        fme_with_participant["study_day"] = (
            fecal_sample_metadata
            .loc[fme_with_participant["fecal_sample_id"], "StudyDayNo"]
            .values
        )

        #Participant-level summary (descriptive only)
        participant_fme_summary = (
            fme_with_participant
            .groupby("participant_id")["fme_score_daily"]
            .agg(
                fme_mean    = "mean",
                fme_std      = "std",
                fme_min      = "min",
                fme_max      = "max",
            )

```



```

        n_diet_days = "count"
    )
    .reset_index()
)

print("Participant-level FME summary (descriptive – 34 participants):")
display(participant_fme_summary.sort_values("fme_mean", ascending=False).head(10))

```

Participant-level FME summary (descriptive – 34 participants):

	participant_id	fme_mean	fme_std	fme_min	fme_max	n_diet_days
1	MCTs03	147.8899	45.7548	68.0651	243.3253	12
5	MCTs07	143.4859	48.3133	86.0332	243.8429	16
2	MCTs04	143.1190	81.0506	36.6199	331.2240	15
6	MCTs08	140.2178	33.6074	84.4009	220.3516	17
14	MCTs16	130.9598	45.2371	69.7540	221.0863	14
27	MCTs31	128.7335	37.7598	83.9027	208.7920	15
23	MCTs26	122.0334	71.3654	24.5678	315.7055	17
16	MCTs19	99.4541	40.7179	14.6490	166.4480	15
17	MCTs20	88.6724	51.9185	24.4219	178.8966	12
24	MCTs27	85.2074	47.0410	5.0752	165.2448	14

```

In [ ]: #Visualize FME scores
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

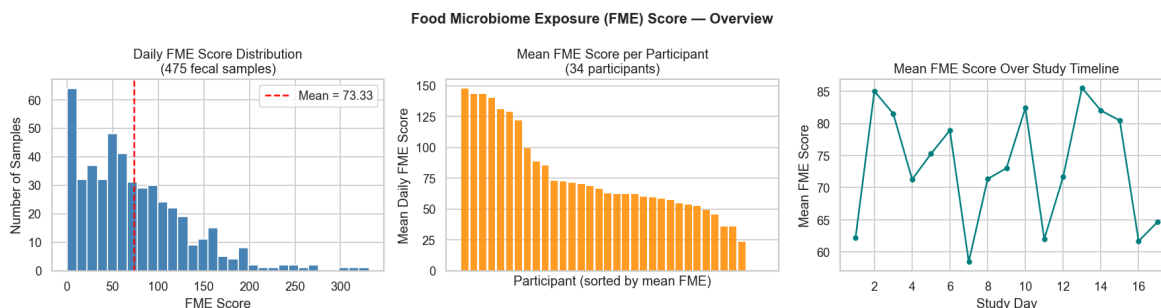
#Distribution of daily FME scores across all fecal samples
axes[0].hist(fme_score_per_sample.values, bins=30,
             color="steelblue", edgecolor="white")
axes[0].axvline(fme_score_per_sample.mean(), color="red", linestyle="--",
               label=f"Mean = {fme_score_per_sample.mean():.2f}")
axes[0].set_title("Daily FME Score Distribution\n(475 fecal samples)")
axes[0].set_xlabel("FME Score")
axes[0].set_ylabel("Number of Samples")
axes[0].legend()

#Mean FME per participant (sorted)
sorted_participants = participant_fme_summary.sort_values("fme_mean", ascending=
axes[1].bar(range(len(sorted_participants)), sorted_participants["fme_mean"],
           color="darkorange", edgecolor="none", alpha=0.85)
axes[1].set_title(f"Mean FME Score per Participant\n({len(sorted_participants)})")
axes[1].set_xlabel("Participant (sorted by mean FME)")
axes[1].set_ylabel("Mean Daily FME Score")
axes[1].set_xticks([])

#FME over study timeline (mean per study day)
fme_by_day = fme_with_participant.groupby("study_day")["fme_score_daily"].mean()
axes[2].plot(fme_by_day.index, fme_by_day.values,
            marker="o", color="teal", linewidth=1.5, markersize=4)
axes[2].set_title("Mean FME Score Over Study Timeline")
axes[2].set_xlabel("Study Day")
axes[2].set_ylabel("Mean FME Score")

```

```
plt.suptitle("Food Microbiome Exposure (FME) Score – Overview", fontsize=13, fontcolor='red')
plt.tight_layout()
plt.show()
```



Section 7 – Gut Microbiome Alpha Diversity per Fecal Sample

Compute **Shannon entropy** for each fecal sample as a measure of gut microbial richness and evenness. This is the primary outcome variable (composition proxy) at the sample level.

```
In [ ]: #Alpha diversity metrics per fecal sample

#Computing Shannon entropy using the formula below, where p_i is the proportion
#Shannon entropy H = -Σ p_i * log(p_i) for non zero taxa
def compute_shannon_entropy(sample_val):
    pos_val = sample_val[sample_val > 0]
    if pos_val.sum() == 0:
        return np.nan
    prop = pos_val / pos_val.sum()
    return float(-np.sum(prop * np.log(prop)))

#Computing richness of taxa per sample
#Richness = number of non-zero taxa
def compute_richness(sample_val):
    pos_val = sample_val[sample_val > 0]
    return int(len(pos_val))

#Computing Simpson diversity using the formula below
#Simpson diversity = 1 / Σ p_i^2
def compute_simpson_diversity(sample_val):
    pos_val = sample_val[sample_val > 0]
    if pos_val.sum() == 0:
        return np.nan
    prop = pos_val / pos_val.sum()
    return float(1 / np.sum(prop ** 2))

#Computing Inverse Simpson using the formula below
#Inverse Simpson = 1 / Σ p_i^2
def compute_inverse_simpson(sample_val):
    pos_val = sample_val[sample_val > 0]
    if pos_val.sum() == 0:
        return np.nan
    prop = pos_val / pos_val.sum()
    denominator = np.sum(prop ** 2)
```

```

    if denominator == 0:
        return np.nan
    return float(1 / denominator)

#Computing Pielou evenness using the formula below
#Pielou evenness = Shannon / log(richness)
def compute_pielou_evenness(sample_val):
    pos_val = sample_val[sample_val > 0]
    richness = len(pos_val)

    if richness <= 1:
        return np.nan

    if pos_val.sum() == 0:
        return np.nan

    prop = pos_val / pos_val.sum()
    shannon = -np.sum(prop * np.log(prop))

    return float(shannon / np.log(richness))

gut_microbiome_analytical = gut_microbiome_abundance[analytical_sample_ids]

shannon_diversity_per_sample = (
    gut_microbiome_analytical
    .apply(compute_shannon_entropy, axis=0)
    .rename("shannon_diversity")
)

richness_per_sample = (
    gut_microbiome_analytical
    .apply(compute_richness, axis=0)
    .rename("richness")
)

simpson_diversity_per_sample = (
    gut_microbiome_analytical
    .apply(compute_simpson_diversity, axis=0)
    .rename("simpson_diversity")
)

inverse_simpson_per_sample = (
    gut_microbiome_analytical
    .apply(compute_inverse_simpson, axis=0)
    .rename("inverse_simpson")
)

pielou_evenness_per_sample = (
    gut_microbiome_analytical
    .apply(compute_pielou_evenness, axis=0)
    .rename("pielou_evenness")
)

alpha_diversity_per_sample = pd.concat(
    [
        shannon_diversity_per_sample,
        richness_per_sample,
        simpson_diversity_per_sample,
        inverse_simpson_per_sample,

```

```

        pielou_evenness_per_sample,
    ],
    axis=1
)

alpha_diversity_per_sample.index.name = "fecal_sample_id"

print(f"Alpha diversity computed for {alpha_diversity_per_sample.shape[0]} samples")
display(alpha_diversity_per_sample.describe().T.round(3))

```

Alpha diversity computed for 475 samples

	count	mean	std	min	25%	50%	75%	
shannon_diversity	475.0000	2.8650	0.4130	1.5720	2.6110	2.9640	3.1430	
richness	475.0000	203.5620	11.9540	140.0000	198.0000	207.0000	212.0000	22
simpson_diversity	475.0000	0.8600	0.0760	0.4650	0.8310	0.8860	0.9100	
inverse_simpson	475.0000	8.7810	3.6000	1.8690	5.9330	8.7990	11.1220	2
pielou_evenness	475.0000	0.5390	0.0750	0.2990	0.4930	0.5560	0.5920	

Section 8 – Gut Microbiome Stability per Participant

Stability = how consistent is a participant's gut microbiome across their sampling days?

We use the **coefficient of variation (CV)** of Shannon diversity across a participant's samples:

$$CV_{participant} = \frac{\sigma(\text{Shannon})}{\mu(\text{Shannon})}$$

Lower CV = more stable microbiome.

This is computed at the **participant level** (34 values) and joined back to the sample-level table as a per-participant attribute.

```

In [ ]: #Map each fecal sample to its participant
sample_to_participant = (
    fecal_sample_metadata
    .loc[analytical_sample_ids, "UserName"]
    .rename("participant_id")
)

shannon_with_participant = pd.DataFrame({
    "fecal_sample_id" : shannon_diversity_per_sample.index,
    "shannon_diversity": shannon_diversity_per_sample.values,
}).set_index("fecal_sample_id")
shannon_with_participant["participant_id"] = sample_to_participant

#Participant-level stability (CV of Shannon across their samples)

```

```

participant_gut_stability = (
    shannon_with_participant
    .groupby("participant_id")["shannon_diversity"]
    .agg(
        participant_shannon_mean = "mean",
        participant_shannon_std = "std",
        participant_n_samples = "count"
    )
    .assign(
        participant_shannon_cv = lambda df:
            df["participant_shannon_std"] / df["participant_shannon_mean"]
    )
    .reset_index()
)

print(f"Gut stability computed for {len(participant_gut_stability)} participants")
print(f"Shannon CV range : {participant_gut_stability['participant_shannon_cv'].
      f"{participant_gut_stability['participant_shannon_cv'].max():.4f}")")
display(participant_gut_stability.sort_values("participant_shannon_cv").head(8))

```

Gut stability computed for 34 participants

Shannon CV range : 0.0202 - 0.1799

	participant_id	participant_shannon_mean	participant_shannon_std	participant_n_samp
5	MCTs07	3.0731	0.0621	
17	MCTs20	3.2696	0.0736	
29	MCTs33	3.0863	0.0876	
11	MCTs13	3.1848	0.0942	
19	MCTs22	2.9985	0.0912	
7	MCTs09	3.0362	0.0948	
30	MCTs34	2.9806	0.1022	
8	MCTs10	3.0279	0.1045	

Section 9 – Assemble the Statistical Analysis Dataset

Merge all variables into one clean table at the **fecal sample level**. Each row = one fecal sample = one dietary day for one participant.

Column group	Columns	Role in statistical model
Identity	fecal_sample_id , participant_id , study_day	Grouping / random effect
Demographics	Age , BMI , Gender	Fixed-effect covariates
FME score	fme_score_daily	Main predictor

Column group	Columns	Role in statistical model
Gut diversity	shannon_diversity	Main outcome
Gut stability	participant_shannon_cv	Secondary outcome
Nutrition	KCAL , FIBE , PROT , TFAT , CARB	Dietary confounders

```
In [ ]: #Base table - fecal sample IDs
statistical_analysis_dataset = pd.DataFrame(
    index=pd.Index(analytical_sample_ids, name="fecal_sample_id")
)

#Participant identity and demographics
participant_meta_cols = ["UserName", "StudyDayNo", "Gender", "Age", "BMI",
                        "Weight", "Supplement", "Medications"]
participant_meta_cols = [c for c in participant_meta_cols
                        if c in fecal_sample_metadata.columns]

statistical_analysis_dataset = statistical_analysis_dataset.join(
    fecal_sample_metadata[participant_meta_cols]
    .rename(columns={"UserName": "participant_id", "StudyDayNo": "study_day"}),
    how="left"
)

#Daily FME score
statistical_analysis_dataset = statistical_analysis_dataset.join(
    fme_score_per_sample.rename("fme_score_daily"),
    how="left"
)

#Participant level and within person FME scores
#The daily FME score is already weighted using cFMD derived microbial richness w
statistical_analysis_dataset["fme_participant_mean"] = (
    statistical_analysis_dataset
    .groupby("participant_id")["fme_score_daily"]
    .transform("mean")
)

statistical_analysis_dataset["fme_within_person"] = (
    statistical_analysis_dataset["fme_score_daily"]
    - statistical_analysis_dataset["fme_participant_mean"]
)

#Lagged and weighted recent FME scores
#The weighted 3 day score gives more weight to the current day and lower weight

fme_time_sorted = statistical_analysis_dataset.sort_values(
    ["participant_id", "study_day"]
).copy()

fme_time_sorted["fme_lag1"] = (
    fme_time_sorted
    .groupby("participant_id")["fme_score_daily"]
    .shift(1)
)

fme_time_sorted["fme_lag2"] = (
```

```

fme_time_sorted
.groupby("participant_id")["fme_score_daily"]
.shift(2)
)

weighted_sum = (
    0.5 * fme_time_sorted["fme_score_daily"].fillna(0)
+ 0.3 * fme_time_sorted["fme_lag1"].fillna(0)
+ 0.2 * fme_time_sorted["fme_lag2"].fillna(0)
)

available_weight = (
    0.5 * fme_time_sorted["fme_score_daily"].notna().astype(float)
+ 0.3 * fme_time_sorted["fme_lag1"].notna().astype(float)
+ 0.2 * fme_time_sorted["fme_lag2"].notna().astype(float)
)

fme_time_sorted["fme_weighted_3day"] = weighted_sum / available_weight

statistical_analysis_dataset[
    ["fme_lag1", "fme_lag2", "fme_weighted_3day"]
] = fme_time_sorted[
    ["fme_lag1", "fme_lag2", "fme_weighted_3day"]
]

#Gut alpha diversity metrics
statistical_analysis_dataset = statistical_analysis_dataset.join(
    alpha_diversity_per_sample,
    how="left"
)

#Participant level gut stability
statistical_analysis_dataset = statistical_analysis_dataset.join(
    participant_gut_stability.set_index("participant_id")[
        ["participant_shannon_cv", "participant_shannon_mean", "participant_n_sa"]
    ],
    on="participant_id",
    how="left"
)

#Daily nutrition totals
nutrition_cols_keep = ["KCAL", "PROT", "TFAT", "CARB", "FIBE",
    "SUGR", "SODI", "D_TOTAL", "D_YOGURT", "D_CHEESE",
    "PF_MEAT", "PF_SEAFD_HI", "G_WHOLE"]
nutrition_cols_keep = [c for c in nutrition_cols_keep
    if c in daily_nutrition_totals.columns]

statistical_analysis_dataset = statistical_analysis_dataset.join(
    daily_nutrition_totals[nutrition_cols_keep],
    how="left"
)

print(f"Statistical dataset shape : {statistical_analysis_dataset.shape}")
print(f"Fecal samples (rows) : {statistical_analysis_dataset.shape[0]}")
print(f"Variables (columns) : {statistical_analysis_dataset.shape[1]}")
print(f"Participants represented : {statistical_analysis_dataset['participant_']")
print(f"Complete rows (no NaN) : {statistical_analysis_dataset.dropna().shap

```

```

Statistical dataset shape : (475, 35)
Fecal samples (rows)      : 475
Variables (columns)       : 35
Participants represented   : 34
Complete rows (no NaN)    : 188

```

```

In [ ]: #Check if the new columns were added correctly
statistical_analysis_dataset[
    [
        "shannon_diversity",
        "richness",
        "simpson_diversity",
        "inverse_simpson",
        "pielou_evenness"
    ]
].head()

```

```

Out[ ]:      shannon_diversity  richness  simpson_diversity  inverse_simpson  pielou_

```

```
fecal_sample_id
```

MCT.f.0002	2.8627	211	0.8720	7.8101
MCT.f.0003	3.5228	208	0.9473	18.9896
MCT.f.0004	3.0312	213	0.8918	9.2447
MCT.f.0005	2.9738	207	0.8825	8.5109
MCT.f.0006	3.0811	209	0.9000	10.0002



```

In [ ]: #Check if there is not null values in the new columns
statistical_analysis_dataset[
    [
        "shannon_diversity",
        "richness",
        "simpson_diversity",
        "inverse_simpson",
        "pielou_evenness"
    ]
].isna().sum()

```

```

Out[ ]: shannon_diversity    0
richness                    0
simpson_diversity           0
inverse_simpson             0
pielou_evenness             0
dtype: int64

```

```

In [ ]: #Check if the new FME columns were added correctly
statistical_analysis_dataset[
    [
        "participant_id",
        "fme_score_daily",
        "fme_participant_mean",
        "fme_within_person"
    ]
].head(10)

```


Out[]:

	participant_id	fme_score_daily	fme_participant_mean	fme_within_perso
fecal_sample_id				
MCT.f.0002	MCTs01	67.1961	71.2710	-4.074
MCT.f.0003	MCTs01	81.2412	71.2710	9.970
MCT.f.0004	MCTs01	126.7711	71.2710	55.500
MCT.f.0005	MCTs01	31.4846	71.2710	-39.786
MCT.f.0006	MCTs01	61.2266	71.2710	-10.044
MCT.f.0007	MCTs01	62.0231	71.2710	-9.247
MCT.f.0008	MCTs01	70.9874	71.2710	-0.283
MCT.f.0009	MCTs01	54.0086	71.2710	-17.262
MCT.f.0010	MCTs01	105.6813	71.2710	34.410
MCT.f.0011	MCTs01	62.2759	71.2710	-8.995

In []:

```
#Check if there are no null values in the new FME columns
statistical_analysis_dataset[
    [
        "fme_score_daily",
        "fme_participant_mean",
        "fme_within_person"
    ]
].isna().sum()
```

Out[]:

```
fme_score_daily      0
fme_participant_mean  0
fme_within_person    0
dtype: int64
```

In []:

```
#Descriptive statistics for the FME score variants
statistical_analysis_dataset[
    [
        "fme_score_daily",
        "fme_participant_mean",
        "fme_within_person"
    ]
].describe().round(3)
```

Out []:

	fme_score_daily	fme_participant_mean	fme_within_person
count	475.0000	475.0000	475.0000
mean	73.3250	73.3250	0.0000
std	58.3070	39.8040	42.6070
min	0.0000	0.0000	-106.4990
25%	29.3220	53.6970	-25.0860
50%	61.7780	62.2510	-3.0860
75%	104.2700	99.4540	17.1070
max	331.2240	147.8900	251.2760

In []: *#Missing value report*

```

missing_report = (
    statistical_analysis_dataset.isnull().sum()
    .reset_index()
    .rename(columns={"index": "variable", 0: "n_missing"})
)
missing_report["pct_missing"] = (
    missing_report["n_missing"] / len(statistical_analysis_dataset) * 100
).round(1)
missing_report = missing_report[missing_report["n_missing"] > 0].sort_values("n_

if missing_report.empty:
    print("No missing values in the statistical dataset.")
else:
    print("Missing values:")
    display(missing_report)

```

Missing values:

	variable	n_missing	pct_missing
7	Medications	255	53.7000
12	fme_lag2	68	14.3000
11	fme_lag1	34	7.2000

In []: *#Descriptive statistics*

```

key_vars = [
    "fme_score_daily",
    "fme_participant_mean",
    "fme_within_person",
    "fme_lag1",
    "fme_lag2",
    "fme_weighted_3day",
    "shannon_diversity",
    "richness",
    "simpson_diversity",
    "inverse_simpson",
    "pielou_evenness",
    "participant_shannon_cv",
    "Age",
    "BMI",

```

```

    "KCAL",
    "FIBE"
]

key_vars = [v for v in key_vars if v in statistical_analysis_dataset.columns]

print("Descriptive statistics - key variables:")
display(statistical_analysis_dataset[key_vars].describe().T.round(3))

```

Descriptive statistics - key variables:

	count	mean	std	min	25%	50%
fme_score_daily	475.0000	73.3250	58.3070	0.0000	29.3220	61.7780
fme_participant_mean	475.0000	73.3250	39.8040	0.0000	53.6970	62.2510
fme_within_person	475.0000	0.0000	42.6070	-106.4990	-25.0860	-3.0860
fme_lag1	441.0000	73.9720	59.2070	0.0000	29.5690	61.6000
fme_lag2	407.0000	74.8860	60.1780	0.0000	30.0080	61.7780
fme_weighted_3day	475.0000	73.4000	49.3200	0.0000	38.8190	64.5830
shannon_diversity	475.0000	2.8650	0.4130	1.5720	2.6110	2.9640
richness	475.0000	203.5620	11.9540	140.0000	198.0000	207.0000
simpson_diversity	475.0000	0.8600	0.0760	0.4650	0.8310	0.8860
inverse_simpson	475.0000	8.7810	3.6000	1.8690	5.9330	8.7990
pielou_evenness	475.0000	0.5390	0.0750	0.2990	0.4930	0.5560
participant_shannon_cv	475.0000	0.0630	0.0380	0.0200	0.0360	0.0490
Age	475.0000	31.1520	10.1560	19.7000	23.9000	28.5000
BMI	475.0000	23.1750	3.2920	17.0000	21.1000	22.9000
KCAL	475.0000	2110.5630	735.4510	722.5520	1599.1800	2028.1680
FIBE	475.0000	22.3370	11.6000	3.6180	14.8500	19.6940

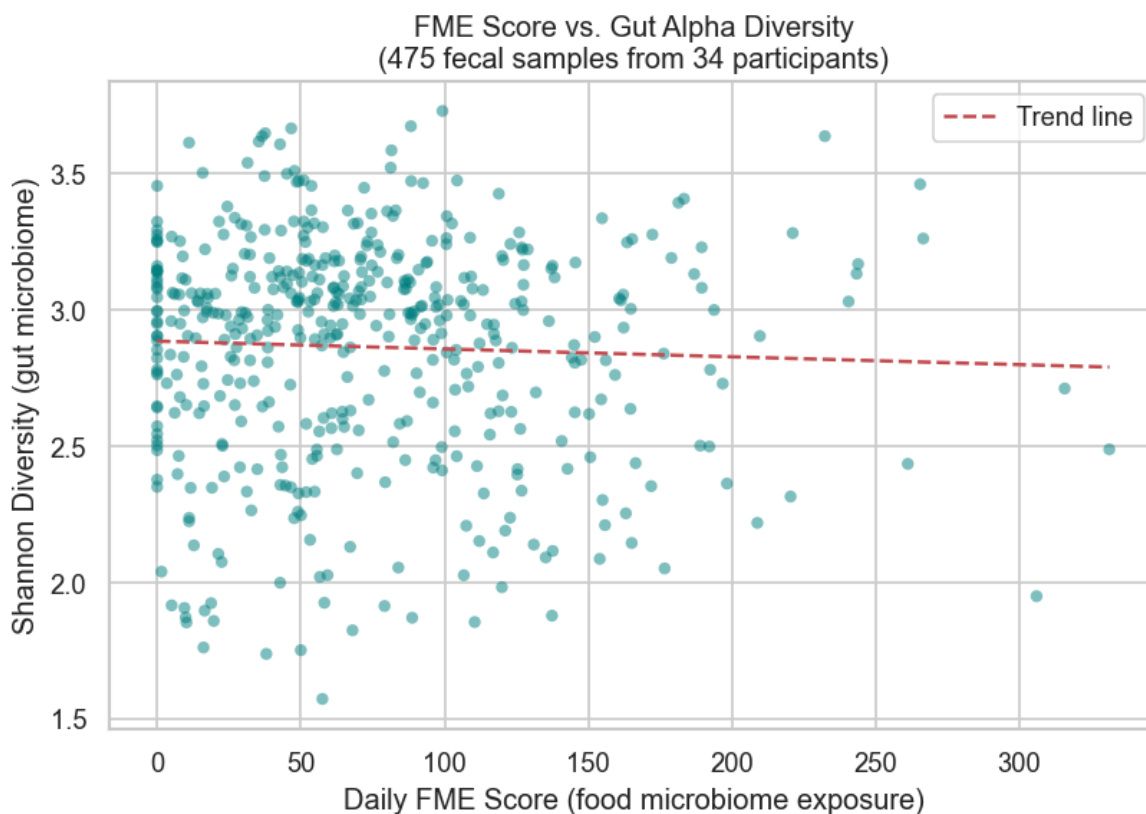
```

In [ ]: #Quick scatter: FME score vs. gut Shannon diversity
plot_df = statistical_analysis_dataset.dropna(
    subset=["fme_score_daily", "shannon_diversity"]
)

plt.figure(figsize=(7, 5))
plt.scatter(plot_df["fme_score_daily"], plot_df["shannon_diversity"],
            alpha=0.5, s=25, c="teal", edgecolors="none")
if len(plot_df) > 2:
    z = np.polyfit(plot_df["fme_score_daily"], plot_df["shannon_diversity"], 1)
    x_line = np.linspace(plot_df["fme_score_daily"].min(),
                          plot_df["fme_score_daily"].max(), 100)
    plt.plot(x_line, np.poly1d(z)(x_line), "r--", lw=1.5, label="Trend line")
plt.xlabel("Daily FME Score (food microbiome exposure)")
plt.ylabel("Shannon Diversity (gut microbiome)")
plt.title("FME Score vs. Gut Alpha Diversity\n(475 fecal samples from 34 partici
plt.legend()

```

```
plt.tight_layout()
plt.show()
```



Section 10 – Save Outputs

```
In [ ]: #Save statistical analysis dataset
statistical_analysis_dataset.to_csv(DATA_DIR / "fme_statistical_dataset_extended
participant_gut_stability.to_csv(DATA_DIR / "participant_gut_stability.csv", ind
fme_with_participant.to_csv(DATA_DIR / "fme_sample_level_scores.csv", index=False

print("Saved:")
print(f" fme_statistical_dataset_extended.csv    {statistical_analysis_dataset.
print(f" participant_gut_stability.csv    {participant_gut_stability.shape} - st
print(f" fme_sample_level_scores.csv    {fme_with_participant.shape} - FME sco
```

Saved:

```
fme_statistical_dataset_extended.csv    (475, 35) - main analytical table
participant_gut_stability.csv    (34, 5) - stability per participant
fme_sample_level_scores.csv    (475, 4) - FME scores with participant labels
```

Summary

What was built

The `fme_statistical_dataset_extended.csv` table is the **primary input for the extended statistical analyses**.

This dataset extends the original statistical table by adding additional gut microbiome alpha diversity metrics, participant-level FME features, and short-term lagged FME

exposure features.

Added alpha diversity metrics

In addition to the original `shannon_diversity`, the following alpha diversity metrics were added:

- `richness` — number of non-zero taxa in each fecal sample
- `simpson_diversity` — dominance-sensitive diversity metric
- `inverse_simpson` — effective diversity based on Simpson index
- `pielou_evenness` — evenness of the microbial community distribution

Added FME features

The original daily FME score was kept as:

- `fme_score_daily`

The daily FME score is already weighted using cFMD-derived microbial richness weights.

In addition, participant-level FME variables were added:

- `fme_participant_mean` — participant-level mean of the daily weighted FME scores
- `fme_within_person` — deviation of each daily FME score from that participant's mean

The participant-level mean is therefore calculated as the average of the already-weighted daily FME scores, not as a new weighting scheme.

Added lagged and short-term FME exposure features

To capture possible short-term dietary effects before or around each fecal sample, additional time-based FME features were added:

- `fme_lag1` — the previous FME score for the same participant, ordered by `study_day`
- `fme_lag2` — the FME score two previous observations back for the same participant, ordered by `study_day`
- `fme_weighted_3day` — a weighted recent FME score using the current score, lag 1, and lag 2

The weighted 3-day FME score is calculated as:

$$\text{fme_weighted_3day} = 0.5 * \text{current FME} + 0.3 * \text{lag1 FME} + 0.2 * \text{lag2 FME}$$

When lag values are missing, for example in the first or second sample of a participant, the weighted average is normalized using only the available values.

Structural note — important for modelling

- **Unit of observation:** fecal sample, 475 rows
- **Grouping variable:** `participant_id`, 34 participants
- Because several fecal samples belong to the same participant, `participant_id` should be included as a **random effect** in sample-level models.

Main predictors

- `fme_score_daily`
- `fme_participant_mean`
- `fme_within_person`
- `fme_lag1`
- `fme_lag2`
- `fme_weighted_3day`

Main outcomes

- `shannon_diversity`
- `richness`
- `simpson_diversity`
- `inverse_simpson`
- `pielou_evenness`
- `participant_shannon_cv` for participant-level stability analysis

Recommended next steps

Mixed-effects model for sample-level alpha diversity using daily FME:

```
alpha_diversity_metric ~ fme_score_daily + Age + BMI + KCAL + FIBE +
PROT + TFAT + CARB + (1 | participant_id)
```

Extended mixed-effects model separating between-person and within-person FME effects:

```
alpha_diversity_metric ~ fme_participant_mean + fme_within_person + Age
+ BMI + KCAL + FIBE + PROT + TFAT + CARB + (1 | participant_id)
```

Short-term exposure mixed-effects model using recent weighted FME:

```
alpha_diversity_metric ~ fme_weighted_3day + Age + BMI + KCAL + FIBE +
PROT + TFAT + CARB + (1 | participant_id)
```

Lagged exposure mixed-effects model:

```
alpha_diversity_metric ~ fme_lag1 + fme_lag2 + Age + BMI + KCAL + FIBE
+ PROT + TFAT + CARB + (1 | participant_id)
```

Participant-level stability model:

```
participant_shannon_cv ~ fme_participant_mean + Age + BMI
```

Interpretation

The extended dataset allows us to test whether FME is associated not only with Shannon diversity, but also with richness, dominance-sensitive diversity metrics, and microbial

evenness.

It also allows us to distinguish between:

- **Between-person effects** — whether participants with higher average FME differ in microbiome diversity
- **Within-person effects** — whether days when a participant has higher-than-usual FME are associated with different microbiome diversity
- **Short-term exposure effects** — whether recent FME values from the current and previous observations are associated with microbiome diversity